

15 - Data Scientist:

Natural Language Processing Specialist Career Path

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsnlp-welcome/modules/dsnlp-welcome-to-the-data-scientist-nlp-specialist-career-path/informationals/welcome-to-the-data-scientist-natural-language-processing-career-path>

Syllabus

1. Introduction to Machine Learning Engineer Career Path

Introduction to the Machine Learning Engineer Career Path

1 Informational, 1 Lesson, 1 Article

2. Machine Learning Fundamentals

Welcome to Machine Learning Fundamentals

1 Informational

Introduction to Machine Learning

Feature Engineering I: Data Transformations

Supervised Learning I : Regressors, Classifiers and Trees

Feature Engineering II : Feature Selection Methods

Unsupervised Learning Algorithms I

Assessment: Machine Learning Fundamentals

[Take assessment](#)

3. Software Engineering for Machine Learning/AI Engineers

Welcome to Software Engineering for Machine Learning/AI Engineers

1 Informational

18% progress

18%

Software Engineering in Python I

4 Lessons, 4 Quizzes, 4 Projects, 1 Video, 3 Articles

5% progress

5%

Learn the Command Line

3 Articles, 4 Lessons, 4 Quizzes, 5 Projects, 1 Informational

Learn Git: Introduction to Version Control

1 Informational, 3 Videos, 3 Lessons, 4 Projects, 3 Quizzes, 1 External resource, 4 Articles

Software Engineering in Python II

4 Lessons, 3 Quizzes, 2 Projects, 2 Articles

Bash Scripting

1 Lesson, 1 Quiz, 1 Project

Learn Git II: Git for Deployment

2 Informationals, 1 Video, 4 Articles, 1 External resource, 2 Lessons, 5 Projects, 2 Quizzes

Assessment: Software Engineering for Machine Learning/AI Engineers

[Take assessment](#)

4. Intermediate Machine Learning

Welcome!

1 Informational

Supervised Learning II: Advanced Regressors and Classifiers

Regularization and Hyperparameter Turning

75% progress

75%

Ensemble Methods in Machine Learning

2 Articles, 2 Lessons, 2 Quizzes, 2 Projects

Assessment: Intermediate Machine Learning

[Take assessment](#)

5. Building Machine Learning Pipelines

Machine Learning Pipelines: Bringing Machine Learning and Engineering together

1 Informational

Build Machine Learning Pipelines

1 Article, 1 Lesson, 1 Project

6. Machine Learning/AI Engineer: Final Portfolio

Machine Learning Engineer Final Portfolio

1 Kanban project

7. Machine Learning Career Path: Next Steps

Next Steps

1 Informational

Welcome to the Data Scientist: NLP Specialist Career Path

Your data science journey begins here!

Natural Language Processing (NLP for short) is an incredible field at the intersection of Linguistics and Programming. It's also a major part of Artificial Intelligence. But the reason it is so incredible is illustrated by a simple example. What does the following sentence mean?

Codey taught the woman with the computer.

If you said that it's ambiguous, you're right! There are two meanings: either Codey has the computer or the woman does. No matter *who* has the computer, the computer wasn't designed to understand ambiguity. That's why Natural Language Processing is so revolutionary—it's a way to tackle ambiguity with computers. You will learn all about what that looks like in this Data Scientist: Natural Language Processing Specialist Career Path.

About the Data Scientist Career Paths

The Data Scientist Career Paths are designed for you to gain the knowledge, skills, and abilities (KSAs) you need to earn a data scientist job. You will develop the statistical knowledge, programming skills, and visualization techniques to work with data responsibly. You will also build a portfolio along the way to showcase your abilities to future employers.

You are currently enrolled in the Data Scientist: Natural Language Processing (NLP) Specialist Career Path. Data Scientists who focus on NLP specialize in working with linguistic data, including both text and speech. As an NLP-based Data Scientist, you might work on a data science team, or fill a specific role on a team like customer support or product. You might be building chatbots, analyzing large amounts of user-generated text, or parsing speech data.

This Career Path is divided into 2 parts. The first section covers all the basics every Data Scientist needs. We call this the Foundations of Data Science. The second section is the specialized KSAs you need to go from being a data generalist to an NLP Data Scientist.

If after learning a bit more about data science, you decide that you'd like to change your specialty, your progress will transfer, allowing you to pick up where you left off. To learn more about the specialties, check out [this blog post](#).

For a quick summary:

Natural Language Processing Specialists focus on getting meaning out of texts, generating human-like text, and interacting with computers for Artificial Intelligence.

Machine Learning Specialists focus on Machine Learning, predictive analytics, creating models and algorithms, and answering questions at scale.

Analytics Specialists focus on answering questions with data, communicating results, and driving decision making.

Inference Specialists focus on finding out why something happened with causal inference, conducting hypothesis tests, and A/B tests, and validating results statistically.

Overview

In the first half of this Career Path, you will learn about Python, SQL, statistics, and data best practices. You will start with a code-free introduction to working with data in the Data Literacy Track and then dive into *SQL* and *Python* in the SQL Fundamentals and Python Fundamentals Tracks. SQL and Python are essential tools that will give you the technical foundation you need to work with data programmatically.

Then you will dive into Data Manipulation with Python Pandas. Pandas is a Python library at the heart of Data Science. It is an essential tool for working with data and has dozens of useful built-in functions. Once you have mastered Pandas, you will apply your new programming skills to Exploratory Data Analysis where you will learn to summarize datasets before mastering the Fundamentals of Statistics where you will learn to spot trends and patterns.

Next, you will learn the basics of machine learning that underpin a wide variety of tasks in NLP such as topic modeling (a type of cluster analysis), and sentiment analysis (often achieved with Logistic Regression). Then you will move on to deep learning, and how neural networks have changed the field of NLP, and apply your new skills to building chatbots. Finally, you will master the most important techniques for transforming linguistic data into numerical data, and understand the assumptions we must make to make that possible.

At this point in your journey, you will have completed the Data Foundations and be ready to move into your specialty.

NLP Specialty

In the NLP Specialty, you will learn the most practical tools and techniques to get meaning from data. You'll use data responsibly to drive decision-making, and give evidence for and against various hypotheses.

But, if you decide that you want to switch specialties after you learn more about Data Science, you can pick from any of our four specialties – and your progress will transfer!

By the end of the Data Scientist: NLP Specialist Career Path, you will be able to:

Create programs with Python 3

Query and manipulate data with SQL and Python pandas

Create data visualizations with Python matplotlib and seaborn

Summarize and analyze datasets

Conduct hypothesis testing

Design experiments

Clean and tidy datasets

Use supervised and unsupervised machine learning to answer questions and develop models

Use statistical methods to evaluate large and small datasets

Transform text into data

Build chatbots

Apply neural networks and deep learning to NLP tasks

Leverage Jupyter Notebooks for experimentation and communication

Along the way, you will build a portfolio of projects to showcase your data science KSA's, and prove your job-ready skills!

Structure

Throughout this Path, you will see lessons, quizzes, articles, and projects. Lessons and articles introduce new tools and concepts, quizzes give you a chance to quickly check your understanding, and projects give you a chance to apply your new skills.

There are a lot of projects, but pay special attention to Portfolio Projects and Cumulative Projects because they give you a chance to apply your skills and build up your portfolio.

The content in this Career Path is cumulative, and we recommend that you take the courses in order. However, if you are familiar with a given technology or idea, feel free to jump ahead, but be aware that skipping modules will affect your

Why Data Science?

Begin Your Journey

Catherine learned and applied a lot during the first two weeks of her job. She:

- Explored churn data using SQL
- Explored Codecademy's datasets
- Analyzed data using pandas, a Python library
- Visualized data using Matplotlib, a Python library

Now it's your turn. By the end of the Data Science Path, you'll be able to do all of the things that Catherine can do. We will also be adding new content every month.

Good luck and happy coding!

Data Types and Quality

Review of Data Types and Quality

Congratulations! You've done the hard work of collecting your data and preparing it for analysis. Along the way you've addressed some data quality issues and considered the relationship between your data and your questions.

While preparing the North Atlantic Tree Census, you've:

- Defined your [variables](#)
- Preview: Docs Loading link description to create tidy datasets
- Classified the variables based on their types
- Reconciled messy data
- Decided how to deal with missing data
- Addressed issues of accuracy
- Aligned your questions to the available data to ensure validity
- Created representative samples

These are important techniques for anyone working with data to always be conscious of. In the end, your human oversight and critical consideration of the data will have the biggest impact on data quality.



Make the Most of Your Codecademy Membership

Learn about the different features Codecademy has to offer!

Codecademy has helped [tens of millions of people](#) to learn to code, everything from `hello world` to getting new tech jobs, but when it comes down to it, you get out what you put in. If you want to be best set up for success, this guide is for you.

Valuable Resources

There are hundreds of hours of free content on Codecademy, and thousands of hours of material available with our paid plans; but no matter your membership level, the key to success is learning intelligently. If you build the habits to use our features, it will be easier for you to master technical skills.

Mobile App

Keep practicing and stay sharp as you go with our free mobile app on [iOS](#) and [Android](#).

Workspaces

[Workspaces](#) allow learners to work in their own [integrated development environment \(IDE\)](#) right inside Codecademy. Build or try whatever you want for free with these unguided sandboxes, share your code, or keep it private. Not sure what to make? Grab code from your coursework and use it as a template.

Cheatsheets

Coding is less about memorization and more about understanding principles; cheatsheets help you quickly refresh your memory and get back to work. You can find them linked in lessons, or head to their [homepage](#) to find them all, broken down by the course section in which you learned them.

Docs

Cheatsheets are a handy companion for consuming coursework, but when coding in the real world, programmers need documentation. With [Docs](#), you have an entirely open-source, free reference material to cover far more than a course can.

Learn What to Learn

Not certain what language to pick up or what career path to choose? Flick through the [Learn What to Learn course](#), where we've consolidated the best resources to help you make a choice with confidence. An hour of planning now could save you days of wasted effort later.

Community

Learning to code is challenging, but it's easier if you can team up. Ask questions, stay connected, and find motivation with the Codecademy Community!:

[Events](#)

[Facebook group](#)

Video Library

Need a break from the IDE but still want to be productive? Watch hundreds of educational and motivational video content on everything from basic coding concepts to career advice, all on-topic and [on-platform](#).

Paid Features

If you want to supercharge your learning, a paid plan may be right for you — here are some of our most popular paid features.

Paths

[Skill Paths](#) and [Career Paths](#) are step-by-step roadmaps that take you through what you need to learn a specific skill or transition to a specific career in tech. They'll tell you what to learn and in what order, cutting out the guesswork and to the chase. Career Paths cover everything that you'll find in a \$15,000 boot camp curriculum, plus more. Pro learners also get access to Interview Prep Skill Paths.

Projects, Challenge Projects, and Portfolio Projects

As you learn, you'll work on guided projects to put the things you're learning to use. You'll find a variety of project types. Some offer step-by-step guidance while others offer more of a challenge by only describing the final outcome of the project. If you take a Path, you'll build your own [Portfolio Projects](#) as well, designed to simulate professional coding tasks. The solutions to these portfolio projects will be unique to you! Check out all the projects available to you via the [Projects Library](#).

Project Tasks

Keep track of your progress by dragging each task from "To Do" to "In Progress" to "Done" as you work on them. You can also click on a task to see more information about it.

To Do	In Progress	Done 0/6 done
Create your files On your computer...		
Set up version control Set up Git tracking in...		
Build the home page The home page will be...		
Build the contact page The contact page incl...		
Style the content Now that you have your...		

Interview prep

If you're taking one of our Career Paths, you'll find interview prep essentials built-in. Learn interview techniques to prepare for technical interviews, including algorithm and data structure practice. Go even further with [dedicated skill paths](#) on acing the interview across a range of languages and disciplines.

Practice & Review

With your Pro membership, you'll find it easier to brush up on what you've learned. Choose for yourself what to practice with links from each module of your coursework. Or let us take the guesswork out with smart practice. Click the practice session link from syllabi and AI will prioritize concepts for you by using spaced repetition — a [scientifically proven](#) way of helping you memorize new ideas.

codecademy / PRO



PRO Career Path

Front-End Engineer

Resume Path

Keep practicing

Front-end engineers work closely with designers to make websites beautiful, functional, and fast. This Career Path will teach you the technologies you need to do just that.

Path Progress

Reset Progress

Continue on your path to unlock your cohort

Earn a certificate of completion

71 Projects

121 Lessons

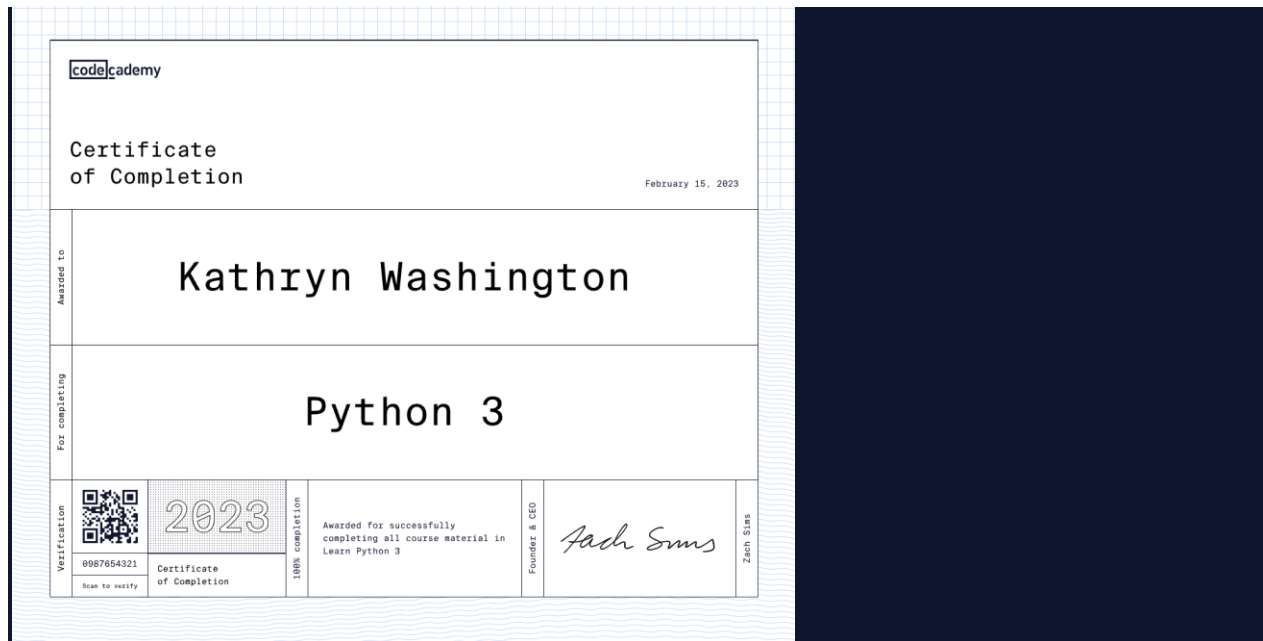
Beginner Friendly

Quizzes

Test yourself as you make your way through a Path or course. Quizzes help you retain all the things you're learning — plus, they'll help you feel confident that you're mastering the material.

Certificates

When you complete a Pro course, you'll earn a certificate. You can share it on your resume or your LinkedIn profile to showcase your skills or prepare for your job search.



In select career paths, we also offer professional certifications. In these paths, you'll earn a professional certification for passing all of the exams in the path.

Code Challenges

With technical interviews (and coding in general), practice makes perfect. Now, you can practice real code challenges from actual interviews to see how your skills stack up, or just solve them for fun. If you get stuck, we'll point you to what you still need to learn. Code challenges are available with Pro.

Conclusion

If you made it this far, you're a highly motivated learner - we're excited to see how far you'll go, and hope some of the features you learned about today help. Please take a moment to [answer two quick questions](#) for feedback on this resource. Happy coding!

Welcome to Data Literacy

Establishing how to think about data will set you up for success when you start analyzing it.

Every job is a data job!

– every Data Scientist ever

Organizations are fueled by data, and data scientists and analysts are at the forefront of working with it. But they don't work alone, and even the most technically sophisticated data scientist will need to communicate about data.

This unit will help you build the conceptual foundation you need both to work with data yourself and present your insights to adjacent teams.

There's no programming yet, but we will get there. For now, join us on a journey to explore the building blocks of data, and establish common vocabulary and ideas.

What will Data Literacy cover?

After this unit, you will be able to:

We're excited for you to start your journey into Data Literacy!

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Case Studies in Data Literacy

Wrap Up

Wowie, we've covered a lot! The road to confident data literacy is full of fascinating examples, and knowledge that helps us make sense of one of the most powerful tools of our age: data.

In this lesson, we covered data quality and data ethics, and saw how interrogating data quality can lead to better outcomes in healthcare. Along with that, we saw how recognizing and addressing bias leads to stronger data and deeper truth.

We learned how people have used

[statistics](#)

Preview: Docs Loading link description

to solve important but nebulous legal problems like hiring discrimination. Knowing how to use statistics has huge consequences for learning about issues that are too big to fully address one-by-one.

We saw how visualizations can make or break a data conclusion when it comes to communicating findings. While most of us aren't making or interpreting life-or-death data viz, learning from the Challenger space shuttle explosion can help everyone to make more intentional, meaningful decisions around data visualization.

Finally, we talked about data analysis. We covered the importance of context when it comes to interpreting data: not just what the numbers are, but what they mean. And we learned about correlation and causation, capping things off with a triumph for modern medicine based on good data visualization and causal analysis.

Hot dog! That's a lot of reasons to get excited about data literacy.

Data Collection methods, ethics and free sources

Learn about data collection methods, ethical considerations, and free dataset sources to make informed decisions in data analysis.

Most people are aware that data visualizations, machine learning algorithms, and analyses require data. But where does that data come from? Does everyone have access to data for analysis?

Understanding data ethics and privacy

Whenever we talk about data collection, we need to discuss data ethics. Much of the data available to us comes from individuals and would be considered **personally identifiable**, meaning we could use it to identify someone. Many people use the acronym PII (pronounced "pie" — yum!) for the term "personally identifiable information". Examples of PII are address, email, phone number, social security numbers, credit card numbers, and medical records. We all have an obligation to protect personally identifiable information.

Ethical issues regarding data collection may be divided into the following categories:

Methods of collecting data

We collect, process, and analyze data to better understand our world and make more informed decisions. The first step in any data work is to collect the data itself. Data can come from a lot of places, including research, governments, technology, observation, or directly from individuals – the list is endless!

We collect this data in many different ways. One way is to seek out information that doesn't yet exist and measure it directly. This can include activities like surveys, observational studies, or recording the results of an experiment. This kind of data might be considered *static*, meaning the information is collected once and does not change. Think about conducting a survey by mail: the survey results are collected and recorded only once.



Data can also be live and ever-changing based on the most up-to-date information. For example, apps and websites can track clicks and time spent on pages across multiple users at the same time without a human actively recording all the data points. Unlike the static data of more traditional methods, sensors and trackers can also continuously update data to include new information in a live feed. Think about weather predictions: the data that goes into weather predictions are updated continuously to get the most accurate predictions.

Finally, rather than collect measurements directly, we can also use existing data that was collected by others or for some other purpose. There are lots of databases that are freely available for public use. We can even compile data from a variety of sources and join them together before an analysis.

Where to find free datasets for analysis?

Many organizations house all kinds of data. Datasets are often kept private or can only be accessed for a fee. This may be done for reasons like protecting the identity of individuals, keeping valuable information from competitors, or making a profit from data collection.

The following list has links and descriptions for websites that provide free access to some interesting datasets. The companies and organizations on this list provide public access to data, allowing anyone with internet access to view this information. The websites vary in how they provide data access: some may have a CSV or Excel file of data that can easily be downloaded to a computer, while others allow access to a database via an API.

[World Health Organization \(WHO\)](#)

[Mental health](#)

[Road traffic mortality](#)

[FiveThirtyEight](#)

[Political polls](#)

[The best NBA players](#)

[Data.gov](#)

[Storm Events Database](#)

[Census data for the United States of America](#)

[Census data for EU countries](#)

[Data Unicef](#)



Looking for something specific? [Google Dataset Search](#) works like a google search bar for datasets. We think the following datasets look really interesting!

[Orchids](#)

[Biodiversity at U.S. national parks](#)

[Revenue of the cosmetic & beauty industry in the U.S.](#)

Conclusion

Understanding data collection methods, ethics, and sources is essential for responsible data analysis. Ethical considerations ensure privacy and fairness, while diverse data collection techniques help gather accurate insights. Free datasets from trusted sources like WHO, Data.gov, and Google Dataset Search provide valuable opportunities for exploration. By applying these principles, you can make informed, ethical decisions in data-driven projects.

Statistical Thinking

Review

Congratulations! We've finished our summary report for the mayor of Melody Metropolis, and you've just completed the lesson on statistical thinking! We covered a lot of information, including:

Summary

[statistics](#)

Preview: Docs Loading link description
give us measurements to describe our data.

Categorical

[variables](#)

Preview: Docs Loading link description
can be described with frequencies and proportions.

Numeric variables can be described using measures of center and spread.

Sometimes we need robust statistics like the median and IQR because our data is skewed or has outliers.

We can aggregate data to get a look at relationships between pairs of variables.

Scatter plots and correlation coefficients help us describe linear relationships between pairs of numeric variables.

And hopefully, the most important discovery you made is that learning statistics doesn't have to be a struggle. We gain valuable skills just by understanding some basic concepts. Enjoy using your new data vocabulary!

Data Visualization Basics

Conclusion

We made it! This lesson covered everything you need to know to start designing data visualizations that look great, communicate effectively, and treat data ethically.

Here's a recap of what we talked about...

- Choosing the right chart type for our data

- Mapping

- [variables](#)

- Preview: Docs Stores data that can be manipulated and referenced throughout the code.

- onto aesthetic properties to communicate using position, shape, size, color

- Making effective color choices for data, design, and accessibility

- Designing accessible visualizations

- Scaffolding the visualization with context for the audience

- Owning the role of author

That's a big chunk of information. Pat yourself on the back for making it to the end of this lesson, then go forth and visualize!

Misleading and Confusing Graphs

Conclusion

That was a short primer on some common misleading parts of data visualizations – pitfalls that get between **the actual truth of the data** and **how the viewer understands it**.

People make misleading or confusing

[graphs](#)

Preview: Docs Loading link description

on purpose and by accident. We've seen how intentional changes to the scaling can really distort what the graph seems to say, and also how going with the status quo on color palettes can be helpful or harmful. Most of us aren't out here trying to make misleading graphs – very often, it's just a question of making a better or worse decision while designing the data viz.

And improving our design skills – knowing when to use different color palettes and how to label our axes clearly – not only makes our graphs better, **it also reduces bias**! When we follow best practices in design, we're not "going rogue" and just making design decisions that make the graph look the most extreme, or suit our needs at that moment. We're following established practices that help to standardize graphs so that we can focus on what the data's really saying.

Long story short: good data viz is always a combination of quality data and effective design choices. If we can avoid confusion or missteps along the way, that'll always be a good thing!

Data Viz Basics Project

Help Janis the intern make a visualization to share with Dandelion City's Backyard Birders Association!

Building Effective Data Visualizations

You've joined Shinji, Paola, and Raj in Dr. Dinkle's ecology lab! The lab is swamped with its latest project after a successful round of funding (award-winning board game designer Claude Tuber wants to turn the lab's work into the tabletop role-playing game of a generation). They're excited to have your data visualization skills!

In a few weeks the lab will present data on birdfeeder birds to the Backyard Birders Association of nearby Dandelion City. One component is a visualization of the wingspans of the three most common backyard birds in Dandelion City: the Northern Cardinal, American Robin, and Blue Jay.

The lab's paid intern Janis is trying her chops at data visualization, and will be drafting all of the visuals for this presentation. With your knowledge of data visualization, your task is to help her make the most effective visualization of wingspan data to present to the backyard birders of Dandelion City.

Here's the data we're working with:

Chart Type

First up... picking the right chart type for our data.

Free response

Which of Janis's drafts best fits the data in the table?

Submit Response

Color Palette

Now that we've gotten the chart type to align with our data, let's move on to the next important consideration: color.

Free response

Help Janis pick the color palette that best represents the information in the data table and will be effective for the broadest audience.

Submit Response

Annotations

With chart type and color palette sorted, let's look on to annotations that will help our audience make the most of the final chart.

Free response

Which version of Janis's annotations are most helpful?

Submit Response

Design

And finally, let's finish up with some design considerations.

Free response

Help Janis choose a design that helps to communicate information beautifully and effectively.

Submit Response

With your guidance, Janis has produced a readable and beautiful visualization for the presentation to the Backyard Birders, and the lab's work moves smoothly

Data Analyses and Conclusions Review

Congratulations on finishing the lesson! We covered a lot of ground and introduced quite a few ideas in this lesson. Here is a recap of what we have learned:

Data analysis is the process of mathematically summarizing and manipulating data to discover useful information, inform conclusions, and support decision-making.

Data analysis can be broken down into 5 main types—Descriptive, Exploratory, Inferential, Causal, and Predictive—that are more or less appropriate depending on the situation.

Descriptive analysis describes major patterns in data through summary

[statistics](#)

Preview: Docs Loading link description
and visualization of measures of central tendency and spread.

Exploratory analysis explores relationships between

[variables](#)

Preview: Docs Loading link description
within a dataset and can group subsets of data. Exploratory analyses might reveal correlations between variables.

Inferential analysis allows us to test hypotheses on a small sample of a population and extend our conclusions to the entire population.

Causal analysis lets us go beyond correlation and actually assign causation when we carefully design and conduct experiments. In addition, causal inference sometimes allows us to determine causal effects even when experimentation is not possible.

Predictive analysis goes beyond understanding the past and present and allows us to make data-driven predictions about the future. The quality of these predictions is deeply dependent on the quality of the data used to generate the predictions.

Having a basic understanding of each type of data analysis, when to use it, and how to interpret results is a big step in data literacy. Now you will be able to better interpret data-driven conclusions presented to you and make the most of your own data!

Bias in Data Analysis

Bias is everywhere in data. The key to combatting bias is knowing what to look out for.

Would you follow your GPS anywhere? Even into a lake? That may sound ridiculous, but a quick [google search](#) brings up dozens of cases where drivers drove into lakes and rivers because their GPS instructed them to. Following GPS instructions against your better judgment is one example of **automation bias**.



As humans, we have many biases, both implicit and explicit. Biases are systematic errors in thinking influenced by cultural and personal experiences. Biases distort our perception and cause us to make incorrect decisions. One bias that many humans share is **automation bias**. Automation bias stems from the idea that computers or machines are more trustworthy than humans because they are more objective. Automation bias is at the root of why people follow their GPS into trouble, even when contradictory information is available.

Computers, data, and [algorithms](#) are not actually completely objective. It is true that data analysis can help us make better decisions, but it is not immune to bias. Humans create technologies and algorithms. As a result, they often have human biases encoded into them. It's clear that we need to pay attention to other information streams (our eyes and ears) when we drive with GPS. Similarly, we need to look at more information sources when we evaluate data analysis results or reports.

If we want to be responsible when we use data and algorithms, we need to understand the different types of bias that show up at each stage of analyzing data. Let's take a closer look at some types of bias that impact data analysis and data-driven decision-making.

Bias in data collection

Before we can analyze data or use machine learning algorithms, we need to collect data. Data collection is subject to **selection bias** (also called sample bias). Selection bias occurs when study subjects (*i.e.*, the sample) are not representative of the population. Selection bias can be due to poor study design if the sample is too small or is not randomized. Selection bias can also crop up when the only data available is influenced by **historical bias** — systematic influence based on historic social and cultural beliefs.

[A Reuters article from 2018](#) highlights how the company Amazon produced a machine-learning algorithm that suffered from such a selection bias. The company designed the algorithm to help recruiters hire top talent. The model was trained on thousands of resumes from people that were or were not hired by Amazon. It learned 50,000 phrases associated with resumes and began to ignore common phrases, such as the names of programming languages. However, the algorithm also learned to downgrade resumes that contained the word "women's." This included resumes that referenced women's colleges, teams, or committees.

This is an example of selection bias because the data used to train the algorithm were not representative of the modern applicant pool. The majority of Amazon's past applicants and employees were male. This means a larger proportion of the successful resumes in the training data came from male applicants. Amazon did not explicitly train the algorithm to use gender. Yet, the algorithm still found and used gender-associated terms to weed out women candidates.

We can do our best to avoid selection bias by doing everything possible to have a representative sample, not just a convenient one. For example, it's a good idea to include data inputs from multiple sources to diversify data. This is easier said than done, however, and we need to acknowledge and address historical bias in data sources and work towards building frameworks to increase inclusivity.

Bias in building and optimizing algorithms

Algorithmic bias arises when an algorithm produces systematic and repeatable errors that lead to unfair outcomes, such as privileging one group over another. Algorithmic bias can be initiated through selection bias and then reinforced and perpetuated by other bias types.

Facial recognition software is an area where algorithmic bias can do a lot of harm. This software is sold to police departments and used to recognize criminals in surveillance footage. If the software systematically makes more mistakes depending on race or gender, people in some groups will be incorrectly pursued more often, which has serious, negative outcomes for individuals.

[The Gender Shades project](#) tested commercial facial recognition software for these kinds of biases. IBM, Microsoft, and Face++ are three companies that offer facial recognition software with a binary gender classifier feature. Researchers assessed the accuracy of these algorithms and discovered

that they suffered from algorithmic bias. The algorithms were good at identifying lighter males, okay at identifying darker males and lighter females, and very bad at identifying darker females.

Each software used proprietary algorithms and did not report performance results with benchmarking datasets. However, the developers probably tested the software on one of two commonly-used benchmarking datasets: Adience or IJB-A. These datasets include few dark-skinned people and especially low proportions of dark-skinned females. Testing an algorithm with a non-representative dataset leads to **evaluation bias**. Testing with a non-representative benchmarking dataset would give high overall accuracy scores, even if the algorithms were inaccurate for certain groups.

Another key point when it comes to algorithmic bias in facial recognition software is that the algorithms are proprietary, making them “black boxes”. In addition to not knowing what data were used to train and test the algorithm, we can’t know how it was designed or how it works. As a result, it’s impossible to evaluate the algorithms themselves.

Avoiding algorithmic bias relies on transparency, especially concerning data used for training and testing an algorithm. In response to the poor performance of facial recognition with darker females, a new benchmarking dataset was developed (PPB) that is more representative of the full spectrum of humanity. This is a big step forward, as long as the new dataset is actually used by companies making and selling facial recognition software.



Data for this plot came from [Buolamwini et al. 2018, Proceedings of Machine Learning Research](#).

Bias in interpreting results and drawing conclusions

Bias also influences the final stages of data analysis: interpreting results and drawing conclusions. The following bias types are ones we should watch out for when evaluating or generating data reports:

Confirmation bias is our tendency to seek out information that supports our views. Confirmation bias influences data analysis when we consciously or unconsciously interpret results in a way that supports our original hypothesis. To limit confirmation bias, clearly state hypotheses and goals before starting an analysis, and then honestly evaluate how they influenced our interpretation and reporting of results.

Overgeneralization bias is inappropriately extending observations made with one dataset to other datasets, leading to overinterpreting results and unjustified extrapolation. To limit overgeneralization bias, be thoughtful when interpreting data, only extend results beyond the dataset used to generate them when it is justified, and only extend results to the proper population.

Reporting bias is the human tendency to only report or share results that affirm our beliefs or hypotheses, also known as “positive” results. Editors, publishers, and readers are also subject to reporting bias as positive results are published, read, and cited more often. To limit reporting bias, report negative results and cite others who do, too.

Conclusions

Data and machine learning algorithms are now ubiquitous. They influence decisions about who is hired or fired, accepted into schools, or allowed to rent houses. They even influence which neighborhoods are more heavily policed and who is granted parole. Therefore, we must recognize that data and algorithms can be biased, just like the humans who create and train them. Learning more about the types of bias that influence how algorithms function will improve our ability to perform and interpret data analyses and will help us make more informed decisions.

The Role of Databases in Data Science

Gear up for the fun you’ll have in SQL Fun-damentals!

As a data scientist or data analyst, you will likely be solving business problems with data (and therefore databases) regularly. Even if you aren’t the person deep into the data, you will have to understand how databases work in order to understand how data is stored. This unit will get you ready for answering business questions with the most popular database language – SQL!

What will SQL Fundamentals cover?

- Manipulating Data
- Querying Data
- Aggregating Data
- Working with Multiple Tables
- Data Acquisition

Why did we build this?

Databases are at the core of Data Science and are where most data is stored. Understanding databases is an essential skill for everyone who works with data.

We decided to introduce you to databases before Python because databases are more foundational to data science and there is a lower barrier to entry. Once you complete this track, you will immediately be able to apply your skills to real data problems and query data to answer questions.

If you would prefer to get started with Python first, feel free to jump ahead and come back to SQL and databases later. It will be here. Databases have been at the core of data science for decades, they'll be here when you get back.

Why Learn SQL?

Begin Your Journey

Catherine learned and applied a lot during the first two weeks of her job. She:

- Wrote basic queries
- Calculated aggregates
- Combined data from multiple tables
- Created usage funnels
- Analyzed user churn
- Determined web traffic attribution

Now it's your turn. By the end of this course, you'll be able to do all of the things that Catherine can do.

Good luck and happy coding!

What is a Relational Database Management System?

Learn about RDBMS and the language used to access large datasets – SQL.



What is a Database?

A *database* is a set of data stored in a computer. This data is usually structured in a way that makes the data easily accessible.

What is a Relational Database?

A *relational database* is a type of database. It uses a structure that allows us to identify and access data *in relation* to another piece of data in the database. Often, data in a relational database is organized into tables.

Tables: Rows and Columns

Tables can have hundreds, thousands, sometimes even millions of rows of data. These rows are often called *records*.

Tables can also have many *columns* of data. Columns are labeled with a descriptive name (say, *age* for example) and have a specific *data type*.

For example, a column called *age* may have a type of *INTEGER* (denoting the type of data it is meant to hold).



In the table above, there are three columns (*name*, *age*, and *country*).

The *name* and *country* columns store string data types, whereas *age* stores integer data types. The set of columns and data types make up the schema of this table.

The table also has four rows, or records, in it (one each for Natalia, Ned, Zenas, and Laura).

What is a Relational Database Management System (RDBMS)?

A relational database management system (RDBMS) is a program that allows you to create, update, and administer a relational database. Most relational database management systems use the SQL language to access the database.

What is SQL?

SQL (Structured Query Language) is a *programming language* used to communicate with data stored in a relational database management system.

SQL syntax is similar to the English language, which makes it relatively easy to write, read, and interpret.

Many RDBMSs use SQL (and variations of SQL) to access the data in tables. For example, SQLite is a relational database management system. SQLite contains a minimal set of SQL commands (which are the same across all RDBMSs). Other RDBMSs may use other variants.

(SQL is often pronounced in one of two ways. You can pronounce it by speaking each letter individually like “S-Q-L”, or pronounce it using the word “sequel”).

Popular Relational Database Management Systems

SQL syntax may differ slightly depending on which RDBMS you are using. Here is a brief description of popular RDBMSs:

MySQL

MySQL is the most popular open source SQL database. It is typically used for web application development, and often accessed using PHP.

The main advantages of MySQL are that it is easy to use, inexpensive, reliable (has been around since 1995), and has a large community of developers who can help answer questions.

Some of the disadvantages are that it has been known to suffer from poor performance when scaling, open source development has lagged since Oracle has taken control of MySQL, and it does not include some advanced features that developers may be used to.

PostgreSQL

PostgreSQL is an open source SQL database that is not controlled by any corporation. It is typically used for web application development.

PostgreSQL shares many of the same advantages of MySQL. It is easy to use, inexpensive, reliable and has a large community of developers. It also provides some additional features such as foreign key support without requiring complex configuration.

The main disadvantage of PostgreSQL is that it can be slower in performance than other databases such as MySQL. It is also slightly less popular than MySQL.

For more information about PostgreSQL including installation instructions, read [this](#) article.

Oracle DB

Oracle Corporation owns Oracle Database, and the code is not open sourced.

Oracle DB is for large applications, particularly in the banking industry. Most of the world’s top banks run Oracle applications because Oracle offers a powerful combination of technology and comprehensive, pre-integrated business applications, including essential functionality built specifically for banks.

The main disadvantage of using Oracle is that it is not free to use like its open source competitors and can be quite expensive.

SQL Server

Microsoft owns SQL Server. Like Oracle DB, the code is close sourced.

Large enterprise applications mostly use SQL Server.

Microsoft offers a free entry-level version called *Express* but can become very expensive as you scale your application.

SQLite

SQLite is a popular open source SQL database. It can store an entire database in a single file. One of the most significant advantages this provides is that all of the data can be stored locally without having to connect your database to a server.

SQLite is a popular choice for databases in cellphones, PDAs, MP3 players, set-top boxes, and other electronic gadgets. The SQL courses on Codecademy use SQLite.

For more info on SQLite, including installation instructions, read [this](#) article.

Using An RDBMS On Codecademy

On Codecademy, we use both SQLite and PostgreSQL. While this may sound confusing, don’t worry! We want to stress that the basic syntax you will learn can be used in both systems. For example, the syntax to create tables, insert data into those tables, and retrieve data from those tables are all identical. That’s one of the nice parts of learning SQL – by learning the fundamentals with one RDBMS, you can easily begin work in another.

That being said, let’s take a look at some of the more subtle details:

File extensions — when working with databases on Codecademy, take a look at the name of the file you’re writing in. If your file ends in `.sqlite`, you’re using a SQLite database. If your file ends in `.sql`, you’re working with PostgreSQL.

Data types — You’ll learn about data types very early into learning a RDBMS. One thing to note is that SQLite and PostgreSQL have slightly different data types. For example, if you want to store text in a SQLite database, you’ll use the `TEXT` data type. If you’re working with PostgreSQL, you have many more options. You could use `varchar(n)`, `char(n)`, or `text`. Each type has its own subtle differences. This is a good example of PostgreSQL being slightly more robust than SQLite, but the core concepts remaining the same.

Built-in tables — As you work your way through more complicated lessons on databases, you’ll start to learn how to access built-in tables. For example, if you take our lesson on indexes, you’ll learn how to look at the table that the system automatically creates to keep track of what indexes exist. Depending on which RDBMS system you are using (in that lesson we’re using PostgreSQL), the syntax for doing that will be different. Any time you’re writing SQL about the database itself, rather than the data, that syntax will likely be unique to the RDBMS you’re using.

Conclusion

Relational databases store data in tables. Tables can grow large and have a multitude of columns and records. Relational database management systems (RDBMSs) use SQL (and variants of SQL) to manage the data in these large tables. The RDBMS you use is your choice and depends on the complexity of your application.

Manipulation

Review

Congratulations! We've learned six commands commonly used to manage data stored in a relational database and how to set constraints on such data. What can we generalize so far?

SQL

Preview: Docs Loading link description

is a programming language designed to manipulate and manage data stored in relational databases.

A *relational database* is a database that organizes information into one or more tables.

A *table* is a collection of data organized into rows and columns.

A *statement* is a string of characters that the database recognizes as a valid command.

CREATE TABLE

Preview: Docs Creates a new table within a database.

creates a new table.

INSERT INTO

Preview: Docs Loading link description

adds a new row to a table.

SELECT

Preview: Docs Loading link description

queries data from a table.

ALTER TABLE

Preview: Docs Loading link description

changes an existing table.

UPDATE

Preview: Docs Loading link description

edits a row in a table.

DELETE FROM

Preview: Docs Loading link description

deletes rows from a table.

Constraints

Preview: Docs Loading link description

add information about how a column can be used.

Instructions

In this lesson, we have learned SQL statements that create, edit, and delete data. In the upcoming lessons, we will learn how to use SQL to retrieve information from a database!

What is SQLite?

Learn what is SQLite, its key features, use cases, and step-by-step installation for Windows, Mac, and Linux.

In this article, we will be exploring the extremely prevalent database engine called **SQLite**. We will describe what it does, its main uses, and then explain how to set it up and use it on your own computer.

What is SQLite?

SQLite is a database engine. It is software that allows users to interact with a **relational database**. In SQLite, a database is stored in a single file — a trait that distinguishes it from other database engines. This fact allows for a great deal of accessibility: copying a database is no more complicated than copying the file that stores the data, sharing a database can mean sending an email attachment.

Drawbacks to SQLite

SQLite's signature portability unfortunately makes it a poor choice when many different users are updating the table at the same time (to maintain integrity of data, only one user can write to the file at a time). It also may require some more work to ensure the security of private data due to the same features that make SQLite accessible. Furthermore, SQLite does not offer the same exact functionality as many other database systems, limiting some advanced features other relational database systems offer. Lastly, SQLite does not validate data types. Where many other database software would reject data that does not conform to a table's schema, SQLite allows users to store data of any type into any column.

SQLite creates schemas, which constrain the type of data in each column, but it does not enforce them. The example below shows that the id column expects to store integers, the name column expects to store text, and the age column expects to store integers:

```
CREATE TABLE celebs (  
  id INTEGER,  
  name TEXT,  
  age INTEGER  
);
```

However, SQLite will not reject values of the wrong type. We could accidentally insert the wrong data types in the columns. Storing different data types in the same column is a bad habit that can lead to errors that are difficult to fix, so it's important to be strict about your schema even though SQLite will not enforce it.

Common use cases for SQLite

Even considering the drawbacks, the benefits of being able to access and manipulate a database without involving a server application are huge. SQLite is used worldwide for testing, development, and in any other scenario where it makes sense for the database to be on the same disk as the application code. SQLite's maintainers consider it to be among the [most replicated pieces of software in the world](#).

How to install SQLite

Binaries for SQLite can be installed at the [SQLite Download](#) page. This page has versions of SQLite for Windows, Mac OS X, and [Linux](#). The last number in each file name is the current SQLite version. Our video walkthrough uses the version number `3200100`. You should download whichever version is listed on the SQLite Download page, and replace `3200100` in the instructions below with the version number you downloaded. For example, if the name of the file you download is `sqlite-tools-win-x64-3490100.zip`, you would change `3200100` in each command to `3490100`.

Installing SQLite on Windows

For Windows machines:

Download the `sqlite-tools-win-x64-3490100.zip` file and unzip it.

From your [git-bash terminal](#), open the directory of the unzipped folder with `cd ~/Downloads/sqlite-tools-win-x64-3490100.zip/sqlite-tools-win-x64-3490100.zip/`. 3. Try running sqlite with the command `winpty ./sqlite3.exe`. If that command opens a `sqlite>` prompt, congratulations! You've installed SQLite.

We want to be able to access this command quickly from elsewhere, so we're going to create an alias to the command. Exit the `sqlite>` prompt by typing in `Ctrl + C`, and in the same git-bash terminal without changing folders, run these commands:

```
echo "alias sqlite3=\"winpty ${PWD}/sqlite3.exe\"" >> ~/.bashrc  
and
```

```
source ~/.bashrc
```

The first command will create the alias `sqlite3` that you can use to open a database. The second command will refresh your terminal so that you can start using this command. Try typing in the command `sqlite3 newdb.sqlite`. If you're presented with a `sqlite>` prompt, you've successfully created the `sqlite3` command for your terminal. Enter `Ctrl + C` to quit. You can also exit by typing `.exit` in the prompt and pressing `Enter`.

Video Tutorial: [Setting Up SQLite Locally \(Windows\)](#)

Installing SQLite on Mac

For Macs, use the Mac OS X (x86) sqlite-tools package:

Install it, and unzip it.

In your terminal, navigate to the directory of the unzipped folder using `cd`.

Run the command `mv sqlite3 /usr/local/bin/`. This will add the command `sqlite3` to your terminal path, allowing you to use the command from anywhere.

Try typing `sqlite3 newdb.sqlite`. If you're presented with a `sqlite>` prompt, you've installed SQLite! Enter `control + d` to quit. You can also exit by typing `.exit` in the prompt and pressing `return`.

Video Tutorial: [Setting Up SQLite Locally \(Mac\)](#)

Installing SQLite on Linux

In Ubuntu or similar distributions:

Open your terminal and run `sudo apt-get install sqlite3`. Otherwise, use your distribution's package managers.

Try typing in the command `sqlite3 newdb.sqlite`. If you're presented with a `sqlite>` prompt, you've successfully created the `sqlite3` command for your terminal. You can exit by typing `.exit` in the prompt and pressing `enter`.

Conclusion

You've installed database software and opened a connection to a database. Now you have the full power of SQL at your fingertips. You'll be able to manage all the data for any application you can dream of writing.

To explore how to use SQLite in real-world applications, check out [Learn Node-SQLite](#). This course teaches you how to integrate SQLite with JavaScript using the `node-sqlite3` package, making it a great next step after learning SQLite basics.

Queries Review

Congratulations!

We just learned how to query data from a database using SQL. We also learned how to filter queries to make the information more specific and useful.

Let's summarize:

SELECT

Preview: Docs Loading link description
is the clause we use every time we want to query information from a database.

AS

Preview: Docs Loading link description
renames a column or table.

DISTINCT

Preview: Docs Loading link description
return unique values.

WHERE

Preview: Docs Loading link description
is a popular command that lets you filter the results of the query based on conditions that you specify.

LIKE

Preview: Docs Loading link description
and

BETWEEN

Preview: Docs Loading link description
are special operators.

AND

Preview: Docs Loading link description
and

OR

Preview: Docs Loading link description
combines multiple conditions.

ORDER BY

Preview: Docs Loading link description
sorts the result.

LIMIT

Preview: Docs Loading link description
specifies the maximum number of rows that the query will return.

CASE

Preview: Docs Loading link description
creates different outputs.

Welcome to the NLP Specialty

Welcome to the second half of your Career Path: the specialization!

You are on your way to becoming a Data Scientist with a Natural Language Processing Specialty. Now that you have a foundation in the general skills a Data Scientist needs, you are ready to complete your NLP education.

In this section of the Career Path, we will focus on the specialized skills that you need. We will return to some topics such as statistics, programming, and math. We will also explore new topics such as Machine Learning, Neural Networks, Text Analysis, and building Chatbots.

Along the way, you'll complete challenge projects to build your skills, and capstone and portfolio projects to build your portfolio. By the end of this Career Path, you will be ready to start your career in Data Science.

But the most successful candidates don't wait until the end to start creating a portfolio. It is a good idea to start thinking about it now. The best place to start is by getting the projects you are already working on portfolio-ready. But more on that in the next article. See you there!

How to Showcase your Data Science Skills

<https://www.codecademy.com/paths/data-science-nlp/tracks/next-step-the-natural-language-processing-specialty/modules/welcome-to-the-natural-language-processing-specialty/articles/dsf-how-to-showcase-your-data-science-skills>

And start building your data science portfolio.

If a tree falls in the forest and no one is around to hear it, does it make a sound?
– Philosophy students everywhere

Hiring managers have their own version of that question: **If a data scientist doesn't have a portfolio, do they really have the skills?**

You need to have a portfolio. A portfolio is evidence that you have the technical skills and knowledge that you say that you do. But what goes into a portfolio and what does that mean in data science?

What Goes Into a Portfolio

Ultimately, exactly what you put in your portfolio is up to you. This [project at the end of our Interview Prep Skill Path](#) will help guide you through making sure that you've covered all of the major categories, but the projects themselves are completely up to you.

We recommend that you demonstrate the ability to:

Throughout this Career Path, you will encounter Capstone and Portfolio projects. These can absolutely be put into your portfolio. However, thousands of people take this Career Path. To really make yourself stand out, you will want to add projects about things you are interested in.

If you like sports, use sports data, if you like science, use scientific data, etc. Showing a little bit of your interests and what you are curious about will not only make the project more fun for you, but also let you showcase your own domain knowledge or curiosity. Interest and enthusiasm in your projects will help set you apart from other candidates when you are applying and interviewing.

There are a lot of places to look for datasets. We have an entire lesson on where to find data later in this Career Path. For right now, the best thing you can do is be curious about the world around you, and keep track of some things that you might want to explore as you build your portfolio.

What a Data Science Portfolio Looks Like

A data science portfolio can take a lot of shapes, you could create your own professional website, upload documents to your LinkedIn or other social media site, or create a repository on GitHub. We highly recommend GitHub. It is simple, lightweight, and by far the most popular place to house a portfolio.

You have a few choices in terms of what kinds of files you want to upload. A general rule is the simpler, the better. You can upload [Jupyter notebooks](#) of your projects, [.Rmd](#) or Python files of your code, or PDFs, or presentations of summary reports.

The kind of files you upload will depend on the kind of Data Scientist role you are applying for. A role that is more programming-focused (i.e., a Machine Learning specialization) should have a more programming-heavy portfolio. A role that is more focused on analysis and communication should have at least one summary report.

The Culture of Portfolios

There are a few unspoken rules about portfolios that you should keep in mind. Throughout the Career Path, we've tried to be explicit about when these things come up, and we have presented best practices throughout. However, we want to call out some of the potential pitfalls and make sure you avoid them.

Datasets

There are a few datasets that are used for teaching a lot. Specifically, the [iris dataset](#) and the [titanic dataset](#) are the most common. We use them in the Path, and you will likely find them in the wild. These are so popular that almost everyone who works in data science has seen them. These do not make good portfolio projects because employers might read them as highly scaffolded student projects or as simply replicating a tutorial, and they won't set you apart from other candidates.

Formatting

It is important to pay close attention to presentation. Using the right file types, and presenting content in the right format is essential to getting that first interview.

For example, if you have written some code to analyze a dataset, it should be stored as a `.py`, `.ipynb`, `.R`, or `.Rmd` file. We teach you how to do this in the Career Paths, but it is worth mentioning again that code should almost never be stored in a text document (e.g., `.txt`, `.doc`, etc.) unless it is formatted as a code snippet and used as an example or evidence.

Another example is Tableau (or PowerBI) visualizations and dashboards. There are two ways to store these that demonstrate proficiency with these tools. First, both platforms offer repositories. [Tableau Public](#) is both a great place to show off your work and see what other people are making. The other way is to embed them into a website with an `iframe`.

You should **not** take screenshots, download them as jpegs, pdfs, or otherwise transform the rich, interactive dashboards you've made into static images.

You can (and should) embed static visualizations (that you make in Python, R, or even Excel or Tableau) into written reports, but the skill you are demonstrating is communicating visually about data (not proficiency in Tableau).

Finally, reports and summaries can be woven into your Jupyter Notebooks, presented as written documents (and stored as PDFs), or compiled into slide decks (also stored as PDFs).

Code Hygiene

It takes a long time and a lot of practice to write clear, well-structured code. However, you can take massive steps in that direction by writing well-commented and organized code. Just like writing an essay, writing good code often requires editing. Not all of your projects need to be edited. However, if you choose to highlight one or two (which you can do by pinning them in GitHub), you'll want to be sure that you've written in some comments and reorganized your code for clarity as much as possible.

Grammar

Just like in your resume and cover letter, you want to proofread your portfolio projects to be sure they are free from grammatical or spelling errors. Unlike in your resume/cover letter, a couple typos probably won't disqualify you. However, every Data Scientist needs to communicate about data, and you want to demonstrate that you can do that clearly. Even simply running any reports you write through a spellchecker will help in this respect.

Conclusion

Ultimately, your portfolio is how you verify that you have all of the skills you've worked so hard to acquire. Have fun with it by including some projects that excite you, and be sure to be ready to talk about those projects with recruiters when you get that first interview!

Learn Python: Tuples

<https://youtu.be/yDvRR8nWMNI>

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-python-fundamentals-part-iii/modules/dscp-python-classes/lessons/data-types/exercises/review>

Modules in Python

Modules Python Review

You've learned:

- what

- [modules](#)

- Preview: Docs Loading link description

- are and how they can be useful

- how to use a few of the most commonly used Python libraries

- what namespaces are and how to avoid polluting your local namespace

- how

- [scope](#)

- Preview: Docs Loading link description

- works for

- [files](#)

- Preview: Docs Files are named locations on disk to store related information that can be used in Python.

- in Python

Programmers can do great things if they are not forced to constantly reinvent tools that have already been built. With the power of modules, we can import any code that someone else has shared publicly.

In this lesson, we covered some of the Python Standard Library, but you can explore all the modules that come packaged with every installation of Python at the Python Standard Library documentation.

This is just the beginning. Using a package manager (like conda or pip3), you can install any modules available on the Python Package Index.

The sky's the limit!

Learn Python: Datetimes

<https://youtu.be/smDBZDvsm0I>

Learn Python: Pipenv

<https://youtu.be/BVr-6Ki96XM>

Introduction: Linear Algebra

Find out what you'll learn about linear algebra and why it's important.

Welcome to Linear Algebra

In this unit, we will cover fundamental linear algebra operations for vectors and matrices—including how to represent and solve linear systems of equations.

Why is this important?

Vectors and matrices are fundamental building blocks of data science as they are used to organize and manipulate data we feed through learning models. Whether you are just starting your journey in data science or have already played around with machine learning models, learning linear algebra fundamentals is key to your success in understanding advanced data science material.

There are many real-world applications of linear algebra, some of which we will cover in the coming exercises and lessons, including:

- Efficient computation of linear transformation on large quantities of data. Computer programs and hardware like GPUs have been

- optimized around these linear algebra data structures to enable much of the high-performance computing advances.

- Computer graphics, especially video games, rely heavily on linear algebra to render images quickly and efficiently.

- Machine learning and advances in deep learning, whose foundations are matrix and vector multiplication and addition.

What will you learn?

At the end of this module, you will be able to:

- Perform basic mathematical operations between scalars, vectors, and matrices.

- Use vectors and matrices to represent and solve linear systems of equations.

Where can you get help and discuss the content?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What is Linear Algebra?

Learn about what linear algebra is and how it is relevant for data science!

Linear algebra focuses on the mathematics surrounding linear operations and solving systems of linear equations. While most of us learned about basic linear operations in algebra classes in high school and middle school, "linear algebra" extends these lessons to apply to multi-dimensional data.

The reality is that many engineering optimization problems can be reduced to systems of linear equations, albeit systems with many equations and many variables. By simplifying problems, we can solve for a solution or set of solutions.

Let's imagine a problem where we can do this. We have a section of town containing four roads and four intersections. We are trying to find the number of cars along the roads in between the intersections and have data about the number of cars departing intersections outside this road system. How can we find out our missing car counts?

To solve the problem mathematically, we can realize that this system can be modeled as a system of linear equations since the number of cars entering an intersection must equal the number leaving an intersection. We can visualize the data on the diagram below.

This creates a straightforward system, written out below— one that we can solve without linear algebra techniques.

$$20+d=50+a \quad 25+a=30+b \quad 60+b=30+c \quad 40+c=35+d$$

Now, what if we extend this problem to the entire city of Manhattan? And now only provide inconsistent intersection counts, requiring us to solve for many unknown road segment car counts? In this case, we can still set the problem up as an extensive system of linear equations, but we quickly realize that solving for those unknown car counts by hand is impossible. This is where linear algebra will come in and allow us to solve these complex problems.

Coming Up

In the upcoming content, we will look at two important linear algebra data structures that can store high-dimensional data. This data can be actual numerical data like one might see on a spreadsheet or representative data like if we need to represent a large system of equations. We will learn how to perform various operations between these data structures and use these techniques to solve systems of linear equations.

As mentioned earlier, linear algebra is fundamental because it allows us to mathematically operate on large amounts of data, which is vital to modern data science techniques. There are many real-world applications of linear algebra, some of which we will cover in the coming exercises and lessons, including:

Happy coding!

Introduction to Linear Algebra

Review

In this lesson, we learned about the fundamental building blocks of linear algebra: vectors and matrices, their important operations, and how to use them to solve important problems.

We discussed that vectors are quantities that include both direction and magnitude, and can hold two or more elements of data. These vectors can be multiplied by scalars and be added together with other vectors of the same dimension. Additionally, we found that the dot product, the amount that one vector goes into the other, can be calculated as the sum of the product of corresponding elements.

We then transitioned to matrices, which are quantities that hold rows and columns of data. Matrices can be constructed by combining vectors as the columns of a matrix. Matrices also have similar operations to vectors in terms of scalar multiplication and the addition of same-sized matrices. We were also introduced to matrix multiplication, where the product of two matrices is determined by taking the dot products of the rows and columns of the two matrices being multiplied.

Finally, we learned how to use both vectors and matrices to efficiently solve systems of linear equations. A traditional set of n equations for n unknown [variables](#)

Preview: Docs Loading link description

can be represented in the form $Ax = b$. Solving for x in this form is done by using Gauss-Jordan elimination. We additionally use Gauss-Jordan elimination to find the potential inverse of square matrices.

Linear Algebra with Python

Review

Congratulations! You have completed your journey into linear algebra operations with NumPy.

The Python library NumPy provides a powerful tool to programmatically perform linear algebra. A fundamental building block of NumPy, the NumPy [array](#)

Preview: Docs Loading link description

, allows us to represent vectors and matrices. These arrays can perform operations directly, such as addition and scalar multiplication using `+` and `*`, respectively.

NumPy also has built-in functions that allow us to perform more advanced operations like dot products, transposes, and matrix multiplication.

Finally, NumPy has a `numpy.linalg` sublibrary that provides more advanced functionality. This includes finding the norm (or magnitude/length) of a vector, finding the inverse of a square matrix, and solving a system of linear equations in $Ax=b$ form.

All this functionality will allow us to use Python and NumPy to solve many linear algebra problems, including linear regression, in future lessons and articles.

Instructions

In `script.py` is a similar system of linear equations as the previous exercise. However, this time there are four unknowns variables instead of three!

Using what you have learned in this lesson, find the unknowns `a`, `b`, `c`, and `d`. If you get stuck, refer back to the previous exercise or take a peek at `solution.py`.

Introduction: Differential Calculus

Find out what you'll learn about differential calculus and why it's important.

Welcome to Differential Calculus

In this unit, we will cover fundamentals of differential calculus and investigate how we can use limits and derivatives to analyze functions.

Why is this important?

Differential calculus deals with the study of rates of change. It allows us to quantify how one variable relates to another. In the field of data science, this is invaluable information to understand. Learning models are built upon quantifying how variables relate to each other to make predictions or understand patterns. Whether you are just starting on your data science journey or have worked with machine learning models, understanding the principles of calculus is an invaluable asset to add to your toolbox!

What will you learn?

At the end of this module, you will be able to:

- Use limits to understand function behavior at specific points.

- Use derivatives to analyze functions and quantify rates of change.

Where can you get help and discuss the content?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#) and on [Discord](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What is Calculus?

Learn about what calculus is and how it is important in the field of data science.

The core task of a data scientist is to make data-driven recommendations on the best way to solve a problem. Along the way, data scientists build and fit statistical models to support their claims, make predictions, and more. In this article, we will look at what calculus is and why it is vital in the data science process.

Rate of Change

Differential calculus is the field that studies rates of change. By quantifying how variables relate to each other, we can better understand our statistical model or physical system. For example, as a data scientist at a t-shirt company, we may want to know how a change in the marketing budget impacts the company's total profit. Spend too little, and our product won't be well known; spend too much, and we might reach a point of diminishing returns where our money would be better spent elsewhere. How can we find the optimal size of our marketing budget?

Optimization

The last question above is the sort of questions asked and answered by the field of *mathematical optimization*, which intertwines closely with calculus. Optimization problems involve looking for the best solution to a problem. Perhaps that's finding the 'optimal' size of the marketing budget for our company, the point where we maximize our profits with a budget that allows us to reach a broad audience without paying too much. Perhaps that means finding the statistical model that 'best' fits our data according to some criteria that we define ourselves— mathematical optimization gives us a way to decide among all our possible options. It is a powerful field that underpins much of the modern machine learning revolution and it is built upon the fundamentals of calculus we will investigate.

Infinitesimal Analysis

One of the fundamental paradigms in calculus is to think via *infinitesimals* (extremely small quantities) to say things about instantaneous rates of change. We will discuss this at length in the upcoming lesson. To give a rough overview, however, the idea is that by 'zooming in' enough and considering how a variable changes over an infinitesimal time frame, we can learn how that variable is changing instantaneously, at a specific moment in time.

Recap

To recap, calculus is essential for a data scientist because it provides a foundation for mathematical optimization, a field used to tune machine learning models to classify and make predictions. With that in mind, let's dive in.

Introduction to Differential Calculus

Review

Congrats! You have finished your exploration of differential calculus. This has been a jam-packed lesson, so pat yourself on the back for making it through the material. Let's review what we have learned throughout the exercises:

Important Concepts

A *limit* is the value of a function approaches as we move to some x value.

The *limit definition of a derivative* shows us how to measure the instantaneous rate of change of a function at a specific point.

A *derivative* is the slope of a tangent line at a specific point. The derivative of a function $f(x)$ is denoted as $f'(x)$.

The derivative of a function allows us to determine when a function is increasing, decreasing, or is at a minimum or maximum value.

Derivative Rules

We learned the following general derivative rules:

$$\frac{d}{dx}cf(x) = c\frac{d}{dx}f(x) \quad \frac{d}{dx}(f(x) + g(x)) = \frac{d}{dx}f(x) + \frac{d}{dx}g(x) \quad \frac{d}{dx}c = 0 \quad f(x) = u(x)v(x) \rightarrow f'(x) = u(x)v'(x) + v(x)u'(x)$$

We also learned how to calculate derivatives for explicit functions:

$$\frac{d}{dx}x^n = nx^{n-1} \quad \frac{d}{dx}\ln(x) = \frac{1}{x} \quad \frac{d}{dx}e^x = e^x \quad \frac{d}{dx}\sin(x) = \cos(x) \quad \frac{d}{dx}\cos(x) = -\sin(x)$$

Derivatives in Python

`np.gradient()` allows us to calculate derivatives of functions represented by arrays.

Introduction to Machine Learning Foundations for Data Science

Get ready to dive into the world of machine learning!

All the pieces are in place, and you are equipped with a solid understanding of Python, math, and statistics, and ready to dive into the world of machine learning. It's a huge accomplishment to get to this point, and we're sure you are excited, so let's see what you are going to learn about in this unit!

What will Machine Learning Foundations cover?

After this unit, you will be able to:

- Explain what machine learning is (and isn't)
- Distinguish between supervised and unsupervised classification
- Analyze features in light of their role in machine learning
- Transform datasets and prepare them for machine learning

Why did we build this?

Preparing a dataset for machine learning is the difference between a usable model that makes valid, reliable predictions, and one that is not those things. You've heard the old adage, "garbage in, garbage out", and this unit will show you exactly how to avoid feeding your models garbage.

Who are my classmates?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Why Use Machine Learning?

Begin Your Journey

DING DING DING!

It was but a dream and there's still time. Understanding

[Machine Learning](#)

Preview: Docs Loading link description

will help you change this bleak future, or be ready for it.

As you begin your journey through our Machine Learning curriculum, you will learn the different applications of this powerful field. You will build your own models, test them, and try to improve them. You will analyze bigger, and more complex data and deliver faster, more accurate results — even on a very large scale using:

- Supervised Learning**
 - Regression
 - Classification
- Unsupervised Learning**

Throughout your exploration of ML, we hope you gain an understanding of how to harness predictive models to do good, and enhance human life rather than replace it!

If you're completely new to data analysis, Python, or pandas, start with [ML/AI Engineering Foundations Skill Path](#) then join us here!

Supervised vs. Unsupervised Learning: what's the difference?

Learn the key differences between supervised and unsupervised learning in machine learning, with real-world examples.

As humans, we have many different ways we learn things. The way you learned calculus, for example, is probably not the same way you learned to stack blocks. The way you learned the alphabet is probably wildly different from the way you learned how to tell if objects are approaching you or going away from you. The latter you might not even realize you learned at all!

Similarly, when we think about making programs that can learn, we have to think about these programs learning in different ways. Two main ways that we can approach machine learning are **Supervised Learning** and **Unsupervised Learning**. Both are useful for different situations or kinds of data available.

What is supervised learning?

Let's imagine you're first learning about different genres in music. Your music teacher plays you an indie rock song and says "This is an indie rock song". Then, they play you a K-pop song and tell you "This is a K-pop song". Then, they play you a techno track and say "This is techno". You go through many examples of these genres.

The next time you're listening to the radio, and you hear techno, you may think "This is similar to the 5 techno tracks I heard in class today. This must be techno!"

Even though the teacher didn't tell you about *this* techno track, she gave you enough examples of songs that were techno, so you could recognize more examples of it.

When we explicitly tell a program what we expect the output to be, and let it learn the rules that produce expected outputs from given inputs, we are performing supervised learning.

A common example of this is **image classification**. Often, we want to build systems that will be able to describe a picture. To do this, we normally show a program thousands of examples of pictures, with labels that describe them. During this process, the program adjusts its internal parameters. Then, when we show it a new example of a photo with an unknown description, it should be able to produce a reasonable description of the photo.

When you complete a **Captcha** and identify the images that have cars, you're labeling images! A supervised machine learning algorithm can now use those pictures that you've tagged to make it's car-image predictor more accurate.

What is unsupervised learning?

Let's say you are an alien who has been observing the meals people eat. You've embedded yourself into the body of an employee at a typical tech startup, and you see people eating breakfasts, lunches, and snacks. Over the course of a couple weeks, you surmise that for breakfast people mostly eat foods like:

Lunch is usually a combination of:

Snacks are usually a piece of fruit or a handful of nuts. No one explicitly *told* you what kinds of foods go with each meal, but you learned from natural observation and put the patterns together. In unsupervised learning, we don't tell the program anything about what we expect the output to be. The program itself analyzes the data it encounters and tries to pick out patterns and group the data in meaningful ways.

An example of this includes **clustering** to create segments in a business's user population. In this case, an unsupervised learning algorithm would probably create groups (or clusters) based on parameters that a human may not even consider.

Conclusion

We have gone over the difference between supervised and unsupervised learning:

EDA Prior To Fitting a Regression Model

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-machine-learning-fun/modules/eda-machine-learning-models/articles/eda-prior-to-fitting-a-regression-model>

Learn about recommended EDA steps before fitting a regression model.

Introduction

Before fitting any model, it is often important to conduct an exploratory data analysis (EDA) in order to check assumptions, inspect the data for anomalies (such as missing, duplicated, or mis-coded data), and inform feature selection/transformation. In this article, we will use **pandas** to explore some of the EDA techniques that are generally employed prior to fitting a regression model.

The data

For our example analysis, we've downloaded a dataset from **Kaggle** which contains data on Major League Baseball (MLB) games from the 2016 season. We've saved this data as a **DataFrame** named **bb**. Suppose that we want to fit a linear regression to predict **attendance** using the following predictors:

game_type — is the game during the day or at night?
day_of_week — what day of the week did the game occur?
temperature — average game temperature (Fahrenheit).
sky — description of sky condition at the time of the game.
total_runs — total runs scored in the game.

Preview the dataset

Any EDA process will probably begin by inspecting a subset of data. For a **pandas DataFrame**, this can be done by using the **.head()** method:

```
bb.head()
```


1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4
5	5	5	5	5	5
6	6	6	6	6	6
7	7	7	7	7	7
8	8	8	8	8	8
9	9	9	9	9	9
10	10	10	10	10	10

By looking at the first few rows of the data, we can often figure out what kind of data we have (eg., discrete or continuous) and get a sense of how they are coded. For example, we can see that `attendance`, `temperature`, and `total_runs` are numbers, while `game_type`, `day_of_week`, and `sky` appear to be text. After our initial inspection, we'll want to dig deeper to investigate the following:

- The data type of each variable.
- How discrete/categorical data is coded (and whether we need to make any changes).
- How the data are scaled.
- Whether there is missing data and how it is coded.
- Whether there are outliers.
- The distributions of continuous features.
- The relationships between pairs of features.

Data types

It is important to check the data type for each feature. The quantitative variables should be read in as numbers — either `int64` or `float64` — and categorical variables should be stored as strings (columns of `strings` have a `dtype` of `object` because of how they are stored in Python). We can check data types of columns in a pandas `DataFrame` using the `.dtypes` property.

```
bb.dtypes
```

The output will look like this:

```
attendance    float64
game_type     object
day_of_week   object
temperature   object
sky           object
total_runs    object
dtype: object
```

From this output, we can see that `temperature` and `total_runs` are both quantitative variables but they were read in as `object` dtypes. This can happen when there is a non-numeric character — such as a letter or punctuation symbol — in the same column. We would need to explore further in order to figure out what's going on. For example, we might inspect a different set of rows and see the following:

1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4
5	5	5	5	5	5
6	6	6	6	6	6
7	7	7	7	7	7
8	8	8	8	8	8
9	9	9	9	9	9
10	10	10	10	10	10

We note that `temperature` has missing data coded as `Unknown` rather than `NaN`. If we fit a regression on this data as is, we will end up treating temperature as a categorical variable and therefore fitting separate slopes for every value of temperature; instead, we probably want a single slope. To fix this, we'll need to replace every `Unknown` with some other value (or remove them from the data altogether) and recode the temperature column as an `int`.

Categorical encoding

EDA is also important during the feature engineering process in order to inform decisions around categorical encoding. This is important because categorical features with many levels are "expensive" to include in a regression model (we need to calculate a separate slope for each level). If one of the levels has only a few observations, we might want to delete those records from the data before fitting the model. We can check this using `.value_counts()`:

```
bb['game_type'].value_counts(dropna=False)
```

Output:

```
Night Game    1664
Day Game       799
Name: game_type, dtype: int64
```

Based on the output, we can see here that there are two levels for `game_type`; about one-third of games are day games and two-thirds are night games.

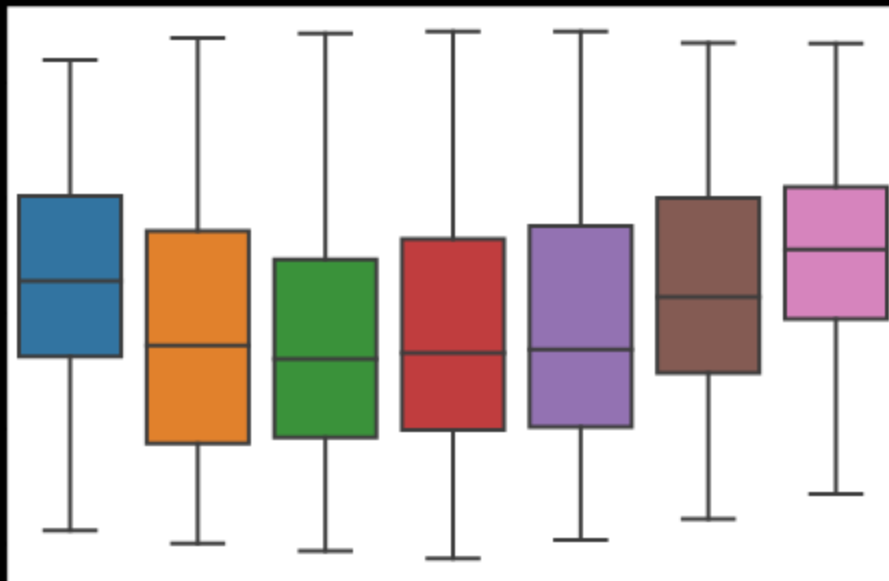
The `.value_counts()` accessor can also illuminate other issues. For example, in the following output, we notice that one instance of 'Tuesday' was miscoded as `Tuesda`. This can either be corrected or removed before proceeding with a regression model.

```
bb['day_of_week'].value_counts(dropna=False)
```

Output:

```
Saturday    396
Friday      394
Sunday      392
Wednesday   379
Tuesday     375
Monday      278
Thursday    248
Tuesda      1
Name: day_of_week, dtype: int64
```

There are a few different options for how we might want to code the `day_of_week` variable. If attendance increases approximately linearly throughout the week, we might argue that `day_of_week` is ordinal and code it as an `int` in our model. However, attendance goes up and down throughout the week, we're better off leaving it as an unordered category (`str`). Finally, if we see that games on Friday-Sunday simply have higher attendance than other days of the week, we might re-code this feature to only have two levels: `Weekend` and `Weekday`. We can check this by using boxplots:



We can see here that attendance on Friday, Saturday, and Sunday is on average higher than the other days of the week. Therefore it may be beneficial to re-code this feature to either `Weekend` or `Weekday`.

Scaling

For quantitative features, it is important to think about how each feature is scaled. Some features will be on vastly different scales than others just based on the nature of what the feature is measuring. For example, let's look at `temperature` and `total_runs` using the `.describe()` method:

```
bb.describe()
```

The output will look like this:

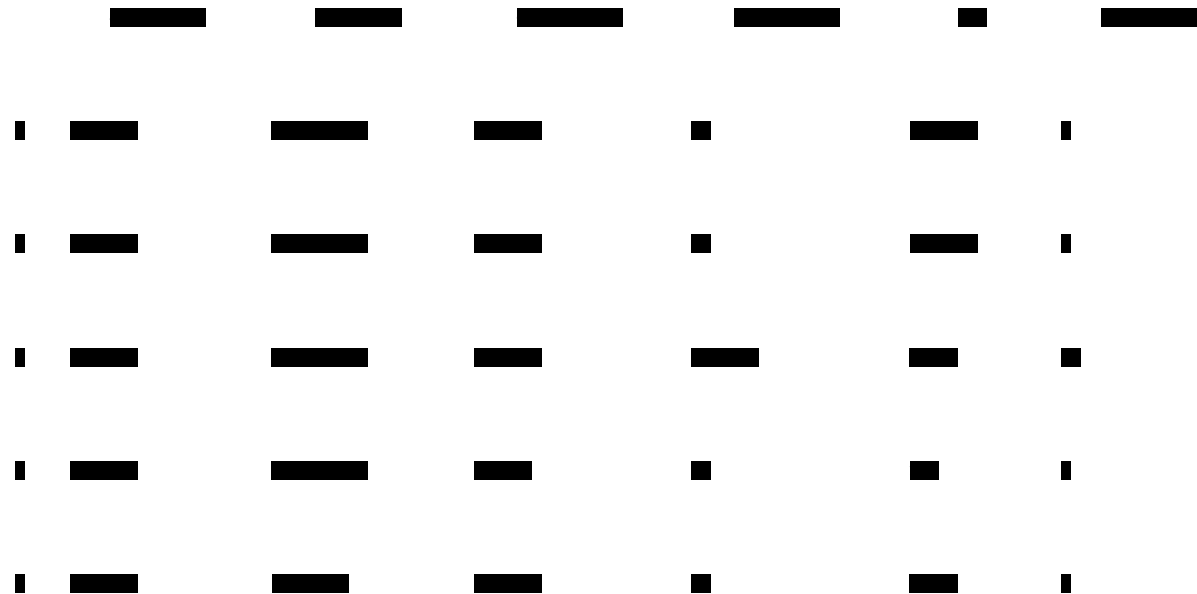
	attendance	temperature	total_runs
count	2457.000000	2457.000000	2457.000000
mean	30380.462352	73.834959	8.949187
std	9874.626652	10.567219	4.579542
min	8766.000000	31.000000	1.000000
25%	22437.000000	67.000000	6.000000
50%	30628.000000	74.000000	8.000000
75%	38412.000000	81.000000	12.000000
max	54449.000000	101.000000	60.000000

These two features are on different scales because what they are measuring are different (`temperature` is in degrees Fahrenheit, `total_runs` is the number of runs scored in a game). Because of this, the ranges of values and the standard deviations for each are very different from one another. We can see here that `temperature` has a standard deviation of about 10.57, while `total_runs` has a standard deviation of about 4.58.

When working with features with largely differing scales, it is often a good idea to *standardize* the features so that they all have a mean of 0 and a standard deviation of 1. A feature without any values close to zero may also make it more difficult to estimate and interpret the intercept of a regression model. Standardizing or otherwise re-scaling the feature can fix this issue.

Missing data

When we initially inspected the data, we saw some evidence that missing data is coded in a few different ways:

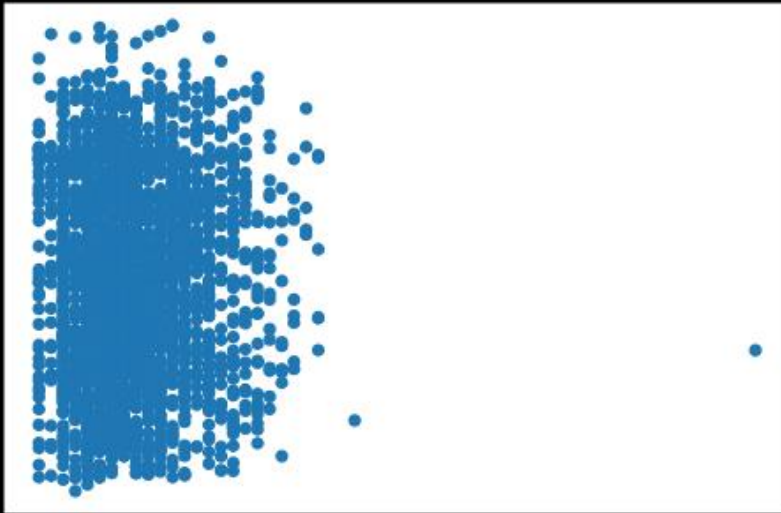


For example, `temperature` uses the term `Unknown`, `sky` uses both `Unknown` and `NaN`, and `total_runs` has `-` to represent a missing value. The observations with missing values will either have to be removed or replaced (with an imputed value or missing data type that Python can recognize, such as `np.NaN`) in order to proceed with fitting a regression model.

Outliers

In our EDA, it is important to check for outliers and skew in the data. One way to check for outliers is to use scatter plots:

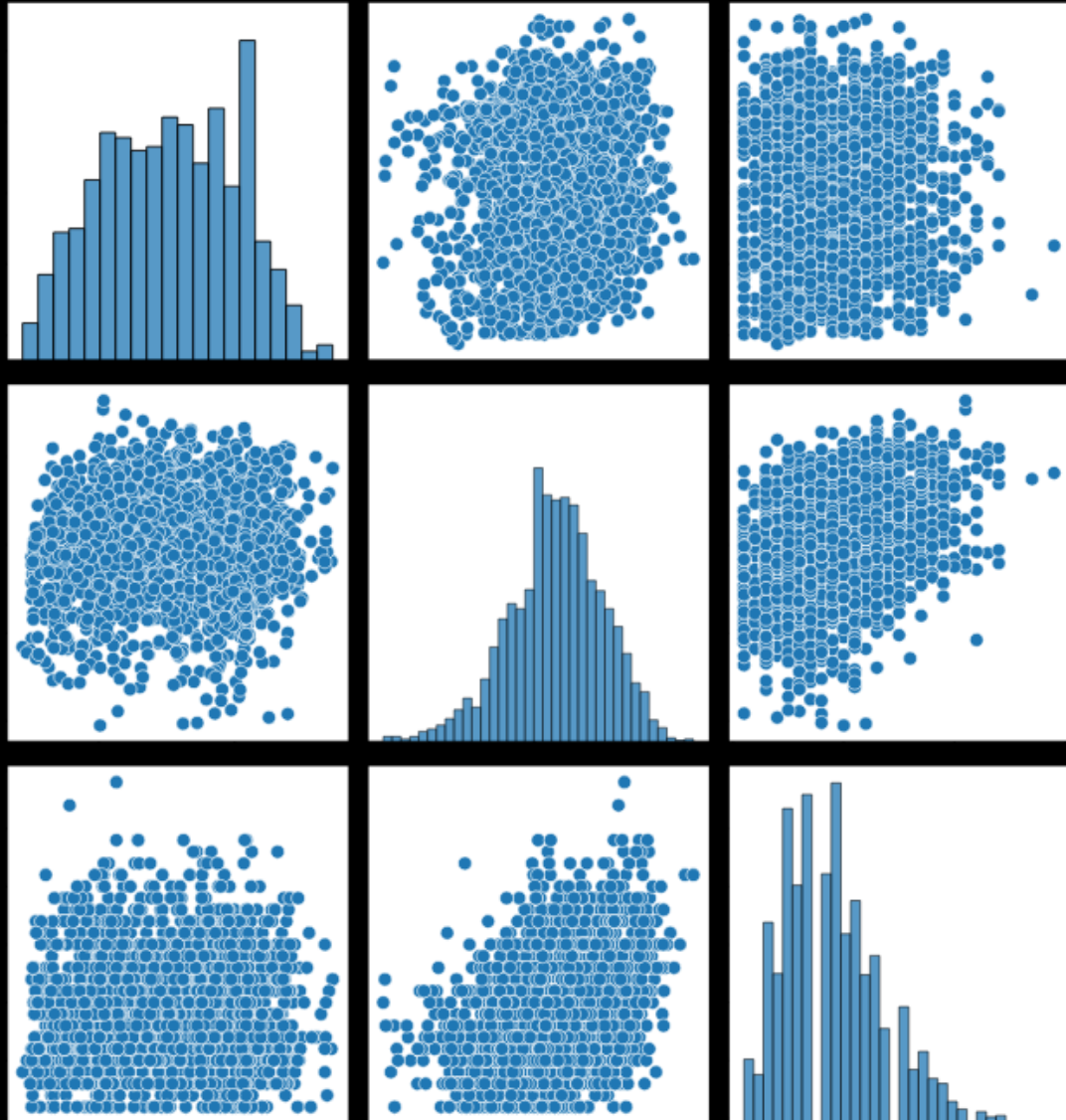
```
bb.plot.scatter(x = 'total_runs', y = 'attendance')
```



We can see here that there is one instance where the total runs in a single game is about 60, which is much larger than in the other games. Depending on the situation, we may first want to verify that this value is correct, then we can decide whether or not to remove it prior to fitting the model.

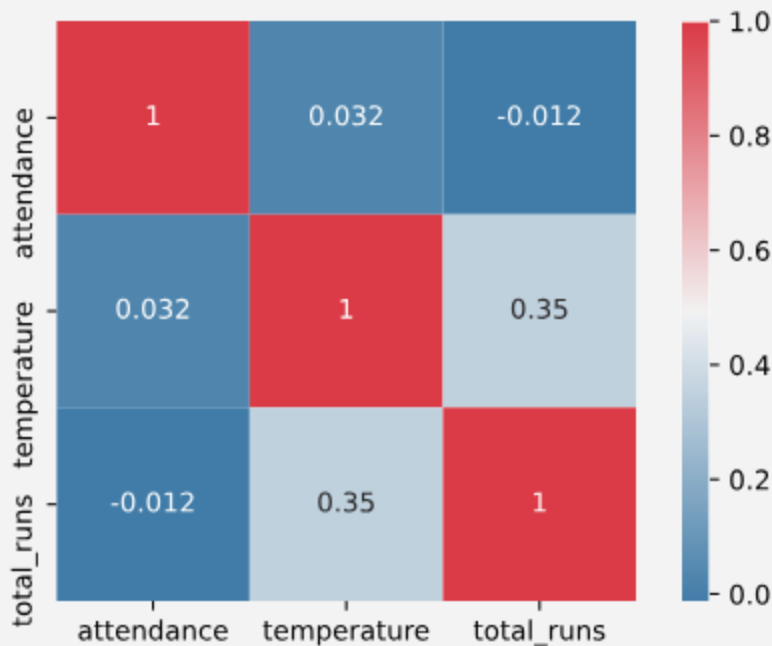
Distributions and associations

Prior to fitting a linear regression model, it can be important to inspect the distributions of quantitative features and investigate the relationships between features. We can visually inspect both of these by using a pair plot:



Looking at the histograms along the diagonal, `total_runs` appears to be somewhat right-skewed. This indicates that we may want to transform this feature to make it more normally distributed.

We can explore the relationships between pairs of features by looking at the scatterplots off of the diagonal. This is useful for a few different reasons. For example, if we see non-linear associations between any of the predictors and the outcome variable, that might lead us to test out polynomial terms in our model. We can also get a sense for which features are most highly related to our outcome variable and check for collinearity. In this example, there appears to be a slight positive linear association between temperature and the total number of runs. We can further investigate this using a heat map of the correlation matrix:



There is a correlation of 0.35 between temperature and the total number of runs. This is not large enough to cause concern; however, if two or more predictors are highly correlated, we may consider leaving only one in our analysis. On the other hand, features that are highly correlated with our outcome variable are especially important to include in the model.

Conclusion

Let's review the ways we were able to explore this data set in preparation for a regression model:

- We previewed the first few rows of the data set using the `.head()` method.
- We checked the data type of each variable in the data set using `.dtypes` and corrected variables with incorrect data types.
- We investigated our categorical data to inform categorical encoding.
- We investigated the scale of our quantitative variables and considered whether standardizing/scaling might be appropriate.
- We investigated missing data.
- We checked for outliers.
- We inspected the distributions of our quantitative variables.
- We looked at the relationships between pairs of features using both scatter plots and box plots.

By going through these steps, we are more prepared to make decisions about feature selection/engineering and have learned valuable information about how to build a more accurate predictive model.

EDA Prior to Fitting a Classification Model

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-machine-learning-fun/modules/eda-machine-learning-models/articles/eda-prior-to-fitting-a-regression-model>

Learn about recommended EDA steps before fitting a classification model.

Introduction

Similar to regression models, it is important to conduct EDA before fitting a classification model. An EDA should check the assumptions of the classification model, inspect how the data are coded, and check for strong relationships between features. In this article, we will explore some of the EDA techniques that are generally employed prior to fitting a classification model.

Data

Suppose we want to build a model to predict whether a patient has heart disease or not based on other characteristics about them. We have downloaded a dataset from the [UCI Machine Learning Repository](#) about heart disease which contains patient information such as:

`age`: age in years

`sex`: male (1) or female (0)

`cp`: chest pain type

`trestbps`: resting blood pressure (mm Hg)

`chol`: cholesterol level

`fbs`: fasting blood sugar level (normal or not)

`restecg`: resting electrocardiograph results

`thalach`: maximum heart rate from an exercise test

`exang`: presence of exercise-induced angina

`oldpeak`: ST depression induced by exercise relative to rest

`slope`: slope of peak exercise ST segment

`ca`: number of vessels colored by flourosopy (0 through 3)

`thal`: type of defect (3, 6 or 7)

The response variable for this analysis will be `heart_disease`, which we have condensed down to either 0 (if the patient does not have heart disease) or 1 (the patient does have heart disease).

EDA is extremely useful to better understand which patient attributes are highly related to heart disease, and ultimately to build a classification model that can accurately predict whether someone has heart disease based on their measurements. By exploring the data, we may be able to see which variables — or which combination of variables — provide the most information about whether or not the patient has heart disease.

Preview the data

Similar to EDA prior to a regression model, it is good to begin EDA with inspecting the first few rows of data:

```
print(heart.head())
```

cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	heart_disease
0	120	120	0	1	150	0	0.0	1	0	3	0
0	140	120	0	1	150	0	0.0	1	0	3	0
0	120	120	0	1	150	0	0.0	1	0	3	0
0	140	120	0	1	150	0	0.0	1	0	3	0
0	120	120	0	1	150	0	0.0	1	0	3	0

By looking at the first rows of data, we can note that all of the columns appear to contain numbers. We can quickly check for missing values and data types by using `.info()`:

```
print(heart.info())
```

Output:

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

```

---  -----  -----  ---
0   age           303 non-null   int64
1   sex           303 non-null   int64
2   cp            303 non-null   int64
3   trestbps      303 non-null   int64
4   chol          303 non-null   int64
5   fbs           303 non-null   int64
6   restecg       303 non-null   int64
7   thalach       303 non-null   int64
8   exang         303 non-null   int64
9   oldpeak       303 non-null   float64
10  slope         303 non-null   int64
11  ca            303 non-null   object
12  thal          303 non-null   object
13  heart_disease 303 non-null   int64
dtypes: float64(1), int64(11), object(2)

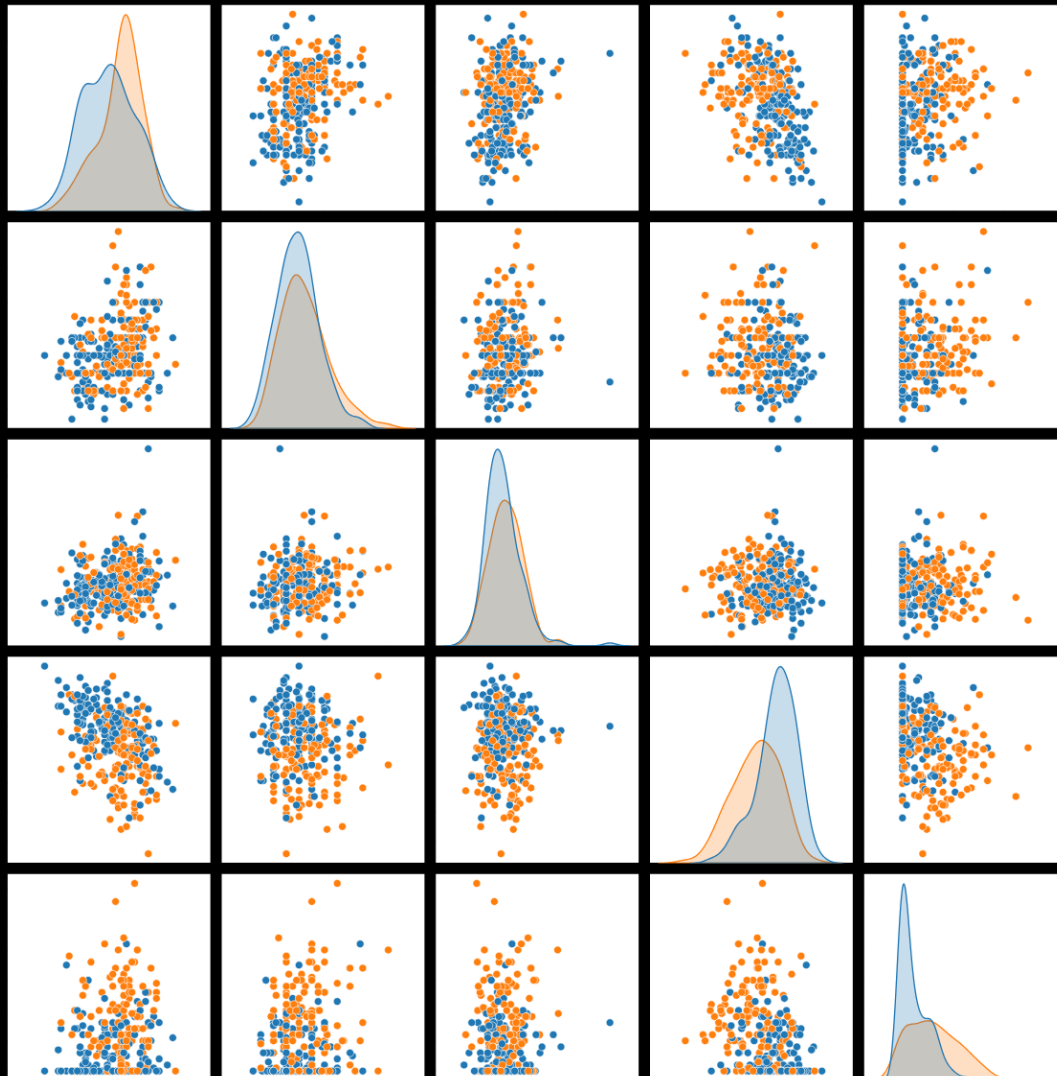
```

We can see that all columns have a count of “303 non-null” values, meaning there are no blank spaces in the dataset. However, there can still be other ways that missing data can be hiding in the data. For example, `ca` and `thal` are `object` data types, indicating that there is at least one character in each of these columns which are preventing the variable from being read as a numeric data type. This could be either an input mistake (such as the letter “o” in place of a “0”), or it can be an indication of how missing data were handled. Depending on which model program is used, you may have to find and remove the observations with characters before proceeding with the model.

We also want to make sure to check how categorical data is encoded before proceeding with model fitting. For example, `cp` is the patient’s chest pain type and is indicated by a number between 1 and 4. These numbers are intended to be treated as groups, so this variable should be changed into an object before continuing into the analysis.

Pair plot

We can explore the relationships between the different numeric variables using a pair plot. If we also color the observations based on heart disease status, we can simultaneously get a sense for a) which features are most associated with heart disease status and b) whether there are any pairs of features that are jointly useful for determining heart disease status:



In this pair plot, we are looking for patterns between the two color groups. Looking at the density plots along the diagonal, there are no features that cleanly separate the groups (age has the most separation). However, looking at the scatterplot for `age` and `thalach` (maximum heart rate from an exercise test), there is more clear separation. It appears that patients who are old and have low `thalach` are more likely to be diagnosed with heart disease than patients who are young and have high `thalach`. This suggests that we want to make sure both of these features are included in our model.

Correlation heat map

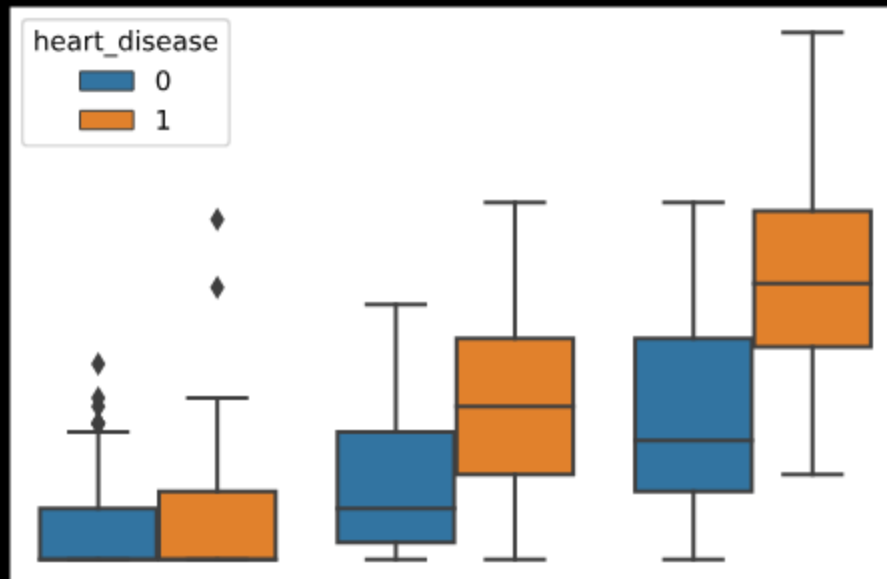
Similar to linear regression, some classification models assume no multicollinearity in the data, meaning that two highly correlated predictors should not be included in the model. We can check this assumption by looking at a correlation heat map:



There is no set value for what counts as "highly correlated", however a general rule is a correlation of 0.7 (or -0.7). There are no pairs of features with a correlation of 0.7 or higher, so we do not need to consider leaving any features out of our model based on multicollinearity.

Further exploration

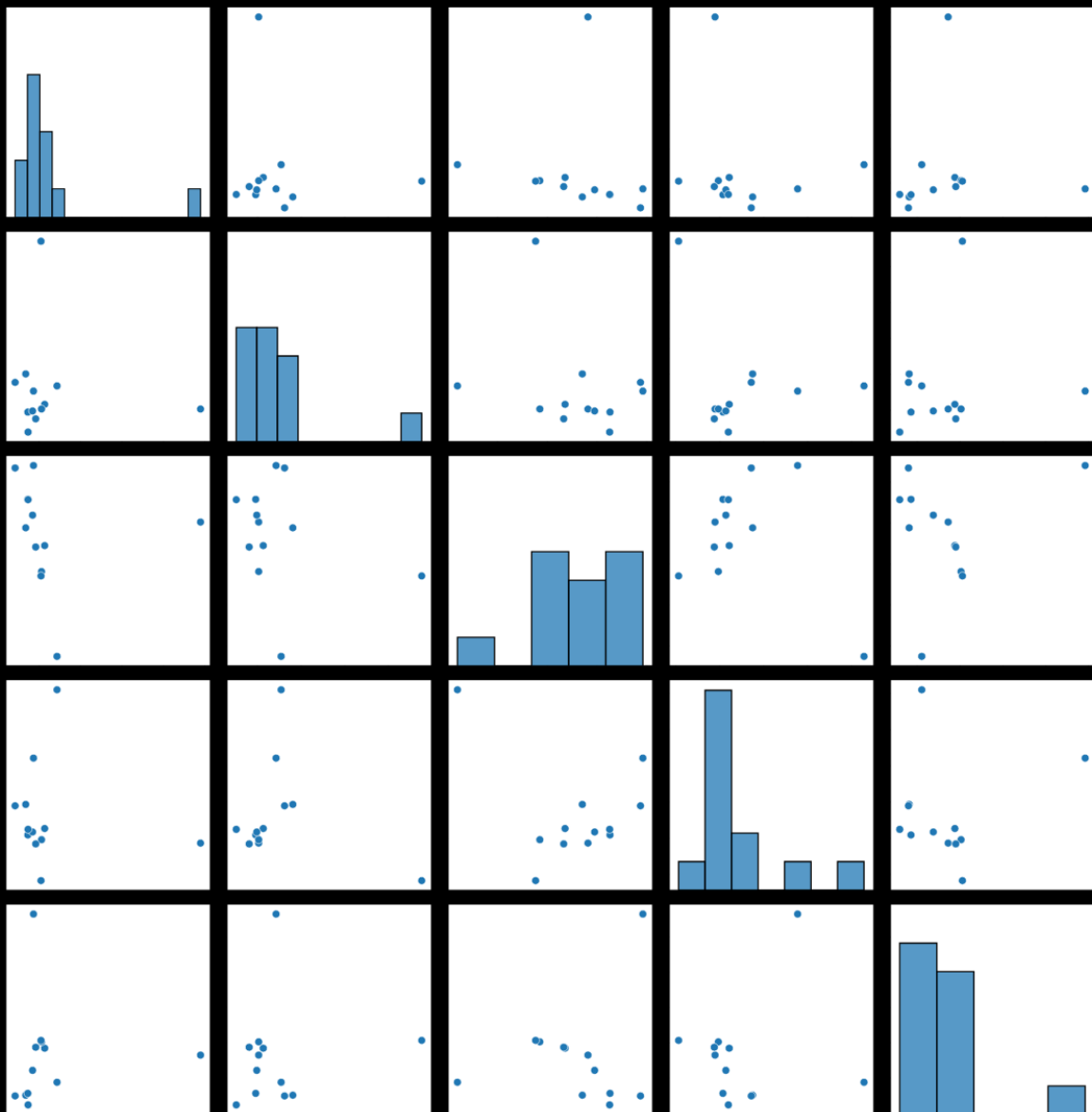
You can use more complex visualizations to examine the relationships between 2 or more features and the response variable at the same time. For example, the following boxplots show the relationship between `oldpeak`, `slope`, and `heart_disease`:



In this boxplot, we can see a pretty distinct difference between those with heart disease and those without at slope level 3. Seeing this distinction indicates that on average `oldpeak` is connected to heart disease at different `slope` levels. This gives insight that it might be beneficial to include an interaction term between `oldpeak` and `slope` in a linear regression model.

Classification model results

After this EDA, we can run a principal component analysis (PCA), which attempts to identify which features (or combination of features) are highly related to heart disease. Ideal results of a PCA show one or more pairs of principal components with some separation between the colored groups:



We can see here that there are not any clear separations, which would indicate that this is not an effective analysis. However, we might use the weights of the components to further explore relationships between features and use that in other analyses.

Conclusion

Exploring the data in the ways outlined above will help prepare you to build an effective classification model. These steps ensure that the data is properly coded and can be useful for both feature selection and model tuning.

EDA Prior to Unsupervised Clustering

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-machine-learning-fun/modules/eda-machine-learning-models/articles/eda-prior-to-fitting-a-classification-model>

Learn the EDA steps that can be helpful prior to creating an unsupervised clustering model.

Introduction

When we want to understand underlying groups for a set of observations but don't know what the group labels should be, we often turn to unsupervised clustering. If we are planning to fit an unsupervised machine learning model, we often want to explore questions such as:

How many groups are there?

How might those groups differ?

In this article, we will demonstrate an exploratory process for beginning to address these questions.

Data

Let's say that you are opening a restaurant and want to be sure to offer a "wide variety" of wines. You get this dataset of chemical differences between types of wines from the [UCI Machine Learning Repository](#) with traits such as:

Alcohol

Malic acid

Ash

Alcalinity of ash

Magnesium

Total phenols

Flavanoids

Nonflavanoid phenols

Proanthocyanins

Color intensity

Hue

OD280/OD315 of diluted wines

Proline

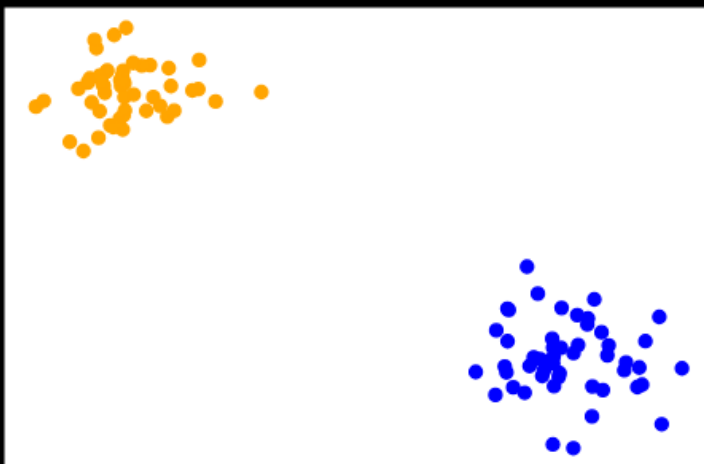
You want to use this information to try to categorize the wine into different groups to ensure that the wines that you decide to buy for the restaurant have a good amount of variety. You want the wines within each group to be similar to each other, and wines in different groups to be less similar. You have no idea how many groups there are or what the group labels should be. So instead, you want to see if the given information has natural groupings based on the characteristics that you have collected data about.

In order to get started, you might need to answer some of the following questions:

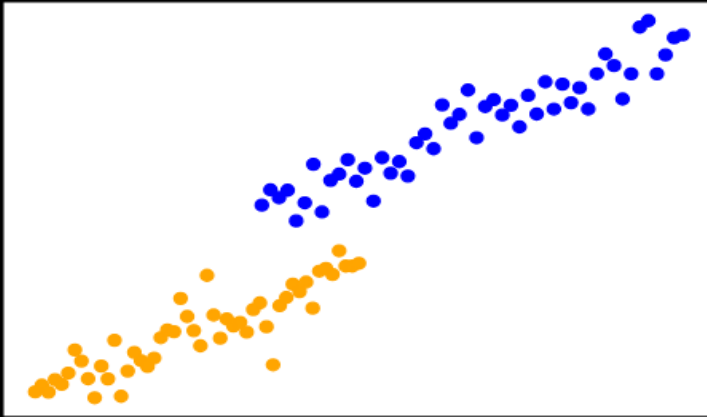
How many different kinds of wine are there?

How do those wine types differ?

For some unsupervised clustering algorithms, you'll need to specify the number of groups ahead of time. Also, different types of algorithms can handle different kinds of groupings more efficiently, so it can be helpful to visualize the shapes of the clusters. For example, k-means algorithms are good at identifying data groups with spherical shapes because they are based on principles of equal variance and distance between data points:



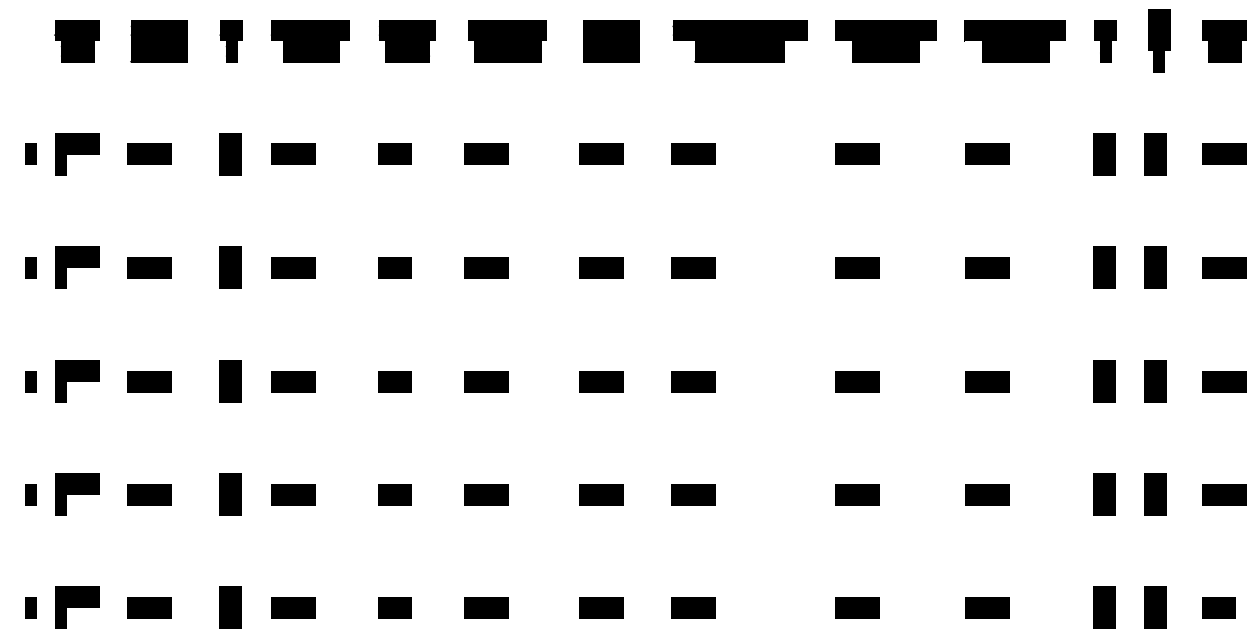
On the other hand, Principal Component Analysis algorithms and BIRCH methods are better at identifying elongated groupings:



There is no method that is the best in every situation. It takes some investigating to know which method will be best for a given set of data.

Prepare the data

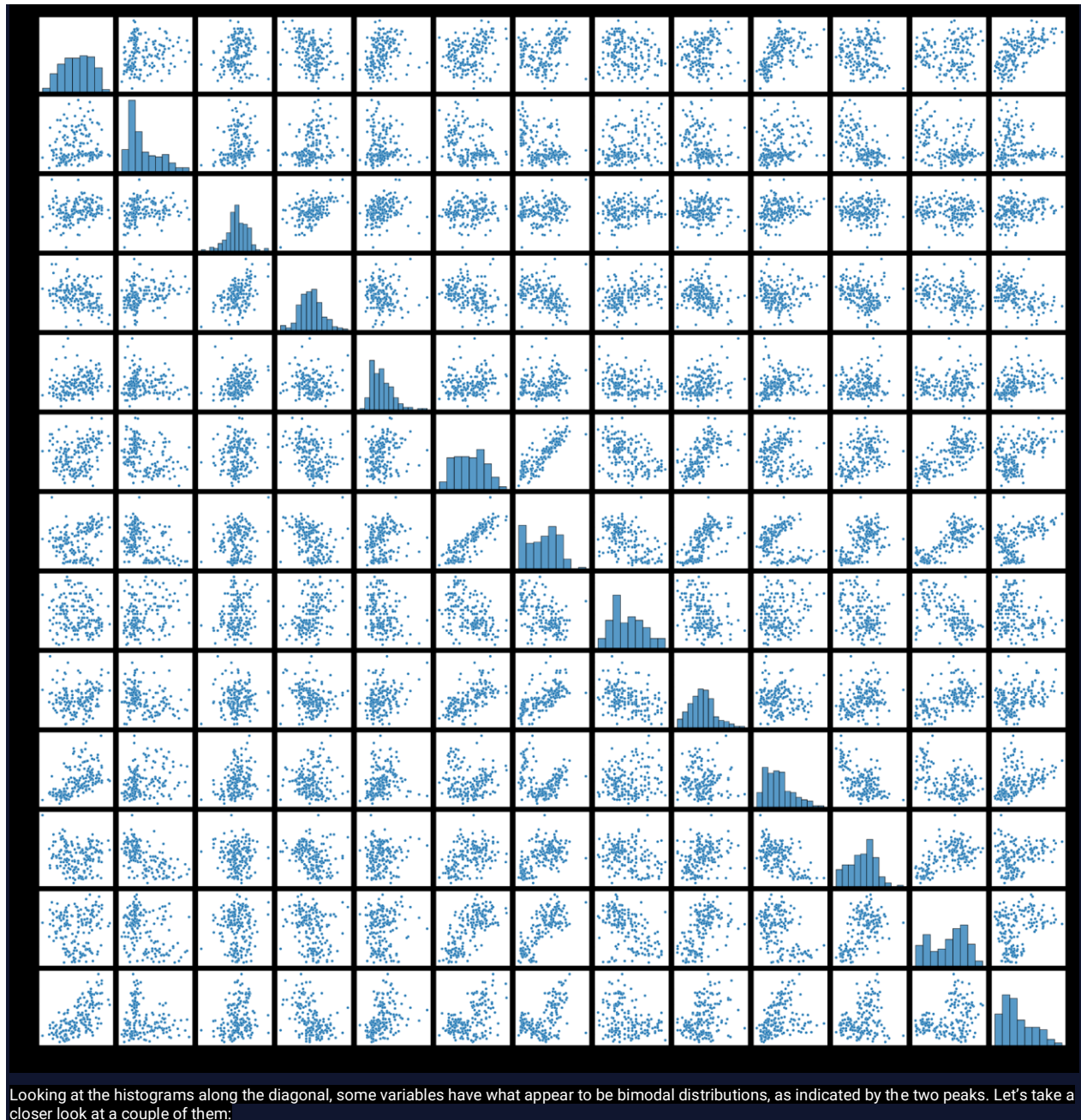
As usual, before we analyze the data, we should go through the process of previewing the data:

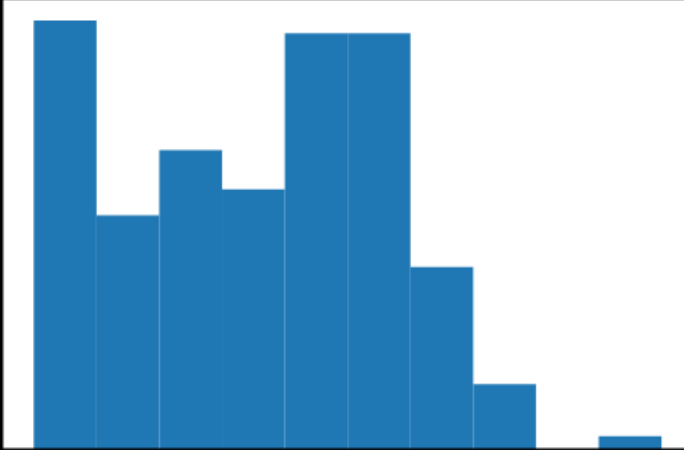


Before continuing with our analysis, we will want to clean and standardize the data, as well as ensure that the variables are coded appropriately as numerical values.

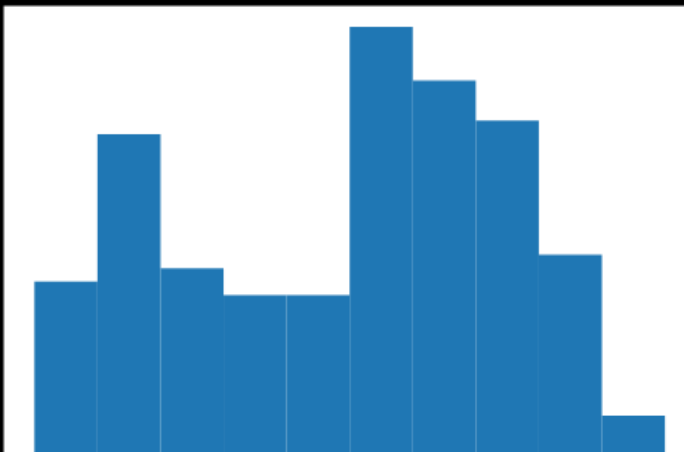
Pairplot to look for clusters

A pairplot of the variables is a good way to look for univariate and bivariate clusters.



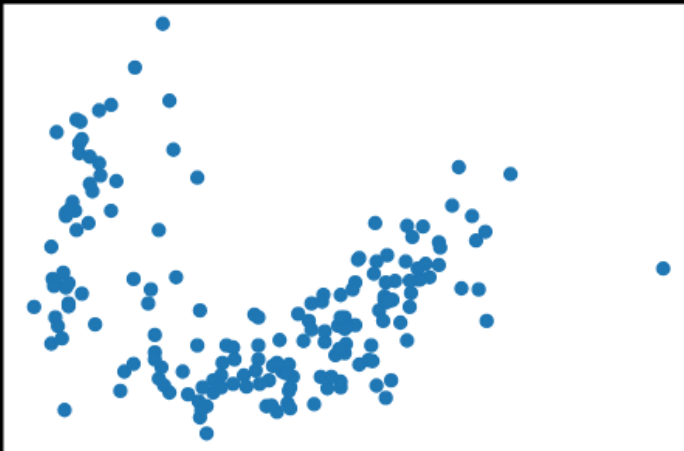


In this histogram of **Flavanoids**, we can see a peak at the very left-hand side as well as between 0 and 1. This is evidence that there could be two groups of wines with respect to **Flavanoids** (low and high flavanoid wines).

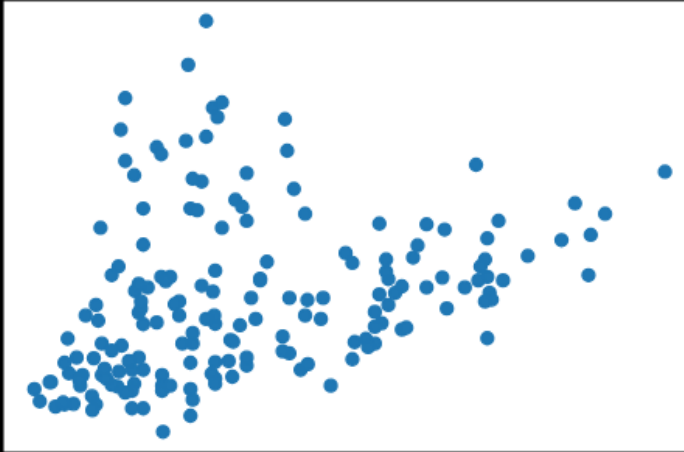


Similarly, the variable of **OD280** has two distinct peaks, one between -1.5 and -1, and another between 0 and 0.5. Thus, we have evidence of at least two wine groups based on **OD280** as well.

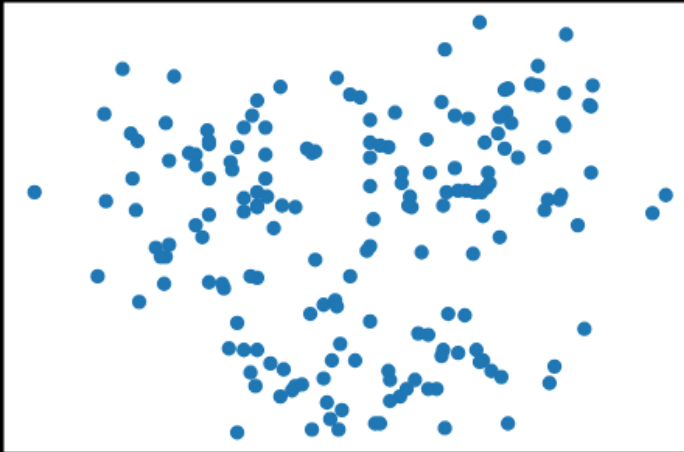
Looking at the rest of the pair plot, it is difficult to see relationships in the scatterplots when there are this many variables. The task of looking at each scatterplot can be tedious, so we have selected three bivariate plots to highlight:



This first scatterplot is between **Flavanoids** and **Color_Intensity**. In this plot, we see at least two clear elongated clusters.



This second scatterplot is between `Proline` and `Color Intensity` and appears to have at least three different clusters.

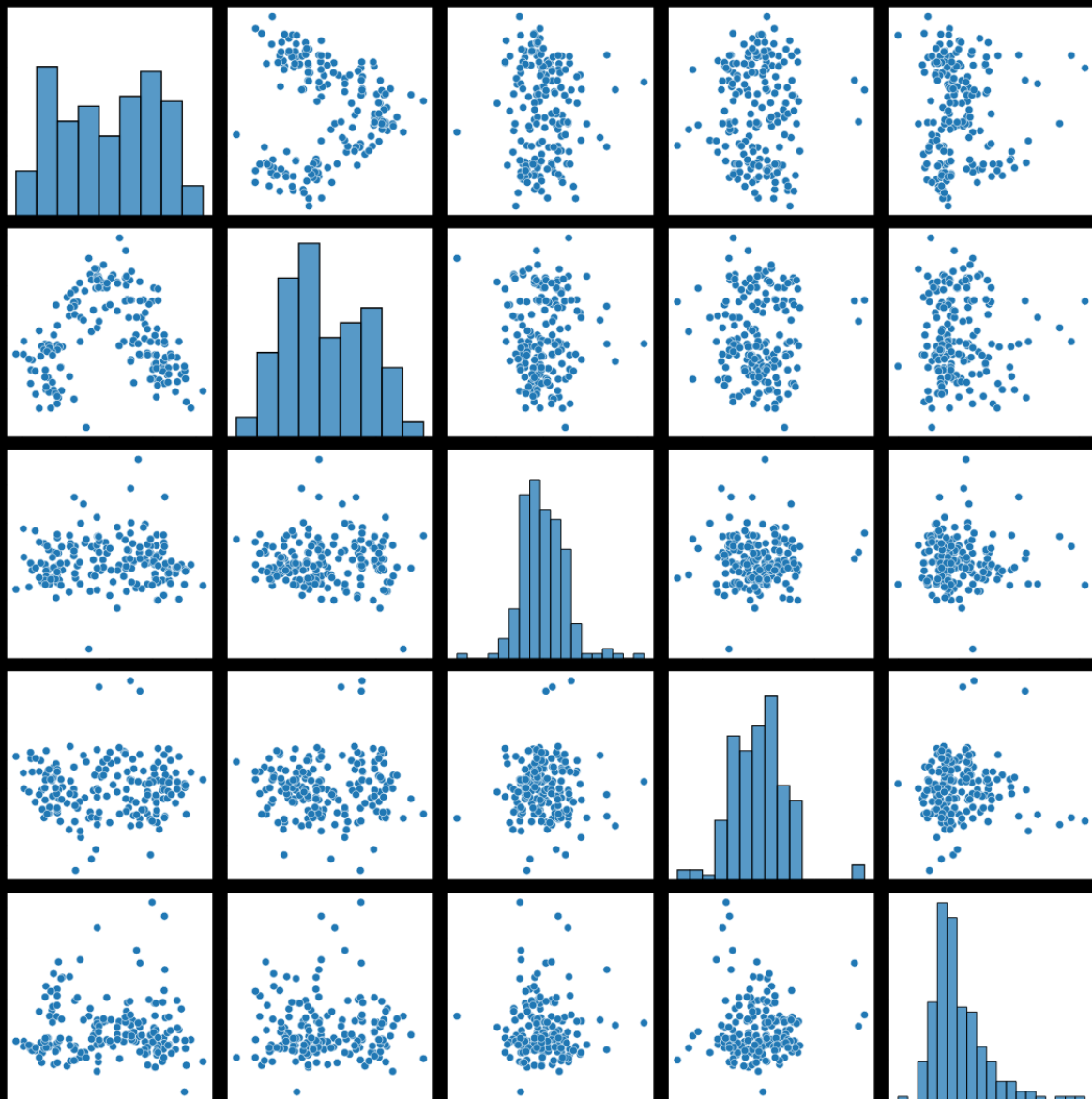


This final scatterplot is between `Alcohol` and `OD280` and contains three distinct round blobs of points.

Based on these plots we can conclude that there are probably at least three different types of wine in this dataset and that `Flavanoids`, `Color Intensity`, `Alcohol`, and `OD280` may be particularly important in distinguishing those groups.

Feature reduction for EDA

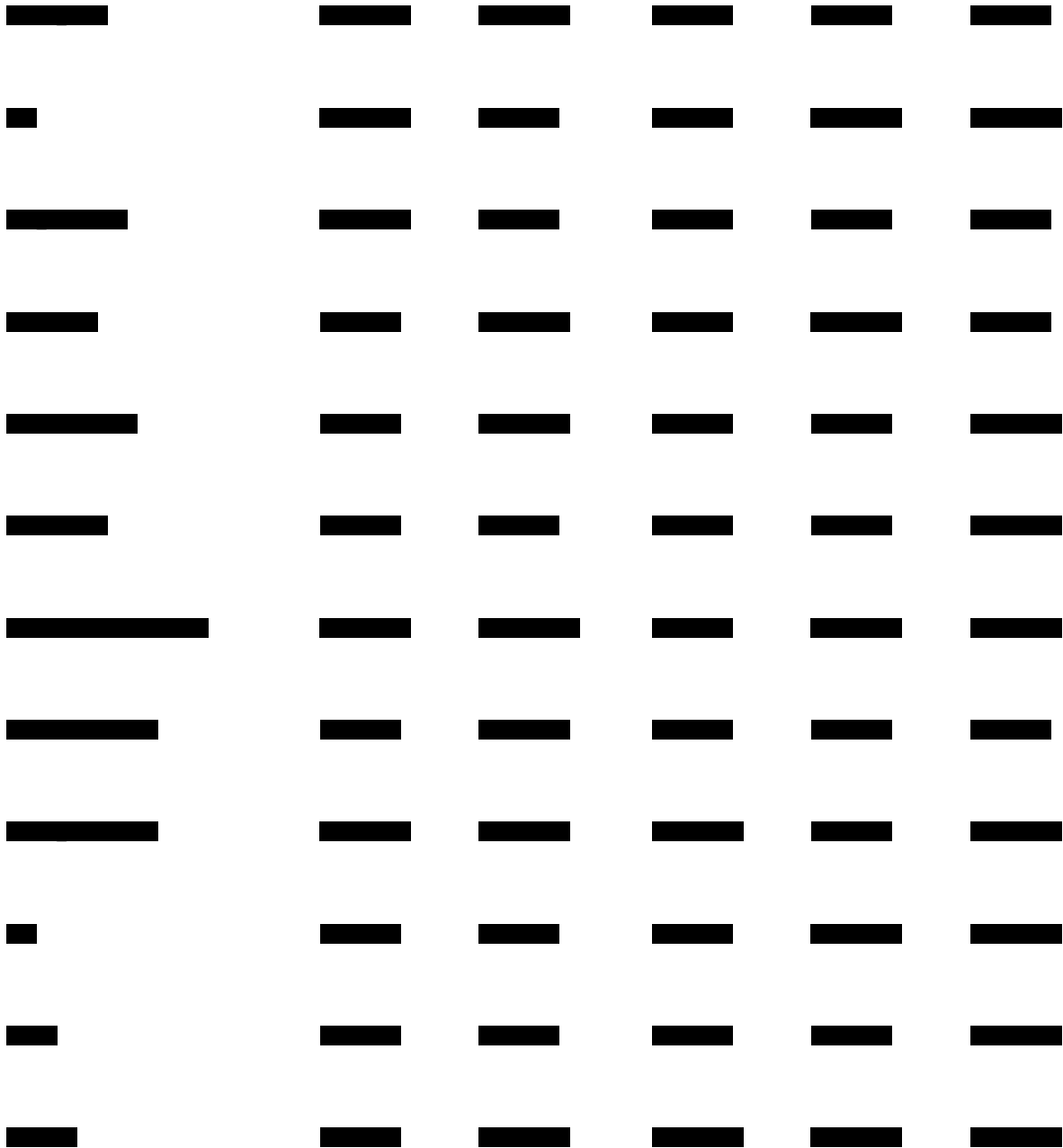
The first pair plot with all of the variables was difficult to inspect, but we can reduce the number of dimensions by transforming our data using PCA. Let's look at the pair plot of the first few principal components:



Now, instead looking at a 13 by 13 pair plot, we can zoom in on a 5 by 5 plot that includes all of the original features. In fact, a single plot can be used to visualize relationships between all of the original features at once, not just two at a time. We can see that there is some distinction between groups in clusters 1 and 2, as well as 2 and 4. We also see three fairly clear groups in the plot of component 1 vs. 2. But what does this mean about our original features?

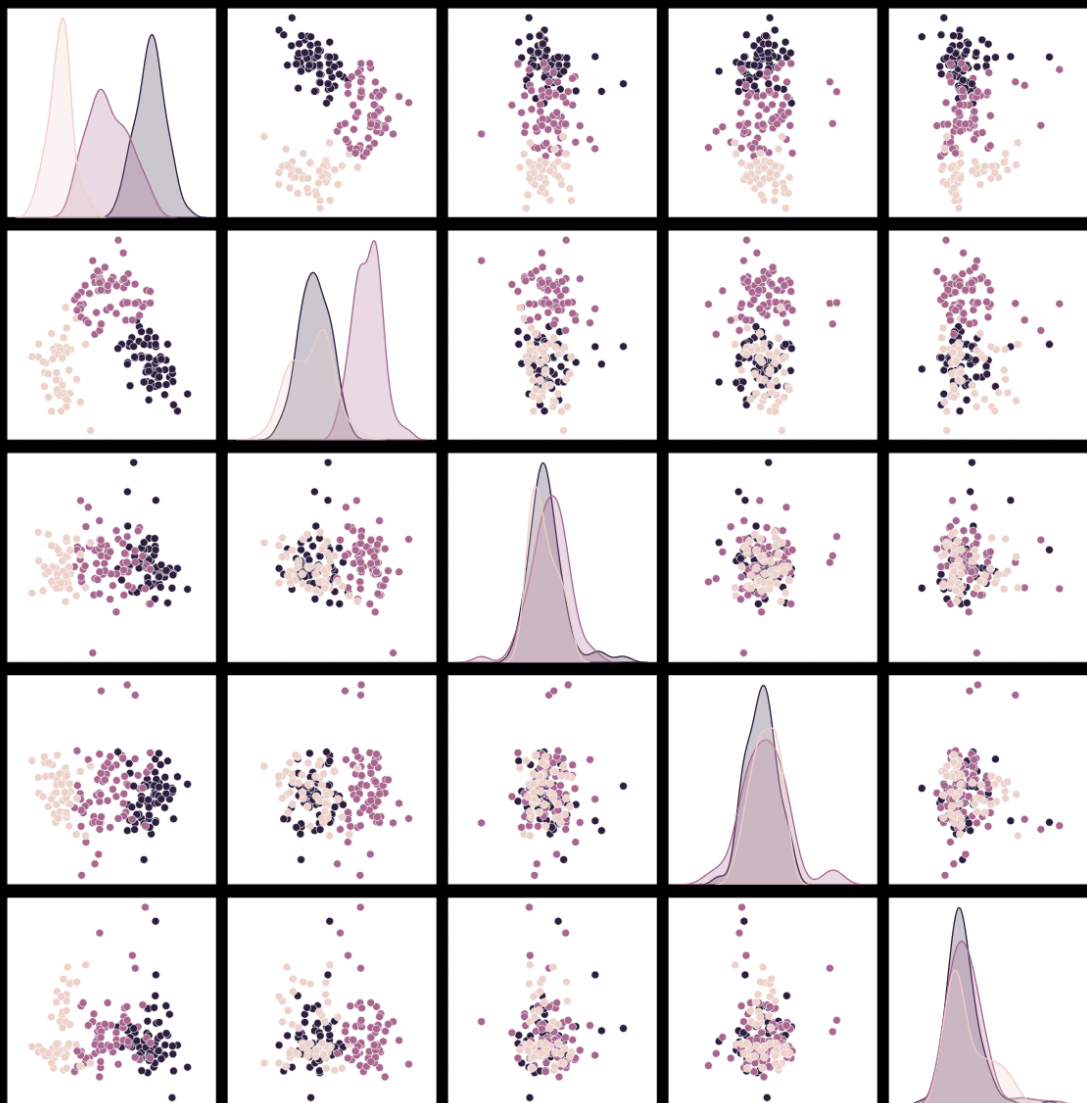
We can look at the weights for each of these components and see which features were most highly weighted in components 1 and 2 (which seem to cluster into three clear groups):





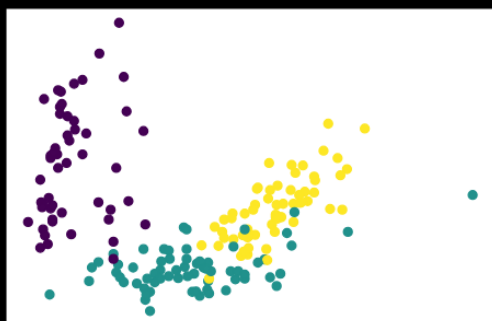
We are looking for the highest weighted feature for each component of interest. For component 1, this would be [Flavonoids](#), component 2, [Color_Intensity](#), and so on. These may be particularly important features to include in our model.

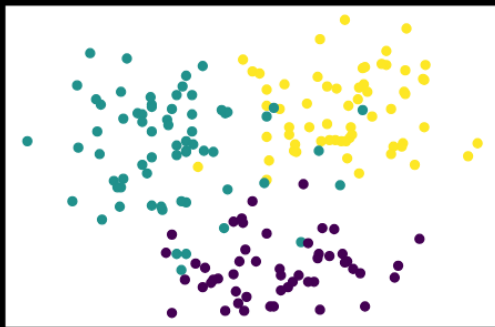
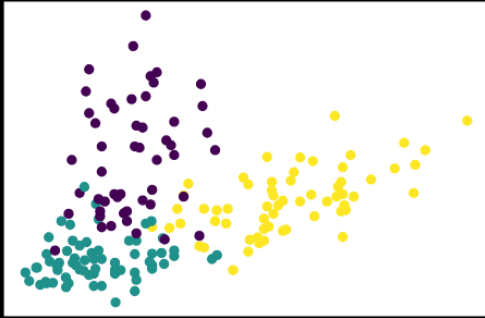
We can also use the transformed data to visually inspect the groups that are produced by other supervised machine learning methods. For example, here is the same pair plot of the transformed data, but the points are colored by the outcome of a k-means analysis with 3 groups:



We can see here that the groupings we saw in the PCA-transformed data align with those produced by the k-means model. Specifically, we see that the pair plot of components 1 and 2 separates the k-means clusters particularly well.

We can also look at the same bivariate scatterplots from earlier, with the added k-means results:





We suspected that these features would be useful in clustering our wines, and now we see that the k-means model is producing groups based on these features. This is also useful for explaining the k-means model to potential stake-holders. For example, we can say that the “purple” group produced by our k-means model is characterized by lower than average amounts of flavanoids and higher than average color intensity.

Conclusion

EDA before and after fitting unsupervised clustering algorithms is extremely helpful for checking model assumptions, choosing an algorithm, determining the number of groups, and explaining the results to potential stake-holders.

Introduction to Feature Engineering

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-machine-learning-fun/modules/dsml-intro-module-to-feature-engineering/articles/fe-introduction-to-feature-engineering>

Learn about feature engineering and its role in machine learning.

Introduction

Feature engineering is an integral part of building and implementing machine learning models. In this article, you'll learn about what feature engineering is, why we need it and where it fits in within the machine learning workflow. Additionally, you'll get a brief overview of the many data science tools involved in feature engineering and how they assist data scientists in model diagnostics.

What is feature engineering?

Before we can talk about feature engineering, we have to define what we mean by 'features'. A feature is a measurable property in a dataset, and a feature can be used as an input to a machine learning model. One way to think about features is as predictor variables that go into a model to predict the outcome variable. For example, if we want a model that predicts precipitation at a given place and time, we might use temperature, humidity, month, altitude, etc. as inputs. These are our features.

Often, when presented with a dataset, it might not be clear what features we should use for a specific model (i.e., should we use 'tree density' as an input to our precipitation model?). Similarly, in large datasets, there might be too many features to manually decide which features to use. Some features might be highly correlated with one another, some might not vary much with the outcome variable, and some might be in the wrong form to be a model input and so on. It is not uncommon that a data scientist might not realize any of this until they begin diagnosing a model that is performing poorly.

Feature engineering is a way to address these challenges. It is an umbrella term for the techniques we use to help us make decisions about features in machine learning model implementations.

Why do we need feature engineering?

To paraphrase Tolstoy, "All functional model implementations resemble one another but every dysfunctional model implementation is dysfunctional in its own way."

There are a lot of different ways that a model implementation can be dysfunctional. So the question is: What would make a model implementation functional or dysfunctional? Consider the following four attributes:

1. Performance:

We would like our machine learning model to perform "well" on our data. If it is not able to predict the outcome variable (to a reasonable degree of accuracy) on known data, it would be unwise to use it to predict outcomes on unknown data.

2. Runtime:

Suppose a model has excellent performance but takes a really long time to run. For a data scientist, depending on their available computational resources, such a model might be impractical for production.

3. Interpretability:

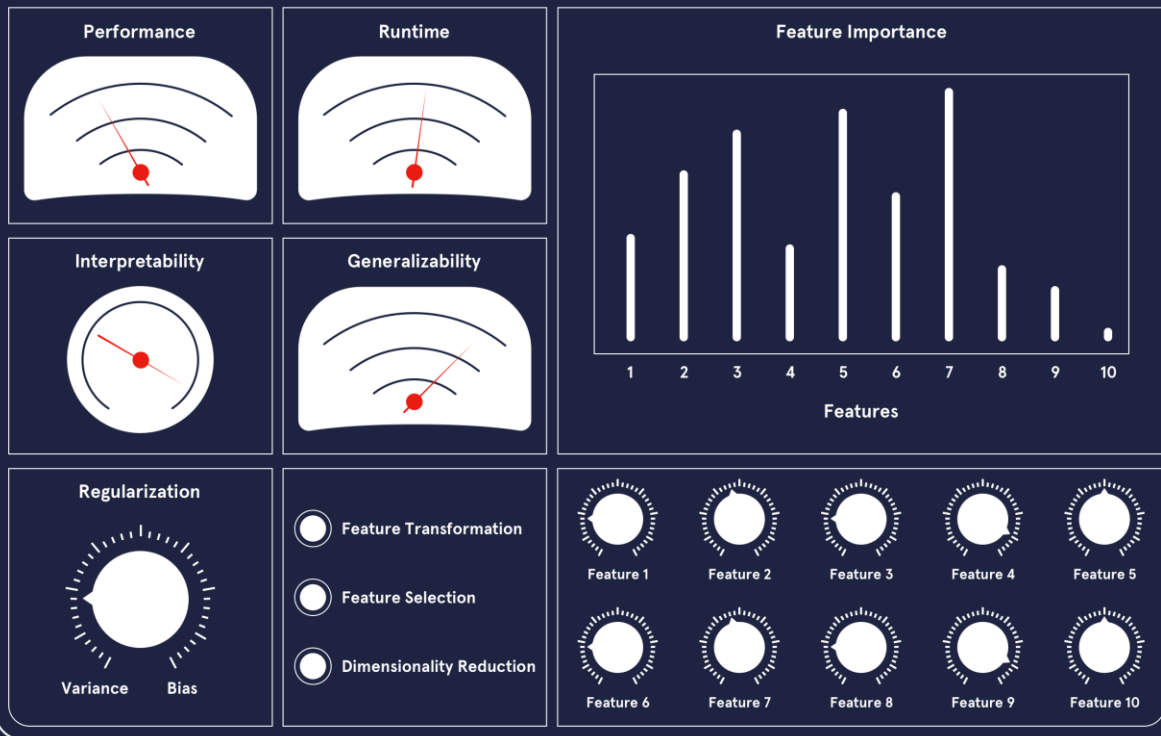
A model is only as good as the insights it helps us glean from the data. Data scientists are often tasked with finding out what factors drive different outcomes. A well-performing model would not be of much help if it's opaque and uninterpretable.

4. Generalizability:

We would like our model to generalize well to unseen data. Often data scientists work with streaming data and need their model to be flexible with new and unknown data..

Feature engineering can be thought of as a toolkit of techniques to use when a model is missing one or more of the above attributes. If we imagine a model diagnostic machine (kind of like a modern version of this one) with meters representing the attributes, feature engineering would be akin to turning knobs and pushing buttons until we arrive upon a model that meets all of our attributes satisfactorily.

FEATURE ENGINEERING TOOLKIT



Feature Engineering and the Machine Learning Workflow

So where does feature engineering fall within the machine learning workflow? Does it go before modeling, after modeling or alongside modeling? The answer is: all of the above! Feature engineering is often introduced as an intermediate step between exploratory data analysis and implementing a machine learning algorithm. In reality, these distinctions are fuzzy and the process is not exactly linear.

Broadly, we can divide feature engineering techniques into three categories:

1. Feature Transformation methods:

These involve numerical transformations methods and ways to encode non-numerical variables. These techniques are applied *before* implementing a machine learning model. They include and are not limited to: scaling, binning, logarithmic transformations, hashing and one hot encoding. These methods typically improve performance, runtime and interpretability of a model.

2. Dimensionality Reduction methods:

Dimensionality of a dataset refers to the number of features within a dataset and reducing dimensionality allows for faster runtimes and (often) better performance. This is an extremely powerful tool in working with datasets with "high dimensionality". For instance, a hundred-feature problem can be reduced to less than ten modified features, saving a lot of computational time and resources while maintaining or even improving performance. Typically, dimensionality reduction methods are machine learning algorithms themselves, such as Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), etc.

These techniques transform the existing feature space into a new subset of features that are ordered by decreasing importance. Since they "extract" new features from high dimensional data they're also referred to as **Feature Extraction** methods. The transformed features do not directly relate to anything in the real world anymore. Rather, they are mathematical objects that are

related to the original features. However, these mathematical objects are often difficult to interpret. The lack of interpretability is one of the drawbacks of dimensionality reduction.

3. Feature Selection methods

Feature selection methods are a set of techniques that allow us to choose among the pool of features available. Unlike dimensionality reduction, these retain the features *as they are* which makes them highly interpretable. They usually belong to one of these three categories:

i. Filter methods:

These are statistical techniques used to “filter” out useful features. Filter methods are completely model agnostic (meaning they can be used with any model) and are useful sanity checks for a data scientist to carry out before deciding on a model. They include and are not limited to: correlation coefficients (Pearson, Spearman, etc) , χ^2 , ANOVA, and Mutual Information calculations.

ii. Wrapper methods:

Wrapper methods search for the best set of features by using a “greedy search strategy”. They pick a subset of features, train a model, evaluate it, try a different subset of features, train a model again, and so on until the best possible set of features and most optimal performance is reached. As this could potentially go on forever, a stopping criterion based on number of features or a model performance metric is typically used. Forward Feature Selection, Backward Feature Elimination and Sequential Floating are some examples of wrapper method algorithms.

iii. Embedded methods:

Embedded methods are implemented *during* the model implementation step. Regularization techniques such as Lasso or Ridge tweak the model to get it to generalize better to new and unknown data. Tree-based feature importance is another widely used embedded method. This method provides insight into the features that are most relevant to the outcome variable while fitting decision trees to the data.

Summary

Here’s a brief summary of the feature engineering methods we’ve covered in this article, the attributes they seek to improve, and where they fit in within the machine learning workflow:

Feature Engineering Method	What	Why	Where
1. Feature Transformation	Transforms Numerical and non-numerical data	Performance, Runtime, Interpretability	Before model implementation
2. Dimensionality Reduction	Extracts a new feature subset	Runtime, Performance	Before model implementation
3. Feature Selection	Filter Methods	Interpretability, Performance	Before model implementation
	Wrapper Methods	Performance, Interpretability	Iterates over model implementation (i.e., before and after)
	Embedded Method: Regularization	Generalizability, Interpretability	Implemented with model implementation
	Embedded Method: Feature Importance	Interpretability, Performance	Implemented with model implementation

We’re now ready to learn about these in-depth!

Introduction: Data Transformations

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-machine-learning-fun/modules/dsml-intro-module-data-transformations/informationals/fe-introduction-data-transformations>

Find out what you will learn in Data Transformations for Feature Analysis!

In this module you will learn how to prepare your data and features prior to running a machine learning model. This is a very essential step that, if skipped, can cause some grief down the line. It is usually performed right after or alongside Exploratory Data Analysis (EDA).

Why did we build this?

Let's take a moment to consider the consequence of not doing data transformations before a model implementation. An obvious one might be that the model code errors out. This is still a benign possibility as it tells you that *something* is wrong. The more insidious possibility is one where it doesn't cause a problem until much later. Multiple diagnoses of many poorly performing models later, it might become apparent that the data might've needed to be transformed in some way at the beginning! Data Transformations are thus an irreplaceable step for data scientist in their workflows.

What will you learn?

You're going to learn about:

numerical data transformations such as standardizing, binning, scaling, etc.

different ways of encoding categorical and non-numerical data.

when, where and how to apply these methods when presented with a dataset

Encoding Categorical Variables

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsml-machine-learning-fun/modules/dsml-data-transformations-for-feature-analysis/articles/fe-encoding-categorical-variables>

Encoding categorical variables with Python.

Introduction

Categorical data is data that has more than one category. When working with that type of data we have two types, nominal and ordinal. *Nominal data* is data that has no particular order or hierarchy to it, and *ordinal data* is categorical data where the categories have order, but the differences between the categories are not important or unclear.

We will be working with a dataset of used cars for this article to truly understand and demonstrate how to work with categorical data. Let's explore it and see what type of data we are working with.

```
import pandas as pd

# import data
cars = pd.read_csv('cars.csv')

# check variable types
print(cars.dtypes)
## OUTPUT
# year          int64
# make          object
# model         object
# trim          object
# body          object
# transmission  object
# vin           object
# state         object
# condition     object
# odometer      float64
# color         object
# interior      object
# seller        object
# mmr           int64
# sellingprice  int64
# saledate      object
```

We can see from the output that we have a lot of features that are `dtype = object` and that tells us those features could be text or a mix of text and numerical values. For our encoding examples, we will explore a few of those object features and transform those values so we can have a data frame ready for machine learning.

The reason we put our time into this level of encoding is that there are many machine learning models that cannot handle text and will only work with numbers. Our data must be encoded into numbers before we even begin to train, test, or evaluate a model.

Ordinal encoding

We mentioned already that *ordinal data* is data that does have order and a hierarchy between its values. Let us take a look at the `condition` feature from our data frame and perform a `value_counts` to see how many times each label is listed in our feature.

```
print(cars['condition'].value_counts())
# #OUTPUT
# New          2881
# Like New     2860
# Good         2027
# Fair         753
# Excellent    186
```

This is definitely an example of ordinal data; the condition of the used cars can easily be put in order of those in the “best” condition to the cars in the “worst” condition. The output printed the labels with the highest counts, but we can assume the following hierarchy:

Excellent
New

Like New
Good
Fair

We need to convert these labels into numbers, and we can do this with two different approaches. First, we can do this by creating a dictionary where every label is the key and the new numeric number is the value. 'Excellent' will get the highest score and 'Fair' will be our lowest score. Then we will map each label from the `condition` column to the numeric value and create a new column called `condition_rating`.

```
# create dictionary of label:values in order
rating_dict = {'Excellent':5, 'New':4, 'Like New':3, 'Good':2,
               'Fair':1}

# create a new column
cars['condition_rating'] = cars['condition'].map(rating_dict)
```

The second approach we will show is how to utilize the `sklearn.preprocessing` library `OrdinalEncoder`. We follow a similar approach: we set our categories as a list, and then we will `.fit_transform` the values in our feature `condition`. We need to make sure we adhere to the shape requirements of a 2-D array, so you'll notice the method `.reshape(-1,1)`.

We'll also note, this method will not work if your feature has `NaN` values. Those need to be addressed prior to running `.fit_transform`.

```
# using scikit-learn
from sklearn.preprocessing import OrdinalEncoder

# create encoder and set category order
encoder = OrdinalEncoder(categories=[['Excellent', 'New', 'Like New',
                                     'Good', 'Fair']])

# reshape our feature
condition_resaped = cars['condition'].values.reshape(-1,1)

# create new variable with assigned numbers
cars['condition_rating'] = encoder.fit_transform(condition_resaped)
```

Label Encoding

Now, we can talk about *nominal data*, and we have to approach this type of data differently than what we did with *ordinal data*. Our `color` feature has a lot of different labels, but here are the top five colors that appear in our data frame.

```
print(cars['color'].nunique())
# #OUTPUT
# 19

print(cars['color'].value_counts()[:5])
# #OUTPUT
# black      2015
# white      1931
# gray       1506
# silver     1503
# blue       869
```

To prepare this feature, we still need to convert our text to numbers, so let's do just that. We will demonstrate two different approaches, with the first one showing how to convert the feature from an `object` type to a `categories` type.

```
# convert feature to category type
cars['color'] = cars['color'].astype('category')

# save new version of category codes
cars['color'] = cars['color'].cat.codes

# print to see transformation
print(cars['color'].value_counts()[:5])
# #OUTPUT
# 2      2015
# 18     1931
# 8      1506
# 15     1503
# 3       869
```

Comparing our newly transformed data to the original top 5 list, we can see Black was transformed to number 2, White was transformed to 18, and so on.

However, we have created a problem for ourselves and potentially our model. We can see that 'Blue' cars now have a value of 3, and our model will assume that 'Blue' has lower precedence over the 'Black' car, whose color has a value of 2. Since 'Blue' cars = 3 and 'White' cars = 18, our model could actually give 'White' cars 6 times more weight than a 'Blue' car simply because of the way we encoded this feature. To combat this ordinal assumption our model will make, we should *one-hot encode* our nominal data, which we will cover in the next section.

One more way we can transform this feature is by using `sklearn.preprocessing` and the `LabelEncoder` library. This method will not work if your feature has `NaN` values. Those need to be addressed prior to running `.fit_transform`.

```
from sklearn.preprocessing import LabelEncoder

# create encoder
encoder = LabelEncoder()

# create new variable with assigned numbers
cars['color'] = encoder.fit_transform(cars['color'])
```

Coding question

Questions

Perform a label encoder on our `make` feature by changing the feature to a category type.

One-hot Encoding

One-hot encoding is when we create a dummy variable for each value of our categorical feature, and a *dummy variable* is defined as a numeric variable with two values: 1 and 0. We will continue to talk about our `color` feature from our used car dataset.

Looking at this visual below, we can see we have ten cars in four different colors. In place of the single `color` column, we create four dummy variables - one new column for each color. Then the values that go into that column are binary, indicating if the car in that row is the color of the column name (1) or not (0).

Cars	Yellow	Green	Pink	Blue
	1	0	0	0
	0	1	0	0
	1	0	0	0
	1	0	0	0
	0	0	0	1
	0	1	0	0
	0	0	0	1
	0	0	1	0
	1	0	0	0
	0	1	0	0

This approach is great for our color feature and will allow the model to see each category as its own feature and not try to create order between a “Black car” and a “Red car”. Here is how we can implement this in Python:

```
import pandas as pd
# use pandas .get_dummies method to create one new column for each color
ohe = pd.get_dummies(cars['color'])

# join the new columns back onto our cars dataframe
cars = cars.join(ohe)
```

A downside to this approach is that it can create a lot of features which can then create a very sparse matrix.

Coding question

Questions

One-hot encode the feature `body` from our cars dataset.

Binary encoding

If we find the need to one-hot encode a lot of categorical features which would, in turn, create a sparse matrix and may cause problems for our model, a strong alternative to this issue is performing a *binary encoder*. A *binary encoder* will find the number of unique categories and then convert each category to its binary representation. Let us take a quick review of binary numbers and keep using our `color` feature. We know that we have 19 unique colors, so the way to represent the numbers from 1 to 19 in binary format is as follows: We can easily see that our highest number 19 is 5 digits long, so our binary encoder will need 5 columns to be able to represent all digits. Here is a sample of how our `color` column will transform each color if we were to perform a binary encoder.

color_1	color_2	color_3	color_4	color_5	color_text
0	0	0	0	1	blue
0	0	0	1	0	—
0	0	0	1	1	gold
0	0	1	0	0	white
0	0	1	0	1	red
0	0	1	1	0	silver
0	0	1	1	1	black
0	1	0	0	0	gray
0	1	0	0	1	burgundy
0	1	0	1	0	brown
0	1	0	1	1	turquoise
0	1	1	0	0	orange
0	1	1	0	1	green
0	1	1	1	0	beige
0	1	1	1	1	yellow
1	0	0	0	0	purple
1	0	0	0	1	off-white
1	0	0	1	0	charcoal
1	0	0	1	1	pink

Our 19th color, pink, has transformed to be represented in the binary form 10011. If we were to utilize this process instead of the traditional one-hot encoder we would have 5 numerical features instead of 19, reducing our features by about 75%!

To make this happen with Python we'll use a library called `category_encoders` and import `BinaryEncoder`. We will determine which column to transform and set `drop_invariant` to `True` so it will keep the five binary columns. If it is set to the default 0, then we would have an additional column full of zeros.

```
from category_encoders import BinaryEncoder

#this will create a new data frame with the color column removed and
replaced with our 5 new binary feature columns
colors = BinaryEncoder(cols = ['color'], drop_invariant =
True).fit_transform(cars)
```

Hashing

Another option we have available to us is an encoding technique called *hashing*. This process is similar to one-hot encoding where it will create new binary columns, but within the parameters, you can decide how many features to output. A huge advantage is reduced dimensionality, but a large disadvantage is that some categories will be mapped to the same values. That is called *collision*.

For example, we have 19 different colored cars. If I were to use the hash encoder and set the number of features to be 5, I will definitely have a few colors with the same hash values.

	col_0	col_1	col_2	col_3
color				
beige	0	1	0	0
black	0	0	0	1
blue	1	0	0	0
brown	0	0	1	0
burgundy	0	1	0	0
charcoal	0	0	1	0
gold	1	0	0	0
gray	0	1	0	0
green	0	0	0	0
off-white	0	0	1	0
orange	1	0	0	0
pink	1	0	0	0
purple	0	0	0	1
red	0	0	1	0
silver	0	1	0	0
turquoise	0	1	0	0
white	0	0	1	0
yellow	0	0	0	0
—	0	1	0	0

We can easily see that brown and charcoal colors have the same hash values. Meaning, we've lost some information and our model won't be able to see the difference between those two colors.

Here is how we can make this work with Python. Our final result of `hash_results` will produce a data frame of just 5 columns, so we will want to concatenate this new data onto our original data frame.

```
from category_encoders import HashingEncoder

# instantiate our encoder
encoder = HashingEncoder(cols='color', n_components=5)

# do a fit transform on our color column and set to a new variable
hash_results = encoder.fit_transform(cars['color'])
```

Now you may be thinking, when would I use this if I'm going to lose information and my model will see brown and charcoal (or some other color combo with the same hash value) as the same thing? Well, this could be a solution to your project and dataset if you are not as interested in assessing the impact of any particular categorical value.

For this example, maybe you aren't interested in knowing which color car had an impact on your final prediction, but you want to be able to get the best performance from your model. This encoding solution may be a good approach.

Target encoding

Target encoding is a Bayesian encoder used to transform categorical features into hashed numerical values and is sometimes called the *mean encoder*. This encoder can be utilized for data sets that are being prepared for regression-based supervised learning, as it needs to take into consideration the mean of the target variable and its correlation between each individual category of our feature. In fact, the numerical values of each category is replaced with a blend of the posterior probability of the target given a particular categorical value and the prior probability of the target over all the training data.

Woah, now that was a lot of Bayesian buzzwords. How would it work with our specific `color` feature? It replaces each color with a blend of the mean price of that car color and the mean price of all the cars. Had it been predicting something categorical, it would've used a Bayesian target statistic.

Some drawbacks to this approach are overfitting and unevenly distributed values that could lead to extremes. Let's review how to implement this in Python and check out what type of numerical values it will return. Again, we'll continue with our color feature - hope you are not yet tired of it!

Say we are preparing our dataset for a regression-based supervised learning algorithm that is trying to predict the selling price.

```
from category_encoders import TargetEncoder

# instantiate our encoder
encoder = TargetEncoder(cols = 'color')

# set the results of our fit_transform to a variable
# the output will be its own pandas series
encoder_results = encoder.fit_transform(cars['color'],
cars['sellingprice'])

print(encoder_results.head())
#    color
# 0 11761.881473
# 1 18007.276995
# 2  8458.251232
# 3 14769.292595
# 4 12691.099747
```

We can examine all the different values our encoder_results holds, and if we look at the output from below we can see our min is about 3,054 and our max is about 18,048. That is quite a big difference!

```
# print all 19 unique values
print(np.sort(encoder_results['color'].unique()))
# OUTPUT
# [ 3054.12209927  8088.87434555  8458.25123153  9276.78571429
#   9867.50002121  9885.8093167   11043.90243902 11247.82608763
#  11761.88147296 11805.06187625 12124.83443709 12376.19047882
#  12691.09974747 13912.83399734 14769.29259451 15496.72704715
#  17174.36440678 17176.25931731 18007.27699531 18048.52540833]
```

Encoding date-time variables

Every data analyst or scientist will have to work with date-time objects at some point in their career. There is so much information to be gained from date-time objects. We will work with the `saledate` feature - yay, a new column to explore! The very first thing we need to do is convert this to a date-time object.

```
print(cars['saledate'].dtypes)
# # OUTPUT
# dtype('O')

cars['saledate'] = pd.to_datetime(cars['saledate'])
# #OUTPUT
# datetime64[ns, tzlocal()]
```

Now that we have our feature set up as a datetime object, let's demonstrate some methods we can utilize to get additional information.

```
# create new variable for month
cars['month'] = cars['saledate'].dt.month

# create new variable for day of the week
cars['dayofweek'] = cars['saledate'].dt.day
```



```
# create new variable for difference between cars model year and year sold
cars['yearbuild_sold'] = cars['saledate'].dt.year - cars['year']
```

Below are other options available through pandas `.dt`

Syntax	Description & Output
<code>df['col'].dt.year</code>	Outputs the year
<code>df['col'].dt.day</code>	Outputs the day number
<code>df['col'].dt.hour</code>	Outputs the hour from the time
<code>df['col'].dt.minute</code>	Outputs the minute from the time
<code>df['col'].dt.second</code>	Outputs the seconds from the time
<code>df['col'].dt.week</code>	Outputs the week ordinal of the year
<code>df['col'].dt.dayofweek</code>	Outputs the day of the week with Monday = 0 & Sunday = 6

Additional methods can be found [here](#).

Review

Wow, we covered a lot in this article! You have so many options when it comes to transforming categorical data into numerical values. We touched in:

- Ordinal encoding
- Label encoding
- One-hot encoding
- Binary encoding
- Hashing
- Target encoding
- Date-time encoding

We hope you can see the importance of getting your categorical features just right before you begin to train and test a model.

Regression vs. Classification

Learn about the two types of Supervised Learning algorithms.

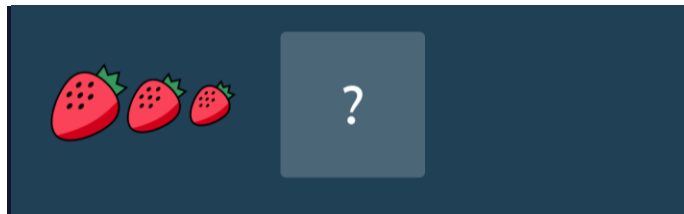
Machine Learning is a set of many different techniques that are each suited to answering different types of questions.

One way of categorizing machine learning algorithms is by using the kind output they produce. In terms of output, two main types of machine learning models exist: those for regression and those for classification.

Regression

Regression is used to predict outputs that are *continuous*. The outputs are quantities that can be flexibly determined based on the inputs of the model rather than being confined to a set of possible labels.

For example:



Linear regression is the most popular regression algorithm. It is often underrated because of its relative simplicity. In a business setting, it could be used to predict the likelihood that a customer will churn or the revenue a customer will generate. More complex models may fit this data better, at the cost of losing simplicity.

Classification

Classification is used to predict a *discrete label*. The outputs fall under a finite set of possible outcomes. Many situations have only two possible outcomes. This is called *binary classification* (True/False, 0 or 1, 🌭 / not 🌭).

For example:

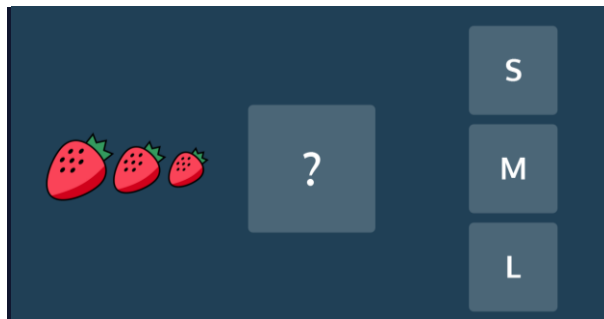
- Predict whether an email is spam or not
- Predict whether it will rain or not
- Predict whether a user is a power user or a casual user

There are also two other common types of classification: *multi-class classification* and *multi-label classification*.

Multi-class classification has the same idea behind binary classification, except instead of two possible outcomes, there are three or more.

For example:

- Predict whether a photo contains a pear, apple, or peach
- Predict what letter of the alphabet a handwritten character is
- Predict whether a piece of fruit is small, medium, or large



An important note about binary and multi-class classification is that in both, each outcome has one specific label. However, in multi-label classification, there are multiple possible labels for each outcome. This is useful for customer segmentation, image categorization, and sentiment analysis for understanding text. To perform these classifications, we use models like Naive Bayes, K-Nearest Neighbors, SVMs, as well as various deep learning models.

An example of multi-label classification is shown below. Here, a cat and a bird are both identified in a photo showing a classification model with more than one label as a result.



Choosing a model is a critical step in the [Machine Learning process](#). It is important that the model fits the question at hand. When you choose the right model, you are already one step closer to getting meaningful and interesting results.

Linear Regression

Review

We have seen how to implement a linear regression

[algorithm](#)

Preview: Docs Loading link description
in Python, and how to use the linear regression model from scikit-learn. We learned:

We can measure how well a line fits by measuring *loss*.

The goal of linear regression is to minimize loss.

To find the line of best fit, we try to find the *b* value (intercept) and the *m* value (slope) that minimize loss.

Convergence refers to when the parameters stop changing with each iteration.

Learning rate refers to how much the parameters are changed on each iteration.

We can use Scikit-learn's `LinearRegression()` model to perform linear regression on a set of points.

These are important tools to have in your toolkit as you continue your exploration of data science.

Instructions

Find another dataset, maybe in [scikit-learn's example datasets](#). Or on [Kaggle](#), a great resource for tons of interesting data.

Try to perform linear regression on your own! If you find any cool linear correlations, make sure to share them!

As a starter, we've loaded in the [Boston housing dataset](#). We made the *X* values the nitrogen oxides concentration (parts per 10 million), and the *y* values the housing prices. See if you can perform regression on these houses!

Multiple Linear Regression

Review

Great work! Let's review the concepts before you move on:

Multiple Linear Regression uses two or more

[variables](#)

Preview: Docs Loading link description

to make predictions about another variable:

$$y=b+m_1x_1+m_2x_2+...+m_nx_n$$

Multiple linear regression uses a set of independent variables and a dependent variable. It uses these variables to learn how to find optimal parameters. It takes a labeled dataset and learns from it. Once we confirm that it's learned correctly, we can then use it to make predictions by plugging in new *x* values.

We can use scikit-learn's `LinearRegression()` to perform multiple linear regression.

Residual Analysis is used to evaluate the regression model's accuracy. In other words, it's used to see if the model has learned the coefficients correctly.

Scikit-learn's `linear_model.LinearRegression` comes with a `.score()`

[method](#)

Preview: Docs Loading link description

that returns the coefficient of determination R^2 of the prediction. The best score is 1.0.

Solving a Regression Problem: Ordinary Least Squares to Gradient Descent

Learn about how the linear regression problem is solved analytically (using Ordinary Least Squares) and algorithmically (using Gradient Descent) and how these two methods are connected!

Linear regression finds a linear relationship between one or more predictor variables and an outcome variable. This article will explore two different ways of finding linear relationships: *Ordinary Least Squares* and *Gradient Descent*.

Ordinary Least Squares

To understand the method of least squares, let's take a look at how to set up the linear regression problem with linear equations. We'll use the [Diabetes dataset](#) as an example. The outcome variable, Y , is a measure of disease progression. There are 10 predictor variables in this dataset, but to simplify things let's take a look at just one of them: BP (average blood pressure). Here are the first five rows of data:

BP	Y
32.1	151
21.6	75
0.5	141
25.3	206
23	135

We can fit the data with the following simple linear regression model with slope m and intercept b :

$$Y = m \cdot \text{BP} + b + \text{error}$$

This equation is actually short-hand for a large number of equations — one for each patient in our dataset. The first five equations (corresponding to the first five rows of the dataset) are:

$$\begin{aligned} 151 &= m \cdot 32.1 + b + \text{error}_1 \\ 75 &= m \cdot 21.6 + b + \text{error}_2 \\ 141 &= m \cdot 0.5 + b + \text{error}_3 \\ 206 &= m \cdot 25.3 + b + \text{error}_4 \\ 135 &= m \cdot 23 + b + \text{error}_5 \end{aligned}$$

When we fit this linear regression model, we are trying to find the values of m and b such that the sum of the squared error terms above (e.g., $\text{error}_1^2 + \text{error}_2^2 + \text{error}_3^2 + \text{error}_4^2 + \text{error}_5^2 + \dots$) is minimized.

We can create a column matrix of Y (the outcome variable), a column matrix of BP (the predictor variable), and a column matrix of the errors and rewrite the five equations above as one matrix equation:

$$\begin{pmatrix} 15175141206135 \end{pmatrix} = m \cdot \begin{pmatrix} 32.121.60.525.31100 \end{pmatrix} + b \cdot \begin{pmatrix} 11111 \end{pmatrix} + \begin{pmatrix} \text{error1} \text{error2} \text{error3} \text{error4} \text{error5} \end{pmatrix}$$

Using the rules of matrix addition and multiplication, it is possible to simplify this to the following.

$$\begin{pmatrix} 15175141206135 \end{pmatrix} = \begin{pmatrix} 132.1121.610.5125.311100 \end{pmatrix} (bm) + \begin{pmatrix} \text{error1} \text{error2} \text{error3} \text{error4} \text{error5} \end{pmatrix}$$

In total we have 4 matrices in this equation:

A one-column matrix on the left hand side of the equation containing the outcome variable values that we will call Y

A two-column matrix on the right hand side that contains a column of 1's and a column of the predictor variable values (BP here) that we will call X .

A one-column matrix containing the intercept b and the slope m , i.e, the solution matrix that we will denote by the Greek letter β . The goal of the regression problem is to find this matrix.

A one-column matrix of the residuals or errors, the error matrix. The regression problem can be solved by minimizing the sum of the squares of the elements of this matrix. The error matrix will be denoted by the Greek letter ϵ .

Using these shorthands, the matrix representation of the regression equation is thus:

$$Y = X\beta + \epsilon$$

Ordinary Least Squares gives us an explicit formula for β . Here's the formula:

$$\beta = (X^T X)^{-1} X^T Y$$

A couple of reminders: X^T is the *transpose* of X . M^{-1} is the *inverse* of a matrix. We won't review these terms here, but if you want to know more you can check the Wikipedia articles for the [transpose](#) and [invertible matrices](#).

This looks like a fairly simple formula. In theory, you should be able to plug in X and Y , do the computations, and get β . But it's not always so simple.

First of all, it's possible that the matrix $X^T X$ might not even have an inverse. This will be the case if there happens to be an exact linear relationship between some of the columns of X . If there is such a relationship between the columns of X , we say that X is *multicollinear*. For example, your data set might contain temperature readings in both Fahrenheit and Celsius. Those columns would be linearly related, and thus $X^T X$ would not have an inverse.

In practice, you also have to watch out for data that is almost multicollinear. For example, a data set might have Fahrenheit and Celsius readings that are rounded to the nearest degree. Due to rounding error, those columns would not be perfectly correlated. In that case it would still be possible to compute the inverse of $X^T X$, but it would lead to other problems. Dealing with situations like that is beyond the scope of this article, but you should be aware that multicollinearity can be troublesome.

Another drawback of the OLS equation for β is that it can take a long time to compute. Matrix multiplication and matrix inversion are both computationally intensive operations. Data sets with a large number of predictor variables and rows can make these computations impractical.

Gradient Descent

Gradient descent is a numerical technique that can determine regression parameters without resorting to OLS. It's an iterative process that uses calculus to get closer and closer to the exact coefficients one step at a time. To introduce the concept, we'll look at a simple example: linear regression with one predictor variable. For each row of data, we have the following equation:

$$y_i = mx_i + b + \epsilon_i$$

The sum of the squared errors is

$$\sum_{i=1}^N \epsilon_i^2 = \sum_{i=1}^N (y_i - (mx_i + b))^2$$

This is the loss function. It depends on two variables: m and b . [Here's](#) an interactive plot where you can tune the parameters m and b and see how it affects the loss.

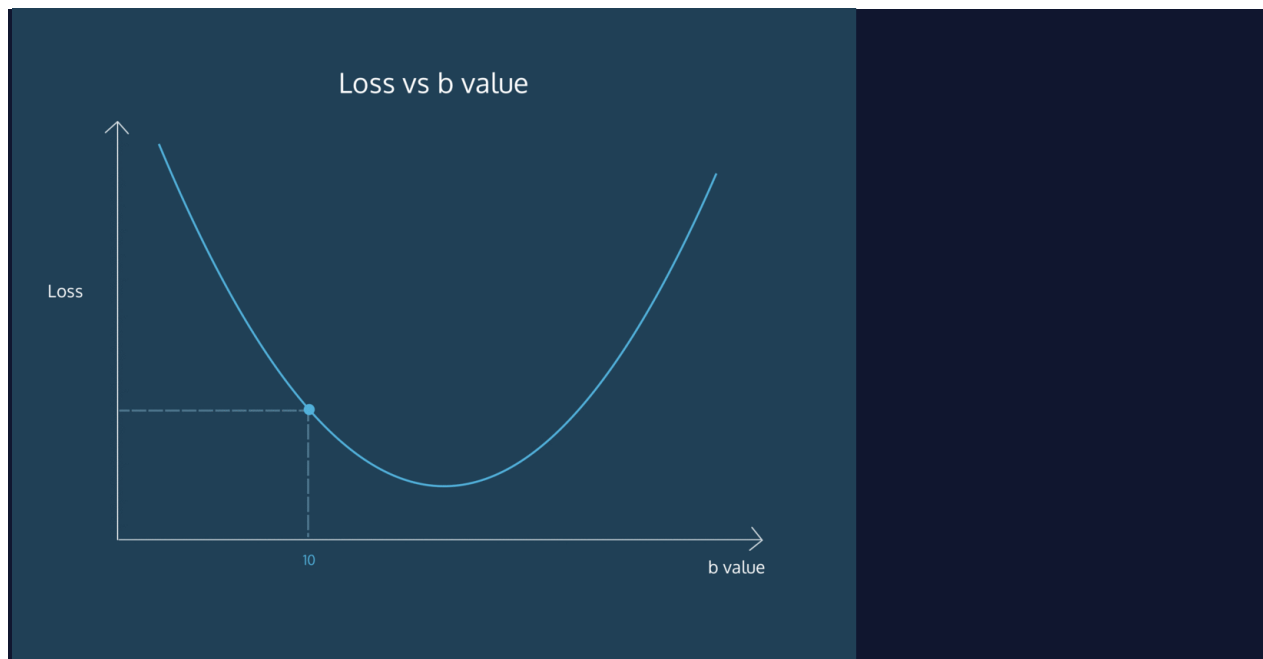
Try changing m and b and observe how those changes affect the loss. You can also try to come up with an algorithm for how to adjust m and b in order to minimize the loss.

As you adjust the sliders and try to minimize the loss, you might notice that there is a sweet spot where changing either m or b too far in either direction will increase the loss. Get too far away from that sweet spot, and small changes in m or b will result in bigger changes to the loss.

If playing with the sliders made you think about rates of change, derivatives, and calculus, then you're well on your way toward understanding gradient descent. The *gradient* of a function is a calculus concept that's very similar to the derivative of a function. We won't cover the technical definition here, but you can think of it as a vector that points uphill in the steepest direction. The steeper the slope, the larger the gradient. If you step in the direction opposite of the gradient, you will move downhill.

That's where the *descent* part of the gradient descent comes in. Imagine you're standing on some undulating terrain with peaks and valleys. Now take a step in the opposite direction of the gradient. If the gradient is large (in other words, if the slope you're on is steep), take a big step. If the gradient is small, take a small step. If you repeat this process enough times, you'll probably end up at the bottom of a valley. By going against the gradient, you've minimized your elevation!

Here's an example of how this works in two dimensions.



Let's take a closer look at how gradient descent works in the case of linear regression with one predictor variable. The process always starts by making starting guesses for m and b . The initial guesses aren't important. You could make random guesses, or just start with $m=0$ and $b=0$. The initial guesses will be adjusted by using gradient formulas.

Here's the gradient formula for b . This formula can be obtained by differentiating the average of the squared error terms with respect to b .

$$-2N \sum_{i=1}^n (y_i - (mx_i + b))$$

In this formula,

Here's the gradient formula for m . Again, this can be obtained by differentiating the average of the squared error terms with respect to m .

$$-2N \sum_{i=1}^n x_i (y_i - (mx_i + b))$$

The next step of gradient descent is to adjust the current guesses for m and b by subtracting a number proportional the gradient.

Our new guess for b is

$$b - \eta (-2N \sum_{i=1}^n (y_i - (mx_i + b)))$$

Our new guess for m is

$$m - \eta (-2N \sum_{i=1}^n x_i (y_i - (mx_i + b)))$$

For now, η is just a constant. We'll explain it in the next section. Once we get new guesses for m and b , we recompute the gradient and continue the process.

Multiple choice

Questions

Which of the following statements is FALSE?

Answer Choices

Gradient descent is an iterative process.

Gradient descent can be preferable to Ordinary Least Squares regression when dealing with a very large number of variables.

In the context of linear regression, Ordinary Least Squares and gradient descent are two different techniques that minimize two different loss functions.

Ordinary Least Squares regression uses a matrix formula to find parameters that minimize loss.

Learning Rate and Convergence

How big should your steps be when doing gradient descent? Imagine you're trying to get to the bottom of a valley, one step at a time. If you step one inch at a time, it could take a very long time to get to the bottom. You might want to take bigger steps. On the other extreme, imagine you could cover a whole mile in a single step. You'd cover a lot of ground, but you might step over the bottom of the valley and end up on a mountain!

The size of the steps that you take during gradient descent depend on the gradient (remember that we take big steps when the gradient is steep, and small steps when the gradient is small). In order to further tune the size of the steps, machine learning algorithms multiply the gradient by a factor called the *learning rate*. If this factor is big, gradient descent will take bigger steps and hopefully reach the bottom of the valley faster. In other words, it "learns" the regression parameters faster. But if the learning rate is too big, gradient descent can overshoot the bottom of the valley and fail to converge.

How do you know when to stop doing gradient descent? Imagine trying to find the bottom of a valley if you were blindfolded. How would you know when you reached the lowest point?

If you're walking downhill (or doing gradient descent on a loss function), sooner or later you'll reach a point where everything flattens out and moving against the gradient will only reduce your elevation by a negligible amount. When that happens, we say that gradient descent *converges*. You might have noticed this when you were adjusting m and b on the interactive graph: when m and b are both near their sweet spot, small adjustments to m and b didn't affect the loss much.

To summarize what we've learned so far, here are the steps of gradient descent. We'll denote the learning rate by η .

Set initial guesses for m and b

Replace m with $m + \eta \cdot (-\text{gradient})$ and replace b with $b + \eta \cdot (-\text{gradient})$

Repeat step 2 until convergence

If the algorithm fails to converge because the loss increases after some steps, the learning rate is probably too large. If the algorithm runs for a long time without converging, then the learning rate is probably too small.

Multiple choice

Questions

Gradient descent is being used to do linear regression. After each iteration of gradient descent, the loss gets larger. Which of the following should you try?

Answer Choices

Make the learning rate larger.

Keep running the gradient descent algorithm until it converges.

Make the learning rate smaller.

Implementation in sci-kit learn

Version 1.0.3 of the scikit-learn library has two different linear regression models: one that uses OLS and another that uses a variation of gradient descent.

The `LinearRegression` model uses OLS. For most applications this is a good approach. Even if a data set has hundreds of predictor variables or thousands of observations, your computer will have no problem computing the parameters using OLS. One advantage of OLS is that it is guaranteed to find the exact optimal parameters for linear regression. Another advantage is that you don't have to worry about what the learning rate is or whether the gradient descent algorithm will converge.

Here's some code that uses `LinearRegression`.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression

# Import the data set
X, y = load_diabetes(return_X_y=True)

# Create the OLS linear regression model
ols = LinearRegression()

# Fit the model to the data
ols.fit(X, y)

# Print the coefficients of the model
print(ols.coef_)

# Print R^2
print(ols.score(X, y))
```

Output:

```
[ -10.01219782 -239.81908937  519.83978679  324.39042769 -792.18416163
  476.74583782  101.04457032  177.06417623  751.27932109   67.62538639]
0.5177494254132934
```

Scikit-learn's `SGDRegressor` model uses a variant of gradient descent called *stochastic gradient descent* (or *SGD* for short). SGD is very similar to gradient descent, but instead of using the actual gradient it uses an approximation of the gradient that is more efficient to compute. This model is also sophisticated enough to adjust the learning rate as the SGD algorithm iterates, so in many cases you won't have to worry about setting the learning rate.

`SGDRegressor` also uses a technique called *regularization* that encourages the model to find smaller parameters. Regularization is beyond the scope of this article, but it's important to note that the use of regularization can sometimes result in finding different coefficients than OLS would have.

If your data set is simply too large for your computer to handle OLS, you can use `SGDRegressor`. It will not find the exact optimal parameters, but it will get close enough for all practical purposes and it will do so without using too much computing power. Here's an example.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import SGDRegressor

# Import the data set
X, y = load_diabetes(return_X_y=True)

# Create the SGD linear regression model
# max_iter is the maximum number of iterations of SGD to try before halting
sgd = SGDRegressor(max_iter = 10000)

# Fit the model to the data
sgd.fit(X, y)

# Print the coefficients of the model
print(sgd.coef_)

# Print R^2
print(sgd.score(X, y))
```

Output:

```
[ 12.19842555 -177.93853188  463.50601685  290.64175509  -33.34621692
 -94.62205923 -202.87056914  129.75873577  386.77536299  123.17079841]
0.5078357600233131
```

Gradient Descent in Other Machine Learning Algorithms

Gradient descent can be used for much more than just linear regression. In fact, it can be used to train any machine learning algorithm as long as the ML algorithm has a loss function that is a differentiable function of the ML algorithm's parameters. In more intuitive terms, gradient descent can be used whenever the loss function looks like smooth terrain with hills and valleys (even if those hills and valleys live in a space with more than 3 dimensions).

Gradient descent (or variations of it) can be used to find parameters in logistic regression models, support vector machines, neural networks, and other ML models. Gradient descent's flexibility makes it an essential part of machine learning.

Logistic Regression

Review

Congratulations! You just learned how a logistic regression model works and how to fit one to a dataset. Here are some of the things you learned:

- Logistic regression is used to perform [binary](#) classification.

- Logistic regression is an extension of linear regression where we use a logit link function to fit a sigmoid curve to the data, rather than a line.

- We can use the coefficients from a logistic regression model to estimate the log odds that a datapoint belongs to the positive class. We can then transform the log odds into a probability.

- The coefficients of a logistic regression model can be used to estimate relative feature importance.

A classification threshold is used to determine the probabilistic cutoff for where a data sample is classified as belonging to a positive or negative class. The default cutoff in sklearn is `0.5`.

We can evaluate a logistic regression model using a confusion matrix or summary [Statistics](#) such as accuracy, precision, recall, and F1 score.

Instructions

Find another dataset for binary classification from [Kaggle](#) or take a look at [sklearn's breast cancer dataset](#). Use `sklearn` to build your own Logistic Regression model on the data and make some predictions. Which features are most important in the model you build?

K-Nearest Neighbors

Review

Congratulations! You just implemented your very own classifier from scratch and used Python's `sklearn` library. In this lesson, you learned some techniques very specific to the K-Nearest Neighbor [Algorithm](#), but some general [machine learning](#) techniques as well. Some of the major takeaways from this lesson include:

- Data with n features can be conceptualized as points lying in n -dimensional space.

- Data points can be compared by using the distance formula. Data points that are similar will have a smaller distance between them.

- A point with an unknown class can be classified by finding the k nearest neighbors

- To verify the effectiveness of a classifier, data with known classes can be split into a training set and a validation set. Validation error can then be calculated.

- Classifiers have parameters that can be tuned to increase their effectiveness. In the case of K-Nearest Neighbors, k can be changed.

- A classifier can be trained improperly and suffer from overfitting or underfitting. In the case of K-Nearest Neighbors, a low k often leads to overfitting and a large k often leads to underfitting.

- Python's `sklearn` library can be used for many classification and machine learning algorithms.

To the right is an interactive visualization of K-Nearest Neighbors. If you move your mouse over the canvas, the location of your mouse will be classified as either green or blue. The nearest neighbors to your mouse are highlighted in yellow. Use the slider to change k to see how the boundaries of the classification change.

K-Nearest Neighbor Regressor

Review

Great work! Here are some of the major takeaways from this lesson:

- The K-Nearest Neighbor [algorithm](#) can be used for regression. Rather than returning a classification, it returns a number.

- By using a weighted average, data points that are extremely similar to the input point will have more of a say in the final result.

- scikit-learn has an implementation of a K-Nearest Neighbor regressor named `KNeighborsRegressor`.

In the browser, you'll find an example of a K-Nearest Neighbor regressor in action. Instead of the training data coming from IMDb ratings, you can create the training data yourself! Rate the movies that you have seen. Once you've rated more than k movies, a K-Nearest Neighbor regressor will train on those ratings. It will then make predictions for every movie that you haven't seen.

As you add more and more ratings, the predictor should become more accurate. After all, the regressor needs information from the user in order to make personalized recommendations. As a result, the system is somewhat useless to brand new users — it takes some time for the system to "warm up" and get enough data about a user. This conundrum is an example of the *cold start problem*.

Decision Trees for Classification and Regression

Learn about decision trees, how they work and how they can be used for classification and regression tasks.

Introduction

Decision trees are a common type of machine learning model used for binary classification tasks. The natural structure of a binary tree lends itself well to predicting a “yes” or “no” target. It is traversed sequentially here by evaluating the truth of each logical statement until the final prediction outcome is reached. Some examples of classification tasks that can use decision trees are: predicting whether a student will pass or fail an exam, whether an email is spam or not, if transaction is fraudulent or legitimate, etc

Decision trees can also be used for regression tasks! Predicting the grade of a student on an exam, the number of spam emails per day, the amount of fraudulent transactions on a platform, etc. are all possible using decision trees. The algorithm works pretty much the same way, with modifications only to the splitting criteria and how the final output it computed. In this article, we will explore both a binary classification and regression model using decision trees with the [Indian Graduate Admissions dataset](#).

Dataset

The data contains features commonly used in determining admission to masters’ degree programs, such as GRE, GPA, and letters of recommendation. The complete list of features is summarized below:

- GRE Scores (out of 340)
- TOEFL Scores (out of 120)
- University Rating (out of 5)
- Statement of Purpose and Letter of Recommendation Strength (out of 5)
- Undergraduate GPA (out of 10)
- Research Experience (either 0 or 1)
- Chance of Admit (ranging from 0 to 1)

We’re going to begin by loading the dataset as a `pandas DataFrame`. Feel free to open up a `jupyter` notebook on the side to implement the code in the article!

```
import pandas as pd
df = pd.read_csv("Admission_Predict.csv")
df.columns = df.columns.str.strip().str.replace(' ','_').str.lower()
```

Decision Trees for Classification: A Recap

As a first step, we will create a binary class (1=admission likely , 0=admission unlikely) from the `chance of admit` – greater than 80% we will consider as likely. The remaining data columns will be used as predictors.

```
X = df.loc[:, 'gre_score': 'research']
y = df['chance_of_admit'] >= .8
```

Fitting and Predicting

We will use `scikit-learn`’s tree module to create, train, predict, and visualize a decision tree classifier. The syntax is the same as other models in `scikit-learn`, once an instance of the model class is instantiated with `dt = DecisionTreeClassifier()`, `.fit()` can be used to fit the model on the training set. After fitting, `.predict()` (and `.predict_proba()`) and `.score()` can be called to generate predictions and score the model on the test data.

As with other `scikit-learn` models, only numeric data can be used (categorical variables and nulls must be handled prior to model fitting). In this case, our categorical features have already been transformed and no missing values are present in the data set.

```
x_train, x_test, y_train, y_test = train_test_split(X, y,
                                                    random_state=0, test_size=0.2)
dt = DecisionTreeClassifier(max_depth=2,
                             ccp_alpha=0.01, criterion='gini')
dt.fit(x_train, y_train)
```

```
y_pred = dt.predict(x_test)
print(dt.score(x_test, y_test))
print(accuracy_score(y_test, y_pred))
```

Output:

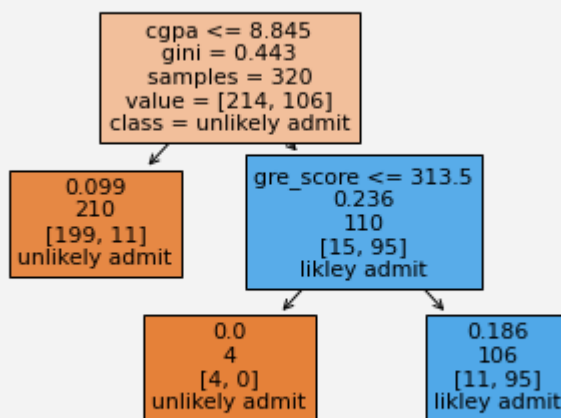
```
0.925
0.925
```

Two methods are available to visualize the tree within the tree module – the first is using `tree.plot` to graphically represent the decision tree. The second uses `export_text` to list the rules behind the splits in the decision tree. There are many other packages available for more visualization options – such as `graphviz`, but may require additional installations and will not be covered here.

```
tree.plot_tree(dt, feature_names = x_train.columns,
               max_depth=3, class_names = ['unlikely admit', 'likely
admit'],
               label='root', filled=True)
print(tree.export_text(dt, feature_names = X.columns.tolist()))
```

Output:

```
|--- cgpa <= 8.85
|   |--- class: False
|--- cgpa > 8.85
|   |--- gre_score <= 313.50
|   |   |--- class: False
|   |--- gre_score > 313.50
|   |   |--- class: True
```



Output:

Split Criteria

For a classification task, the default split criteria is Gini impurity – this gives us a measure of how “impure” the groups are. At the root node, the first split is then chosen as the one that maximizes the information gain, i.e. decreases the Gini impurity the most. Our

tree has already been built for us, but how was the split `cgpa<=8.845` determined? `cgpa` is a continuous variable, which adds an extra complication, as the split can occur for ANY value of `cgpa`.

To verify, we will use the defined functions `gini` and `info_gain`. By running `gini(y_train)`, we get the same Gini impurity value as printed in the tree at the root node, `0.443`.

```
def gini(data):
    """Calculate the Gini Impurity Score
    """
    data = pd.Series(data)
    return 1 - sum(data.value_counts(normalize=True)**2)

gi = gini(y_train)
print(f'Gini impurity at root: {round(gi,3)}')
```

Output:

```
Gini impurity at root: 0.443
```

Next, we are going to verify how the split on `cgpa` was determined, i.e. where did the `8.845` value come from:

```
def info_gain(left, right, current_impurity):
    """Information Gain associated with creating a node/split data.
    Input: left, right are data in left branch, right branch,
    respectively
    current_impurity is the data impurity before splitting into left,
    right branches
    """
    # weight for gini score of the left branch
    w = float(len(left)) / (len(left) + len(right))
    return current_impurity - w * gini(left) - (1 - w) * gini(right)
```

We will use `info_gain` over ALL values of `cgpa` to determine the information gain when split on each value. This is stored in a table and sorted, and voila, the top value for the split is `cgpa<=8.845`! This is also done for every other feature (and for those continuous ones, every value), to find the top split overall.

```
info_gain_list = []
for i in x_train.cgpa.unique():
    left = y_train[x_train.cgpa<=i]
    right = y_train[x_train.cgpa>i]
    info_gain_list.append([i, info_gain(left, right, gi)])

ig_table = pd.DataFrame(info_gain_list, columns=['split_value',
'info_gain']).sort_values('info_gain',ascending=False)
ig_table.head(10)
```

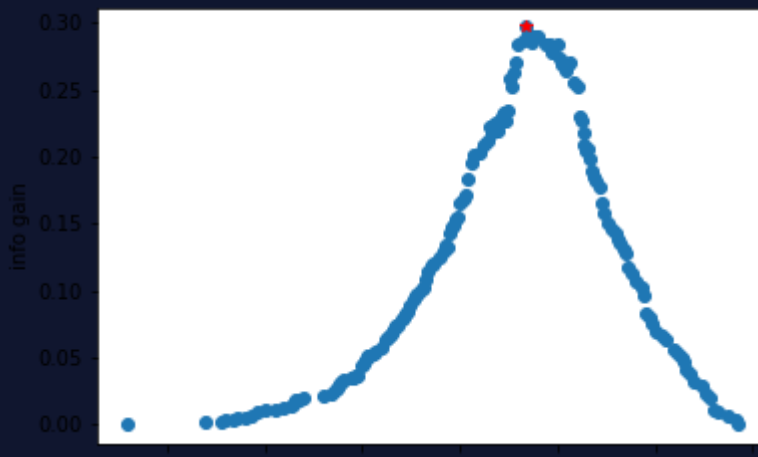
Output:

```
| |split_value |   info_gain |
|---|-----|-----|
| 10 | 8.84 | 0.296932 |
| 124 | 8.85 | 0.291464 |
| 139 | 8.88 | 0.290704 |
```

18	8.90	0.290054
98	8.83	0.287810
110	8.87	0.286050
152	8.94	0.284714
57	8.96	0.284210
96	8.80	0.283371
21	9.00	0.283364

Plot the results:

```
plt.plot(ig_table['split_value'], ig_table['info_gain'], 'o')
plt.plot(ig_table['split_value'].iloc[0],
ig_table['info_gain'].iloc[0], 'r*')
plt.xlabel('cgpa split value')
plt.ylabel('info gain')
```



Output:

After this process is repeated, and there is no further info gain by splitting, the tree is finally built. Last to evaluate, any sample traverses through tree and appropriate splits until it reaches a leaf node, and then assigned the majority class of that leaf (or weighted majority).

Regression

For the regression problem, we will use the unaltered `chance_of_admit` target, which is a floating point value between 0 and 1.

```
X = df.loc[:, 'gre_score': 'research']
y = df['chance_of_admit']
```

Fitting and Predicting

The syntax is identical as the decision tree classifier, except the target, `y`, must be real-valued and the model used must be `DecisionTreeRegressor()`. As far as the model hyperparameters go, almost all are the same, except for the split criterion. The split criterion now needs to be suitable for a regression task – the default for regression is Mean Squared Error (or MSE). Let's investigate this:

```
x_train, x_test, y_train, y_test = train_test_split(X, y,
random_state=0, test_size=0.2)
dt = DecisionTreeRegressor(max_depth=3, ccp_alpha=0.001)
```

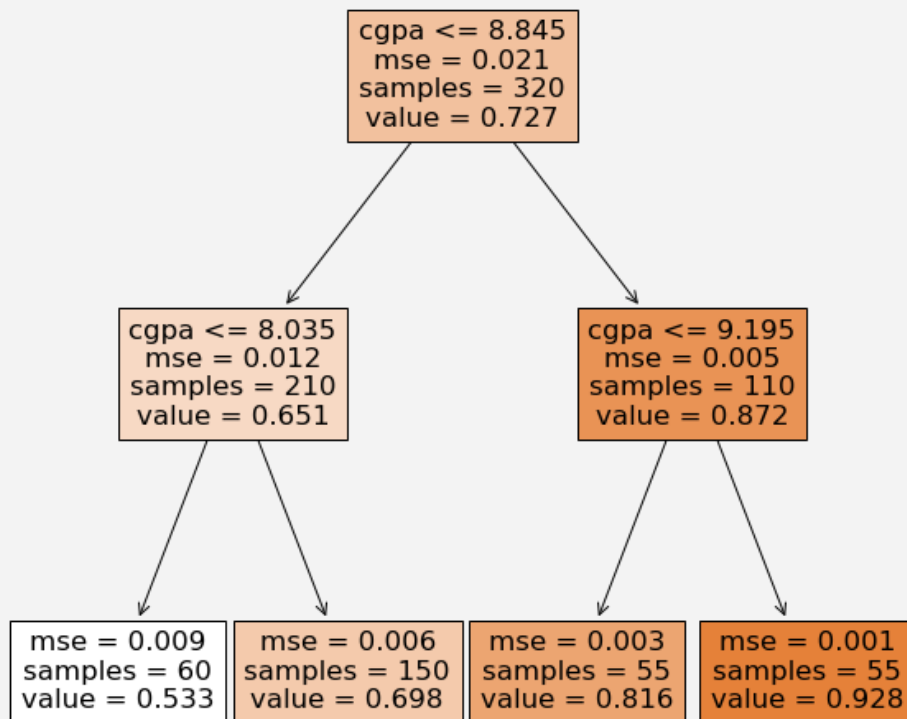


```
dt.fit(x_train, y_train)
y_pred = dt.predict(x_test)
print(dt.score(x_test, y_test))
```

Similarly, the tree can be visualized using `tree.plot_tree` – keeping in mind the splitting criteria is `mse` and the `value` is the average `chance_of_admit` of all samples in that leaf.

```
plt.figure(figsize=(10,10))
tree.plot_tree(dt, feature_names = x_train.columns,
               max_depth=2, filled=True);
```

Output:



Split Criteria

Unlike the classification problem, there are no longer classes to split the tree by. Instead, at each level, the value is the average of all samples that fit the logical criteria. In terms of evaluating the split, the default method is MSE. For example, the root node, the average target value is 0.727 (verify `y_train.mean()`). Then the MSE (mean-squared error) if we were to use 0.727 as the value for all samples, would be:

```
np.mean((y_train - y_train.mean())**2) = 0.02029
```

Now to determine the split, for each value of `cgpa`, the information gain, or decrease in MSE after the split, is calculated and then values are sorted. Like before, we can modify our functions for the regression version, and see the best split is again `cgpa<=8.84`.

The below code walks you through the details – in the regression version, instead of Gini impurity, MSE is used, and the information gain function is modified to `mse_gain`.

```
def mse(data):
    """Calculate the MSE of a data set
    """
```

```

    return np.mean((data - data.mean())**2)

def mse_gain(left, right, current_mse):
    """Information Gain (MSE) associated with creating a node/split
    data based on MSE.
    Input: left, right are data in left branch, right branch,
    respectively
    current_impurity is the data impurity before splitting into left,
    right branches
    """
    # weight for gini score of the left branch
    w = float(len(left)) / (len(left) + len(right))
    return current_mse - w * mse(left) - (1 - w) * mse(right)

m = mse(y_train)
print(f'MSE at root: {round(m,3)}')

mse_gain_list = []
for i in x_train.cgpa.unique():
    left = y_train[x_train.cgpa<=i]
    right = y_train[x_train.cgpa>i]
    mse_gain_list.append([i, mse_gain(left, right, m)])

mse_table = pd.DataFrame(mse_gain_list, columns=['split_value',
'info_gain']).sort_values('info_gain', ascending=False)
print(mse_table.head(10))

```

Output:

```

MSE at root: 0.021
   split_value  info_gain
10           8.84   0.011065
96           8.80   0.011037
98           8.83   0.011023
124          8.85   0.010985
125          8.73   0.010939
110          8.87   0.010932
139          8.88   0.010895
1           8.70   0.010894
17          8.76   0.010858
140         8.74   0.010850

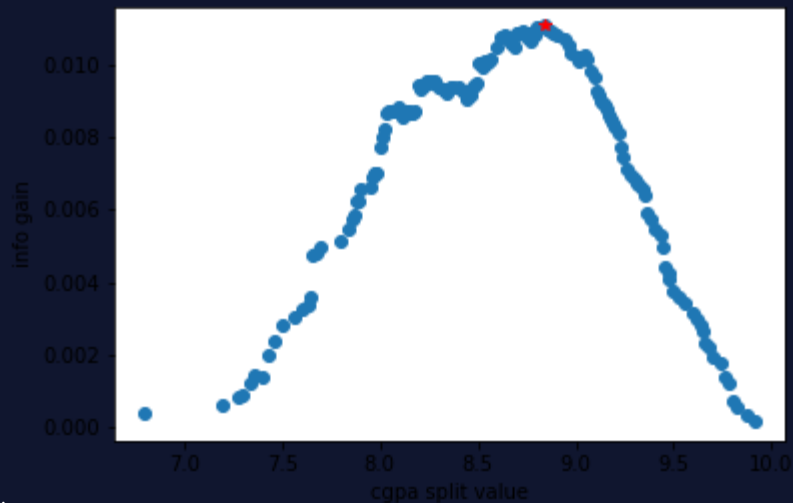
```

Plot the results:

```

plt.plot(mse_table['split_value'], mse_table['info_gain'], 'o')
plt.plot(mse_table['split_value'].iloc[0],
mse_table['info_gain'].iloc[0], 'r*')
plt.xlabel('cgpa split value')
plt.ylabel('info gain')

```



Output:

Again, the process will continue until there is no increase in information gain by splitting. Now that the tree has been built, evaluation occurs in pretty much the same way. Any sample traverses through the tree until it reaches a leaf node and is then assigned the average value of the samples in leaf. Depending on the depth of the tree, *the predicted values can be limited*. In this example, only four unique predicted values are possible, which we can verify. This is something to be aware of when using a decision tree regressor, unlike linear/logistic regression, not all output values may be possible.

```
np.unique(dt.predict(x_train))
```

Output:

```
array([0.          , 0.00588235, 0.28571429, 0.32          , 0.63636364,
        0.96428571])
```

Summary

We've seen how decision trees can be used for both classification and regression tasks.

The fundamental difference is that for classification, splits are based on Gini impurity error calculations whereas for regression, Mean Squared Error minimization is used.

Tree traversal based on information gain and evaluation works pretty much the same way for both tasks.

Decision tree regressors differ from other regressors in that all output values may not be possible and it depends on the depth of the tree.

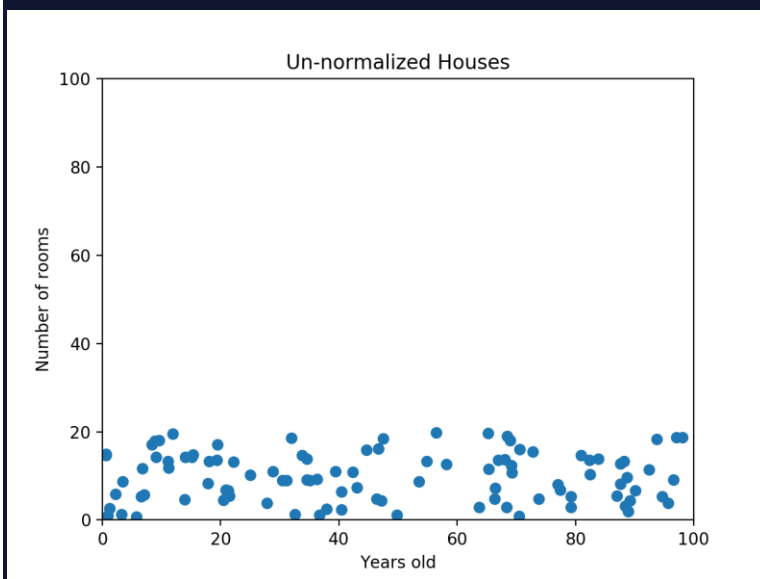
Normalization

This article describes why normalization is necessary. It also demonstrates the pros and cons of min-max normalization and z-score normalization.

Why Normalize?

Many machine learning algorithms attempt to find trends in the data by comparing features of data points. However, there is an issue when the features are on drastically different scales.

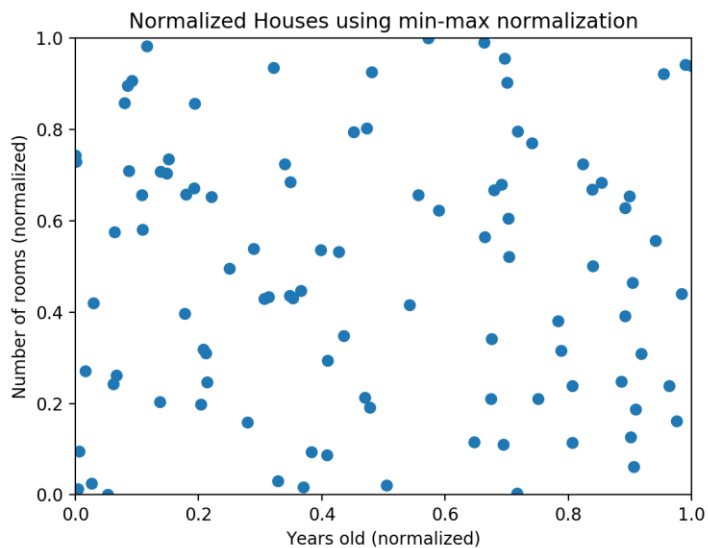
For example, consider a dataset of houses. Two potential features might be the number of rooms in the house, and the total age of the house in years. A machine learning algorithm could try to predict which house would be best for you. However, when the algorithm compares data points, the feature with the larger scale will completely dominate the other. Take a look at the image below:



When the data looks squished like that, we know we have a problem. The machine learning algorithm should realize that there is a huge difference between a house with 2 rooms and a house with 20 rooms. But right now, because two houses can be 100 years apart, the difference in the number of rooms contributes less to the overall difference.

As a more extreme example, imagine what the graph would look like if the x-axis was the cost of the house. The data would look even more squished; the difference in the number of rooms would be even less relevant because the cost of two houses could have a difference of thousands of dollars.

The goal of normalization is to make every datapoint have the same scale so each feature is equally important. The image below shows the same house data normalized using min-max normalization.



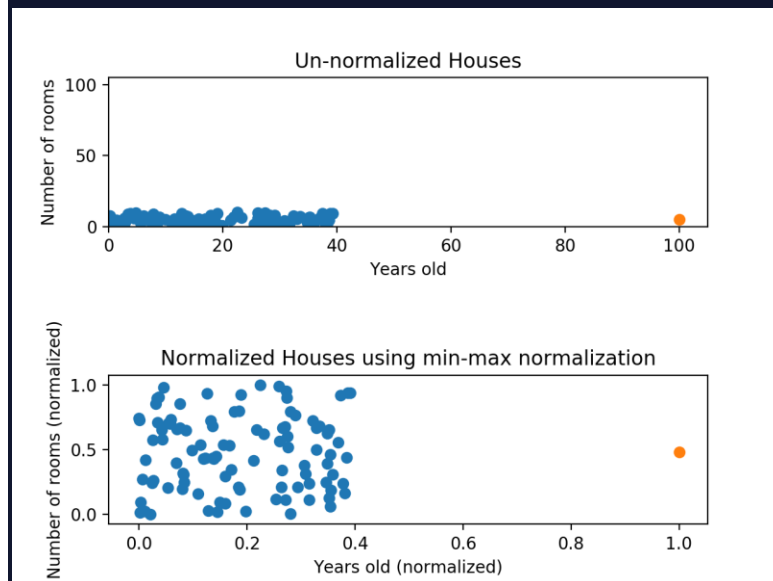
Min-Max Normalization

Min-max normalization is one of the most common ways to normalize data. For every feature, the minimum value of that feature gets transformed into a 0, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between 0 and 1.

For example, if the minimum value of a feature was 20, and the maximum value was 40, then 30 would be transformed to about 0.5 since it is halfway between 20 and 40. The formula is as follows:

$$(\text{value} - \text{min}) / (\text{max} - \text{min})$$

Min-max normalization has one fairly significant downside: it does not handle outliers very well. For example, if you have 99 values between 0 and 40, and one value is 100, then the 99 values will all be transformed to a value between 0 and 0.4. That data is just as squished as before! Take a look at the image below to see an example of this.



Normalizing fixed the squishing problem on the y-axis, but the x-axis is still problematic. Now if we were to compare these points, the y-axis would dominate; the y-axis can differ by 1, but the x-axis can only differ by 0.4.

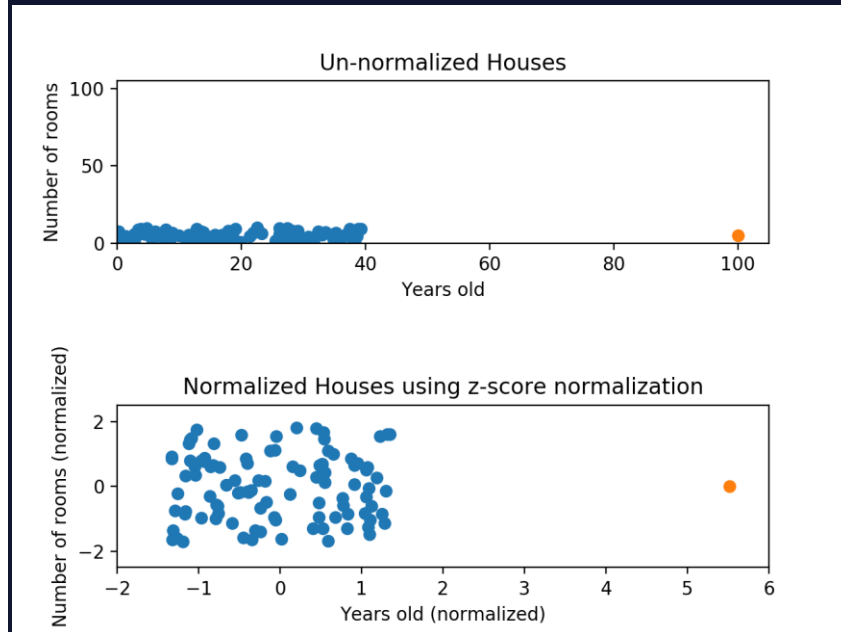
Z-Score Normalization

Z-score normalization is a strategy of normalizing data that avoids this outlier issue. The formula for Z-score normalization is below:

$$(\text{value} - \mu) / \sigma$$

Here, μ is the mean value of the feature and σ is the standard deviation of the feature. If a value is exactly equal to the mean of all the values of the feature, it will be normalized to 0. If it is below the mean, it will be a negative number, and if it is above the mean it will be a positive number. The size of those negative and positive numbers is determined by the standard deviation of the original feature. If the unnormalized data had a large standard deviation, the normalized values will be closer to 0.

Take a look at the graph below. This is the same data as before, but this time we're using z-score normalization.



While the data still *looks* squished, notice that the points are now on roughly the same scale for both features — almost all points are between -2 and 2 on both the x-axis and y-axis. The only potential downside is that the features aren't on the *exact* same scale.

With min-max normalization, we were guaranteed to reshape both of our features to be between 0 and 1. Using z-score normalization, the x-axis now has a range from about -1.5 to 1.5 while the y-axis has a range from about -2 to 2. This is certainly better than before; the x-axis, which previously had a range of 0 to 40, is no longer dominating the y-axis.

Review

Normalizing your data is an essential part of machine learning. You might have an amazing dataset with many great features, but if you forget to normalize, one of those features might completely dominate the others. It's like you're throwing away almost all of your information! Normalizing solves this problem. In this article, you learned the following techniques to normalize:

Min-max normalization: Guarantees all features will have the exact same scale but does not handle outliers well.

Z-score normalization: Handles outliers, but does not produce normalized data with the *exact* same scale.

Training, Validation and Test Datasets

This article teaches the importance of splitting a dataset into training, validation and test sets.

Supervised machine learning models have the potential to learn patterns from labeled datasets and make predictions. But how do we know if the predictions our models are making are even useful? After all, it is possible that every prediction our email spam classifier makes is actually wrong, or that every housing price prediction our regression model makes is orders of magnitude off!

Luckily, we can leverage the fact that supervised machine learning models require a dataset of pre-labeled observations to help us determine the performance of our model. To do so, we will have to split our labeled dataset into three chunks – a training, validation and test (or holdout) set.

Training-Validation-Test Split

Before we even begin to think about writing the code we will use to fit our machine learning model to the data, we must perform the split of our dataset to create our training set, validation set, and test set.

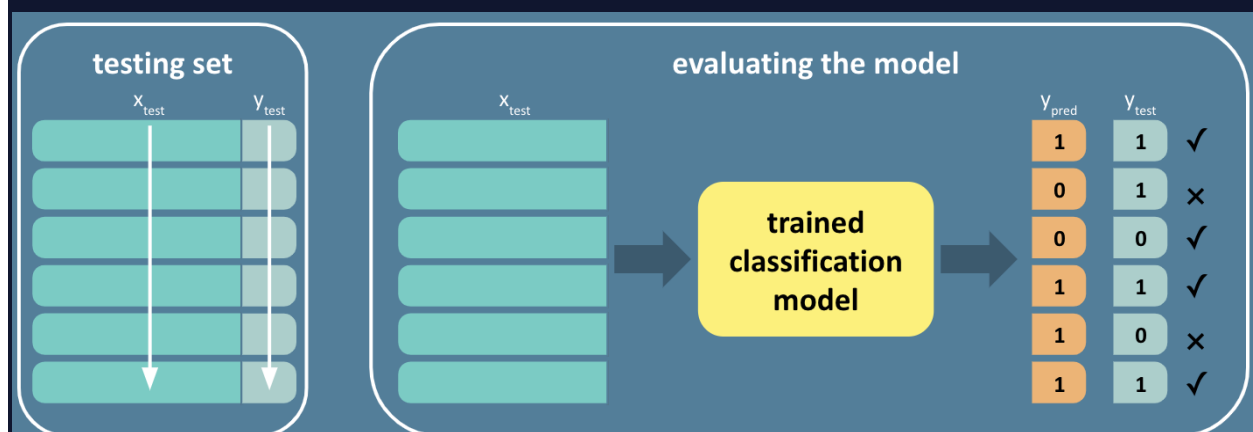
The *training set* is the data that the model will learn how to make predictions from. Learning looks different depending on which algorithm we are using. With Linear Regression, the observations in the training set are used to fit the parameters of a line of best fit. For choose different model (classification), the observations in the training set are used to fit the parameters being fit.

The *validation set* is the data that will be used during the training phase to evaluate the interim performance of the model, guide the tuning of hyper-parameters, and assist in other model improvement capacities (for example, feature selection). Some common metrics used to calculate the performance of machine learning models are [accuracy, recall, precision, and F1-Score](#). The metric we choose to use will vary depending on our particular use case.

The *test set* is the data that will determine the performance of our final model so we can estimate how our model will fare in the real world. To avoid introducing any bias to the final measurements of performance, we do not want the test set anywhere near the model training or tuning processes. That is why the test set is often referred to as the holdout set.

Evaluating the Model

During model fitting, both the features (X) and the true labels (y) of the training set (X_{train} , y_{train}) are used to learn. When evaluating the performance of the model with the validation (X_{val} , y_{val}) or test (X_{test} , y_{test}) set, we are going to temporarily pretend like we do not know the true label of every observation. If we use the observation features in our validation (X_{val}) or test (X_{test}) sets as inputs to the trained model, we can receive a prediction as output for each observation (y_{pred}). We can now compare each of the true labels (y_{val} or y_{test}) with each of the predicted labels (y_{pred}) and get a quantitative evaluation on the performance of the model.



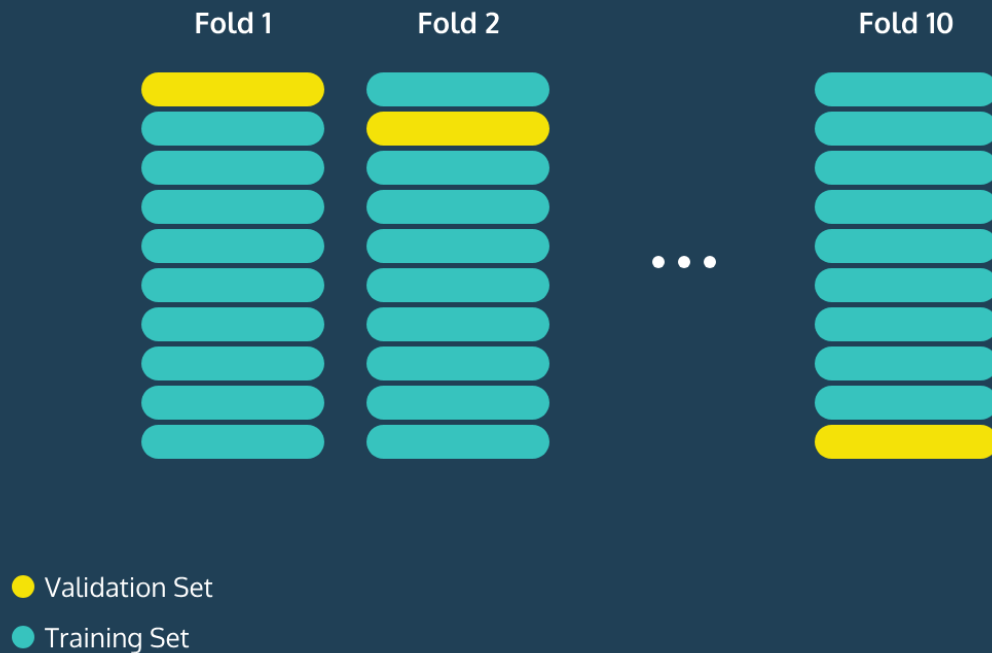
How to Split

Figuring out how much of our data should be split into training, validation, and test sets is a tricky question and there is no simple answer. If our training set is too small, then the model might not have enough data to effectively learn. On the other hand, if our validation and test sets are too small, then the performance metrics could have a large variance. In general, putting 70% of the data into the training set and 15% of the data into each of the validation and test sets is a good place to start.

N-Fold Cross-Validation

Sometimes our dataset is so small that splitting it into training, validation, and test sets that are appropriate sizes is unfeasible. A potential solution is to perform N-Fold Cross-Validation. While we still first split the dataset into a training and test set, we are going to further split the training set into N chunks. In each iteration (or fold), N-1 of the chunks are treated as the training set and 1 of the chunks is treated as the validation set over which the evaluation metrics are calculated.

10 Fold Cross Validation



This process is repeated N times cycling through each chunk acting as the validation set and the evaluation metrics from each fold are averaged. For example, in 10-fold cross-validation, we'll make the validation set the first 10% of the training set and calculate our evaluation metrics. We'll then make the validation set the second 10% of the data and calculate these statistics once again. We can do this process 10 times, and every time the validation set will be a different chunk of the data. If we then average all of the accuracies, we will have a better sense of how our model does on average.

An Introduction to Unsupervised Learning

Learn about Unsupervised Learning!

Unsupervised Learning describes a class of algorithms that find patterns from unlabelled or untagged data. In Supervised Learning, we deal with data that is labelled or tagged. For example, we predict continuous outcomes in *regression* (for instance, predicting housing prices) or categorical outcomes in *classification* (spam versus not spam, for example). However, often training data isn't labelled in this manner and this is where unsupervised learning comes in! It relies on using the underlying distributions of features within the data to figure out clusters of similarity.

Why did we build this?

Unsupervised learning methods are extremely important to the functioning of many real-world algorithms - think image recognition, ride-shares anticipating demands, snapchat filters and many more! There are three primary ways they're used:

- *Clustering*: Identifying clusters within a dataset like identifying disease outbreak clusters or in natural language processing, in creating word clouds that are semantically related, etc.
- *Dimensionality Reduction/Feature Extraction*: They can be used to condense the number of features in a dataset with a high number of features before applying a supervised learning algorithm.
- *Automated Labelling/Tagging*: Unsupervised learning algorithms are immensely useful in categorizing uncategorized data and one can then perform the familiar classification/regression tasks using supervised learning.

What will you learn?

In this module we will focus on two of the most commonly used unsupervised learning techniques - Principal Component Analysis (PCA) and K-Means Clustering. The former is most often used for dimensionality reduction and the latter is used in clustering problems primarily. After this module you will be able to:

- Perform dimensionality reduction using PCA
- Classify images using PCA
- Find clusters within data using K-Means
- Extract features using PCA and K-Means

Who are my classmates?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

K-Means Clustering Try It On Your Own

Now it is your turn!

In this review section, find another dataset from one of the following:

The [scikit-learn](#) library

[UCI Machine Learning Repo](#)

Codecademy

[GitHub](#)

Preview: Docs Github is a UI-based version control platform that uses Git.

Repo (coming soon!)

Import the `pandas` library as `pd`:

```
import pandas as pd
```

Copy to Clipboard

Load in the data with `read_csv()`:

```
digits = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/optdigits/optdigits.tra", header=None)
```

Copy to Clipboard

Note that if you download the data like this, the data is already split up into a training and a test set, indicated by the extensions **.tra** and **.tes**. You'll need to load in both files.

With the command above, you only load in the training set.

Happy Coding!

Instructions

1. Implement k-means clustering on another dataset and see what you can find. Here are some questions to get you started.
 - How does the model perform?
 - How did you choose the number of clusters?

If you think you found something interesting, let us know by posting it on Facebook, Twitter, or Instagram.

What is PCA?

Learn how, when, and why to use **Principal Component Analysis (PCA)** as part of the **Machine Learning** lifecycle.

Introduction to PCA

In the world of Machine Learning, any model that we implement will be more valuable when the features are engineered to suit the question we're trying to answer. With many datasets, we can simply include all available features, which gives us the full picture about our observations. For example, it's straightforward to see a correlation between height and weight for a patient dataset. Some datasets, however, have very large numbers of features. If our example patient dataset expanded to include 20 different features, how would we visualize and correlate this data? When it comes time to actually process the data and train the model, we often hit computational or complexity limits. How do we leverage correlations within the data to make fewer, better features without losing the information included in the dataset?

Situations like this are a great use case for implementing Principal Component Analysis. PCA is a technique where we can reduce the number of features in a dataset without losing any of the information we have. Sounds pretty great right? This article will cover various aspects of PCA, so let's dive in.

Laying the groundwork for PCA

Before we dive into the specifics of PCA, we need to understand the importance of information. In particular, we need to understand how variance plays into the level of information in a dataset. For the purposes of this article, we will be looking at a synthetic dataset about local pizza stores. Let's see what data we have:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('pizza.csv')

print(df.columns)
```

Output:

```
Index(['revenue', 'total_customers', 'amt_flour', 'amt_tomatoes',
      'amt_cheese'],
      dtype='object')
```

Each row of data pertains to an individual store, and gives information about how the store is doing overall with inventory and sales. Suppose we look at just **revenue** and **total_customers**, and we see the following information:

	revenue	total_customers
12345		500
13425		500
10872		500
9561		500

In this scenario, the value for `total_customers` has a value of 500 for every row. While every row has a value, and therefore has *data*, this column does not provide a lot of *information*, due to the lack of variance in the values. While we could include this feature in our downstream analytics, it doesn't provide any additional value, because each row would have the same data.

Now let's look at what the real data shows us for these two columns:

```
df[['revenue', 'total_customers']].head()
```

Output:

```

   revenue  total_customers
0  9931.860710  615.336682
1  12397.798907   725.440590
2  11983.079340   630.987797
3  13910.984353   746.264763
4  13083.859701   689.060436

```

As we can see, the real dataset has far more variance in these two columns. Since each feature has significant variance, these features provide valuable information about our observations, and should therefore be included in our analysis.

Variance alone is one indicator of the level of information in a dataset, but is not the only factor. To expand on the idea of variance within a dataset, we will look at the Coefficient of Variance, or CV for short. The premise here is that variance must be taken into context with the central tendencies of that dataset. For example, if a dataset has a variance of 5, that will mean very different things if the mean is 2 vs. a dataset with a mean of 100.

Now, let's actually calculate the Coefficient of Variance for each of our columns.

```

import numpy as np

#define function to calculate cv
cv = lambda x: np.std(x, ddof=1) / np.mean(x) * 100

print(df.apply(cv))

```

Output:

```

revenue          10.001034
total_customers   5.138628
amt_flour         9.128946
amt_tomatoes      9.926973
amt_cheese        6.401035

```

```
dtype: float64
```

All of the features in this dataset have enough variance where they will be useful in analysis. Since variance is an important factor to PCA, these features will ultimately be ordered by the level of information (i.e. variance) they have. For this dataset, that means, in order of importance, PCA will look at `revenue`, `amt_tomatoes`, `amt_flour`, `amt_cheese`, and then `total_customers`. While the results of PCA won't resemble our original features, they will be a mathematical representation of the information contained in the original features, which has value for analytical purposes.

Coding question

Questions

The kind of information we have can vary from dataset to dataset, and thus can the Coefficient of Variance. Use what you just learned on a new set of synthetic pizza store data, `pizza_new.csv`. Calculate the Coefficients of Variance for each feature in the dataset. Then, create a ranked order Python list for the features in the dataset in terms of information for PCA, from most important to least important.

Code

1
2

Run

Run your code to check your answer

The Math Behind PCA

At this point, we need to address how we can actually take information from multiple features and distill it down into a smaller number of features. Let's dive deeper into each of the steps that lead to PCA.

Data Matrix

First, we need to isolate a *Data Matrix*, another name for a dataset. This data matrix holds all of the features and information that we are interested in. Many datasets will have columns that hold information (i.e. features), and other columns that we want to predict (i.e. labels). Using our previous pizza dataset, we have 5 features in our data matrix.

revenue	total_customers	amt_flour	amt_tomatoes	amt_cheese
9931.860710	615.336682	37.662830	174.102712	139.402208
12397.798907	725.440590	44.424509	239.119556	168.425842
11983.079340	630.987797	40.259276	224.084121	146.612426
13910.984353	746.264763	43.633485	227.096619	170.726464
13083.859701	689.060436	48.964844	221.383478	154.786070

Covariance Matrix

From here, the next step of PCA is to calculate a *covariance matrix*. Essentially, a covariance matrix is calculating how much a feature changes with changes in every other feature, i.e., we're looking at the relative variance between any two features. Mathematically, the formula for covariance between two features X and Y is:

$$\text{Cov}(X,Y) = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})$$

We will do this equation for the relationship between each of our features, ultimately resulting in a covariance matrix that shows relationships for the entire dataset. Simplifying our example dataset, we could think about our pizza dataset having five individual features with the names a , b , c , d , and e . Our ultimate covariance matrix, thus, would end up looking like this:

[Cov_{a,a},Cov_{a,b},Cov_{a,c},Cov_{a,d},Cov_{a,e},Cov_{b,a},Cov_{b,b},Cov_{b,c},Cov_{b,d},Cov_{b,e},Cov_{c,a},Cov_{c,b},Cov_{c,c},Cov_{c,d},Cov_{c,e},Cov_{d,a},Cov_{d,b},Cov_{d,c},Cov_{d,d},Cov_{d,e},Cov_{e,a},Cov_{e,b},Cov_{e,c},Cov_{e,d},Cov_{e,e}]

Luckily, with the `pandas` package, we can calculate a covariance matrix with the `.cov()` method. For our pizza dataset, this results in the following:

```

revenue    total_customers  amt_flour  amt_tomatoes  amt_cheese
revenue    1.563517e+06    31853.053820  3713.664277  19980.869509    9152.568482
total_customers  3.185305e+04    1295.096885  105.909335  577.443015    256.641069
amt_flour    3.713664e+03    105.909335  16.894551  66.165738    29.130750
amt_tomatoes  1.998087e+04    577.443015  66.165738  500.715936    162.221734
amt_cheese   9.152568e+03    256.641069  29.130750  162.221734    105.370280

```

One important point to note is that along the primary diagonal (from top-left to bottom-right), we see the same variance values that we calculated for each individual column earlier on.

Matrix Factorization, Eigenvalues, and Eigenvectors

We now have a matrix of variance values for our features. The next step in PCA revolves around *matrix factorization*. Without going into too much detail, our goal with matrix factorization is to find a pair of smaller matrices whose product would equal our covariance matrix. Another way of thinking about it: we want to find a smaller matrix that captures the majority of our information.

An important part of this matrix factorization are *Eigenvectors*. Eigenvectors are vectors (mathematical concepts that have direction and magnitude) that do not change direction when a transformation is applied to them. In the context of data matrices, these eigenvectors give us a direction to "rotate" the dataset in n -dimensional space so we can look at the entire dataset from a simplified perspective. The *eigenvalues* are related to the relative variation described by each principal component.

For a matrix A , the eigenvectors and eigenvalues are the solutions to the following equation:

$$\det(A - \lambda I)$$

After some linear algebra, for our covariance matrix, we are looking for the solution to the following matrix, which will be our eigenvectors and eigenvalues.

```

det[1.563517e+06-λ31853.0538203713.66427719980.8695099152.5684823.185305e+041295.096885-λ
105.909335577.443015256.6410693.713664e+03105.90933516.894551-λ66.16573829.1307501.998087
e+04577.44301566.165738500.715936-λ162.2217349.152568e+03256.64106929.130750162.22173410
5.370280-λ]

```

Principal Components

All of the underlying math behind PCA results in *principal components*, but what exactly are they? Principal components are a linear combination of all the input features from the original dataset. By using the eigenvectors we calculated earlier, we can "rotate" our dataset features from an n -dimensional space into a 2-dimensional space, which is easier for us to understand and analyze.

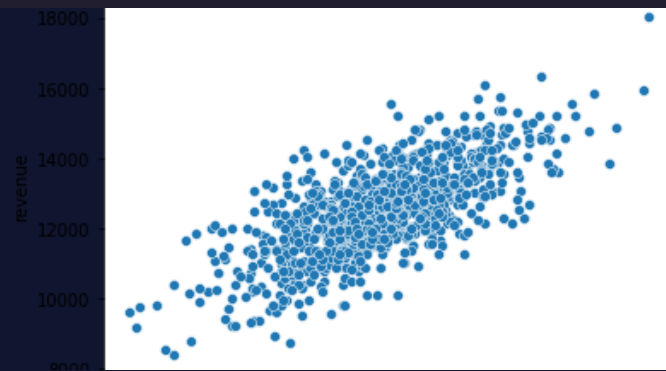
To illustrate this point, let's return to our pizza dataset. We can observe the correlation between our `revenue` and `total_customers` features.

```

sns.scatterplot(x='total_customers',
               y='revenue',

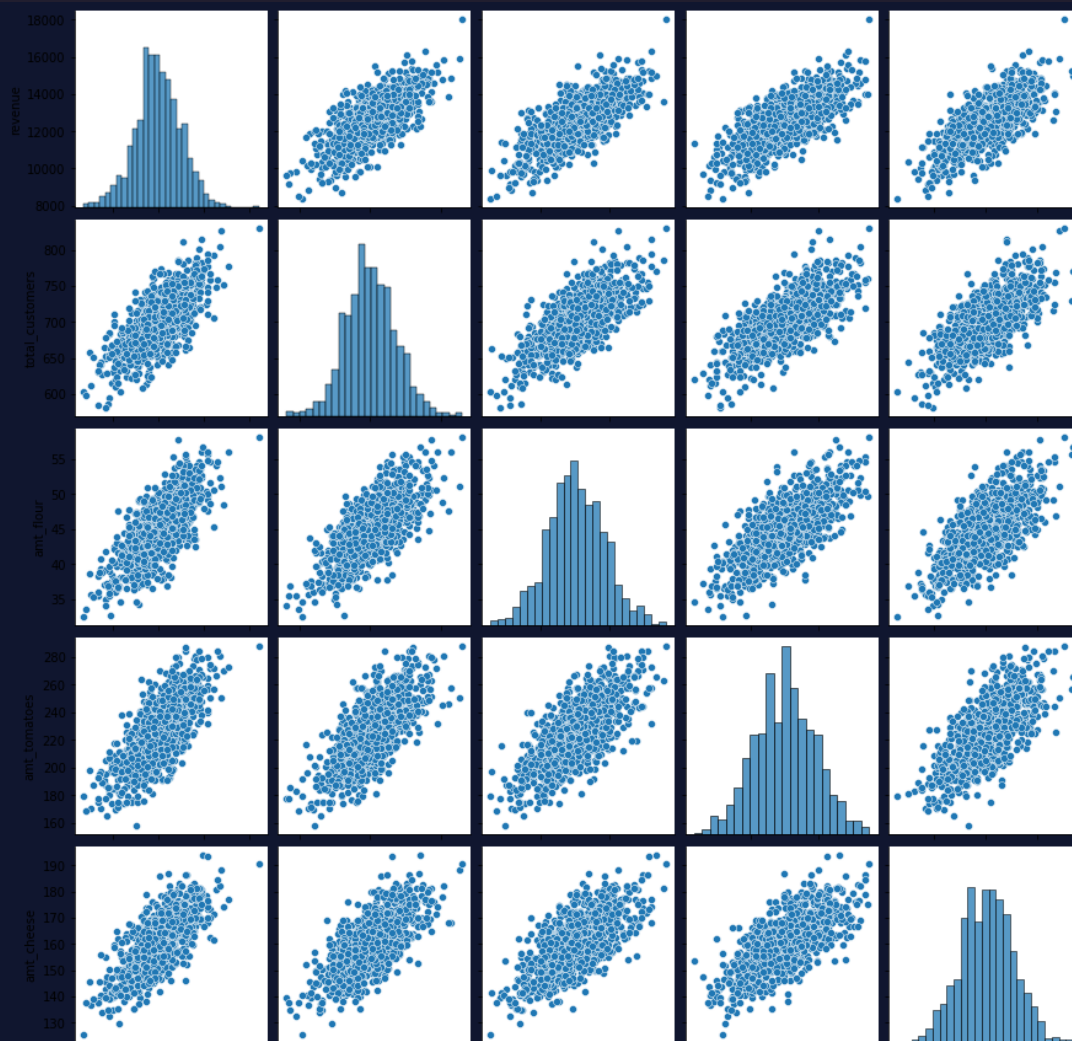
```

```
data=df)
```



We can see a positive correlation between these two features, and could use that information to guide any analysis we perform. We can also do correlation plots for every combination of features, like so:

```
sns.pairplot(df)
```



Each individual combination of features will have its own correlation and variance, both of which provide valuable information about that relationship. When comparing two features at a time, these relationships are more understandable. If we wanted to, however, look at all of the feature relationships and information at once, it would be very difficult to decipher, as we cannot visualize data in a 5-dimensional space.

By using PCA, however, we can reduce the dimensionality of our dataset into a 2-dimensional dataset, allowing for better visualization. Let's see the result.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_array = pca.fit_transform(df)
```

Output:

```
array([[ -2572.64663126,  -37.43225524],
       [  -104.21066949,   31.54952593],
       [  -521.0563251 ,  -54.19045713],
       ...,
       [  1429.58053669,  -5.98229122],
       [-3550.23561932,   23.8935932 ],
       [  -481.85213117,  -34.14891261]])
```

As we can see, by running PCA on our original dataset, we were able to take our 5 features and reduce the dimensions down to 2 principal components. With 2 dimensions, we can now plot the data on a single scatterplot:

```
sns.scatterplot(pca_array[:,0], pca_array[:,1])
```



While it can be difficult to interpret what this new data matrix is showing us, it does hold valuable information that can be used in a variety of contexts. The axes of this chart are the two most impactful principal components as part of our analysis, and were the two that we decided to keep.

Fill in the blank

Questions

The below statements outline the overall process of PCA. Fill in the blanks with the correct terms to validate your understanding.

Code

```
For a given dataset, we start by calculating a
blank 1
for all of our features.
```

```
Afterwards, we perform
blank 2
, which will separate out the dataset and give us two results:
  1)
blank 3
, also known as Principal Components, which define the direction, or
"rotation", of our new data space
  2)
blank 4
, which determine the magnitude of that new data space
```

Answer Choices

principal values
eigenvalues
correlation matrix
matrix decomposition
matrix factorization
covariance matrix
principal vectors
eigenvectors

Click or drag and drop to fill in the blank

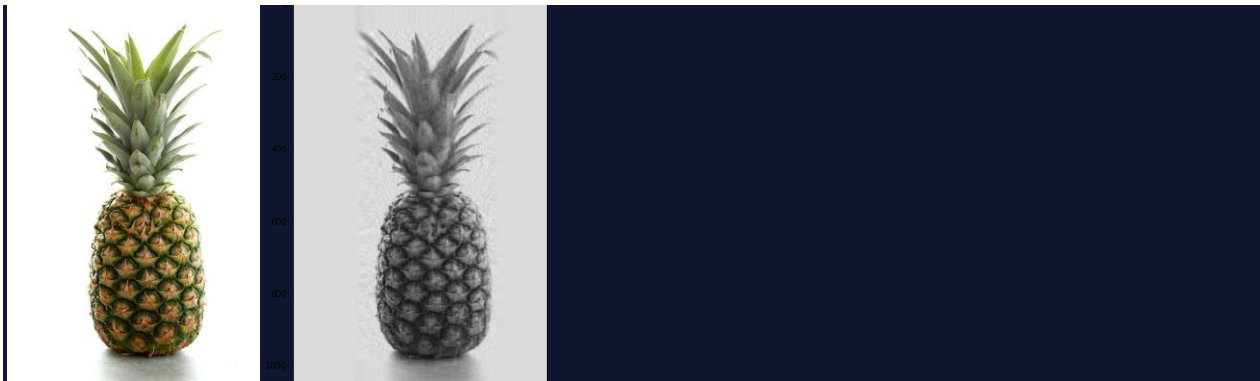
The How, Where, and Why of PCA

PCA serves an important role in many different parts of data science and analytics in general, as this process allows us to maximize the amount of information we can extract from data while reducing computational time down the line. We just saw a common use case for PCA with our pizza dataset. We took a higher dimensional dataset (5 dimensions in our case), and reduced it down to 2 dimensions. This two-dimensional dataset can now be an input to a variety of Machine Learning models. For example, we could use this new dataset as part of a forecasting model, or perform linear regression. These techniques would have been much more difficult prior to performing PCA.

PCA is also inherently an unsupervised learning algorithm and can be used to identify clusters in data on its own. Very similar to the popular **k-means** algorithms, PCA will look at overall similarities between the different features in a dataset. When we set the number of principal components to keep, we are defining the number of similar "rotations" of our dataset, which will act very much like a cluster of their own. Typically, many practitioners will implement PCA as a precursor to other clustering algorithms to augment the accuracy, but it is an interesting application to do clustering with PCA alone!

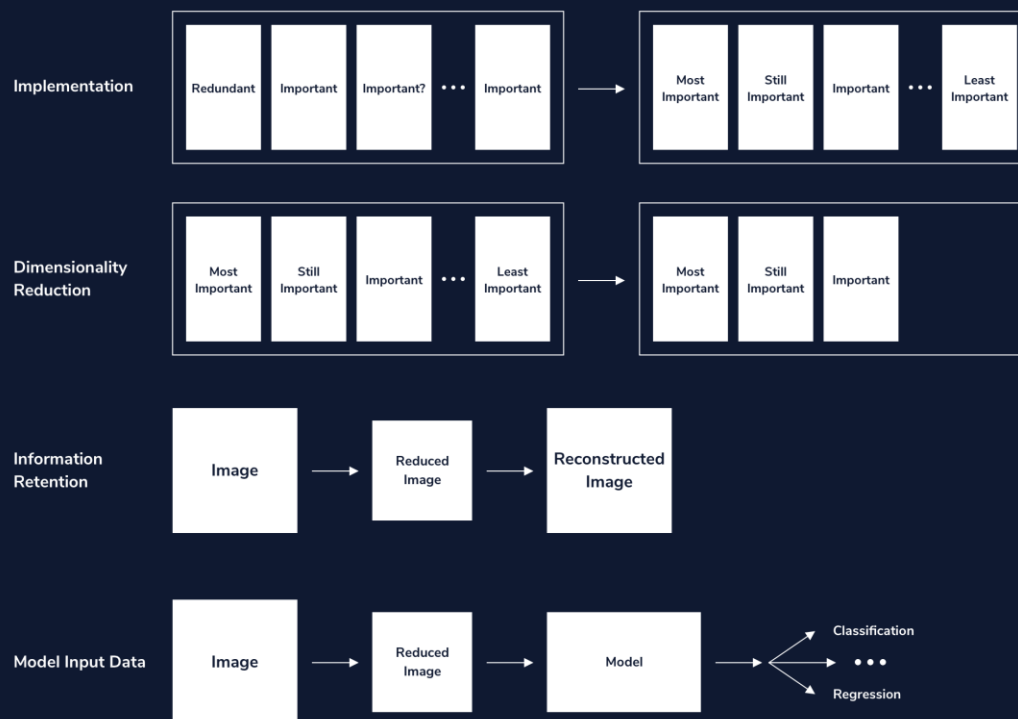
Another, very powerful, application of PCA is with image processing. Images hold a vast amount of information in each file, and analyzing this information can have very useful applications. Image classification, for example, uses algorithms to detect the subject of an image, or find a particular object within the image. Overall, it can be very costly to process image data, due to the high dimensionality it has. By applying PCA, however, practitioners can reduce the number of features for the image with minimal information loss and continue their processing.

Below are two images from a Fruit and Vegetable Image dataset. The first image is the original image, and the second is the reconstructed image using the calculated eigenvectors after performing PCA. Note that, in this very quick example, there is minimal information loss, meaning that data could be effectively used for analytics.



Summary

In this article, you learned the underlying mathematics behind Principal Component Analysis (PCA) and got a bird's eye view of how, when, and where to implement PCA. Principal Component Analysis is a powerful tool that provides a mechanism to reduce dimensionality and simplify datasets without losing the valuable information they inherently contain. The following image summarizes the different applications of PCA and we are now ready to delve into these in detail!



Implementing PCA in Python

Exercise # 9: Review

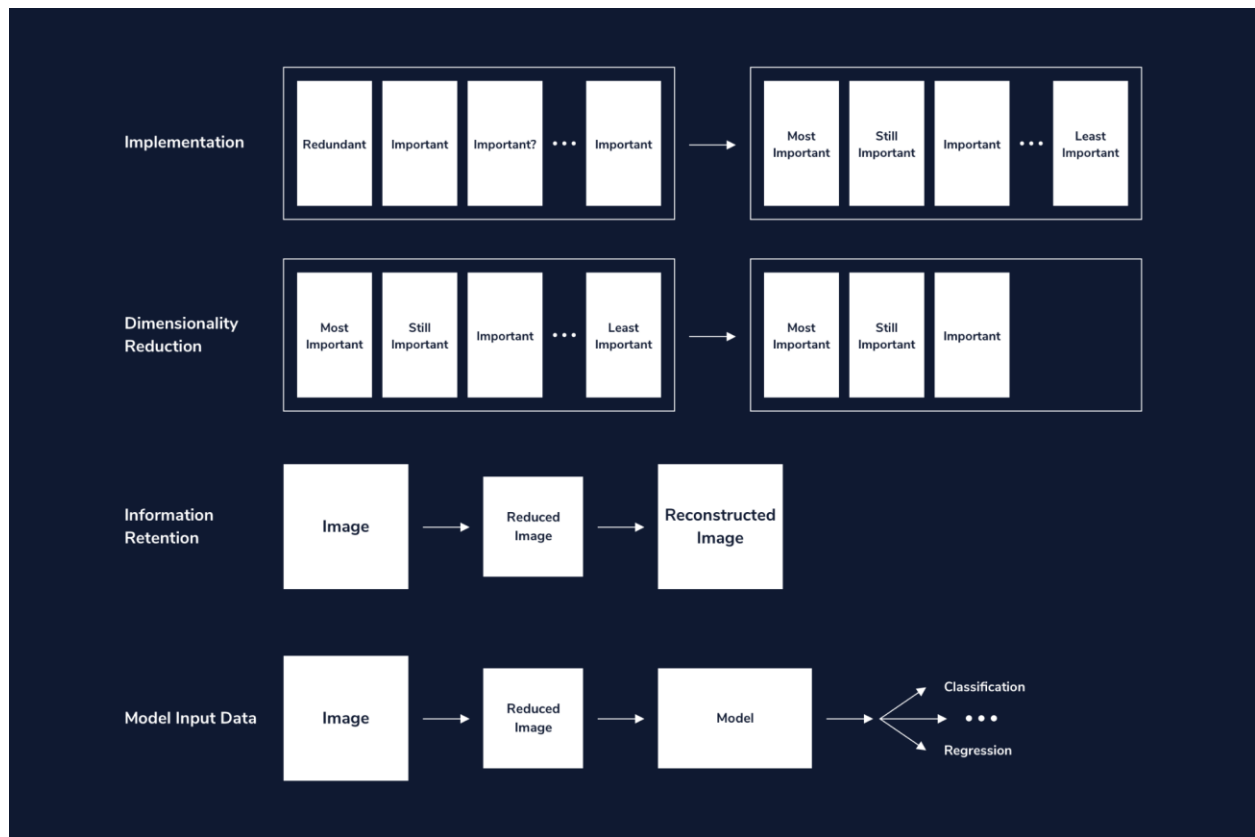
In this lesson, we have seen how PCA can be implemented using NumPy and scikit-learn. In particular, we have seen how:

Implementation: scikit-learn provides a more in-depth set of methods and attributes that extend the number of ways to perform PCA or display the percentage of variance for each principal axis.

Dimensionality reduction: We visualized the data projected onto the principal axes, known as principal components.

Image classification: We performed PCA on images of faces to visually understand how principal components still retain nearly all the information in the original dataset.

Improved algorithmic speed/accuracy: Using principal components as input to a classifier, we observed how we could achieve equal or better results with lower dimensional data. Having lower dimensionality also speeds the training.



Support Vector Machines

Review

Great work! Here are some of the major takeaways from this lesson on SVMs:

SVMs are supervised

[machine learning](#)

Preview: Docs Loading link description
models used for classification.

An SVM uses support vectors to define a decision boundary. Classifications are made by comparing unlabeled points to that decision boundary.

Support vectors are the points of each class closest to the decision boundary. The distance between the support vectors and the decision boundary is called the margin.

SVMs attempt to create the largest margin possible while staying within an acceptable amount of error.

The C

[parameter](#)

Preview: Docs Loading link description

controls how much error is allowed. A large C allows for little error and creates a hard margin. A small C allows for more error and creates a soft margin.

SVMs use kernels to classify points that aren't linearly separable.

Kernels transform points into higher dimensional space. A polynomial kernel transforms points into three dimensions while an rbf kernel transforms points into infinite dimensions.

An rbf kernel has a γ parameter. If γ is large, the training data is more relevant, and as a result overfitting can occur.

Random Forests

Review

Nice work! Here are some of the major takeaways about random forests:

A random forest is an ensemble

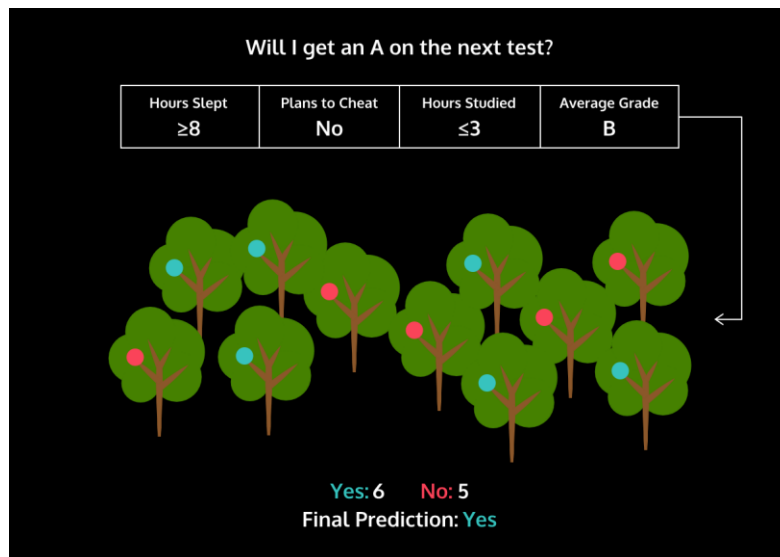
[machine learning](#)

model. It makes a classification by aggregating the classifications of many decision trees.

Random forests are used to avoid overfitting. By aggregating the classification of multiple trees, having overfitted trees in a random forest is less impactful.

Every decision tree in a random forest is created by using a different subset of data points from the training set. Those data points are chosen at random with replacement, which means a single data point can be chosen more than once. This process is known as bagging.

When creating a tree in a random forest, a randomly selected subset of features are considered as candidates for the best splitting feature. If your dataset has n features, it is common practice to randomly select the square root of n features.



Bayes' Theorem

Review

In this course, we learned several new definitions:

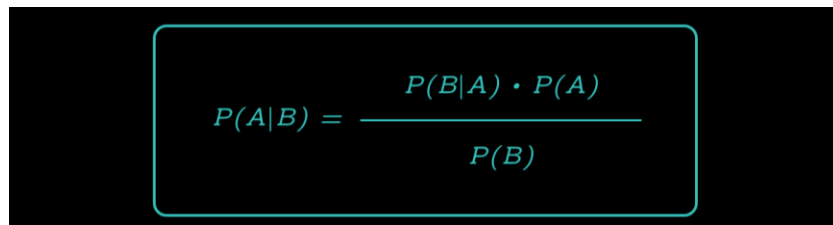
Two events are *independent* if the occurrence of one event does not affect the probability of the second event

If two events are independent then:

$$P(A \cap B) = P(A) \times P(B)$$

Bayes' Theorem is the following:

$$P(A|B) = P(B|A) \cdot P(A) / P(B)$$


$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Naive Bayes Classifier

Review

In this lesson, you've learned how to leverage Bayes' Theorem to create a supervised

[machine learning](#)

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[algorithm](#)

Preview: Docs Loading link description

. Here are some of the major takeaways from the lesson:

A tagged dataset is necessary to calculate the probabilities used in Bayes' Theorem.

In this example, the features of our dataset are the words used in a product review. In order to apply Bayes' Theorem, we assume that these features are independent.

Using Bayes' Theorem, we can find $P(\text{class}|\text{data point})$ for every possible class. In this example, there were two classes – positive and negative. The class with the highest probability will be the algorithm's prediction.

Even though our algorithm is running smoothly, there's always more that we can add to try to improve performance. The following techniques are focused on ways in which we process data before feeding it into the Naive Bayes classifier:

Remove punctuation from the training set. Right now in our dataset, there are 702 instances of "great!" and 2322 instances of "great ". We should probably combine those into 3024 instances of "great".

Lowercase every word in the training set. We do this for the same reason why we remove punctuation. We want "Great" and "great" to be the same.

Use a bigram or trigram model. Right now, the features of a review are individual words. For example, the features of the point "This crib is great" are "This", "crib", "is", and "great". If we used a bigram model, the features would be "This crib", "crib is", and "is great". Using a bigram model makes the assumption of independence more reasonable.

These three improvements would all be considered part of the field *Natural Language Processing*.

You can find the baby product review dataset, along with many others, on [Dr. Julian McAuley's website](#).

Instructions

Checkpoint 1 Passed

1.

In the code editor, we've included three Naive Bayes classifiers that have been trained on different datasets. The training sets used are the baby product reviews, reviews for Amazon Instant Videos, and reviews about video games.

Try changing `review` again and see how the different classifiers react!

Group Project: OKCupid Date-A-Scientist

You can learn a lot on your own, but you can learn even more when you collaborate with others!

Group Projects in the Data Scientist Path

As part of this Career Path, you will have the opportunity to connect with other learners to practice and apply your new skills. This is optional! Throughout the career path, we will sometimes prompt you to work with peers, but you should feel free to work with others as often as you like to gain practice!

How to Conduct a Group Project

Step One – Understand the Project

Select what you would like to work on together. We suggest [OKCupid Date-A-Scientist](#), which will help you to collaborate on creating a full machine learning project with scikit-learn and a real-world dataset. Alternatively, come up with your own project that uses scikit-learn to implement machine learning algorithms or NLP.

Assess the prompt or project requirements:

What technical skill sets do you need to complete the project? Will you need data analysis, data engineering, visualization and subject matter expertise?

How advanced a skillset in each is required and how much work is there to go around?

From here you'll know which roles your project needs and how many people are necessary. Your project may only need one other person or could have four or more collaborators, depending on the size and complexity of the project.

Step Two – Find Your Teammates

Find collaborators! [Visit the projects forum](#). You can also find collaborators in the [data-science channel in Codecademy's Discord server](#). If still looking for others to work with, check out the Codecademy [Facebook group](#), in a [Codecademy Chapter](#), or at local meetups.

When you arrive, be sure to introduce yourself with the following information:

What % of the path you have completed

What timezone or country you're in

A link to your Codecademy Profile

One fun fact about yourself

Step Three – Build Your Team

Now that you've found people to work with, the next step is to figure out how each of you would like to contribute to this project. These are some different roles that you and your teammates could take. Also, one person can have more than one role and more than one person could have the same role.

Product Manager who also codes. This person makes decisions about what features will be built and in which order. Data Science Product Managers keep the team organized and on track to be sure everything gets done. To learn a lot more about being a team leader, [please see this post](#).

Data Engineer. This person manages the data, making sure it is clean, reliable, and accessible. This role typically maintains the data standards and ensures consistency across a project.

Data Scientist. This person draws on one or more data sets to generate insights, and uses hypothesis testing to answer questions. They may create visualizations, conduct exploratory analyses, or use summary statistics to describe data. They may also use predictive statistics and machine learning to make predictions from the data.

Subject Matter Expert who also codes. This person situates datasets within their real-world context to ensure that the team is asking the right questions and interpreting the results accurately.

Think you may want to build something more complex? [Check out this article to see what a more specialized team looks like.](#)

Step Four – Get to Work

Get to it! Have a kick-off meeting, ideally over a video chat platform, so you can all meet one another and discuss the project. Remember that if and when you get stuck, you can lean on each other in order to get through problems. In the real world, data scientists troubleshoot their own work first with tried and true methods, but they often ask teammates for help too. If your teammates can't help, reach out in the forums or in [the Discord server](#).

Sometimes, life happens. Even in the "real world" workplace, people get sick or leave projects. If a learner leaves and you need a replacement, post in the Discord server and the community will try to help you out.

Step Five – Share!

If you completed the [OKCupid Date-A-Scientist](#) Project, share it with fellow learners on its [forum page](#). If you choose to do another project, post your project in [the Projects category](#) and share it with the other learners in the forums or on Discord! Getting feedback is a vital part of the development process and in growing your skills. Remember to give back to other people wanting feedback or guidance, not just because it's a good thing to do, but you'll also benefit by explaining to others what you've learned.

If you're feeling generous, don't just share your work but share your insights and guidance for the next team tackling something similar. Paying it forward and giving back is how the developer world works – [entire systems have been built this way](#).

In recent years, there has been a massive rise in the usage of dating apps to find love. Many of these apps use sophisticated data science techniques to recommend possible matches to users and to

optimize the user experience. These apps give us access to a wealth of information that we've never had before about how different people experience romance.

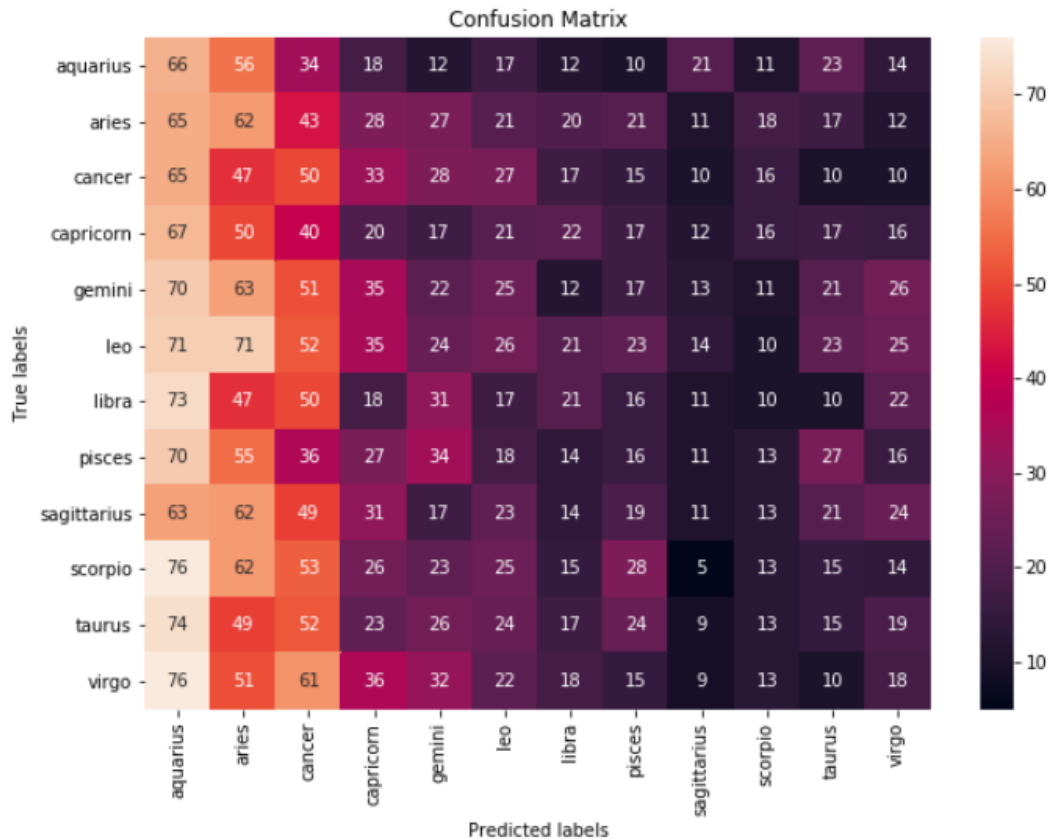
In this portfolio project, you will analyze some data from OKCupid, an app that focuses on using multiple choice and short answers to match users.

You will also create a presentation about your findings from this OKCupid dataset.

The purpose of this project is to practice formulating questions and implementing machine learning techniques to answer those questions. However, the questions you ask and how you answer them are entirely up to you.

We're excited to see the different topics you explore.

Example Project



Project Objectives:

- Complete a project to add to your portfolio
- Use Jupyter Notebook to communicate findings
- Build, train, and evaluate a machine learning model

Prerequisites:

- Natural Language Processing
- Supervised Machine Learning
- Unsupervised Machine Learning

Project Tasks

Keep track of your progress by dragging each task from "To Do" to "In Progress" to "Done" as you work on them. You can also click on a task to see more information about it.

To Do

In Progress

Done

12 / 12 done

✓ **Project Setup**
First we will have to download ou...

✓ **Setting up your Git Repository** ↗
Create a new Git repository for t...

✓ **Project Scoping**

✓ **Select ML-Solvable Problem**
First select a problem that you ca...

✓ **Load and Check Data**
If supervised learning will be used...

✓ **Explore and Explain Data**
Once you have your data, it's a go...

✓ **Preprocess Data**
Once you have a good understand...

✓ **Build Model(s)**
Once the data is preprocessed, it...

✓ **Evaluate Model(s)**
Once the model(s) are created, y...

✓ **Create a Slide Deck**
Once you've performed your anal...

✓ **Next Steps**
Lastly, being a data scientist is ab...

✓ **Conclusions**
Finally we can wrap up the projec...

What Is Deep Learning?

A quick overview of deep learning and its applications

What is Deep Learning?

Have you ever wondered what powers ChatGPT? What technology developers are using to create self-driving cars? How your phone can recognize faces? Why it seems like your photos app is better at recognizing your friends' faces than even you can? What is behind all of this? Is it magic? Well, not exactly; it is a powerful technology called *deep learning* (DL). Let's dive in and see what this is all about!



Deep Learning vs. Machine Learning

First, let's focus on *learning*. If you have come across [machine learning](#) before, you might be familiar with the concept of a *learning model*. Learning describes the process by which models analyze data and find patterns. A machine learning algorithm learns from patterns to find the best representation of this data, which it then uses to make predictions about new data that it has never seen before.

Deep learning is a subfield of machine learning, and the concept of *learning* is pretty much the same.

- We create our model carefully
- Throw relevant data at it
- Train it on this data
- Have it make predictions for data it has never seen

Deep learning models are used with many different types of data, such as text, images, audio, and more, making them applicable to many different domains.

What does “deep” mean?

That leaves the question: what is this “deep” aspect of deep learning? It separates deep learning from typical machine learning models and why it is a powerful tool that is becoming more prevalent in today's society.

The deep part of deep learning refers to the numerous “layers” that transform data. This architecture mimics the structure of the brain, where each successive layer attempts to learn progressively complex patterns from the data fed into the model. This may seem a bit abstract, so let's look at a concrete example, such as facial recognition. With facial recognition, a deep learning model takes in a photo as an input, and numerous layers perform specific steps to identify whose face is in the picture. The steps taken by each layer might be the following:

- Analyze the facial features (eyes, nose, mouth, etc.).
- Compare against faces within a repository.
- Output a prediction!

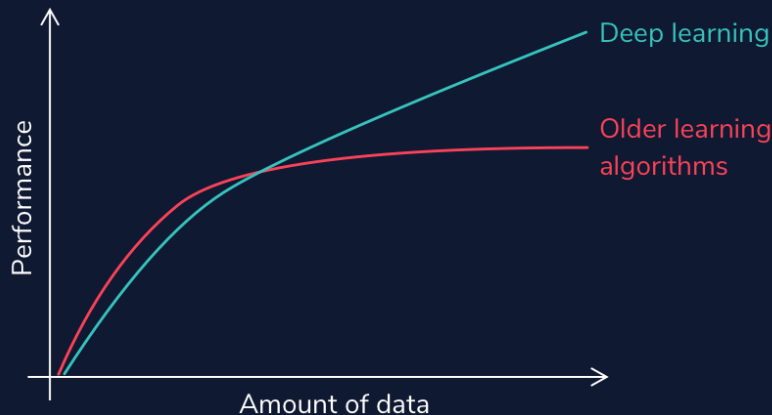


1. Find face in the image using edge detection
2. Analyze facial features
3. Compare against known faces
4. Make a prediction

This structure of many abstract layers makes deep learning incredibly powerful. Feeding high volumes of data into the model makes the connections between layers more intricate. Deep learning models tend to perform better with more massive amounts of data than other learning algorithms.

High Volume of Data

Notice that without large amounts of data, deep learning models are no more powerful (and maybe even less accurate) than less complex learning models. However, with large amounts of data, deep learning models can improve performance to the point that they outperform humans in tasks such as classifying objects or faces in images or driving. Deep learning is fundamentally a future learning system (also known as representation learning). It learns from raw data without human intervention. Hence, given massive amounts of data, a deep learning system will perform better than traditional machine learning systems that rely on feature extractions from developers.



Autonomous vehicles, such as Tesla, have to process thousands, even millions of stop signs, to understand that cars are supposed to stop when they see a red hexagon with "Stop" written on them (and this is only for U.S. stop signs!). Think about the enormous number of situations a self-driving vehicle must train on to ensure safety.

With Deep Learning Comes Deep Responsibility

Even beyond identifying objects, deep learning models can generate audio and visual content that is deceptively real. They can modify existing images, such as in [this cool applet](#) that allows you to add in trees or alter buildings in a set of photos. However, they can have a darker side. DL models can produce artificial media in which the identity of someone in an image, video, or audio is replaced with someone else. These are known as [deepfakes](#), and they can have scary implications, such as [financial fraud](#) and the distribution of fake news and hoaxes.



Graphics Processing Units

One final thing to note about deep learning is that with large amounts of data and layers of complexity, you may imagine that this takes a lot of time and processing power. That intuition would be correct. These models often require high-performance *GPUs*

(graphics processing units) to run in a reasonable amount of time. These processors have a large memory bandwidth and can process multiple computations simultaneously. CPUs can run deep learning models as well; however, they will be much slower. The development of GPUs has been critical to the success of deep learning. It is interesting to note that one of the driving factors for this development was not the need for better deep learning tools, but the [demand for better video game graphics](#). It just so happened that GPUs are perfect for processing large datasets. This makes them a perfect tool for learning models and has put deep learning specifically at the forefront of machine learning conversations. We have only just begun our dive into deep learning. Let's keep digging in and see where our journey takes us!

Deep Learning Math

Review

This overview completes the necessary mathematical intuition you need to move forward and begin coding your own learning models! To recap all the things we have learned (so many things!!!):

Scalars, vectors, matrices, and tensors

A *scalar* is a singular quantity like a number.

A *vector* is an

[array](#)

Preview: Docs Stores elements of various data types in an ordered collection. of numbers (scalar values).

A *matrix* is a grid of information with rows and columns.

A *tensor* is a multidimensional array and is a generalized version of a vector and matrix.

Matrix Algebra

In *scalar multiplication*, every entry of the matrix is multiplied by a scalar value.

In *matrix addition*, corresponding matrix entries are added together.

In *matrix multiplication*, the dot product between the corresponding rows of the first matrix and columns of the second matrix is calculated.

A *matrix transpose* turns the rows of a matrix into columns.

In *forward propagation*, data is sent through a neural network to get initial outputs and error values.

Weights are the learning parameters of a deep learning model that determine the strength of the connection between two nodes.

A *bias node* shifts the activation function either left or right to create the best fit for the given data in a deep learning model.

Activation Functions are used in each layer of a neural network and determine whether neurons should be "fired" or not based on output from a weighted sum.

Loss functions are used to calculate the error between the predicted values and actual values of our training set in a learning model.

In *backpropagation*, the gradient of the loss function is calculated with respect to the weight parameters within a neural network.

Gradient descent updates our weight parameters by iteratively minimizing our loss function to increase our model's accuracy.

Stochastic gradient descent is a variant of gradient descent, where instead of using all data points to update parameters, a random data point is selected.

Adam optimization is a variant of SGD that allows for adaptive learning rates.

Mini-batch gradient descent is a variant of GD that uses random batches of data to update parameters instead of a random datapoint.

This has been a ton of material, so do not fret if you are confused or still wrapping your head around some of the concepts. Many of this will be reviewed as you begin coding models. Get excited to see how these concepts are powered with TensorFlow!

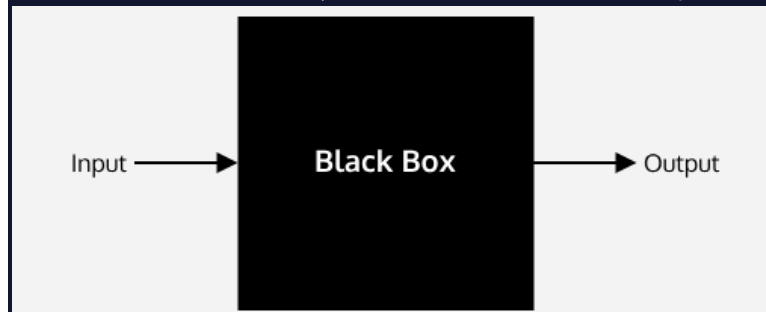
Dangers of the Black Box

Deep learning models have deep implications.

What Makes Deep Learning Models Dangerous?

When talking about machine learning, deep learning, and artificial intelligence, people tend to focus on the progress and amazing feats we could potentially achieve. While it is true that these disciplines have the potential to change the world we live in and allow us to perform otherwise impossible feats, there are often unintended consequences.

We live in an imperfect world, and the learning algorithms we design are not immune to these imperfections. Before we dive into creating our models, we must review some of the issues and implications that they can have on people's lives. Deep Learning models can be especially scary as they are some of the most powerful, and they are becoming more commonplace in society. They are also most often *black boxes* (hard for users to understand as to why and how specific results occur).



When to Use Them

Due to this nature of deep learning models, we should only ever use them if there is an apparent, significant reason for using one. If there is not a clear reason, we can use basic machine learning or statistical analysis approaches depending on what suffices.

When choosing solutions to a problem, we have to juggle many numerous factors, including

- speed
- accuracy
- training time
- interpretability
- maintenance
- enhancement
- size of the trained model

along with even more before beginning to prototype. Asking questions along these lines is vital because we design learning algorithms that can harm individuals or even entire communities in their daily lives if we do not craft them with care and awareness.

Misconceptions

There are often misconceptions about AI and deep learning models and what implications they have on our world. Science fiction has illustrated the dangers as some sort of robot apocalypse where humans are outsmarted and overpowered. This depiction does not reflect the real risks that are already present from the growing dependence on learning models in our everyday lives. Healthcare, banking, and the criminal justice system have all seen a massive rise in the reliance on learning algorithms to make decisions in recent years. While these have led to increased efficiency and developments, trusting these systems to make high-risk decisions has also led to various risks.

Key Issues

There are some key things to address about machine learning models that can lead to potentially problematic implications. Let's dive into this through the lens of healthcare.

Machine learning algorithms can only be as good as the data it is trained on. If we train the data on a model that does not match environmental data well, accuracy will not translate into the real world. A good example is this [study](#) on personalized risk assessments for acute kidney injury. While patient data evolved and disease patterns changed, the predictive model became less accurate and, therefore, much less useful. It is crucial for developers to monitor outputs periodically and account for data-shift problems over time. The work is not complete after we deploy a model; it must be managed and continuously refined to account for continuous environmental developments.

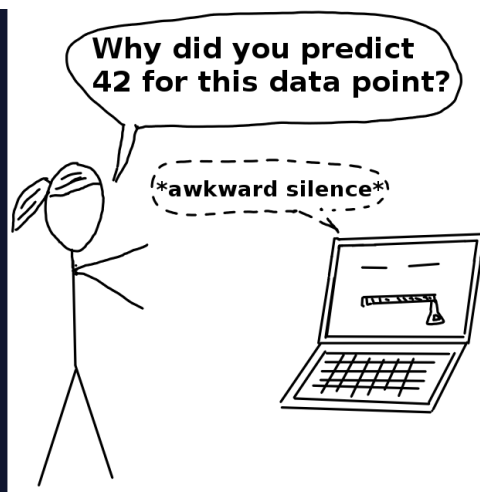
There is also a history of the health industry not including enough women and people of color in medical studies. Different

demographics can have unique manifestations and risk factors with diseases. If training data is not diverse and representative of all potential sample individuals, inaccuracies and misdiagnoses could occur. It is vital to have orthogonal data in that it is both high in volume and diversity. Without attention to this, social health disparities that are already present could become widened.

Machine learning models do not understand the impact of a false negative vs. a false positive diagnostic (at least not like humans can). When diagnosing patients, doctor's often "err on the side of caution." For example, being falsely diagnosed with a malignant tumor (false positive) is much less dangerous than being incorrectly diagnosed with a benign tumor (false negative). Further testing would occur in the former situation, whereas the latter could cause much scarier consequences with a malignant tumor being unaddressed.

Models may not have this "err on the side of caution" attitude, especially if we do not design them with this implication in mind. If we solely train it to be as accurate as possible, it is at the expense of missing malignant diagnoses. Developers can create custom loss functions that can account for the implications of false negatives vs. false positives. However, to do this, they must understand the domain well.

For many of the clinicians and the patients, the models are a black box. Stakeholders only can make decisions based on the outcome, and the predictions are not open to inspection. Therefore, it is hard for anyone to determine that there are patterns of inaccurate predictions until prolonged use has already occurred. For example, imagine an X-ray analysis model that is inaccurate under certain conditions because it was not present in the training data. The doctor would not identify this until continuously observing incorrect diagnoses because the only aspect of the model focuses on is the outcome.

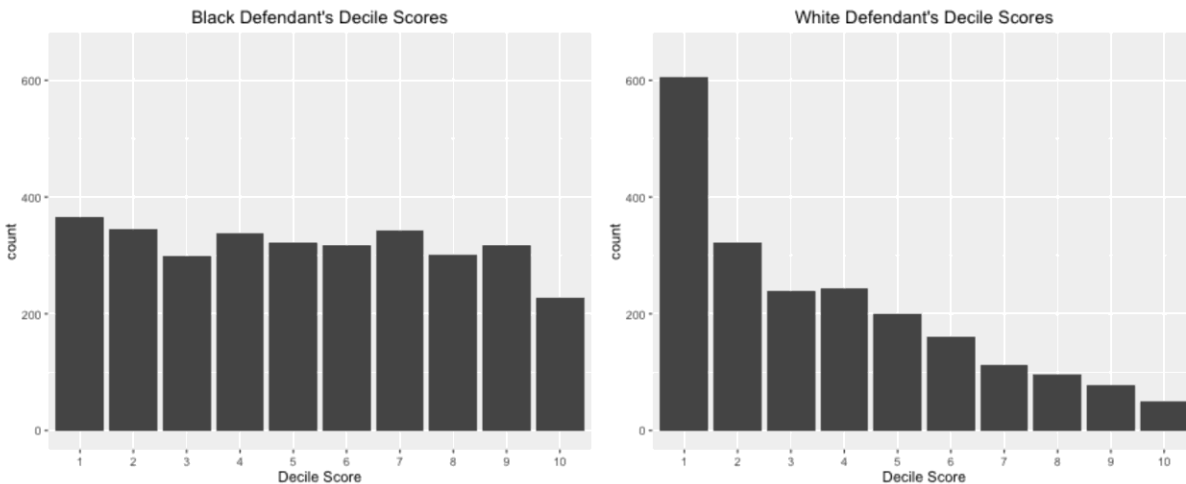


(Source: [Christoph Molnar](#))

More Examples

This is just the tip of the iceberg for concerns about the use of learning models in the healthcare industry, and we have yet to even go into implications within other sectors. Facial recognition has shown implicit bias within Google's algorithm, which [incorrectly identified a woman and her friend as gorillas](#). An error like this leads to emotional harm, and it shows that learning models are not immune to social issues and can reproduce or even exacerbate them.

Employing black box technology becomes more of an issue when used in contexts without transparency. For example, in criminal justice or banking, biased data is used to deny people of color loans at a higher rate or label them as "high risk" repeat offenders. A real-life example of this is a machine learning questionnaire algorithm known as COMPAS (Correctional Offender Management Profiling for Alternative Sanctions). It was used to determine the risk of whether an arrestee would be a repeat offender. A [study](#) showed that this algorithm was racially-biased despite never explicitly asking about race. Individuals were asked questions that modeled existing social inequalities, and minorities, particularly blacks, were more likely to be labeled "high-risk" repeat offenders. The graph below shows the distribution of risk scores for black defendants and white defendants. You can read more about the analysis of this report [here](#).



(Source: [ProPublica](#))

Models like these can even go beyond mirroring existing inequity. They can go onto perpetuate them and contribute to vicious cycles. For example, if we were to use systems like these to determine patrol routes, this could bias crime data for individuals in those communities. As we add more data to the learning model, the problem is exacerbated. What is even scarier about these models is that they are often beyond dispute or appeal. Given the result of a mathematical formula, we usually take it as fact. Someone who ends up negatively affected by this cannot argue against it or reason with a machine. They cannot explain the full reality to a computer. Instead, a machine will define its own reality to justify its results. While an outside observer can question, “why did this individual get a high-risk score despite only a minor crime?” a machine is merely operating under its historical data and findings.

Personal Responsibility

This is not to say that we should not use machine learning models in ways that impact everyday lives. It is meant to outline some of the issues that can arise when used to make high-stakes decisions and how they can cause harm. As you move forward to developing your own neural networks, consider the social implications of what you have made. As developers, we must work to ensure that our models are free from bias and will not misrepresent certain populations. The Institute for Ethical AI and Machine Learning has a [framework for developing responsible machine learning](#) that we encourage you to read up on.

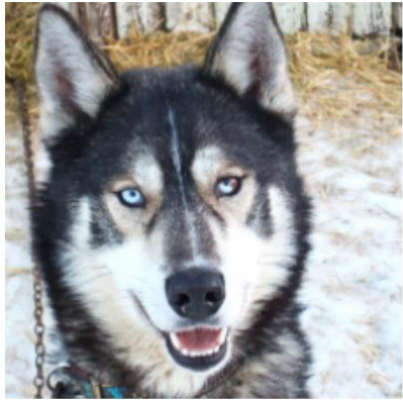
Interpretability and Transparency

It is also imperative that the models we build are used in ways that are transparent to potential stakeholders. If someone is impacted due to the output of a learning model, they should understand:

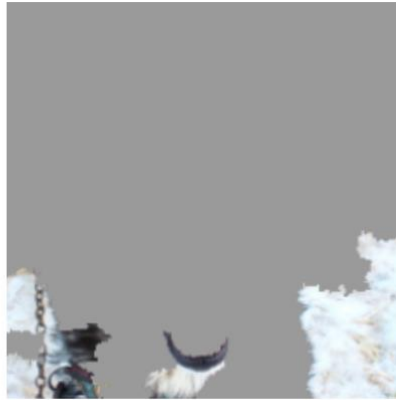
- why the model is being used
- how their personal data is being used
- what personal data is being used

The final thing to consider is making your model’s inner workings understandable for your users, also known as *interpretable machine learning*. Making the inner workings of a model understandable allows users to identify inaccuracies earlier and explain results to potential stakeholders. A great example of this is a simple classifier model that identifies huskies vs. wolves. In the case below, a husky is incorrectly classified as a wolf. With the implementation of a system called [LIME](#), predictions are explained and can be understood by users. In this case, the explanation is that if a picture contains snow, it will be classified as a wolf. This

information gives developers an indicator of why their model contains inaccuracies and clarifies how to improve it.



(a) Husky classified as wolf



(b) Explanation

(Source: [arXiv.org](https://arxiv.org))

In Conclusion

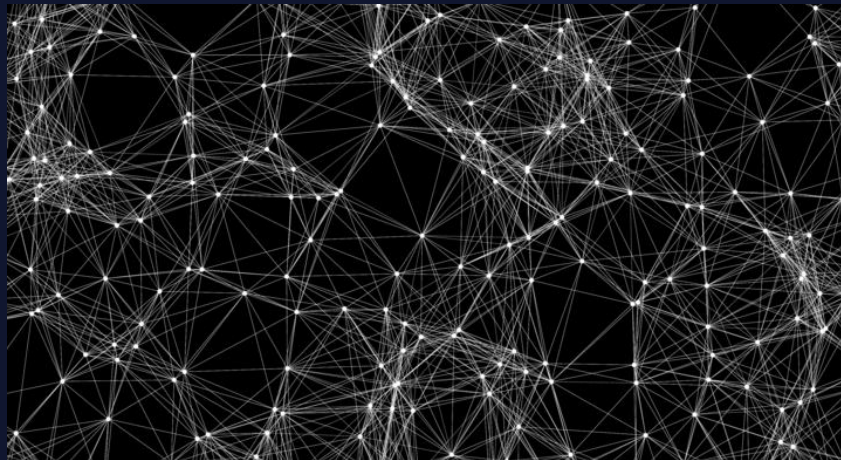
There is no doubt that machine learning can help us make waves of progress. However, in a world riddled with inequity, developers must attempt design systems so that no one is left behind. By designing learning models that account for limitations of interpretability and likelihood of bias, we can take strides to ensure that individuals and communities are not unjustly harmed.

What are Neural Networks?

An artificial neural network is an interconnected group of nodes, an attempt to mimic to the vast network of neurons in a brain.

Understanding Neural Networks

As you are reading this article, the very same brain that sometimes forgets why you walked into a room is magically translating these pixels into letters, words, and sentences – a feat that puts the world's fastest supercomputers to shame. Within the brain, thousands of neurons are firing at incredible speed and accuracy to help us recognize text, images, and the world at large.



A **neural network** is a programming model inspired by the human brain. Let's explore how it came into existence.

The Birth of an Artificial Neuron

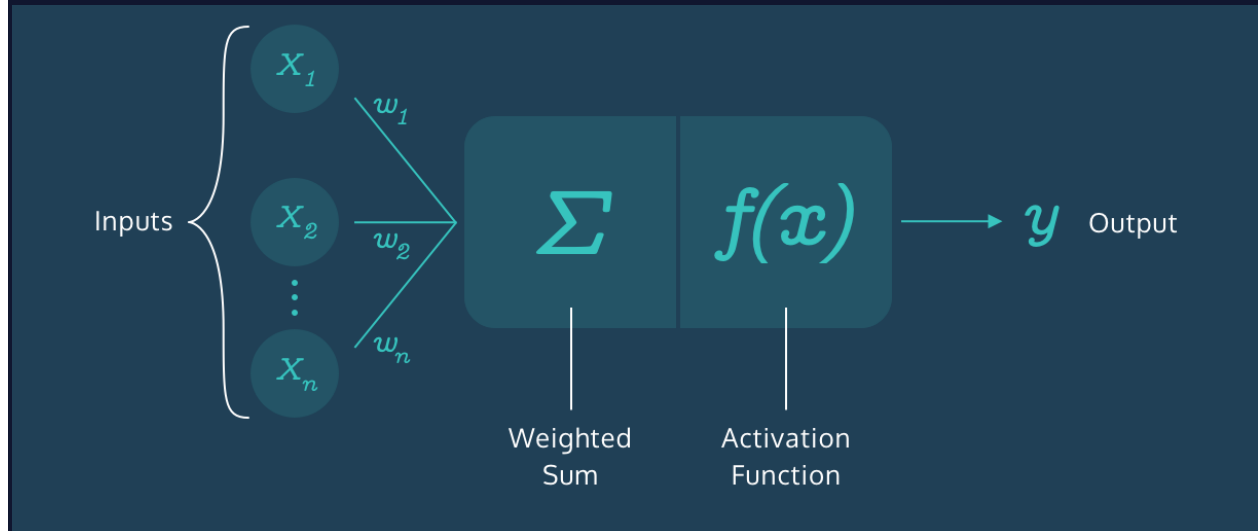
Computers have been designed to excel at number-crunching tasks, something that most humans find terrifying. On the other hand, humans are naturally wired to effortlessly recognize objects and patterns, something that computers find difficult.

This juxtaposition brought up two important questions in the 1950s:

"How can computers be better at solving problems that humans find effortless?"

"How can computers solve such problems in the way a human brain does?"

In 1957, [Frank Rosenblatt](#) explored the second question and invented the **Perceptron** algorithm that allowed an artificial neuron to simulate a biological neuron! The artificial neuron could take in an input, process it based on some rules, and fire a result. But computers had been doing this for years – what was so remarkable?

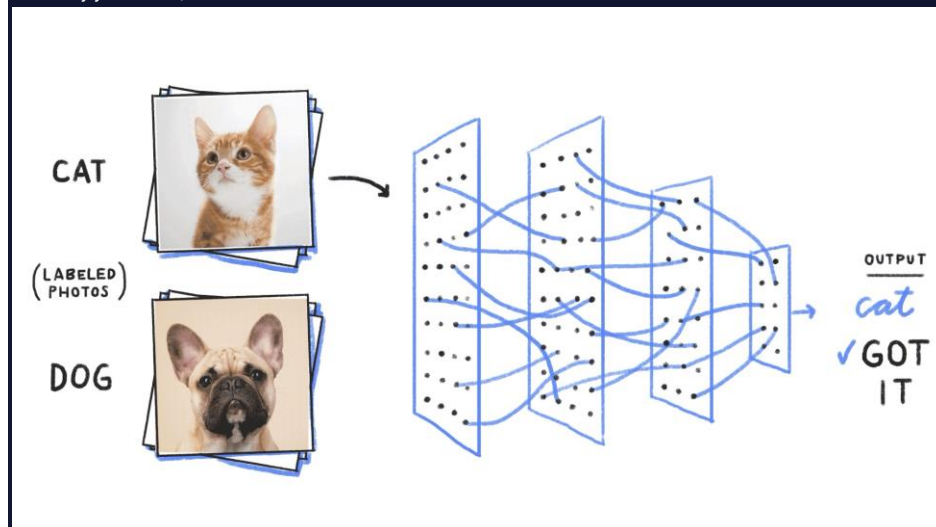


There was a final step in the Perceptron algorithm that would give rise to the incredibly mysterious world of Neural Networks – the artificial neuron could *train itself based on its own results, and fire better results in the future*. In other words, it could learn by trial and error, just like a biological neuron.

More Neurons

The Perceptron Algorithm used multiple artificial neurons, or perceptrons, for image recognition tasks and opened up a whole new way to solve computational problems. However, as it turns out, this wasn't enough to solve a wide range of problems, and interest in the Perceptron Algorithm along with Neural Networks waned for many years.

But many years later, the neurons fired back.



It was found out that creating multiple layers of neurons – with one layer feeding its output to the next layer as input – could process a wide range of inputs, make complex decisions, and still produce meaningful results. With some tweaks, the algorithm became known as the **Multilayer Perceptron**, which led to the rise of Feedforward Neural Networks.

60 Years Later...

With Feedforward Networks, the results improved. But it was only recently, with the development of high-speed processors, that neural networks finally got the necessary computing power to seamlessly integrate into daily human life.

Today, the applications of neural networks have become widespread – from simple tasks like speech recognition to more complicated tasks like self-driving vehicles.

In 2012, [Alex Krizhevsky](#) and his team at University of Toronto entered the ImageNet competition (the annual Olympics of computer vision) and trained a **deep convolutional neural network** [pdf]. No one truly understood how it made the decisions it did, but it worked better than any other traditional classifier, by a huge 10.8% margin.

Summary

The neurons have come a long way. They have braved the AI winter and remained patient amidst the lack of computing power in the 20th century. They have now taken the world by storm and deservedly so.

Neural networks are ridiculously good at generating results but also mysteriously complex; the apparent complexity of the decision-making process makes it difficult to say exactly how neural networks arrive at their superhuman level of accuracy.

Let's dive into the world of Neural Networks and relish in all its mystery!

Perceptron

What's Next? Neural Networks

Congratulations! You have now built your own perceptron from scratch.

Let's step back and think about what you just accomplished and see if there are any limits to a single perceptron.

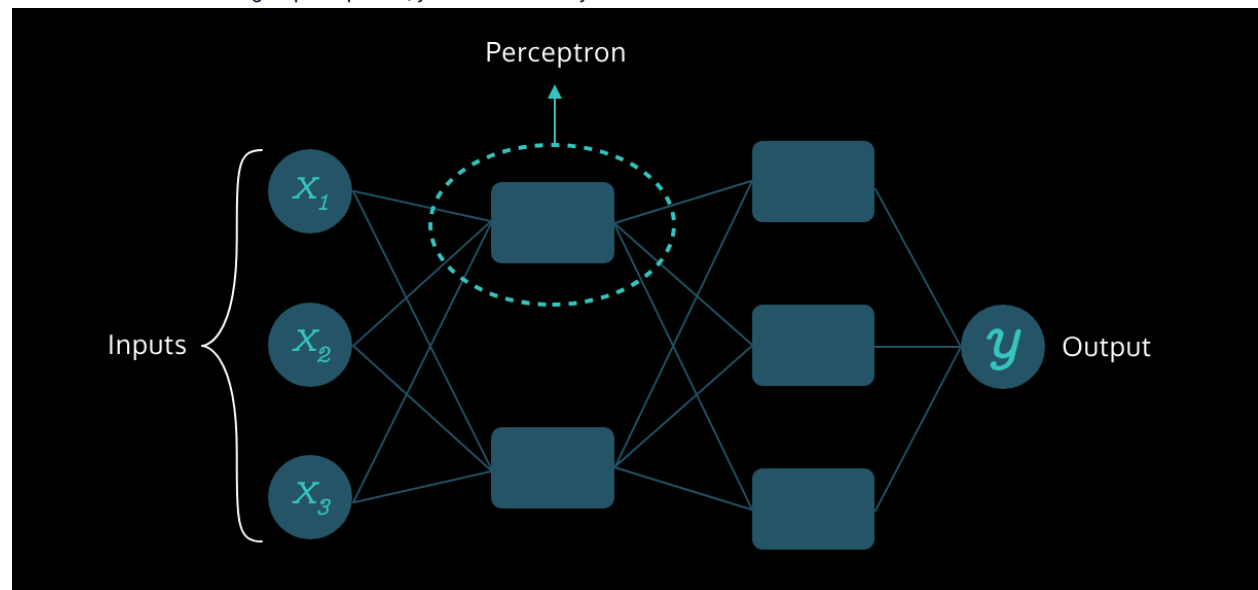
Earlier, the data points in the training set were *linearly separable* i.e. a single line could easily separate the two dissimilar sets of points.

What would happen if the data points were scattered in such a way that a line could no longer classify the points? A single perceptron with only two inputs wouldn't work for such a scenario because it cannot represent a *non-linear decision boundary*.

That's when more perceptrons and features come into play!

By increasing the number of features and perceptrons, we can give rise to the Multilayer Perceptrons, also known as Neural Networks, which can solve much more complicated problems.

With a solid understanding of perceptrons, you are now ready to dive into the incredible world of Neural Networks!



Introduction: Getting Started with Natural Language Processing

Find out what you'll learn in [Getting Started with Natural Language Processing](#).

Goals of this Unit

The goal of this unit is to introduce the field of natural language processing and provide an overview of common applications, techniques, and challenges.

After this unit, you will be able to:

- Understand what natural language processing is.

Gain an introduction to common applications and challenges within natural language processing.

Identify several natural language processing techniques and how they relate to each other.

Try out a few natural language processing techniques using Python.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Natural Language Processing

Getting Started with Natural Language Processing

Intro to NLP

Look at the technologies around us:

- Spellcheck and autocorrect
- Auto-generated video captions
- Virtual assistants like Amazon's Alexa
- Autocomplete
- Your news site's suggested articles

What do they have in common?

All of these handy technologies exist because of **natural language processing**! Also known as **NLP**, the field is at the intersection of linguistics, [artificial intelligence](#)

Preview: Docs Simulates human intelligence in computers, enabling learning, reasoning, and problem-solving to provide solutions across various tasks.

, and computer science. The goal? Enabling computers to interpret, analyze, and approximate the generation of human languages (like English or Spanish).

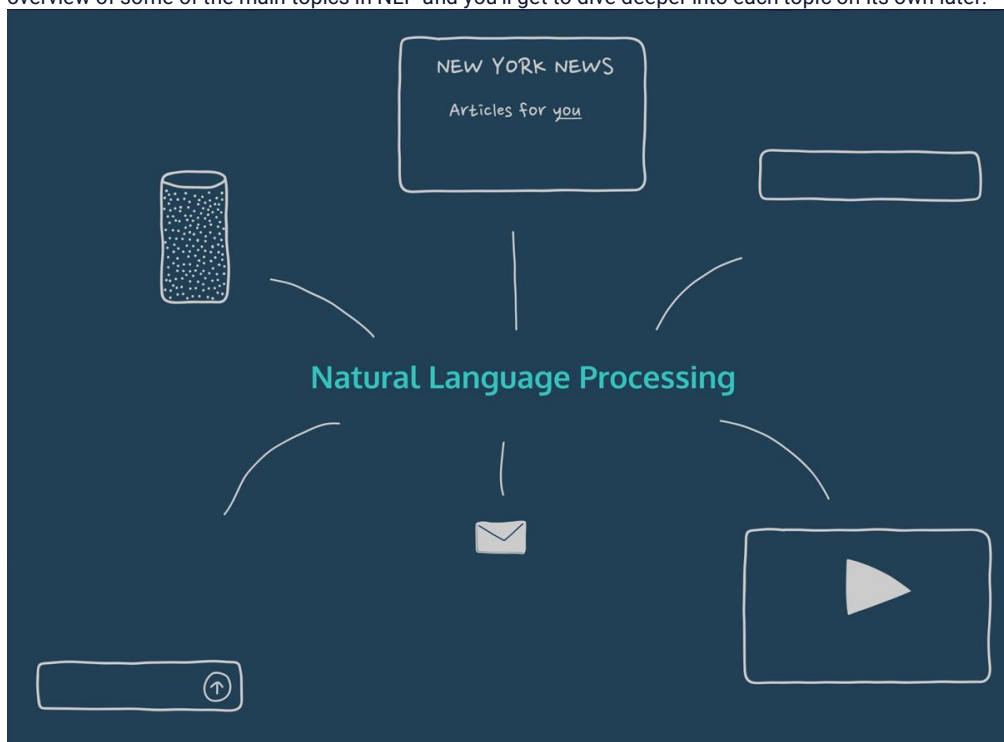
NLP got its start around 1950 with Alan Turing's test for artificial intelligence evaluating whether a computer can use language to fool humans into believing it's human.

But approximating human speech is only one of a wide range of applications for NLP! Applications from detecting spam emails or bias in tweets to improving accessibility for people with disabilities all rely heavily on natural language processing techniques.

NLP can be conducted in several programming languages. However, Python has some of the most extensive open-source NLP libraries, including the [Natural Language Toolkit](#) or **NLTK**. Because of this, you'll be using Python to get your first taste of NLP.

Instructions

Don't worry if you don't understand much of the content right now — you don't need to at this point! This lesson is an introductory overview of some of the main topics in NLP and you'll get to dive deeper into each topic on its own later.



Text Preprocessing

"You never know what you have... until you clean your data."

~ Unknown (or possibly made up)

Cleaning and preparation are crucial for many tasks, and NLP is no exception. **Text preprocessing** is usually the first step you'll take when faced with an NLP task.

Without preprocessing, your computer interprets "the", "The", and "<p>The" as entirely different words. There is a LOT you can do here, depending on the formatting you need. Lucky for you, [Regex](#) and NLTK will do most of it for you! Common tasks include:

Noise removal — stripping text of formatting (e.g., HTML tags).

Tokenization — breaking text into individual words.

Normalization — cleaning text data in any other way:

Stemming is a blunt axe to chop off word prefixes and suffixes. "booing" and "booed" become "boo", but "computer" may become "comput" and "are" would remain "are."

Lemmatization is a scalpel to bring words down to their root forms. For example, NLTK's savvy lemmatizer knows "am" and "are" are related to "be."

Other common tasks include lowercasing, [stopwords](#) removal, spelling correction, etc.

Instructions

Checkpoint 1 Passed

1.

We used NLTK's PorterStemmer to normalize the text — run the code to see how it does. (It may take a few seconds for the code to run.)

Checkpoint 2 Passed

2.

In the output terminal you'll see our program counts "go" and "went" as different words! Also, what's up with "mani" and "hardli"? A lemmatizer will fix this. Let's do it.

Where `lemmatizer` is defined, replace `None` with `WordNetLemmatizer()`.

Where we defined `lemmatized`, replace the empty list with a list comprehension that uses `lemmatizer` to `lemmatize()` each token in `tokenized`.

(Don't know Python that well? No problem. Just check the hints for help throughout the lesson.)

```
lemmatized = [lemmatizer.lemmatize(token) for token in tokenized]
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Why are the lemmatized verbs like "went" still conjugated? By default `lemmatize()` treats every word as a noun.

Give `lemmatize()` a second argument: `get_part_of_speech(token)`. This will tell our lemmatizer what part of speech the word is.

Run your code again to see the result!

```
lemmatized = [lemmatizer.lemmatize(token, get_part_of_speech(token)) for token in tokenized]
```

```
# regex for removing punctuation!
import re
# nltk preprocessing magic
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
# grabbing a part of speech function:
from nltk.part_of_speech import get_part_of_speech

text = "So many squids are jumping out of suitcases these days that you can barely go anywhere
without seeing one burst forth from a tightly packed valise. I went to the dentist the other day,
and sure enough I saw an angry one jump out of my dentist's bag within minutes of arriving. She
hardly even noticed."

cleaned = re.sub('\W+', ' ', text)
tokenized = word_tokenize(cleaned)

stemmer = PorterStemmer()
```

```

stemmed = [stemmer.stem(token) for token in tokenized]

## -- CHANGE these -- ##
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(token, get_part_of_speech(token)) for token in tokenized]

print("Stemmed text:")
print(stemmed)
print("\nLemmatized text:")
print(lemmatized)

Stemmed text:
['So', 'mani', 'squid', 'are', 'jump', 'out', 'of', 'suitcas', 'these', 'day', 'that', 'you', 'can', 'bare', 'go', 'anywh',
'without', 'see', 'one', 'burst', 'forth', 'from', 'a', 'tightli', 'pack', 'valis', 'I', 'went', 'to', 'the', 'dentist', 'the',
'other', 'day', 'and', 'sure', 'enough', 'I', 'saw', 'an', 'angri', 'one', 'jump', 'out', 'of', 'my', 'dentist', 's', 'bag',
'within', 'minut', 'of', 'arriv', 'she', 'hardli', 'even', 'notic']

Lemmatized text:
['So', 'many', 'squid', 'be', 'jump', 'out', 'of', 'suitcase', 'these', 'day', 'that', 'you', 'can', 'barely', 'go', 'anywhere',
'without', 'see', 'one', 'burst', 'forth', 'from', 'a', 'tightly', 'pack', 'valise', 'I', 'go', 'to', 'the', 'dentist', 'the',
'other', 'day', 'and', 'sure', 'enough', 'I', 'saw', 'an', 'angry', 'one', 'jump', 'out', 'of', 'my', 'dentist', 's', 'bag',
'within', 'minute', 'of', 'arrive', 'She', 'hardly', 'even', 'notice']

```

Getting Started with Natural Language Processing

Parsing Text

You now have a preprocessed, clean list of words. Now what? It may be helpful to know how the words relate to each other and the underlying syntax (grammar). **Parsing** is an NLP process concerned with segmenting text based on syntax.

You probably do not want to be doing any parsing by hand and NLTK has a few tricks up its sleeve to help you out:

Part-of-speech tagging (POS tagging) identifies parts of speech (verbs, nouns, adjectives, etc.). NLTK can do it faster (and maybe more accurately) than your grammar teacher.

Named entity recognition (NER) helps identify the proper nouns (e.g., “Natalia” or “Berlin”) in a text. This can be a clue as to the topic of the text and NLTK captures many for you.

Dependency grammar trees help you understand the relationship between the words in a sentence. It can be a tedious task for a human, so the Python library spaCy is at your service, even if it isn't always perfect.

In English we leave a lot of ambiguity, so syntax can be tough, even for a computer program. Take a look at the following sentence:

I saw a cow under a tree with binoculars

Do I have the binoculars? Does the cow have binoculars? Does the tree have binoculars?

Regex parsing, using Python's `re` library, allows for a bit more nuance. When coupled with POS tagging, you can identify specific phrase chunks. On its own, it can find you addresses, emails, and many other common patterns within large chunks of text.

Instructions

Checkpoint 1 Passed

1.

Run the code to see the silly squid sentences parsed into dependency trees visually!

Checkpoint 2 Passed

2.

Change `my_sentence` to a sentence of your choosing and run the code again to see it parsed out as a tree!

```
squids_text = "So many squids are jumping out of suitcases these days. You can barely go anywhere without seeing one. I went to the dentist the other day. Sure enough, I saw an angry one jump out of my dentist's bag. She hardly even noticed."
```

```
import spacy
from nltk import Tree
from squids import squids_text

dependency_parser = spacy.load('en')

parsed_squids = dependency_parser(squids_text)

# Assign my_sentence a new value:
my_sentence = "How does spaCy know how to parse this sentence?"
my_parsed_sentence = dependency_parser(my_sentence)

def to_nltk_tree(node):
    if node.n_lefts + node.n_rights > 0:
        parsed_child_nodes = [to_nltk_tree(child) for child in node.children]
        return Tree(node.orth_, parsed_child_nodes)
    else:
        return node.orth_

for sent in parsed_squids.sents:
    to_nltk_tree(sent.root).pretty_print()

for sent in my_parsed_sentence.sents:
    to_nltk_tree(sent.root).pretty_print()
```

jumping

out

squids of days

So are . many suitcases these

go

without

seeing

You can barely anywhere . one

went

to

dentist day

```

I . the the other
      saw
      |
      | | | | | jump
      | | | | | |
      | | | | | out
      | | | | | |
      | | | | | of
      | | | | | |
      | | | | | bag
      | | | | | |
      | | | | | dentist
      | | | | | |
      , I . Sure an angry my 's

      noticed
      |
      |
She hardly even .

      know
      |
      | | | | | parse
      | | | | | |
      | | | | | sentence
      | | | | | |
      | | | | |
How does spaCy ? how to this

```

Getting Started with Natural Language Processing

Language Models: Bag-of-Words

How can we help a machine make sense of a bunch of word tokens? We can help computers make predictions about language by training a language model on a *corpus* (a bunch of example text).

Language models are probabilistic computer models of language. We build and use these models to figure out the likelihood that a given sound, letter, word, or phrase will be used. Once a model has been trained, it can be tested out on new texts.

One of the most common language models is the unigram model, a statistical language model commonly known as **bag-of-words**. As its name suggests, bag-of-words does not have much order to its chaos! What it does have is a tally count of each instance for each word. Consider the following text example:

The squids jumped out of the suitcases.



Provided some initial preprocessing, bag-of-words would result in a mapping like:

```
{"the": 2, "squid": 1, "jump": 1, "out": 1, "of": 1, "suitcase": 1}
```

Copy to Clipboard

Now look at this sentence and mapping: "Why are your suitcases full of jumping squids?"

```
{"why": 1, "be": 1, "your": 1, "suitcase": 1, "full": 1, "of": 1, "jump": 1, "squid": 1}
```

Copy to Clipboard

You can see how even with different word order and sentence structures, "jump," "squid," and "suitcase" are shared topics between the two examples. Bag-of-words can be an excellent way of looking at language when you want to make predictions concerning topic or sentiment of a text. When grammar and word order are irrelevant, this is probably a good model to use.

Instructions

Checkpoint 1 Passed

1.

We've turned a passage from *Through the Looking Glass* by Lewis Carroll into a list of words (aside from stopwords, which we've removed) using `nltk` preprocessing. Run your code to see the full list.

Checkpoint 2 Passed

2.

Now let's turn this list into a bag-of-words using `Counter()`!

Comment out the print statement and set `bag_of_looking_glass_words` equal to a call of `Counter()` on `normalized`. Print `bag_of_looking_glass_words`. What are the most common words?

```
bag_of_looking_glass_words = Counter(normalized)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Try changing `text` to another string of your choosing and see what happens!

```
looking_glass_text = ""
    However, the egg only got larger and larger, and more and more human: when she had come within a few yards of it, she
    saw that it had eyes and a nose and mouth; and when she had come close to it, she saw clearly that it was HUMPTY DUMPTY
    himself. It cant be anybody else! she said to herself. Im as certain of it, as if his name were written all over his
    face.

    It might have been written a hundred times, easily, on that enormous face. Humpty Dumpty was sitting with his legs
    crossed, like a Turk, on the top of a high wallsuch a narrow one that Alice quite wondered how he could keep his
    balanceand, as his eyes were steadily fixed in the opposite direction, and he didnt take the least notice of her, she
    thought he must be a stuffed figure after all.

    And how exactly like an egg he is! she said aloud, standing with her hands ready to catch him, for she was every moment
    expecting him to fall.

    Its very provoking, Humpty Dumpty said after a long silence, looking away from Alice as he spoke, to be called an
    eggVery!

    I said you looked like an egg, Sir, Alice gently explained. And some eggs are very pretty, you know she added, hoping
    to turn her remark into a sort of a compliment.

    Some people, said Humpty Dumpty, looking away from her as usual, have no more sense than a baby!

    Alice didnt know what to say to this: it wasnt at all like conversation, she thought, as he never said anything to her;
    in fact, his last remark was evidently addressed to a treeso she stood and softly repeated to herself:

        Humpty Dumpty sat on a wall:
        Humpty Dumpty had a great fall.
        All the Kings horses and all the Kings men
        Couldnt put Humpty Dumpty in his place again.

    That last line is much too long for the poetry, she added, almost out loud, forgetting that Humpty Dumpty would hear
    her.

    Dont stand there chattering to yourself like that, Humpty Dumpty said, looking at her for the first time, but tell me
    your name and your business.

    My name is Alice, but

    Its a stupid enough name! Humpty Dumpty interrupted impatiently. What does it mean?

    Must a name mean something? Alice asked doubtfully.
```


Of course it must, Humpty Dumpty said with a short laugh: my name means the shape I am and a good handsome shape it is, too. With a name like yours, you might be any shape, almost.

Why do you sit out here all alone? said Alice, not wishing to begin an argument.

Why, because theres nobody with me! cried Humpty Dumpty. Did you think I didnt know the answer to that? Ask another.

Dont you think youd be safer down on the ground? Alice went on, not with any idea of making another riddle, but simply in her good-natured anxiety for the queer creature. That wall is so very narrow!

What tremendously easy riddles you ask! Humpty Dumpty growled out. Of course I dont think so! Why, if ever I did fall off which theres no chance of but if I did here he pursed his lips and looked so solemn and grand that Alice could hardly help laughing. If I did fall, he went on, The King has promised me with his very own mouth to

To send all his horses and all his men, Alice interrupted, rather unwisely.

Now I declare thats too bad! Humpty Dumpty cried, breaking into a sudden passion. Youve been listening at doors and behind trees and down chimneys or you couldnt have known it!

I havent, indeed! Alice said very gently. Its in a book.

Ah, well! They may write such things in a book, Humpty Dumpty said in a calmer tone. Thats what you call a History of England, that is. Now, take a good look at me! Im one that has spoken to a King, I am: mayhap youll never see such another: and to show you Im not proud, you may shake hands with me! And he grinned almost from ear to ear, as he leant forwards (and as nearly as possible fell off the wall in doing so) and offered Alice his hand. She watched him a little anxiously as she took it. If he smiled much more, the ends of his mouth might meet behind, she thought: and then I dont know what would happen to his head! Im afraid it would come off!

Yes, all his horses and all his men, Humpty Dumpty went on. Theyd pick me up again in a minute, they would! However, this conversation is going on a little too fast: lets go back to the last remark but one.

Im afraid I cant quite remember it, Alice said very politely.

In that case we start fresh, said Humpty Dumpty, and its my turn to choose a subject (He talks about it just as if it was a game! thought Alice.) So heres a question for you. How old did you say you were?

Alice made a short calculation, and said Seven years and six months.

Wrong! Humpty Dumpty exclaimed triumphantly. You never said a word like it!

I though you meant How old are you? Alice explained.

If Id meant that, Id have said it, said Humpty Dumpty.
""

```
# importing regex and nltk
import re, nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# importing Counter to get word counts for bag of words
from collections import Counter

# importing a passage from Through the Looking Glass
from looking_glass import looking_glass_text

# importing part-of-speech function for lemmatization
from part_of_speech import get_part_of_speech

# Change text to another string:
text = "Such an excellent bag of words and an excellent word 'bags'."

cleaned = re.sub('[\W+]', ' ', text).lower()
tokenized = word_tokenize(cleaned)

stop_words = stopwords.words('english')
filtered = [word for word in tokenized if word not in stop_words]

normalizer = WordNetLemmatizer()
```

```
normalized = [normalizer.lemmatize(token, get_part_of_speech(token)) for token in filtered]
# Comment out the print statement below
#print(normalized)

# Define bag_of_looking_glass_words & print:
bag_of_looking_glass_words = Counter(normalized)
print(bag_of_looking_glass_words)
```

Part of speech - library content

```
from nltk.corpus import wordnet
from collections import Counter
def get_part_of_speech(word):
    probable_part_of_speech = wordnet.synsets(word)
    pos_counts = Counter()
    pos_counts["n"] = len([ item for item in probable_part_of_speech if item.pos()=="n" ])
    pos_counts["v"] = len([ item for item in probable_part_of_speech if item.pos()=="v" ])
    pos_counts["a"] = len([ item for item in probable_part_of_speech if item.pos()=="a" ])
    pos_counts["r"] = len([ item for item in probable_part_of_speech if item.pos()=="r" ])

    most_likely_part_of_speech = pos_counts.most_common(1)[0][0]
    return most_likely_part_of_speech
```

Output-only Terminal

```
Counter({'excellent': 2, 'bag': 2, 'word': 2})
```

Getting Started with Natural Language Processing

Language Models: N-Gram and NLM

For parsing entire phrases or conducting language prediction, you will want to use a model that pays attention to each word's neighbors. Unlike bag-of-words, the ***n-gram*** model considers a sequence of some number (n) units and calculates the probability of each unit in a body of language given the preceding sequence of length n . Because of this, n -gram probabilities with larger n values can be impressive at language prediction.

Take a look at our revised squid example: "The squids jumped out of the suitcases. The squids were furious."

A bigram model (where n is 2) might give us the following count frequencies:

```
{(' ', 'the'): 2, ('the', 'squids'): 2, ('squids', 'jumped'): 1, ('jumped', 'out'): 1, ('out', 'of'): 1, ('of', 'the'): 1, ('the', 'suitcases'): 1, ('suitcases', ' '): 1, ('squids', 'were'): 1, ('were', 'furious'): 1, ('furious', ' '): 1}
```

Copy to Clipboard

There are a couple problems with the n gram model:

How can your language model make sense of the sentence "The cat fell asleep in the mailbox" if it's never seen the word "mailbox" before? During training, your model will probably come across test words that it has never encountered before (this issue also pertains to bag of words). A tactic known as *language smoothing* can help adjust probabilities for unknown words, but it isn't always ideal.

For a model that more accurately predicts human language patterns, you want n (your sequence length) to be as large as possible. That way, you will have more natural sounding language, right? Well, as the sequence length grows, the number of examples of each sequence within your training corpus shrinks. With too few examples, you won't have enough data to make many predictions.

Enter **neural language models (NLMs)**! Much recent work within NLP has involved developing and training neural networks to approximate the approach our human brains take towards language. This deep learning approach allows computers a much more adaptive tack to processing human language. Common NLMs include LSTMs and transformer models.

Instructions

Checkpoint 1 Passed

1.

If you run the code, you'll see the 10 most commonly used words in *Through the Looking Glass* parsed with NLTK's `ngrams` module — if you're thinking this looks like a bag of words, that's because it is one!

Checkpoint 2 Passed

2.

What do you think the most common phrases in the text are? Let's find out...

Where `looking_glass_bigrams` is defined, change the second argument to `2` to see bigrams. Change `n` to `3` for `looking_glass_trigrams` to see trigrams.

The `ngrams()` function takes two arguments: the text you want to use and the `n` value.

Checkpoint 3 Passed

3.

Change `n` to a number greater than `3` for `looking_glass_ngrams`. Try increasing the number.

At what `n` are you just getting lines from poems repeated in the text? This is where there may be too few examples of each sequence within your training corpus to make any helpful predictions.

```
import nltk, re
from nltk.tokenize import word_tokenize
# importing ngrams module from nltk
from nltk.util import ngrams
from collections import Counter
from looking_glass import looking_glass_full_text

cleaned = re.sub('\W+', ' ', looking_glass_full_text).lower()
tokenized = word_tokenize(cleaned)

# Change the n value to 2:
looking_glass_bigrams = ngrams(tokenized, 2)
looking_glass_bigrams_frequency = Counter(looking_glass_bigrams)

# Change the n value to 3:
looking_glass_trigrams = ngrams(tokenized, 3)
looking_glass_trigrams_frequency = Counter(looking_glass_trigrams)

# Change the n value to a number greater than 3:
looking_glass_ngrams = ngrams(tokenized, 7)
looking_glass_ngrams_frequency = Counter(looking_glass_ngrams)

print("Looking Glass Bigrams:")
print(looking_glass_bigrams_frequency.most_common(10))

print("\nLooking Glass Trigrams:")
print(looking_glass_trigrams_frequency.most_common(10))

print("\nLooking Glass n-grams:")
print(looking_glass_ngrams_frequency.most_common(10))

Looking Glass Bigrams:
[('of', 'the'), 101], (('said', 'the'), 98), (('in', 'a'), 97), (('in', 'the'), 90), (('as', 'she'), 82), (('you', 'know'), 72), (('a', 'little'), 68), (('the', 'queen'), 67), (('said', 'alice'), 67), (('to', 'the'), 66)]

Looking Glass Trigrams:
[('the', 'red', 'queen'), 54], (('the', 'white', 'queen'), 31), (('said', 'in', 'a'), 21), (('she', 'went', 'on'), 18), (('said', 'the', 'red'), 17), (('thought', 'to', 'herself'), 16), (('the', 'queen', 'said'), 16), (('said', 'to', 'herself'), 14), (('said', 'humpty', 'dumpty'), 14), (('the', 'knight', 'said'), 14)]

Looking Glass n-grams:
[('one', 'and', 'one', 'and', 'one', 'and', 'one', 'and', 'one', 'and'), 7], (('and', 'one', 'and', 'one', 'and', 'one', 'and'), 6), (('twas', 'brillig', 'and', 'the', 'slithy', 'toves', 'did'), 3), (('brillig', 'and', 'the', 'slithy', 'toves', 'did', 'gyre'), 3), (('and', 'the', 'slithy', 'toves', 'did', 'gyre', 'and'), 3), (('the', 'slithy', 'toves', 'did', 'gyre', 'and', 'gimble'), 3), (('slithy', 'toves', 'did', 'gyre', 'and', 'gimble', 'in'), 3), (('toves', 'did', 'gyre', 'and', 'gimble', 'in', 'the'), 3), (('did', 'gyre', 'and', 'gimble', 'in', 'the', 'wabe'), 3), (('gyre', 'and', 'gimble', 'in', 'the', 'wabe', 'all'), 3)]
```

Topic Models

We've touched on the idea of finding topics within a body of language. But what if the text is long and the topics aren't obvious?

Topic modeling is an area of NLP dedicated to uncovering latent, or hidden, topics within a body of language. For example, one Codecademy curriculum developer [used topic modeling to discover patterns within Taylor Swift songs](#) related to love and heartbreak over time.

A common technique is to deprioritize the most common words and prioritize less frequently used terms as topics in a process known as **term frequency-inverse document frequency (tf-idf)**. Say what?! This may sound counter-intuitive at first. Why would you want to give more priority to less-used words? Well, when you're working with a lot of text, it makes a bit of sense if you don't want your topics filled with words like "the" and "is." The Python libraries `gensim` and `sklearn` have modules to handle tf-idf.

Whether you use your plain bag of words (which will give you term frequency) or run it through tf-idf, the next step in your topic modeling journey is often **latent Dirichlet allocation (LDA)**. LDA is a statistical model that takes your documents and determines which words keep popping up together in the same contexts (i.e., documents). We'll use `sklearn` to tackle this for us.

If you have any interest in visualizing your newly minted topics, **word2vec** is a great technique to have up your sleeve. word2vec can map out your topic model results spatially as vectors so that similarly used words are closer together. In the case of a language sample consisting of "The squids jumped out of the suitcases. The squids were furious. Why are your suitcases full of jumping squids?", we might see that "suitcase", "jump", and "squid" were words used within similar contexts. This word-to-vector mapping is known as a *word embedding*.

Instructions

Checkpoint 1 Passed

1.

Check out how the bag of words model and tf-idf models stack up when faced with a new Sherlock Holmes text!

Run the code as is to see what topics they uncover...

Checkpoint 2 Passed

2.

Tf-idf has some interesting findings, but the regular bag of words is full of words that tell us very little about the topic of the texts!

Let's fix this. Add some words to `stop_list` that don't tell you much about the topic and then run your code again. Do this until you have at least 10 words in `stop_list` so that the bag of words LDA model has some interesting topics.

Some words you may want to add to the `stop_list`:

```
"say", "see", "holmes", "shall", "say",  
"man", "upon", "know", "quite", "one",  
"well", "could", "would", "take", "may",  
"think", "come", "go", "little", "must",  
"look"
```

Copy to Clipboard

```
import nltk, re  
from sherlock_holmes import bohemia_ch1, bohemia_ch2, bohemia_ch3, boscombe_ch1,  
boscombe_ch2, boscombe_ch3  
from preprocessing import preprocess_text  
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer  
from sklearn.decomposition import LatentDirichletAllocation  
  
# preparing the text  
corpus = [bohemia_ch1, bohemia_ch2, bohemia_ch3, boscombe_ch1, boscombe_ch2, boscombe_ch3]  
preprocessed_corpus = [preprocess_text(chapter) for chapter in corpus]  
  
# Update stop_list:  
stop_list = ["say", "see", "holmes", "shall", "say", "man", "upon", "know", "quite", "one",  
"well", "could", "would", "take", "may", "think", "come", "go", "little", "must", "look"]  
# filtering topics for stop words  
def filter_out_stop_words(corpus):  
    no_stops_corpus = []  
    for chapter in corpus:  
        no_stops_chapter = " ".join([word for word in chapter.split(" ") if word not in  
stop_list])  
        no_stops_corpus.append(no_stops_chapter)  
    return no_stops_corpus  
filtered_for_stops = filter_out_stop_words(preprocessed_corpus)
```

```

# creating the bag of words model
bag_of_words_creator = CountVectorizer()
bag_of_words = bag_of_words_creator.fit_transform(filtered_for_stops)

# creating the tf-idf model
tfidf_creator = TfidfVectorizer(min_df = 0.2)
tfidf = tfidf_creator.fit_transform(preprocessed_corpus)

# creating the bag of words LDA model
lda_bag_of_words_creator = LatentDirichletAllocation(learning_method='online',
n_components=10)
lda_bag_of_words = lda_bag_of_words_creator.fit_transform(bag_of_words)

# creating the tf-idf LDA model
lda_tfidf_creator = LatentDirichletAllocation(learning_method='online', n_components=10)
lda_tfidf = lda_tfidf_creator.fit_transform(tfidf)

print("~~~ Topics found by bag of words LDA ~~~")
for topic_id, topic in enumerate(lda_bag_of_words_creator.components_):
    message = "Topic #{}: ".format(topic_id + 1)
    message += " ".join([bag_of_words_creator.get_feature_names()[i] for i in
topic.argsort()[::-5 :-1]])
    print(message)

print("\n\n~~~ Topics found by tf-idf LDA ~~~")
for topic_id, topic in enumerate(lda_tfidf_creator.components_):
    message = "Topic #{}: ".format(topic_id + 1)
    message += " ".join([tfidf_creator.get_feature_names()[i] for i in topic.argsort()[::-5 :-
1]])
    print(message)

~~~ Topics found by bag of words LDA ~~~
Topic #1: holmes say upon know
Topic #2: holmes know could see
Topic #3: hand holmes could answer
Topic #4: upon mccarthy see say
Topic #5: say holmes know upon
Topic #6: holmes man see know
Topic #7: say come know upon
Topic #8: holmes could drive man
Topic #9: say one holmes look
Topic #10: say man holmes cigar

~~~ Topics found by tf-idf LDA ~~~
Topic #1: listen maid always daughter
Topic #2: gain park future path
Topic #3: mccarthy obvious tall cooe
Topic #4: holmes majesty king say
Topic #5: respectable high scarlet seat
Topic #6: watch costume marriage probably
Topic #7: whether briony window day
Topic #8: reveal sure pink trick
Topic #9: mccarthy father say man
Topic #10: power watch dress lady

```

Getting Started with Natural Language Processing

Text Similarity

Most of us have a good autocorrect story. Our phone's messenger quietly swaps one letter for another as we type and suddenly the meaning of our message has changed (to our horror or pleasure). However, addressing **text similarity** — including spelling correction — is a major challenge within natural language processing.

Addressing word similarity and misspelling for spellcheck or autocorrect often involves considering the **Levenshtein distance** or minimal edit distance between two words. The distance is calculated through the minimum number of insertions, deletions, and substitutions that would need to occur for one word to become another. For example, turning "bees" into "beans" would require one substitution ("a" for "e") and one insertion ("n"), so the Levenshtein distance would be two.

Phonetic similarity is also a major challenge within speech recognition. English-speaking humans can easily tell from context whether someone said "euthanasia" or "youth in Asia," but it's a far more challenging task for a machine! More advanced autocorrect and spelling correction technology additionally considers key distance on a keyboard and **phonetic similarity** (how much two words or phrases sound the same).

It's also helpful to find out if texts are the same to guard against plagiarism, which we can identify through **lexical similarity** (the degree to which texts use the same vocabulary and phrases). Meanwhile, **semantic similarity** (the degree to which documents contain similar meaning or topics) is useful when you want to find (or recommend) an article or book similar to one you recently finished.

Instructions

Checkpoint 1 Passed

1.

Assign the variable `three_away_from_code` a word with a Levenshtein distance of 3 from "code". Assign `two_away_from_chunk` a word with a Levenshtein distance of 2 from "chunk".

Each insertion, deletion, or substitution counts as an edit distance of 1.

```
import nltk
# NLTK has a built-in function
# to check Levenshtein distance:
from nltk.metrics import edit_distance

def print_levenshtein(string1, string2):
    print("The Levenshtein distance from '{0}' to '{1}' is {2}!".format(string1, string2,
edit_distance(string1, string2)))

# Check the distance between
# any two words here!
print_levenshtein("fart", "target")

# Assign passing strings here:
three_away_from_code = "cloud"

two_away_from_chunk = "dunk"

print_levenshtein("code", three_away_from_code)
print_levenshtein("chunk", two_away_from_chunk)
```

Output-only Terminal

Output:

```
The Levenshtein distance from 'fart' to 'target' is 3!
The Levenshtein distance from 'code' to 'cloud' is 3!
The Levenshtein distance from 'chunk' to 'dunk' is 2!
```

Language Prediction & Text Generation

<https://www.codecademy.com/paths/data-science-nlp/tracks/nlp-getting-started-with-nlp/modules/getting-started-with-nlp-module/lessons/getting-started-with-nlp/exercises/language-prediction-and-generation>

How does your favorite search engine complete your search queries? How does your phone's keyboard know what you want to type next? **Language prediction** is an application of NLP concerned with predicting text given preceding text. Autosuggest, autocomplete, and suggested replies are common forms of language prediction.

Your first step to language prediction is picking a language model. Bag of words alone is generally not a great model for language prediction; no matter what the preceding word was, you will just get one of the most commonly used words from your training corpus.

If you go the n -gram route, you will most likely rely on **Markov chains** to predict the statistical likelihood of each following word (or character) based on the training corpus. Markov chains are memory-less and make statistical predictions based entirely on the current n -gram on hand.

For example, let's take a sentence beginning, "I ate so many grilled cheese". Using a trigram model (where n is 3), a Markov chain would predict the following word as "sandwiches" based on the number of times the sequence "grilled cheese sandwiches" has appeared in the training data out of all the times "grilled cheese" has appeared in the training data.

A more advanced approach, using a neural language model, is the **Long Short Term Memory (LSTM)** model. LSTM uses deep learning with a network of artificial "cells" that manage memory, making them better suited for text prediction than traditional neural networks.

Instructions

Checkpoint 1 Passed

1.

Add three short stories by your favorite author or the lyrics to three songs by your favorite artist to **document1.py**, **document2.py**, and **document3.py**. Then run **script.py** to see a short example of text prediction.

Does it look like something by your favorite author or artist?

If you accidentally close one of the files, just click the file folder in the top left corner of the code editor to find the file and re-open it.

If you need some text to play around with, [Project Gutenberg](#) is an excellent resource.

```
import nltk, re, random
from nltk.tokenize import word_tokenize
from collections import defaultdict, deque
from document1 import training_doc1
from document2 import training_doc2
from document3 import training_doc3

class MarkovChain:
    def __init__(self):
        self.lookup_dict = defaultdict(list)
        self._seeded = False
        self.__seed_me()

    def __seed_me(self, rand_seed=None):
        if self._seeded is not True:
            try:
                if rand_seed is not None:
                    random.seed(rand_seed)
                else:
                    random.seed()
                self._seeded = True
            except NotImplementedError:
                self._seeded = False

    def add_document(self, str):
        preprocessed_list = self._preprocess(str)
        pairs = self._generate_tuple_keys(preprocessed_list)
```

```

    for pair in pairs:
        self.lookup_dict[pair[0]].append(pair[1])

def _preprocess(self, str):
    cleaned = re.sub(r'\W+', ' ', str).lower()
    tokenized = word_tokenize(cleaned)
    return tokenized

def __generate_tuple_keys(self, data):
    if len(data) < 1:
        return

    for i in range(len(data) - 1):
        yield [ data[i], data[i + 1] ]

def generate_text(self, max_length=50):
    context = deque()
    output = []
    if len(self.lookup_dict) > 0:
        self.__seed_me(rand_seed=len(self.lookup_dict))
        chain_head = [list(self.lookup_dict)[0]]
        context.extend(chain_head)

        while len(output) < (max_length - 1):
            next_choices = self.lookup_dict[context[-1]]
            if len(next_choices) > 0:
                next_word = random.choice(next_choices)
                context.append(next_word)
                output.append(context.popleft())
            else:
                break
        output.extend(list(context))
    return " ".join(output)

my_markov = MarkovChain()
my_markov.add_document(training_doc1)
my_markov.add_document(training_doc2)
my_markov.add_document(training_doc3)
generated_text = my_markov.generate_text()
print(generated_text)

```

Output-only Terminal

Output:

alice could see how she had caught the cook tulip roots instead of hers that very diligently to make out
silence its head began in to put it or small but there was in it was enough of little sister on with you make
their slates she ran across her

```

training_doc1 = """
Alice was beginning to get very tired of sitting by her sister on the bank, and of having nothing to do: once or twice she had
peeped into the book her sister was reading, but it had no pictures or conversations in it, and what is the use of a book,
thought Alice without pictures or conversations?

So she was considering in her own mind (as well as she could, for the hot day made her feel very sleepy and stupid), whether the
pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit
with pink eyes ran close by her.

There was nothing so very remarkable in that; nor did Alice think it so very much out of the way to hear the Rabbit say to
itself, Oh dear! Oh dear! I shall be late! (when she thought it over afterwards, it occurred to her that she ought to have
wondered at this, but at the time it all seemed quite natural); but when the Rabbit actually took a watch out of its waistcoat-
pocket, and looked at it, and then hurried on, Alice started to her feet, for it flashed across her mind that she had never
before seen a rabbit with either a waistcoat-pocket, or a watch to take out of it, and burning with curiosity, she ran across the
field after it, and fortunately was just in time to see it pop down a large rabbit-hole under the hedge.

In another moment down went Alice after it, never once considering how in the world she was to get out again.

The rabbit-hole went straight on like a tunnel for some way, and then dipped suddenly down, so suddenly that Alice had not a
moment to think about stopping herself before she found herself falling down a very deep well.

Either the well was very deep, or she fell very slowly, for she had plenty of time as she went down to look about her and to
wonder what was going to happen next. First, she tried to look down and make out what she was coming to, but it was too dark to
see anything; then she looked at the sides of the well, and noticed that they were filled with cupboards and book-shelves; here
and there she saw maps and pictures hung upon pegs. She took down a jar from one of the shelves as she passed; it was labelled
ORANGE MARMALADE, but to her great disappointment it was empty: she did not like to drop the jar for fear of killing somebody, so
managed to put it into one of the cupboards as she fell past it.

```


Well! thought Alice to herself, after such a fall as this, I shall think nothing of tumbling down stairs! How brave they'll all think me at home! Why, I wouldn't say anything about it, even if I fell off the top of the house! (Which was very likely true.)

Down, down, down. Would the fall never come to an end! I wonder how many miles I've fallen by this time? she said aloud. I must be getting somewhere near the centre of the earth. Let me see; that would be four thousand miles down, I think (for, you see, Alice had learnt several things of this sort in her lessons in the schoolroom, and though this was not a very good opportunity for showing off her knowledge, as there was no one to listen to her, still it was good practice to say it over) yes, that's about the right distance! but then I wonder what Latitude or Longitude I've got to? (Alice had no idea what Latitude was, or Longitude either, but thought they were nice grand words to say.)

Presently she began again. I wonder if I shall fall right through the earth! How funny it'll seem to come out among the people that walk with their heads downward! The Antipathies, I think (she was rather glad there was no one listening, this time, as it didn't sound at all the right word) but I shall have to ask them what the name of the country is, you know. Please, Maam, is this New Zealand or Australia? (and she tried to curtsey as she spoke—fancy *curtseying* as you're falling through the air! Do you think you could manage it?) And what an ignorant little girl she'll think me for asking! No, it'll never do to ask; perhaps I shall see it written up somewhere.

Down, down, down. There was nothing else to do, so Alice soon began talking again. Dinah! I miss me very much to-night, I should think! (Dinah was the cat.) I hope they'll remember her saucer of milk at tea-time. Dinah my dear! I wish you were down here with me! There are no mice in the air, I'm afraid, but you might catch a bat, and that's very like a mouse, you know. But do cats eat bats, I wonder? And here Alice began to get rather sleepy, and went on saying to herself, in a dreamy sort of way, Do cats eat bats? Do cats eat bats? and sometimes, Do bats eat cats? for, you see, as she couldn't answer either question, it didn't much matter which way she put it. She felt that she was dozing off, and had just begun to dream that she was walking hand in hand with Dinah, and saying to her very earnestly, Now, Dinah, tell me the truth: did you ever eat a bat? when suddenly, thump! thump! down she came upon a heap of sticks and dry leaves, and the fall was over.

Alice was not a bit hurt, and she jumped up on to her feet in a moment: she looked up, but it was all dark overhead; before her was another long passage, and the White Rabbit was still in sight, hurrying down it. There was not a moment to be lost: away went Alice like the wind, and was just in time to hear it say, as it turned a corner, Oh my ears and whiskers, how late it's getting! She was close behind it when she turned the corner, but the Rabbit was no longer to be seen: she found herself in a long, low hall, which was lit up by a row of lamps hanging from the roof.

There were doors all round the hall, but they were all locked; and when Alice had been all the way down one side and up the other, trying every door, she walked sadly down the middle, wondering how she was ever to get out again.

Suddenly she came upon a little three-legged table, all made of solid glass; there was nothing on it except a tiny golden key, and Alice's first thought was that it might belong to one of the doors of the hall; but, alas! either the locks were too large, or the key was too small, but at any rate it would not open any of them. However, on the second time round, she came upon a low curtain she had not noticed before, and behind it was a little door about fifteen inches high: she tried the little golden key in the lock, and to her great delight it fitted!

Alice opened the door and found that it led into a small passage, not much larger than a rat-hole: she knelt down and looked along the passage into the loveliest garden you ever saw. How she longed to get out of that dark hall, and wander about among those beds of bright flowers and those cool fountains, but she could not even get her head through the doorway; and even if my head would go through, thought poor Alice, it would be of very little use without my shoulders. Oh, how I wish I could shut up like a telescope! I think I could, if I only knew how to begin. For, you see, so many out-of-the-way things had happened lately, that Alice had begun to think that very few things indeed were really impossible.

There seemed to be no use in waiting by the little door, so she went back to the table, half hoping she might find another key on it, or at any rate a book of rules for shutting people up like telescopes: this time she found a little bottle on it, (which certainly was not here before, said Alice,) and round the neck of the bottle was a paper label, with the words DRINK ME beautifully printed on it in large letters.

It was all very well to say Drink me, but the wise little Alice was not going to do that in a hurry. No, I'll look first, she said, and see whether it's marked poison or not; for she had read several nice little histories about children who had got burnt, and eaten up by wild beasts and other unpleasant things, all because they would not remember the simple rules their friends had taught them: such as, that a red-hot poker will burn you if you hold it too long; and that if you cut your finger very deeply with a knife, it usually bleeds; and she had never forgotten that, if you drink much from a bottle marked poison, it is almost certain to disagree with you, sooner or later.

However, this bottle was not marked poison, so Alice ventured to taste it, and finding it very nice, (it had, in fact, a sort of mixed flavour of cherry-tart, custard, pine-apple, roast turkey, toffee, and hot buttered toast,) she very soon finished it off.

* * * * *

* * * * *

* * * * *

What a curious feeling! said Alice; I must be shutting up like a telescope.

And so it was indeed: she was now only ten inches high, and her face brightened up at the thought that she was now the right size for going through the little door into that lovely garden. First, however, she waited for a few minutes to see if she was going to shrink any further: she felt a little nervous about this; for it might end, you know, said Alice to herself, in my going out altogether, like a candle. I wonder what I should be like then? And she tried to fancy what the flame of a candle is like after the candle is blown out, for she could not remember ever having seen such a thing.

After a while, finding that nothing more happened, she decided on going into the garden at once; but, alas for poor Alice! when she got to the door, she found she had forgotten the little golden key, and when she went back to the table for it, she found she could not possibly reach it: she could see it quite plainly through the glass, and she tried her best to climb up one of the legs of the table, but it was too slippery; and when she had tired herself out with trying, the poor little thing sat down and cried.

Come, there's no use in crying like that! said Alice to herself, rather sharply; I advise you to leave off this minute! She generally gave herself very good advice, (though she very seldom followed it), and sometimes she scolded herself so severely as to bring tears into her eyes; and once she remembered trying to box her own ears for having cheated herself in a game of croquet she was playing against herself, for this curious child was very fond of pretending to be two people. But its no use now, thought poor Alice, to pretend to be two people! Why, there's hardly enough of me left to make one respectable person!

Soon her eye fell on a little glass box that was lying under the table: she opened it, and found in it a very small cake, on which the words EAT ME were beautifully marked in currants. Well, I'll eat it, said Alice, and if it makes me grow larger, I can reach the key; and if it makes me grow smaller, I can creep under the door; so either way I'll get into the garden, and I don't care which happens!

She ate a little bit, and said anxiously to herself, Which way? Which way?, holding her hand on the top of her head to feel which way it was growing, and she was quite surprised to find that she remained the same size: to be sure, this generally happens when one eats cake, but Alice had got so much into the way of expecting nothing but out-of-the-way things to happen, that it seemed quite dull and stupid for life to go on in the common way.

So she set to work, and very soon finished off the cake.

```

"""
training doc2 = """
A large rose-tree stood near the entrance of the garden: the roses growing on it were white, but there were three gardeners at it, busily painting them red. Alice thought this a very curious thing, and she went nearer to watch them, and just as she came up to them she heard one of them say, Look out now, Five! Don't go splashing paint over me like that!

I couldn't help it, said Five, in a sulky tone; Seven jogged my elbow.

On which Seven looked up and said, That's right, Five! Always lay the blame on others!

You'd better not talk! said Five. I heard the Queen say only yesterday you deserved to be beheaded!

What for? said the one who had spoken first.

That's none of your business, Two! said Seven.

```

Yes, it is his business! said Five, and I'll tell him it was for bringing the cook tulip-roots instead of onions.

Seven flung down his brush, and had just begun Well, of all the unjust things when his eye chanced to fall upon Alice, as she stood watching them, and he checked himself suddenly: the others looked round also, and all of them bowed low.

Would you tell me, said Alice, a little timidly, why you are painting those roses?

Five and Seven said nothing, but looked at Two. Two began in a low voice, Why the fact is, you see, Miss, this here ought to have been a red rose-tree, and we put a white one in by mistake; and if the Queen was to find it out, we should all have our heads cut off, you know. So you see, Miss, were doing our best, afore she comes, to At this moment Five, who had been anxiously looking across the garden, called out The Queen! The Queen! and the three gardeners instantly threw themselves flat upon their faces. There was a sound of many footsteps, and Alice looked round, eager to see the Queen.

First came ten soldiers carrying clubs; these were all shaped like the three gardeners, oblong and flat, with their hands and feet at the corners: next the ten courtiers; these were ornamented all over with diamonds, and walked two and two, as the soldiers did. After these came the royal children; there were ten of them, and the little dears came jumping merrily along hand in hand, in couples: they were all ornamented with hearts. Next came the guests, mostly Kings and Queens, and among them Alice recognised the White Rabbit: it was talking in a hurried nervous manner, smiling at everything that was said, and went by without noticing her. Then followed the Knave of Hearts, carrying the Kings crown on a crimson velvet cushion; and, last of all this grand procession, came THE KING AND QUEEN OF HEARTS.

Alice was rather doubtful whether she ought not to lie down on her face like the three gardeners, but she could not remember ever having heard of such a rule at processions; and besides, what would be the use of a procession, thought she, if people had all to lie down upon their faces, so that they couldnt see it? So she stood still where she was, and waited.

When the procession came opposite to Alice, they all stopped and looked at her, and the Queen said severely Who is this? She said it to the Knave of Hearts, who only bowed and smiled in reply.

Idiot! said the Queen, tossing her head impatiently; and, turning to Alice, she went on, Whats your name, child?

My name is Alice, so please your Majesty, said Alice very politely; but she added, to herself, Why, theyre only a pack of cards, after all. I neednt be afraid of them!

And who are these? said the Queen, pointing to the three gardeners who were lying round the rosetree; for, you see, as they were lying on their faces, and the pattern on their backs was the same as the rest of the pack, she could not tell whether they were gardeners, or soldiers, or courtiers, or three of her own children.

How should I know? said Alice, surprised at her own courage. Its no business of mine.

The Queen turned crimson with fury, and, after glaring at her for a moment like a wild beast, screamed Off with her head! Off

Nonsense! said Alice, very loudly and decidedly, and the Queen was silent.

The King laid his hand upon her arm, and timidly said Consider, my dear: she is only a child!

The Queen turned angrily away from him, and said to the Knave Turn them over!

The Knave did so, very carefully, with one foot.

Get up! said the Queen, in a shrill, loud voice, and the three gardeners instantly jumped up, and began bowing to the King, the Queen, the royal children, and everybody else.

Leave off that! screamed the Queen. You make me giddy. And then, turning to the rose-tree, she went on, What have you been doing here?

May it please your Majesty, said Two, in a very humble tone, going down on one knee as he spoke, we were trying

I see! said the Queen, who had meanwhile been examining the roses. Off with their heads! and the procession moved on, three of the soldiers remaining behind to execute the unfortunate gardeners, who ran to Alice for protection.

You shant be beheaded! said Alice, and she put them into a large flower-pot that stood near. The three soldiers wandered about for a minute or two, looking for them, and then quietly marched off after the others.

Are their heads off? shouted the Queen.

Their heads are gone, if it please your Majesty! the soldiers shouted in reply.

Thats right! shouted the Queen. Can you play croquet?

The soldiers were silent, and looked at Alice, as the question was evidently meant for her.

Yes! shouted Alice.

Come on, then! roared the Queen, and Alice joined the procession, wondering very much what would happen next.

Itsits a very fine day! said a timid voice at her side. She was walking by the White Rabbit, who was peeping anxiously into her face.

Very, said Alice: wheres the Duchess?

Hush! Hush! said the Rabbit in a low, hurried tone. He looked anxiously over his shoulder as he spoke, and then raised himself upon tiptoe, put his mouth close to her ear, and whispered Shes under sentence of execution.

What for? said Alice.

Did you say What a pity!? the Rabbit asked.

No, I didnt, said Alice: I dont think its at all a pity. I said What for?

She boxed the Queens ears the Rabbit began. Alice gave a little scream of laughter. Oh, hush! the Rabbit whispered in a frightened tone. The Queen will hear you! You see, she came rather late, and the Queen said

Get to your places! shouted the Queen in a voice of thunder, and people began running about in all directions, tumbling up against each other; however, they got settled down in a minute or two, and the game began. Alice thought she had never seen such a curious croquet-ground in her life; it was all ridges and furrows; the balls were live hedgehogs, the mallets live flamingoes, and the soldiers had to double themselves up and to stand on their hands and feet, to make the arches.

The chief difficulty Alice found at first was in managing her flamingo: she succeeded in getting its body tucked away, comfortably enough, under her arm, with its legs hanging down, but generally, just as she had got its neck nicely straightened out, and was going to give the hedgehog a blow with its head, it would twist itself round and look up in her face, with such a puzzled expression that she could not help bursting out laughing; and when she had got its head down, and was going to begin again, it was very provoking to find that the hedgehog had unrolled itself, and was in the act of crawling away; besides all this, there was generally a ridge or furrow in the way wherever she wanted to send the hedgehog to, and, as the doubled-up soldiers were always getting up and walking off to other parts of the ground, Alice soon came to the conclusion that it was a very difficult game indeed.

The players all played at once without waiting for turns, quarrelling all the while, and fighting for the hedgehogs; and in a very short time the Queen was in a furious passion, and went stamping about, and shouting Off with his head! or Off with her head! about once in a minute.

Alice began to feel very uneasy: to be sure, she had not as yet had any dispute with the Queen, but she knew that it might happen any minute, and then, thought she, what would become of me? Theyre dreadfully fond of beheading people here; the great wonder is, that theres any one left alive!

She was looking about for some way of escape, and wondering whether she could get away without being seen, when she noticed a curious appearance in the air: it puzzled her very much at first, but, after watching it a minute or two, she made it out to be a grin, and she said to herself Its the Cheshire Cat: now I shall have somebody to talk to.

How are you getting on? said the Cat, as soon as there was mouth enough for it to speak with.

Alice waited till the eyes appeared, and then nodded. Its no use speaking to it, she thought, till its ears have come, or at least one of them. In another minute the whole head appeared, and then Alice put down her flamingo, and began an account of the game, feeling very glad she had someone to listen to her. The Cat seemed to think that there was enough of it now in sight, and no more of it appeared.

I dont think they play at all fairly, Alice began, in rather a complaining tone, and they all quarrel so dreadfully one cant hear oneself speakand they dont seem to have any rules in particular; at least, if there are, nobody attends to themand youve no idea how confusing it is all the things being alive; for instance, theres the arch Ive got to go through next walking about at the other end of the groundand I should have croqueted the Queens hedgehog just now, only it ran away when it saw mine coming!

How do you like the Queen? said the Cat in a low voice.

Not at all, said Alice: shes so extremely Just then she noticed that the Queen was close behind her, listening: so she went on, likely to win, that its hardly worth while finishing the game.

The Queen smiled and passed on.

Who are you talking to? said the King, going up to Alice, and looking at the Cats head with great curiosity.

Its a friend of minea Cheshire Cat, said Alice: allow me to introduce it.

I dont like the look of it at all, said the King: however, it may kiss my hand if it likes.

Id rather not, the Cat remarked.

Dont be impertinent, said the King, and dont look at me like that! He got behind Alice as he spoke.

A cat may look at a King, said Alice. Ive read that in some book, but I dont remember where.

Well, it must be removed, said the King very decidedly, and he called the Queen, who was passing at the moment, My dear! I wish you would have this cat removed!

The Queen had only one way of settling all difficulties, great or small. Off with his head! she said, without even looking round.

Ill fetch the executioner myself, said the King eagerly, and he hurried off.

Alice thought she might as well go back, and see how the game was going on, as she heard the Queens voice in the distance, screaming with passion. She had already heard her sentence three of the players to be executed for having missed their turns, and she did not like the look of things at all, as the game was in such confusion that she never knew whether it was her turn or not. So she went in search of her hedgehog.

The hedgehog was engaged in a fight with another hedgehog, which seemed to Alice an excellent opportunity for croquetting one of them with the other: the only difficulty was, that her flamingo was gone across to the other side of the garden, where Alice could see it trying in a helpless sort of way to fly up into a tree.

By the time she had caught the flamingo and brought it back, the fight was over, and both the hedgehogs were out of sight: but it doesnt matter much, thought Alice, as all the arches are gone from this side of the ground. So she tucked it away under her arm, that it might not escape again, and went back for a little more conversation with her friend.

When she got back to the Cheshire Cat, she was surprised to find quite a large crowd collected round it: there was a dispute going on between the executioner, the King, and the Queen, who were all talking at once, while all the rest were quite silent, and looked very uncomfortable.

The moment Alice appeared, she was appealed to by all three to settle the question, and they repeated their arguments to her, though, as they all spoke at once, she found it very hard indeed to make out exactly what they said.

The executioners argument was, that you couldnt cut off a head unless there was a body to cut it off from: that he had never had to do such a thing before, and he wasnt going to begin at his time of life.

The Kings argument was, that anything that had a head could be beheaded, and that you werent to talk nonsense.

The Queens argument was, that if something wasnt done about it in less than no time shed have everybody executed, all round. (It was this last remark that had made the whole party look so grave and anxious.)

Alice could think of nothing else to say but It belongs to the Duchess: youd better ask her about it.

Shes in prison, the Queen said to the executioner: fetch her here. And the executioner went off like an arrow.

The Cats head began fading away the moment he was gone, and, by the time he had come back with the Duchess, it had entirely disappeared; so the King and the executioner ran wildly up and down looking for it, while the rest of the party went back to the game.

""

training doc3 = ""

Here! cried Alice, quite forgetting in the flurry of the moment how large she had grown in the last few minutes, and she jumped up in such a hurry that she tipped over the jury-box with the edge of her skirt, upsetting all the jurymen on to the heads of the crowd below, and there they lay sprawling about, reminding her very much of a globe of goldfish she had accidentally upset the week before.

Oh, I beg your pardon! she exclaimed in a tone of great dismay, and began picking them up again as quickly as she could, for the accident of the goldfish kept running in her head, and she had a vague sort of idea that they must be collected at once and put back into the jury-box, or they would die.

The trial cannot proceed, said the King in a very grave voice, until all the jurymen are back in their proper placesall, he repeated with great emphasis, looking hard at Alice as he said do.

Alice looked at the jury-box, and saw that, in her haste, she had put the Lizard in head downwards, and the poor little thing was waving its tail about in a melancholy way, being quite unable to move. She soon got it out again, and put it right; not that it signifies much, she said to herself; I should think it would be quite as much use in the trial one way up as the other.

As soon as the jury had a little recovered from the shock of being upset, and their slates and pencils had been found and handed back to them, they set to work very diligently to write out a history of the accident, all except the Lizard, who seemed too much overcome to do anything but sit with its mouth open, gazing up into the roof of the court.

What do you know about this business? the King said to Alice.

Nothing, said Alice.

Nothing whatever? persisted the King.

Nothing whatever, said Alice.

Thats very important, the King said, turning to the jury. They were just beginning to write this down on their slates, when the White Rabbit interrupted: Unimportant, your Majesty means, of course, he said in a very respectful tone, but frowning and making faces at him as he spoke.

Unimportant, of course, I meant, the King hastily said, and went on to himself in an undertone,

importantunimportantunimportantimportant as if he were trying which word sounded best.

Some of the jury wrote it down important, and some unimportant. Alice could see this, as she was near enough to look over their slates; but it doesnt matter a bit, she thought to herself.

At this moment the King, who had been for some time busily writing in his note-book, cackled out Silence! and read out from his book, Rule Forty-two. All persons more than a mile high to leave the court.

Everybody looked at Alice.

Im not a mile high, said Alice.

You are, said the King.

Nearly two miles high, added the Queen.

Well, I shant go, at any rate, said Alice; besides, thats not a regular rule; you invented it just now.

Its the oldest rule in the book, said the King.

Then it ought to be Number One, said Alice.

The King turned pale, and shut his note-book hastily. Consider your verdict, he said to the jury, in a low, trembling voice.

Theres more evidence to come yet, please your Majesty, said the White Rabbit, jumping up in a great hurry; this paper has just been picked up.

Whats in it? said the Queen.

I havent opened it yet, said the White Rabbit, but it seems to be a letter, written by the prisoner toto somebody.

It must have been that, said the King, unless it was written to nobody, which isnt usual, you know.

Who is it directed to? said one of the jurymen.

It isnt directed at all, said the White Rabbit; in fact, theres nothing written on the outside. He unfolded the paper as he spoke, and added It isnt a letter, after all: its a set of verses.

Are they in the prisoners handwriting? asked another of the jurymen.

No, theyre not, said the White Rabbit, and thats the queerest thing about it. (The jury all looked puzzled.)

He must have imitated somebody elses hand, said the King. (The jury all brightened up again.)

Please your Majesty, said the Knave, I didnt write it, and they cant prove I did: theres no name signed at the end.

If you didnt sign it, said the King, that only makes the matter worse. You must have meant some mischief, or else youd have signed your name like an honest man.

There was a general clapping of hands at this: it was the first really clever thing the King had said that day.

That proves his guilt, said the Queen.

It proves nothing of the sort! said Alice. Why, you dont even know what theyre about!

Read them, said the King.

The White Rabbit put on his spectacles. Where shall I begin, please your Majesty? he asked.

Begin at the beginning, the King said gravely, and go on till you come to the end: then stop.

These were the verses the White Rabbit read:

They told me you had been to her,
And mentioned me to him:
She gave me a good character,
But said I could not swim.

He sent them word I had not gone
(We know it to be true):
If she should push the matter on,
What would become of you?

I gave her one, they gave him two,
You gave us three or more;
They all returned from him to you,
Though they were mine before.

If I or she should chance to be
Involved in this affair,
He trusts to you to set them free,
Exactly as we were.

My notion was that you had been
(Before she had this fit)
An obstacle that came between
Him, and ourselves, and it.

Dont let him know she liked them best,
For this must ever be
A secret, kept from all the rest,
Between yourself and me.

Thats the most important piece of evidence weve heard yet, said the King, rubbing his hands; so now let the jury

If any one of them can explain it, said Alice, (she had grown so large in the last few minutes that she wasnt a bit afraid of interrupting him,) Ill give him sixpence. I dont believe theres an atom of meaning in it.

The jury all wrote down on their slates, She doesnt believe theres an atom of meaning in it, but none of them attempted to explain the paper.

If theres no meaning in it, said the King, that saves a world of trouble, you know, as we neednt try to find any. And yet I dont know, he went on, spreading out the verses on his knee, and looking at them with one eye; I seem to see some meaning in them, after all. said I could not swim you cant swim, can you? he added, turning to the Knave.

The Knave shook his head sadly. Do I look like it? he said. (Which he certainly did not, being made entirely of cardboard.)

All right, so far, said the King, and he went on muttering over the verses to himself: We know it to be true thats the jury, of course! gave her one, they gave him two why, that must be what he did with the tarts, you know

But, it goes on they all returned from him to you, said Alice.

Why, there they are! said the King triumphantly, pointing to the tarts on the table. Nothing can be clearer than that. Then againbefore she had this fit you never had fits, my dear, I think? he said to the Queen.

Never! said the Queen furiously, throwing an inkstand at the Lizard as she spoke. (The unfortunate little Bill had left off writing on his slate with one finger, as he found it made no mark; but he now hastily began again, using the ink, that was trickling down his face, as long as it lasted.)

Then the words dont fit you, said the King, looking round the court with a smile. There was a dead silence.

Its a pun! the King added in an offended tone, and everybody laughed, Let the jury consider their verdict, the King said, for about the twentieth time that day.

No, no! said the Queen. Sentence first verdict afterwards.

Stuff and nonsense! said Alice loudly. The idea of having the sentence first!

Hold your tongue! said the Queen, turning purple.

I won't! said Alice.

Off with her head! the Queen shouted at the top of her voice. Nobody moved.

Who cares for you? said Alice, (she had grown to her full size by this time.) You're nothing but a pack of cards!

At this the whole pack rose up into the air, and came flying down upon her: she gave a little scream, half of fright and half of anger, and tried to beat them off, and found herself lying on the bank, with her head in the lap of her sister, who was gently brushing away some dead leaves that had fluttered down from the trees upon her face.

Wake up, Alice dear! said her sister; Why, what a long sleep you've had!

Oh, I've had such a curious dream! said Alice, and she told her sister, as well as she could remember them, all these strange Adventures of hers that you have just been reading about; and when she had finished, her sister kissed her, and said, It was a curious dream, dear, certainly: but now run in to your tea; it's getting late. So Alice got up and ran off, thinking while she ran, as well she might, what a wonderful dream it had been.

But her sister sat still just as she left her, leaning her head on her hand, watching the setting sun, and thinking of little Alice and all her wonderful Adventures, till she too began dreaming after a fashion, and this was her dream:

First, she dreamed of little Alice herself, and once again the tiny hands were clasped upon her knee, and the bright eager eyes were looking up into hers: she could hear the very tones of her voice, and see that queer little toss of her head to keep back the wandering hair that would always get into her eyes: and still as she listened, or seemed to listen, the whole place around her became alive with the strange creatures of her little sister's dream.

The long grass rustled at her feet as the White Rabbit hurried by: the frightened Mouse splashed his way through the neighbouring pools: she could hear the rattling of the teacups as the March Hare and his friends shared their never-ending meal, and the shrill voice of the Queen ordering off her unfortunate guests to execution: once more the pig-baby was sneezing on the Duchess's knee, while plates and dishes crashed around it: once more the shriek of the Gryphon, the squeaking of the Lizards slate-pencil, and the choking of the suppressed guinea-pigs, filled the air, mixed up with the distant sobs of the miserable Mock Turtle.

So she sat on, with closed eyes, and half believed herself in Wonderland, though she knew she had but to open them again, and all would change to dull reality: the grass would be only rustling in the wind, and the pool rippling to the waving of the reeds: the rattling teacups would change to tinkling sheep-bells, and the Queen's shrill cries to the voice of the shepherd boy: and the sneeze of the baby, the shriek of the Gryphon, and all the other queer noises, would change (she knew) to the confused clamour of the busy farm-yard: while the lowing of the cattle in the distance would take the place of the Mock Turtle's heavy sobs.

Lastly, she pictured to herself how this same little sister of hers would, in the after-time, be herself a grown woman; and how she would keep, through all her riper years, the simple and loving heart of her childhood: and how she would gather about her other little children, and make their eyes bright and eager with many a strange tale, perhaps even with the dream of Wonderland of long ago; and how she would feel with all their simple sorrows, and find a pleasure in all their simple joys, remembering her own child-life, and the happy summer days.

Getting Started with Natural Language Processing

Advanced NLP Topics

Believe it or not, you've just scratched the surface of natural language processing. There are a slew of advanced topics and applications of NLP, many of which rely on deep learning and neural networks.

Naive Bayes classifiers are supervised

[machine learning](#)

Preview: Docs Loading link description

algorithms that leverage a probabilistic theorem to make predictions and classifications. They are widely used for sentiment analysis (determining whether a given block of language expresses negative or positive feelings) and spam filtering.

We've made enormous gains in **machine translation**, but even the most advanced translation software using neural networks and LSTM still has far to go in accurately translating between languages.

Some of the most life-altering applications of NLP are focused on improving **language accessibility** for people with disabilities. Text-to-speech functionality and speech recognition have improved rapidly thanks to neural language models, making digital spaces far more accessible places.

NLP can also be used to detect bias in writing and speech. Feel like a political candidate, book, or news source is biased but can't put your finger on exactly how? Natural language processing [can help you identify the language at issue](#).

Instructions

Checkpoint 1 Passed

1.

Assign `review` a string with a brief review of this lesson so far. Next, run your code. Is the Naive Bayes Classifier accurately classifying your review?

```
from reviews import counter, training_counts
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Add your review:
review = ""
review_counts = counter.transform([review])

classifier = MultinomialNB()
training_labels = [0] * 1000 + [1] * 1000
classifier.fit(training_counts, training_labels)
```

```

neg = (classifier.predict_proba(review_counts)[0][0] * 100).round()
pos = (classifier.predict_proba(review_counts)[0][1] * 100).round()

if pos > 50:
    print("Thank you for your positive review!")
elif neg > 50:
    print("We're sorry this hasn't been the best possible lesson for you! We're always looking to improve.")
else:
    print("Naive Bayes cannot determine if this is negative or positive. Thank you or we're sorry?")

print("\nAccording to our trained Naive Bayes classifier, the probability that your review was negative was {}% and the probability it was positive was {}%.".format(neg, pos))

```

Naive Bayes cannot determine if this is negative or positive. Thank you or we're sorry?

According to our trained Naive Bayes classifier, the probability that your review was negative was 50.0% and the probability it was positive was 50.0%.

Getting Started with Natural Language Processing

Challenges and Considerations

As you've seen, there are a vast

[array](#)

Preview: Docs Loading link description

of applications for NLP. However, as they say, “with great language processing comes great responsibility” (or something along those lines). When working with NLP, we have several important considerations to take into account:

Different NLP tasks may be more or less difficult in different languages. Because so many NLP tools are built by and for English speakers, these tools may lag behind in processing other languages. The tools may also be programmed with cultural and linguistic biases specific to English speakers.

What if your Amazon Alexa could only understand wealthy men from coastal areas of the United States? English itself is not a homogeneous body. English varies by person, by dialect, and by many sociolinguistic factors. When we build and train NLP tools, are we only building them for one type of English speaker?

You can have the best intentions and still inadvertently program a bigoted tool. While NLP can limit bias, it can also propagate bias. As an NLP developer, it's important to consider biases, both within your code and within the training corpus. A machine will learn the same biases you teach it, whether intentionally or unintentionally.

As you become someone who builds tools with natural language processing, it's vital to take into account your users' privacy. There are many powerful NLP tools that come head-to-head with privacy concerns. Who is collecting your data? How much data is being collected and what do those companies plan to do with your data?

Instructions

Checkpoint 1 Enabled

1.

Test out different slang on the Naive Bayes Classifier! What happens when you use the word “lit” to mean “wonderful” or “fun”?

Is the sentiment prediction accurate? Test out different slang.

```

from reviews import counter, training_counts
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Add your review:
review = "It was fun!"
review_counts = counter.transform([review])

classifier = MultinomialNB()
training_labels = [0] * 1000 + [1] * 1000

classifier.fit(training_counts, training_labels)

neg = (classifier.predict_proba(review_counts)[0][0] * 100).round()
pos = (classifier.predict_proba(review_counts)[0][1] * 100).round()

if pos > 50:
    print("Naive Bayes classifies this as positive.")
elif neg > 50:
    print("Naive Bayes classifies this as negative.")
else:
    print("Naive Bayes cannot determine if this is negative or positive.")

```

```
print("\nAccording to our trained Naive Bayes classifier, the probability that your review was negative was {0}% and the probability it was positive was {1}%".format(neg, pos))
Naive Bayes classifies this as positive.
```

```
According to our trained Naive Bayes classifier, the probability that your review was negative was 16.0% and the probability it was positive was 84.0%.
```

Getting Started with Natural Language Processing

NLP Review

Check out how much you've learned about natural language processing!

Natural language processing combines computer science, linguistics, and [artificial intelligence](#)

Preview: Docs Loading link description
to enable computers to process human languages.

NLTK is a Python library used for NLP.

Text preprocessing is a stage of NLP focused on cleaning and preparing text for other NLP tasks.

Parsing is an NLP technique concerned with breaking up text based on syntax.

Language models are probabilistic machine models of language use for NLP comprehension tasks. Common models include bag-of-words, *n*-gram models, and neural language modeling.

Topic modeling is the NLP process by which hidden topics are identified given a body of text.

Text similarity is a facet of NLP concerned with semblance between instances of language.

Language prediction is an application of NLP concerned with predicting language given preceding language.

There are many social and ethical considerations to take into account when designing NLP tools.

Instructions

You can build a lot of fun tools with NLP knowledge and a bit of Python. This is just the beginning.

Feel free to test out the plagiarism classifier we built in the code editor (does it work?) or use the space to play around with other NLP code you've encountered in this lesson!

```
import nltk
# Levenshtein distance:
from nltk.metrics import edit_distance

# an arbitrary plagiarism classifier:
def is_plagiarized(text1, text2):
    n = 7
    if edit_distance(text1.lower(), text2.lower()) > ((len(text1) + len(text2)) / n):
        return False
    return True

doc1 = "is this plagiarized"
doc2 = "maybe it's plagiarized"

print(is_plagiarized(doc1, doc2))

False
```

 Recommended Reading



Read the following:

Algorithms of Oppression: How Search Engines Reinforce Racism by Safiya Umoja Noble

Read Now 

In this book, you will learn how a biased set of search algorithms privilege whiteness and discriminate against people of color, specifically women of color. This is helpful if you want to think about how to mitigate the impact of biased algorithms in your data science work.

Review: Getting Started with Natural Language Processing

See all you've learned in [Getting Started with Natural Language Processing](#).

Review

Congratulations! The goal of this unit was to introduce the field of natural language processing and provide an overview of common applications, techniques, and challenges.

Having completed this unit, you are now able to:

- Understand what natural language processing is.
- Gain an introduction to common applications and challenges within natural language processing.
- Identify several natural language processing techniques and how they relate to each other.
- Try out a few natural language processing techniques using Python.

If you are interested in learning more about these topics, here are some additional resources:

Documentation: [NLTK](#) Book: [Algorithms of Oppression: How Search Engines Reinforce Racism, Safiya Umoja Noble](#)

Remember, you will put all of this knowledge into practice with an upcoming Portfolio Project. If you ever get stuck while working on the Project, you can come back to this Unit and review what you have learned.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Introduction: Text Preprocessing

See what you'll learn in [Text Preprocessing](#).

Goals of this Unit

The goal of this unit is to introduce several methods to prepare your text data for most NLP tasks.

After this unit, you will be able to:

- Recognize several techniques to prepare text for various NLP tasks.
- Use Python and regex (which you learned in the Data Wrangling track) to remove unnecessary formatting from your text.
- Split text into tokens using NLTK.
- Normalize text with Python, regex, and NLTK by removing affixes, changing case, and removing common words.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Text Preprocessing

Introduction

Text preprocessing is an approach for cleaning and preparing text data for use in a specific context. Developers use it in almost all natural language processing (NLP) pipelines, including voice recognition software, search engine lookup, and

[machine learning](#)

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

model training. It is an essential step because text data can vary. From its format (website, text message, voice recognition) to the people who create the text (language, dialect), there are plenty of things that can introduce noise into your data.

The ultimate goal of cleaning and preparing text data is to reduce the text to only the words that you need for your NLP goals.

In this lesson, you will learn strategies for preparing text data. While this list is not exhaustive, we will cover a few common approaches for cleaning and processing text data. They include:

- Using Regex & NLTK libraries

- Noise Removal – Removing unnecessary characters and formatting

- [Tokenization](#)

- Preview: Docs Loading link description

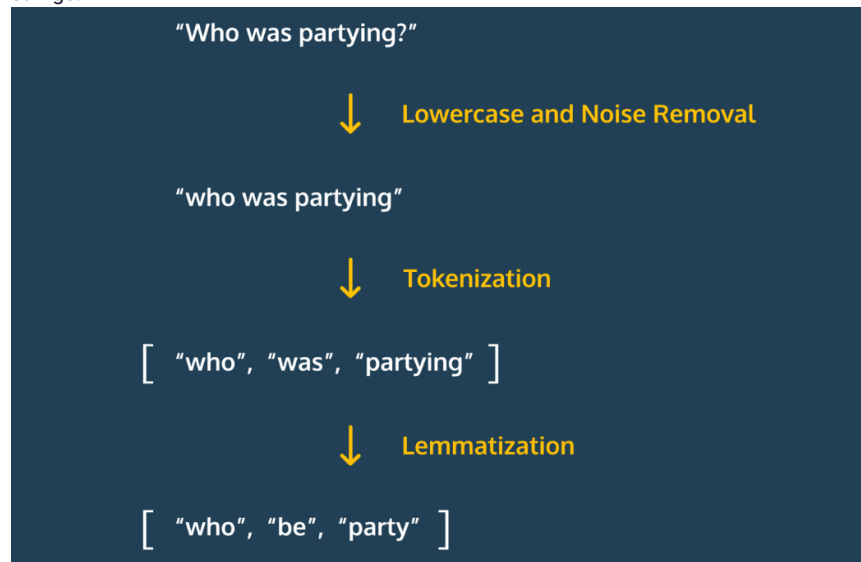
- break multi-word strings into smaller components

- Normalization – a catch-all term for processing data; this includes stemming and lemmatization

Instructions

In the gif to the right, you can see an example of using noise removal, tokenization, and lemmatization to change the string "Who was partying?" into a list with the words "who", "be", and "party".

In this lesson, you will learn how to use built-in and NLTK functions to apply these same text preprocessing approaches to your own strings.



Text Preprocessing

Noise Removal

Text cleaning is a technique that developers use in a variety of domains. Depending on the goal of your project and where you get your data from, you may want to remove unwanted information, such as:

- Punctuation and accents

- Special characters
- Numeric digits
- Leading, ending, and vertical whitespace
- HTML formatting

The type of noise that you need to remove from text usually depends on its source. For example, you could access data via the Twitter API, scraping a webpage, or voice recognition software. Fortunately, you can use the `.sub()` method in Python's regular expression (`re`) library for most of your noise removal needs.

The `.sub()` method has three required arguments:

- `pattern` – a regular expression that is searched for in the input string. There must be an `r` preceding the string to indicate it is a raw string, which treats backslashes as literal characters.
- `replacement_text` – text that replaces all matches in the input string
- `input` – the input string that will be edited by the `.sub()` method

The method returns a string with all instances of the `pattern` replaced by the `replacement_text`. Let's see a few examples of using this method to remove and replace text from a string.

Examples

First, let's consider how to remove HTML `<p>` tags from a string:

```
import re

text = "<p>    This is a paragraph</p>"
result = re.sub(r'<.*?p>', '', text)
print(result)
#    This is a paragraph
```

Copy to Clipboard

Notice, we replace the tags with an empty string `''`. This is a common approach for removing text.

Next, let's remove the whitespace from the beginning of the text. The whitespace consists of four spaces.

```
import re

text = "    This is a paragraph"
result = re.sub(r'\s{4}', '', text)
print(result)
# This is a paragraph
```

Copy to Clipboard

Take a look at Codecademy's [Parsing with Regular Expressions](#) lesson if you want to learn more regular expression syntax and tricks.

Instructions

Checkpoint 1 Passed

1.

At the top of `script.py`, import the regular expression library.

You can import a module with the following syntax:

```
from package import module
```

Copy to Clipboard

Checkpoint 2 Passed

2.

We used a package called Beautiful Soup to scrape [The Onion website](#) from October of 2019. We saved one of the headlines to a variable called `headline_one`.

Remove the opening and closing `h1` tags from `headline_one`. Save the value to `headline_no_tag`.

Use the `.sub()` method to find `<h1>` and `</h1>` tags and replace them with `''`. You can use `'<.*?h1>'` to target both the opening and closing tags at one time.

Checkpoint 3 Passed

3.

We also saved a Tweet to the variable `tweet`. Remove all `@` characters. Save the result to `tweet_no_at`

Use the `.sub()` method to find `@` characters and replace them with `''`.

```
import re

headline_one = '<h1>Nation\'s Top Pseudoscientists Harness High-Energy Quartz Crystal Capable Of Reversing Effects Of Being Gemini</h1>'
```

```

tweet = '@fat_meats, veggies are better than you think.'

headline_no_tag = re.sub(r'<.?h1>', '', headline_one)

tweet_no_at = re.sub(r'@', '', tweet)

try:
    print(headline_no_tag)
except:
    print('No variable called `headline_no_tag`')
try:
    print(tweet_no_at)
except:
    print('No variable called `tweet_no_at`')

```

Text Preprocessing

Tokenization

For many natural language processing tasks, we need access to each word in a

[string](#)

Preview: Docs Stores a sequence of indexed characters that can be of any length and is contained within a pair of single or double quotes.

. To access each word, we first have to break the text into smaller components. The

[method](#)

Preview: Docs Loading link description

for breaking text into smaller components is called

[tokenization](#)

Preview: Docs Loading link description

and the individual components are called *tokens*.

A few common operations that require tokenization include:

- Finding how many words or sentences appear in text
- Determining how many times a specific word or phrase exists
- Accounting for which terms are likely to co-occur

While tokens are usually individual words or terms, they can also be sentences or other size pieces of text.

To tokenize individual words, we can use `nltk's word_tokenize()` function. The function accepts a string and returns a list of words:

```

from nltk.tokenize import word_tokenize

text = "Tokenize this text"
tokenized = word_tokenize(text)

print(tokenized)
# ["Tokenize", "this", "text"]

```

Copy to Clipboard

To tokenize at the sentence level, we can use `sent_tokenize()` from the same module.

```

from nltk.tokenize import sent_tokenize

text = "Tokenize this sentence. Also, tokenize this sentence."
tokenized = sent_tokenize(text)

print(tokenized)
# ['Tokenize this sentence.', 'Also, tokenize this sentence.']

```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Import the `word_tokenize()` and `sent_tokenize()` functions from Python's NLTK package.

Checkpoint 2 Passed

2.

Tokenize `ecg_text` by word and save the result to `tokenized_by_word`.

Use `word_tokenize` to tokenize `ecg_text` and save the result to `tokenized_by_word`.

Checkpoint 3 Passed

3.

Tokenize `ecg_text` by sentence and save the result to `tokenized_by_sentence`.

Use `sent_tokenize` to tokenize `ecg_text` and save the result to `tokenized_by_sentence`.

```
from nltk.tokenize import word_tokenize, sent_tokenize

ecg_text = 'An electrocardiogram is used to record the electrical conduction through a person\'s heart. The readings can be used to diagnose cardiac arrhythmias.'

tokenized_by_word = word_tokenize(ecg_text)

tokenized_by_sentence = sent_tokenize(ecg_text)

try:
    print('Word Tokenization:')
    print(tokenized_by_word)
except:
    print('Expected a variable called `tokenized_by_word`')
try:
    print('Sentence Tokenization:')
    print(tokenized_by_sentence)
except:
    print('Expected a variable called `tokenized_by_sentence`')
```

Text Preprocessing

Normalization

[Tokenization](#)

Preview: Docs Tokenization is the process of breaking down text into smaller units called tokens, which are used in natural language processing and text analysis.

and noise removal are staples of almost all text pre-processing pipelines. However, some data may require further processing through text normalization. Text *normalization* is a catch-all term for various text pre-processing tasks. In the next few exercises, we'll cover a few of them:

- Upper or lowercasing

- Stopword removal

- Stemming* – bluntly removing prefixes and suffixes from a word

- Lemmatization* – replacing a single-word token with its root

The simplest of these approaches is to change the case of a

[string](#)

Preview: Docs Loading link description

. We can use Python's built-in String methods to make a string all uppercase or lowercase:

```
my_string = 'tHiS HaS a MiX oF cAsEs'

print(my_string.upper())
# 'THIS HAS A MIX OF CASES'

print(my_string.lower())
# 'this has a mix of cases'
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Make all the characters in `brands` lowercase and save the results to `brands_lower`.

Use the built-in `.lower()` string method to make all the letters in `brands` lowercase.

Checkpoint 2 Passed

2.

Make all the letters in `brands` uppercase and save the results to `brands_upper`.

Use the built-in `.upper()` string method to make all the letters in `brands` uppercase.

```
brands = 'Salvation Army, YMCA, Boys & Girls Club of America'

brands_lower = brands.lower()
brands_upper = brands.upper()

try:
    print(f'Lowercased brands: {brands_lower}')
except:
    print('Expected a variable called `brands_lower`')
try:
    print(f'Uppercased brands: {brands_upper}')
except:
    print('Expected a variable called `brands_upper`')
```

Text Preprocessing

Stopword Removal

Stopwords are words that we remove during preprocessing when we don't care about sentence structure. They are usually the most common words in a language and don't provide any information about the tone of a statement. They include words such as "a", "an", and "the".

NLTK provides a built-in library with these words. You can import them using the following statement:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
```

Copy to Clipboard

We create a set with the stop words so we can check if the words are in a list below.

Now that we have the words saved to `stop_words`, we can use

[tokenization](#)

Preview: Docs Loading link description

and a list comprehension to remove them from a sentence:

```
nbc_statement = "NBC was founded in 1926 making it the oldest major broadcast network in the USA"

word_tokens = word_tokenize(nbc_statement)
# tokenize nbc_statement

statement_no_stop = [word for word in word_tokens if word not in stop_words]

print(statement_no_stop)
# ['NBC', 'founded', '1926', 'making', 'oldest', 'major', 'broadcast', 'network', 'USA']
```

Copy to Clipboard

In this code, we first tokenized our

[string](#)

Preview: Docs Loading link description

, `nbc_statement`, then used a list comprehension to return a list with all of the stopwords removed.

Instructions

Checkpoint 1 Passed

1.

At the top of your script, import stopwords from NLTK. Save all English stopwords, as a set, to a variable called `stop_words`.

Use the following code to import stopwords:

```
from nltk.corpus import stopwords
```

Copy to Clipboard

Then, create a set of English stopwords with the following:

```
stop_words = set(stopwords.words('english'))
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Tokenize the text in `survey_text` and save the result to `tokenized_survey`.

Use `word_tokenize()` to tokenize `survey_text`, by word.

Checkpoint 3 Passed

3.

Remove stop words from `tokenized_survey` and save the result to `text_no_stops`.

Use a list comprehension of the following form:

```
my_var = [w for w in word_tokens if not w in stop_words]
```

Copy to Clipboard

Substitute `tokenized_survey` for `word_tokens`.

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

survey_text = 'A YouGov study found that American\'s like Italian food more than any other
country\'s cuisine.'

tokenized_survey = word_tokenize(survey_text)

text_no_stops = [w for w in tokenized_survey if not w in stop_words]

try:
    print(f'Stopwords type: {type(stop_words)}')
except:
    print('Expected a variable called `stop_words`')
try:
    print(f'Words Tokenized: {tokenized_survey}')
except:
    print('Expected a variable called `tokenized_survey`')
try:
    print(f'Text without Stops: {text_no_stops}')
except:
    print('Expected a variable called `text_no_stops`')
```

Output-only Terminal

Output:

```
Stopwords type: <class 'set'>
Words Tokenized: ['A', 'YouGov', 'study', 'found', 'that', 'American', '"s', 'like', 'Italian', 'food', 'more',
'than', 'any', 'other', 'country', '"s', 'cuisine', '.']
Text without Stops: ['A', 'YouGov', 'study', 'found', 'American', '"s', 'like', 'Italian', 'food', 'country',
'"s', 'cuisine', '.']
```

Text Preprocessing

Stemming

In natural language processing, *stemming* is the text preprocessing normalization task concerned with bluntly removing word affixes (prefixes and suffixes). For example, stemming would cast the word “going” to “go”. This is a common

[method](#)

Preview: Docs Loading link description

used by search engines to improve matching between user input and website hits.

NLTK has a built-in stemmer called PorterStemmer. You can use it with a list comprehension to stem each word in a tokenized list of words.

First, you must import and initialize the stemmer:

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
```

Copy to Clipboard

Now that we have our stemmer, we can apply it to each word in a list using a list comprehension:

```
tokenized = ['NBC', 'was', 'founded', 'in', '1926', '.', 'This', 'makes', 'NBC', 'the', 'oldest',
'major', 'broadcast', 'network', '.']

stemmed = [stemmer.stem(token) for token in tokenized]

print(stemmed)
# ['nbc', 'wa', 'found', 'in', '1926', '.', 'thi', 'make', 'nbc', 'the', 'oldest', 'major',
'broadcast', 'network', '.']
```

Copy to Clipboard

Notice, the words like ‘was’ and ‘founded’ became ‘wa’ and ‘found’, respectively. The fact that these words have been reduced is useful for many language processing applications. However, you need to be careful when stemming strings, because words can often be converted to something unrecognizable.

Instructions

Checkpoint 1 Passed

1.

At the top of **script.py**, import `PorterStemmer`, then initialize an instance of it and save the object to a variable called `stemmer`.

Checkpoint 2 Passed

2.

Tokenize `populated_island` and save the result to `island_tokenized`.

Checkpoint 3 Passed

3.

Use a list comprehension to stem each word in `island_tokenized`. Save the result to a variable called `stemmed`.

Use code similar to the following list comprehension to stem each word in `island_tokenized`.

```
stemmed = [stemmer.stem(token) for token in tokenized]
```

Copy to Clipboard

Replace `tokenized` in the code above with `island_tokenized`.

```
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()

populated_island = 'Java is an Indonesian island in the Pacific Ocean. It is the most populated
island in the world, with over 140 million people.'

island_tokenized = word_tokenize(populated_island)

stemmed = [stemmer.stem(token) for token in island_tokenized]

try:
    print('A stemmer exists:')
    print(stemmer)
except:
    print('Expected a variable called `stemmer`')
```

```
try:
    print('Words Tokenized:')
    print(island_tokenized)
except:
    print('Expected a variable called `island_tokenized`')
try:
    print('Stemmed Words:')
    print(stemmed)
except:
    print('Expected a variable called `stemmed`')
```

Output-only Terminal

Output:

```
A stemmer exists:
<PorterStemmer>
Words Tokenized:
['Java', 'is', 'an', 'Indonesian', 'island', 'in', 'the', 'Pacific', 'Ocean', '.', 'It', 'is',
'the', 'most', 'populated', 'island', 'in', 'the', 'world', ',', 'with', 'over', '140',
'million', 'people', '.']
Stemmed Words:
['java', 'is', 'an', 'indonesian', 'island', 'in', 'the', 'pacif', 'ocean', '.', 'It', 'is',
'the', 'most', 'popul', 'island', 'in', 'the', 'world', ',', 'with', 'over', '140', 'million',
'peopl', '.']
```

Text Preprocessing

Lemmatization

Lemmatization is a

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

for casting words to their root forms. This is a more involved process than stemming, because it requires the method to know the part of speech for each word. Since lemmatization requires the part of speech, it is a less efficient approach than stemming.

In the next exercise, we will consider how to tag each word with a part of speech. In the meantime, let's see how to use NLTK's lemmatize operation.

We can use NLTK's `WordNetLemmatizer` to lemmatize text:

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
```

Copy to Clipboard

Once we have the `lemmatizer` initialized, we can use a list comprehension to apply the lemmatize operation to each word in a list:

```
tokenized = ["NBC", "was", "founded", "in", "1926"]

lemmatized = [lemmatizer.lemmatize(token) for token in tokenized]

print(lemmatized)
# ["NBC", "wa", "founded", "in", "1926"]
```

Copy to Clipboard

The result saved to `lemmatized` contains 'wa', while the rest of the words remain the same. Not too useful. This happened because `lemmatize()` treats every word as a noun. To take advantage of the power of lemmatization, we need to tag each word in our text with the most likely part of speech. We'll do that in the next exercise.

Instructions

Checkpoint 1 Passed

1.

At the top of **script.py**, import `WordNetLemmatizer`, then initialize an instance of it and save the result to `lemmatizer`.
Checkpoint 2 Passed

2.

Tokenize the string saved to `populated_island`. Save the result to `tokenized_string`.

Checkpoint 3 Passed

3.

Use a list comprehension to lemmatize every word in `tokenized_string`. Save the result to the variable `lemmatized_words`.

Use a list comprehension in the following form to lemmatize every word in the tokenized list. Substitute `tokenized_string` for `tokenized_sentence`:

```
lemmatized = [lemmatizer.lemmatize(token) for token in tokenized_sentence]
```

```
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

populated_island = 'Indonesia was founded in 1945. It contains the most populated island in the world, Java, with over 140 million people.'

tokenized_string = word_tokenize(populated_island)

lemmatized_words = [lemmatizer.lemmatize(token) for token in tokenized_string]

try:
    print(f'A lemmatizer exists: {lemmatizer}')
except:
    print('Expected a variable called `lemmatizer`')
try:
    print(f'Words Tokenized: {tokenized_string}')
except:
    print('Expected a variable called `tokenized_string`')
try:
    print(f'Lemmatized Words: {lemmatized_words}')
except:
    print('Expected a variable called `lemmatized_words`')
```

Output-only Terminal

Output:

```
A lemmatizer exists: <WordNetLemmatizer>
Words Tokenized: ['Indonesia', 'was', 'founded', 'in', '1945', '.', 'It', 'contains', 'the', 'most', 'populated', 'island', 'in', 'the', 'world', ',', 'Java', ',', 'with', 'over', '140', 'million', 'people', '.']
Lemmatized Words: ['Indonesia', 'wa', 'founded', 'in', '1945', '.', 'It', 'contains', 'the', 'most', 'populated', 'island', 'in', 'the', 'world', ',', 'Java', ',', 'with', 'over', '140', 'million', 'people', '.']
```

Text Preprocessing

Part-of-Speech Tagging

To improve the performance of lemmatization, we need to find the part of speech for each word in our

[string](#)

Preview: Docs Stores a sequence of indexed characters that can be of any length and is contained within a pair of single or double quotes.

. In **script.py**, to the right, we created a part-of-speech tagging function. The function accepts a word, then returns the most common part of speech for that word. Let's break down the steps:

1. Import wordnet and Counter

```
from nltk.corpus import wordnet
from collections import Counter
```

Copy to Clipboard

`wordnet` is a [database](#)
Preview: Docs Loading link description
that we use for contextualizing words
`Counter` is a container that stores elements as [dictionary](#)
Preview: Docs Loading link description
keys

2. Get synonyms

Inside of our function, we use the `wordnet.synsets()` function to get a set of synonyms for the word:

```
def get_part_of_speech(word):  
    probable_part_of_speech = wordnet.synsets(word)
```

Copy to Clipboard

The returned synonyms come with their part of speech.

3. Use synonyms to determine the most likely part of speech

Next, we create a `Counter()` object and set each value to the count of the number of synonyms that fall into each part of speech:

```
pos_counts["n"] = len([ item for item in probable_part_of_speech if item.pos()=="n" ] )  
...
```

Copy to Clipboard

This line counts the number of nouns in the synonym set.

4. Return the most common part of speech

Now that we have a count for each part of speech, we can use the `.most_common()` counter [method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

to find and return the most likely part of speech:

```
most_likely_part_of_speech = pos_counts.most_common(1)[0][0]
```

Copy to Clipboard

Now that we can find the most probable part of speech for a given word, we can pass this into our lemmatizer when we find the root for each word. Let's take a look at how we would do this for a tokenized string:

```
tokenized = ["How", "old", "is", "the", "country", "Indonesia"]  
  
lemmatized = [lemmatizer.lemmatize(token, get_part_of_speech(token)) for token in tokenized]  
  
print(lemmatized)  
# ['How', 'old', 'be', 'the', 'country', 'Indonesia']  
# Previously: ['How', 'old', 'is', 'the', 'country', 'Indonesia']
```

Copy to Clipboard

Because we passed in the part of speech, "is" was cast to its root, "be." This means that words like "was" and "were" will be cast to "be".

Instructions

Checkpoint 1 Enabled

1.

Navigate to the file `script.py`. At the top of the file, we imported `get_part_of_speech` for you. Use

`get_part_of_speech()` to improve your lemmatizer.

Under the line where `tokenized_string` is defined, use the `get_part_of_speech()` function in a list comprehension to lemmatize all the words in `tokenized_string`. Save the result to a new variable called `lemmatized_pos`.

Use a list comprehension with the following form to pass the token and part of speech into the lemmatizer:

```
lemmatized = [lemmatizer.lemmatize(token, get_part_of_speech(token)) for token in  
tokenized]
```

```
import nltk  
from nltk.corpus import wordnet  
from collections import Counter
```

```
def get_part_of_speech(word):
    probable_part_of_speech = wordnet.synsets(word)

    pos_counts = Counter()

    pos_counts["n"] = len([ item for item in probable_part_of_speech if item.pos()=="n" ])
    pos_counts["v"] = len([ item for item in probable_part_of_speech if item.pos()=="v" ])
    pos_counts["a"] = len([ item for item in probable_part_of_speech if item.pos()=="a" ])
    pos_counts["r"] = len([ item for item in probable_part_of_speech if item.pos()=="r" ])

    most_likely_part_of_speech = pos_counts.most_common(1)[0][0]
    return most_likely_part_of_speech
```

Text Preprocessing

Review

This lesson is not an exhaustive introduction to text preprocessing. However, it does show a few of the most common tricks for cleaning your data. Before building a text preprocessing pipeline, it's most important to have an idea of how you want your data formatted and why you want it formatted that way. Once you know what you want, you can use these tools to get you there.

Let's review what we covered in this lesson:

- Text preprocessing is all about cleaning and prepping text data so that it's ready for other NLP tasks.
- Noise removal is a text preprocessing step concerned with removing unnecessary formatting from our text.
- [Tokenization](#)
- Preview: Docs Tokenization is the process of breaking down text into smaller units called tokens, which are used in natural language processing and text analysis.
- is a text preprocessing step devoted to breaking up text into smaller units (usually words or discrete terms).
- Normalization is the name we give most other text preprocessing tasks, including stemming, lemmatization, upper and lowercasing, and stopword removal.
- Stemming is the normalization preprocessing task focused on removing word affixes.
- Lemmatization is the normalization preprocessing task that more carefully brings words down to their root forms.



Recommended Reading

Read the following:

Natural Language Processing with Python, Chapter 3 by Steven Bird, Ewan Klein, and Edward Loper

[Read Now](#) 

In this book, you will learn about processing raw text from files and the web. This is helpful if you would like to analyze the textual data using NLTK (Natural Language Toolkit) tools.

Read the following selection: "Natural Language Processing with Python - Analyzing Text with the Natural Language Toolkit", Steven Bird, Ewan Klein, and Edward Loper, Chapter 3.

<https://www.nltk.org/book/ch03.html>

NLTK with Python 3 for Natural Language Processing

External videos that teach NLTK for text preprocessing.

We recommend watching the following videos for helpful tutorials on using NLTK for text preprocessing:

[Natural Language Processing With Python and NLTK p.1 Tokenizing words and Sentences](#)

In this video, you will get a tutorial on tokenizing text using NLTK. This is helpful if you want to see a demo of how to break text data up into words or sentences.

[Stop Words - Natural Language Processing With Python and NLTK p.2](#)

In this video, you will get a tutorial on removing stopwords from text data using NLTK, which is helpful if you're preparing text for sentiment analysis, topic modeling, or other tasks where common words are not helpful.

[Stemming - Natural Language Processing With Python and NLTK p.3](#)

[Lemmatizing - Natural Language Processing With Python and NLTK p.8](#)

In these videos, you will get tutorials on stemming and lemmatization using NLTK, further explaining the differences between these two techniques.

Review: Text Preprocessing

See what you've learned in Text Preprocessing.

Review

Congratulations! The goal of this unit was to introduce regular expressions (regex) and several methods to prepare your text data for most NLP tasks.

Having completed this unit, you are now able to:

- Recognize several techniques to prepare text for various NLP tasks.

- Use Python and regex (which you learned before in Data Wrangling) to remove unnecessary formatting from your text.

- Split text into tokens using NLTK.

- Normalize text with Python, regex, and NLTK by removing affixes, changing case, and removing common words.

If you are interested in learning more about these topics, here are some additional resources:

Book: [Speech and Language Processing, Chapter 2, Daniel Jurafsky & James H. Martin](#)

Video Playlist: [NLTK with Python 3 for Natural Language Processing](#)

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Introduction: Language Parsing

See what you'll learn in Language Parsing.

Goals of this Unit

The goal of this unit is to apply regular expressions (regex) and other natural language parsing tactics to find meaning and insights in text data.

After this unit, you will be able to:

- Work with regex patterns to find specific strings in your data.

- Use NLTK to identify parts of speech.

- Group words together by part-of-speech tags.

- Remove specific chunks of text based on part-of-speech tags.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Natural Language Parsing with Regular Expressions

Introduction

Discovering new code words in declassified CIA documents may seem like a mission for a foreign intelligence service, and detecting gender biases in the Harry Potter novels a task for a literature professor. Yet by utilizing **natural language parsing with regular expressions**, the power to perform such analyses is in your own hands!

While you may not put much explicit thought into the structure of your sentences as you write, the syntax choices you make are critical in ensuring your writing has meaning. Analyzing such sentence structure as well as word choice can not only provide insights into the connotation of a piece of text, but can also [highlight the biases of its author](#) or [uncover additional insights that even a deep, rigorous reading of the text might not reveal](#).

By using [Python's regular expression module](#) `re` and the [Natural Language Toolkit](#), known as NLTK, you can find keywords of interest, discover where and how often they are used, and discern the parts-of-speech patterns in which they appear to understand the sometimes hidden meaning in a piece of writing. Let's get started!

Instructions

Checkpoint 1 Enabled

1.

The code in the workspace performs natural language parsing with regular expressions on L. Frank Baum's classic novel *The Wonderful Wizard of Oz* . Run the code to view the output, which gives the frequency of different phrases that appear in the text.

Proceed to the next exercise when you are ready to learn how to perform such parsing yourself!

```

from nltk import RegexpParser
from pos_tagged_oz import pos_tagged_oz
from np_chunk_counter import np_chunk_counter

# define noun-phrase chunk grammar here
chunk_grammar = "NP: {<DT>?<JJ>*<NN>}"

# create RegexpParser object here
chunk_parser = RegexpParser(chunk_grammar)

# create a list to hold noun-phrase chunked sentences
np_chunked_oz = list()

# create a for-loop through each pos-tagged sentence in pos_tagged_oz here
for pos_tagged_sentence in pos_tagged_oz:
    # chunk each sentence and append to np_chunked_oz here
    np_chunked_oz.append(chunk_parser.parse(pos_tagged_sentence))

# store and print the most common np-chunks here
most_common_np_chunks = np_chunk_counter(np_chunked_oz)
print(most_common_np_chunks)

[(((('i', 'NN'),), 326), (((('dorothy', 'NN'),), 222), (((('the', 'DT'), ('scarecrow', 'NN'),), 213), (((('the', 'DT'), ('lion', 'NN'),), 148), (((('the', 'DT'), ('tin', 'NN'),), 123), (((('woodman', 'NN'),), 112), (((('oz', 'NN'),), 86), (((('toto', 'NN'),), 73), (((('head', 'NN'),), 59), (((('the', 'DT'), ('woodman', 'NN'),), 59), (((('the', 'DT'), ('wicked', 'JJ'), ('witch', 'NN'),), 58), (((('the', 'DT'), ('emerald', 'JJ'), ('city', 'NN'),), 51), (((('the', 'DT'), ('witch', 'NN'),), 49), (((('the', 'DT'), ('girl', 'NN'),), 46), (((('the', 'DT'), ('road', 'NN'),), 41), (((('room', 'NN'),), 29), (((('nothing', 'NN'),), 29), (((('the', 'DT'), ('air', 'NN'),), 29), (((('the', 'DT'), ('country', 'NN'),), 26), (((('the', 'DT'), ('land', 'NN'),), 24), (((('a', 'DT'), ('heart', 'NN'),), 24), (((('the', 'DT'), ('west', 'NN'),), 23), (((('axe', 'NN'),), 23), (((('the', 'DT'), ('sun', 'NN'),), 22), (((('the', 'DT'), ('little', 'JJ'), ('girl', 'NN'),), 22), (((('course', 'NN'),), 22), (((('the', 'DT'), ('cowardly', 'JJ'), ('lion', 'NN'),), 21), (((('aunt', 'NN'),), 21), (((('the', 'DT'), ('house', 'NN'),), 21), (((('the', 'DT'), ('door', 'NN'),), 21)])]
```

Natural Language Parsing with Regular Expressions

Compiling and Matching

Before you dive into more complex syntax parsing, you'll begin with basic

[regular expressions](#).

Preview: Docs Regular expressions, often shortened to regex or regexp, is a language used for pattern-matching text content. in Python using the [re module](#) as a regex refresher.

The first

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

you will explore is `.compile()`. This method takes a regular expression pattern as an

[argument](#)

Preview: Docs An argument is the actual value of a variable passed into a function.

and compiles the pattern into a regular expression object, which you can later use to find matching text. The regular expression object below will exactly match 4 upper or lower case characters.

```
regular_expression_object = re.compile("[A-Za-z]{4}")
```

Copy to Clipboard

Regular expression objects have a `.match()` method that takes a

[string](#)

Preview: Docs Loading link description

of text as an argument and looks for a *single* match to the regular expression that *starts at the beginning* of the string. To see if your regular expression matches the string "Toto" you can do the following:

```
result = regular_expression_object.match("Toto")
```


Copy to Clipboard

If `.match()` finds a match that starts at the beginning of the string, it will return a match object. The match object lets you know what piece of text the regular expression matched, and at what

[index](#)

Preview: Docs Loading link description

the match begins and ends. If there is no match, `.match()` will return `None`.

With the match object stored in `result`, you can access the matched text by calling `result.group(0)`. If you use a regex containing capture groups, you can access these groups by calling `.group()` with the appropriately numbered capture group as an argument.

Instead of compiling the regular expression first and then looking for a match in separate lines of code, you can simplify your match to one line:

```
result = re.match("[A-Za-z]{4}", "Toto")
```

Copy to Clipboard

With this syntax, `re's .match()` method takes a regular expression pattern as the first argument and a string as the second argument.

Instructions

Checkpoint 1 Passed

1.

The `re` module has been imported for you at the top of the workspace. `.compile()` a regular expression object named `regular_expression` that will match any 7 character string of word characters.

You can `.compile()` a regular expression object with the below syntax:

```
regular_expression_object = re.compile("[A-Za-z]{4}")
```

Copy to Clipboard

Update the regex pattern to match any 7 character string of word characters:

```
\w{7}
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Use `regular_expression's .match()` method to check if the regex matches the string stored in `character_1`. Save the result to `result_1` and print it.

Checkpoint 3 Passed

3.

Access the match in `result_1` using its `.group()` method with an argument of `0`. Save the result to `match_1` and print it.

Checkpoint 4 Passed

4.

In one line, use `re's .match()` method to compile a regular expression that will match any string of characters of length 7 and check if the regex matches the string stored in `character_2`. Save the result to `result_2` and print it. Was a match found?

```
import re

# characters are defined
character_1 = "Dorothy"
character_2 = "Henry"

# compile your regular expression here
regular_expression = re.compile("\w{7}")

# check for a match to character_1 here
result_1 = regular_expression.match(character_1)
print(result_1)

# store and print the matched text here
match_1 = result_1.group(0)
print(match_1)

# compile a regular expression to match a 7 character string of word characters and check for a
match to character_2 here
```

```
result_2 = re.match("\w{7}", character_2)
print(result_2)

<_sre.SRE_Match object; span=(0, 7), match='Dorothy'>
Dorothy
None
```

Natural Language Parsing with Regular Expressions

Searching and Finding

You can make your regular expression matches even more dynamic with the help of the `.search()`

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

. Unlike `.match()` which will only find matches at the start of a

[string](#)

Preview: Docs Loading link description

, `.search()` will look left to right through an entire piece of text and return a match object for the first match to the regular expression given. If no match is found, `.search()` will return `None`. For example, to search for a sequence of 8 word characters in the string `Are you a Munchkin?`:

```
result = re.search("\w{8}", "Are you a Munchkin?")
```

Copy to Clipboard

Using `.search()` on the string above will find a match of `"Munchkin"`, while using `.match()` on the same string would return `None`!

So far you have used methods that only return one piece of matching text. What if you want to find all the occurrences of a word or keyword in a piece of text to determine a frequency count? Step in the `.findall()` method!

Given a regular expression as its first

[argument](#)

Preview: Docs Loading link description

and a string as its second argument, `.findall()` will return a list of all *non-overlapping* matches of the regular expression in the string. Consider the below piece of text:

```
text = "Everything is green here, while in the country of the Munchkins blue was the favorite color. But the people do not seem to be as friendly as the Munchkins, and I'm afraid we shall be unable to find a place to pass the night."
```

Copy to Clipboard

To find all *non-overlapping* sequences of 8 word characters in the sentence you can do the following:

```
list_of_matches = re.findall("\w{8}", text)
```

Copy to Clipboard

`.findall()` will thus return the list `['Everythi', 'Munchkin', 'favorite', 'friendly', 'Munchkin']`.

Instructions

Checkpoint 1 Passed

1.

The entire text of L. Frank Baum's *The Wonderful Wizard of Oz* has been stored in `oz_text`. `.search()` for the occurrence of `"wizard"` in `oz_text`. Store the result in `found_wizard`, and print it.

The syntax to `.search()` a string for a match to a regular expression is given below:

```
first_match = re.search('regex pattern here', text_youre_searching_through)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Find all occurrences of `"lion"` in `oz_text` and store the result in `all_lions`. Print `all_lions`.

The syntax to `.findall()` occurrences of matches to a regular expression in a string is given below:

```
all_occurences = re.search('regex pattern here', text_youre_searching_through)
```

Copy to Clipboard

3.

It's important to note that the number of words in an entire text can impact the importance of a given word's frequency!

```
length_of_list = len(_____)
```

Replace the blank with the name of the list you want to find the length of!

Part-of-Speech Tagging

It may have been a while since you've been in English class, so let's review [the nine parts of speech](#) with an example:

Noun: the name of a person (Ramona, class), place, thing (textbook), or idea (NLP)

Pronoun: a word used in place of a noun (her, she)

Determiner: a word that introduces, or “determines”, a noun (the)

Verb: expresses action (studying) or being (are, has)

Adjective: modifies or describes a noun or pronoun (new)

Adverb: modifies or describes a verb, an adjective, or another adverb (happily)

Preposition: a word placed before a noun or pronoun to form a phrase modifying another word in the sentence (on)

Conjunction: a word that joins words, phrases, or clauses (and)

Interjection: a word used to express emotion (Wow)

Preview: Docs Loading link description

, a list of words in the order they appear in a sentence, and returns a list of tuples, where the first entry in the

[tuple](#)

Preview: Docs Loading link description

is a word and the second is the part-of-speech tag.

Given the sentence split into a list of words below:

```
word_sentence = ['do', 'you', 'suppose', 'oz', 'could', 'give', 'me', 'a', 'heart', '?']
```

Copy to Clipboard

you can tag the parts of speech as follows:

```
part_of_speech_tagged_sentence = pos_tag(word_sentence)
```

Copy to Clipboard

The call to `pos_tag()` will return the following:

```
[('do', 'VB'), ('you', 'PRP'), ('suppose', 'VB'), ('oz', 'NNS'), ('could', 'MD'), ('give', 'VB'), ('me', 'PRP'), ('a', 'DT'), ('heart', 'NN'), ('?', '.')]
```

Copy to Clipboard

Abbreviations are given instead of the full part of speech name. Some common abbreviations include: `NN` for nouns, `VB` for verbs, `RB` for adverbs, `JJ` for adjectives, and `DT` for determiners. A [complete list of part-of-speech tags and their abbreviations can be found here](#).

Instructions

Checkpoint 1 Passed

1.

Provided to you in the workspace is the text of *The Wonderful Wizard of Oz*, broken down into individual words on a sentence by sentence basis in a process known as tokenization. These sentences are called word tokenized sentences, which are stored in `word_tokenized_oz`.

Save the value stored at index 100 of `word_tokenized_oz` to a variable named `witches_fate`, and print it. You should see a sentence from the novel, split into individual words, print to the terminal.

To access a sentence split into individual words in `word_tokenized_oz`:

```
sentence = word_tokenized_oz[_____]
```

Copy to Clipboard

Replace the blank with the index 100!

Checkpoint 2 Passed

2.

Since the text has been broken down to individual words on a sentence by sentence level, you now can part-of-speech tag each word tokenized sentence in *The Wonderful Wizard of Oz*! Begin by creating an empty list named `pos_tagged_oz` to hold the part-of-speech tagged sentences.

Checkpoint 3 Passed

3.

Create a for-loop through each word tokenized sentence in `word_tokenized_oz`. Within the for-loop, part-of-speech tag each word tokenized sentence and append the result to `pos_tagged_oz`.

You can loop through each word tokenized sentence in `word_tokenized_oz` as follows:

```
for word_tokenized_sentence in word_tokenized_oz:
```

Copy to Clipboard

Within the loop call `pos_tag()` on each word tokenized sentence! Don't forget to append the result to `pos_tagged_oz`.

```
pos_tagged_oz.append(pos_tag(sentence_to_tokenize))
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Save the part-of-speech tagged sentence at index 100 to a variable named `witches_fate_pos`, and print it.

To access a part-of-speech tagged sentence in `pos_tagged_oz`:

```
sentence = pos_tagged_oz[_____]
```

Copy to Clipboard

Replace the blank with the index 100!

Wondering what `MD` stands for in the output? The answer is a *modal verb*! Modal verbs express necessity or possibility.

```
import nltk
from nltk import pos_tag
from word_tokenized_oz import word_tokenized_oz
```

```
# save and print the sentence stored at index 100 in word_tokenized_oz here
witches_fate = word_tokenized_oz[100]
print(witches_fate)

# create a list to hold part-of-speech tagged sentences here
pos_tagged_oz = list()

# create a for loop through each word tokenized sentence in word_tokenized_oz here
for word_tokenized_sentence in word_tokenized_oz:
    # part-of-speech tag each sentence and append to pos_tagged_oz here
    pos_tagged_oz.append(pos_tag(word_tokenized_sentence))

# store and print the part-of-speech tagged sentence at index 100 in witches_fate_pos here
witches_fate_pos = pos_tagged_oz[100]
print(witches_fate_pos)

[('', 'the', 'house', 'must', 'have', 'fallen', 'on', 'her', '.')]
[(('', ''), ('the', 'DT')), ('house', 'NN'), ('must', 'MD'), ('have', 'VB'), ('fallen', 'VBN'), ('on', 'IN'), ('her', 'PRP'), (',', '.')]

```

Natural Language Parsing with Regular Expressions

Introduction to Chunking

You have made it to the juicy stuff! Given your part-of-speech tagged text, you can now use regular expressions to find patterns in sentence structure that give insight into the meaning of a text. This technique of grouping words by their part-of-speech tag is called **chunking**.

With chunking in `nltk`, you can define a pattern of parts-of-speech tags using a modified notation of regular expressions. You can then find non-overlapping matches, or *chunks* of words, in the part-of-speech tagged sentences of a text.

The regular expression you build to find chunks is called *chunk grammar*. A piece of chunk grammar can be written as follows:

```
chunk_grammar = "AN: {<JJ><NN>}"
```

Copy to Clipboard

`AN` is a user-defined name for the kind of chunk you are searching for. You can use whatever name makes sense given your chunk grammar. In this case `AN` stands for adjective-noun

A pair of curly braces `{ }` surround the actual chunk grammar

`<JJ>` operates similarly to a regex character class, matching any adjective

`<NN>` matches any noun, singular or plural

The chunk grammar above will thus match any adjective that is followed by a noun.

To use the chunk grammar defined, you must create a `nltk.RegexpParser` object and give it a piece of chunk grammar as an argument.

```
chunk_parser = RegexpParser(chunk_grammar)
```

Copy to Clipboard

You can then use the `RegexpParser` object's `.parse()` method, which takes a list of part-of-speech tagged words as an argument, and identifies where such chunks occur in the sentence!

Consider the part-of-speech tagged sentence below:

```
pos_tagged_sentence = [('where', 'WRB'), ('is', 'VBZ'), ('the', 'DT'), ('emerald', 'JJ'), ('city', 'NN'), ('?', '.')]

```

Copy to Clipboard

You can chunk the sentence to find any adjectives followed by a noun with the following:

```
chunked = chunk_parser.parse(pos_tagged_sentence)
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Define a piece of chunk grammar named `chunk_grammar` that will chunk a single adjective followed by a single noun. Name the chunk `AN`.

Remember `<JJ>` matches any adjective and `<NN>` matches any noun, singular or plural.

To define a piece of chunk grammar:

```
your_chunk_grammar = "_____ : { _____ }"
```

Copy to Clipboard

Make sure to replace the first blank with the name of the chunk and the second blank with a regular expression to match an adjective and a noun!

Checkpoint 2 Passed

2.

Create a `RegexParser` object called `chunk_parser` using `chunk_grammar` as an argument.

To create a `RegexParser` object using chunk grammar:

```
regex_parser_object = RegexParser(_____)
```

Copy to Clipboard

Fill the blank with the `chunk grammar` you just defined!

Checkpoint 3 Passed

3.

The part-of-speech tagged novel `pos_tagged_oz` from the previous exercise has been given to you in the workspace.

Chunk the part-of-speech tagged sentence stored at index 282 in `pos_tagged_oz` using `chunk_parser`'s `.parse()`

method. Save the result to a variable named `scaredy_cat`, and print it. The chunked sequences of an adjective followed by a noun will be indicated with an `AN`, the chunk name you defined earlier.

Checkpoint 4 Passed

4.

`nltk` allows you to better visualize a chunked sentence with the `.pretty_print()` function. Uncomment the last line in the workspace and run the code to view the chunked sentence. Expand the output terminal all the way to the left to get a better view!

```
from nltk import RegexpParser, Tree
from pos_tagged_oz import pos_tagged_oz

# define adjective-noun chunk grammar here
chunk_grammar = "AN:{<JJ><NN>}"

# create RegexpParser object here
chunk_parser = RegexpParser(chunk_grammar)

# chunk the pos-tagged sentence at index 282 in pos_tagged_oz here
scaredy_cat = chunk_parser.parse(pos_tagged_oz[282])
print(scaredy_cat)

# pretty_print the chunked sentence here
Tree.fromstring(str(scaredy_cat)).pretty_print()
```

```
(S ``/`` where WRB is VBZ the DT (AN emerald JJ city NN) ?/. ' ' /')
      S
      |
      |
      | | | | | |
      | | | | | | AN
      | | | | | |
      | | | | | |
``/`` where WRB is VBZ the DT ?/. ' ' /' emerald JJ city NN
```

Natural Language Parsing with Regular Expressions

Chunking Noun Phrases

While you are able to chunk any sequence of parts of speech that you like, there are certain types of chunking that are linguistically helpful for determining meaning and bias in a piece of text. One such type of chunking is *NP-chunking*, or **noun phrase chunking**. A *noun phrase* is a phrase that contains a noun and operates, as a unit, as a noun.

A popular form of noun phrase begins with a *determiner* **DT**, which specifies the noun being referenced, followed by any number of *adjectives* **JJ**, which describe the noun, and ends with a noun **NN**.

Consider the part-of-speech tagged sentence below:

```
[('we', 'PRP'), ('are', 'VBP'), ('so', 'RB'), ('grateful', 'JJ'), ('to', 'TO'), ('you', 'PRP'), ('for', 'IN'), ('having', 'VBG'), ('killed', 'VBN'), ('the', 'DT'), ('wicked', 'JJ'), ('witch', 'NN'), ('of', 'IN'), ('the', 'DT'), ('east', 'NN'), (',', ','), ('and', 'CC'), ('for', 'IN'), ('setting', 'VBG'), ('our', 'PRP$'), ('people', 'NNS'), ('free', 'VBP'), ('from', 'IN'), ('bondage', 'NN'), ('.', '.')]

Copy to Clipboard
```

Copy to Clipboard

Can you spot the three noun phrases of the form described above? They are:

```
((('the', 'DT'), ('wicked', 'JJ'), ('witch', 'NN'))
(('the', 'DT'), ('east', 'NN'))
(('bondage', 'NN'))
```

With the help of a regular expression defined chunk grammar, you can easily find all the *non-overlapping* noun phrases in a piece of text! Just like in normal regular expressions, you can use quantifiers to indicate how many of each part of speech you want to match.

The chunk grammar for a noun phrase can be written as follows:

```
chunk_grammar = "NP: {<DT>?<JJ>*<NN>}"
```

Copy to Clipboard

NP is the user-defined name of the chunk you are searching for. In this case **NP** stands for noun phrase

<DT> matches any determiner

? is an *optional quantifier*, matching either **0** or **1** determiners

<JJ> matches any adjective

***** is the *Kleene star* quantifier, matching **0** or more occurrences of an adjective

<NN> matches any noun, singular or plural

By finding all the NP-chunks in a text, you can perform a frequency analysis and identify important, recurring noun phrases. You can also use these NP-chunks as pseudo-topics and tag articles and documents by their highest count NP-chunks! Or perhaps your analysis has you looking at the adjective choices an author makes for different nouns.

It is ultimately up to you, with your knowledge of the text you are working with, to interpret the meaning and use-case of the NP-chunks and their frequency of occurrence.

Instructions

Checkpoint 1 Passed

1.

Define a piece of chunk grammar named `chunk_grammar` that will chunk a noun phrase. Name the chunk **NP**.

Remember a noun phrase consists of an optional *determiner* **DT**, followed by any number of *adjectives* **JJ**, followed by a noun **NN**.

Checkpoint 2 Passed

2.

Create a `RegexParser` object called `chunk_parser` using `chunk_grammar` as an argument.

To create a `RegexParser` object using `chunk_grammar`:

```
regex_parser_object = RegexParser(_____)
```

Copy to Clipboard

Fill the blank with the `chunk_grammar` you just defined!

Checkpoint 3 Passed

3.

That part-of-speech tagged novel `pos_tagged_oz` you previously created has been imported for you in the workspace. Create a for loop through each part-of-speech tagged sentence in `pos_tagged_oz`. Within the for loop, NP-chunk each part-of-speech tagged sentence using `chunk_parser's` `.parse()` method and append the result to `np_chunked_oz`. Each item in `np_chunked_oz` will now be a noun phrase chunked sentence from *The Wonderful Wizard of Oz*!

You can loop through each part-of-speech tagged sentence in `pos_tagged_oz` as follows:

```
for pos_tagged_sentence in pos_tagged_oz:
```

Copy to Clipboard

Within the for-loop, call `chunk_parser.parse()` method on each part-of-speech tagged sentence. Don't forget to append the result to `np_chunked_oz`!

```
np_chunked_oz.append(chunk_parser.parse(pos_tagged_sentence))
```

Copy to Clipboard

Checkpoint 4 Passed

4.

A customized function `np_chunk_counter` that returns the 30 most common NP-chunks from a list of chunked sentences has been imported to the workspace for you. Call `np_chunk_counter` with `np_chunked_oz` as an argument and save the result to a variable named `most_common_np_chunks`.

Print `most_common_np_chunks`. What sticks out to you about the most common noun phrase chunks? Are you surprised by anything? Open the hint to see our analysis.

Want to see how `np_chunk_counter` works? Use the file navigator to open `np_chunk_counter.py` and inspect the function.

You can call `np_chunk_counter()` to count the most common noun phrase chunks as follows:

```
most_common_np_chunks = np_chunk_counter(np_chunked_oz)
```

Copy to Clipboard

Print `most_common_np_chunks` to view the 30 most common chunks in descending order, given in the form (NP-Chunk, Count).

Analysis

Looking at `most_common_np_chunks`, you can identify characters of importance in the text, such as `dorothy`, `the scarecrow` and `toto`, based on their high frequency count. For other characters you can see not only that they are important, but gain an understanding of who they are, such as `the wicked witch` and `the cowardly lion`, due to the inclusion of an adjective. A place of importance, `the emerald city`, also becomes evident.

```
from nltk import RegexpParser
from pos_tagged_oz import pos_tagged_oz
from np_chunk_counter import np_chunk_counter

# define noun-phrase chunk grammar here
chunk_grammar = "NP: {<DT>?<JJ>*<NN>}"

# create RegexpParser object here
chunk_parser = RegexpParser(chunk_grammar)

# create a list to hold noun-phrase chunked sentences
np_chunked_oz = list()

# create a for loop through each pos-tagged sentence in pos_tagged_oz here
for pos_tagged_sentence in pos_tagged_oz:
    # chunk each sentence and append to np_chunked_oz here
    np_chunked_oz.append(chunk_parser.parse(pos_tagged_sentence))

# store and print the most common np-chunks here
most_common_np_chunks = np_chunk_counter(np_chunked_oz)
print(most_common_np_chunks)

[ (('i', 'NN'), 326), (('dorothy', 'NN'), 222), (('the', 'DT'), ('scarecrow', 'NN'), 213), (('the', 'DT'), ('lion', 'NN'), 148), (('the', 'DT'), ('tin', 'NN'), 123), (('woodman', 'NN'), 112), (('oz', 'NN'), 86), (('toto', 'NN'), 73), (('head', 'NN'), 59), (('the', 'DT'), ('woodman', 'NN'), 59), (('the', 'DT'), ('wicked', 'JJ'), ('witch', 'NN'), 58), (('the', 'DT'), ('emerald', 'JJ'), ('city', 'NN'), 51), (('the', 'DT'), ('witch', 'NN'), 49), (('the', 'DT'), ('girl', 'NN'), 46), (('the', 'DT'), ('road', 'NN'), 41), (('room', 'NN'), 29), (('nothing', 'NN'), 29), (('the', 'DT'), ('air', 'NN'), 29), (('the', 'DT'), ('country', 'NN'), 26), (('the', 'DT'), ('land', 'NN'), 24), (('a', 'DT'), ('heart', 'NN'), 24), (('the', 'DT'), ('west', 'NN'), 23), (('axe', 'NN'), 23), (('the', 'DT'), ('sun', 'NN'), 22), (('the', 'DT'), ('little', 'JJ'), ('girl', 'NN'), 22), (('course', 'NN'), 22), (('the', 'DT'), ('cowardly', 'JJ'), ('lion', 'NN'), 21), (('aunt', 'NN'), 21), (('the', 'DT'), ('house', 'NN'), 21), (('the', 'DT'), ('door', 'NN'), 21)]
```

Natural Language Parsing with Regular Expressions

Chunking Verb Phrases

Another popular type of chunking is *VP-chunking*, or **verb phrase chunking**. A *verb phrase* is a phrase that contains a verb and its complements, objects, or modifiers.

Verb phrases can take a variety of structures, and here you will consider two. The first structure begins with a verb `VB` of any tense, followed by a noun phrase, and ends with an optional adverb `RB` of any form. The second structure switches the order of the verb and the noun phrase, but also ends with an optional adverb.

Both structures are considered because verb phrases of each form are essentially the same in meaning. For example, consider the part-of-speech tagged verb phrases given below:

```
((('said', 'VBD'), ('the', 'DT'), ('cowardly', 'JJ'), ('lion', 'NN'))  
('the', 'DT'), ('cowardly', 'JJ'), ('lion', 'NN')), (('said', 'VBD'),
```

The chunk grammar to find the first form of verb phrase is given below:

```
chunk_grammar = "VP: {<VB.*><DT>?<JJ>*<NN><RB.??>}"
```

Copy to Clipboard

`VP` is the user-defined name of the chunk you are searching for. In this case `VP` stands for verb phrase

`<VB.*>` matches any verb using the `.` as a wildcard and the `*` quantifier to match 0 or more occurrences of any character. This ensures matching verbs of any tense (ex. `VB` for present tense, `VBD` for past tense, or `VBN` for past participle)

`<DT>?<JJ>*<NN>` matches any noun phrase

`<RB.??>` matches any adverb using the `.` as a wildcard and the *optional quantifier* to match 0 or 1 occurrence of any character. This ensures matching any form of adverb (regular `RB`, comparative `RBR`, or superlative `RBS`)

`?` is an *optional quantifier*, matching either 0 or 1 adverbs

The chunk grammar for the second form of verb phrase is given below:

```
chunk_grammar = "VP: {<DT>?<JJ>*<NN><VB.*><RB.??>}"
```

Copy to Clipboard

Just like with NP-chunks, you can find all the VP-chunks in a text and perform a frequency analysis to identify important, recurring verb phrases. These verb phrases can give insight into what kind of action different characters take or how the actions that characters take are described by the author.

Once again, this is the part of the analysis where you get to be creative and use your own knowledge about the text you are working with to find interesting insights!

Instructions

Checkpoint 1 Passed

1.

Define a piece of chunk grammar named `chunk_grammar` that will chunk a verb-phrase of the following form: verb `VB`, followed by a noun phrase, followed by an optional adverb `RB`. Name the chunk `VP`.

You can match any verb with the notation `<VB.*>` and any adverb with `<RB.??>`.

A noun phrase is defined as an optional *determiner* `DT`, followed by any number of *adjectives* `JJ`, followed by a noun `NN`.

Checkpoint 2 Passed

2.

Create a `RegexParser` object called `chunk_parser` using `chunk_grammar` as an argument.

To create a `RegexParser` object using `chunk_grammar`:

```
regex_parser_object = RegexParser(_____)
```

Copy to Clipboard

Fill the blank with the `chunk_grammar` you just defined!

Checkpoint 3 Passed

3.

That part-of-speech tagged novel `pos_tagged_oz` you previously created has been imported for you in the workspace. Create a for loop through each part-of-speech tagged sentence in `pos_tagged_oz`. Within the for loop, VP-chunk each part-of-speech tagged sentence using `chunk_parser's` `.parse()` method and append the result to `vp_chunked_oz`. Each item in `vp_chunked_oz` will now be a verb phrase chunked sentence from *The Wonderful Wizard of Oz*!

You can loop through each part-of-speech tagged sentence in `pos_tagged_oz` as follows:

```
for pos_tagged_sentence in pos_tagged_oz:
```

Copy to Clipboard

Within the for loop, call `chunk_parser's` `.parse()` method on each part-of-speech tagged sentence. Don't forget to append the result to `vp_chunked_oz`!

```
vp_chunked_oz.append(chunk_parser.parse(pos_tagged_sentence))
```

Copy to Clipboard

Checkpoint 4 Passed

4.

A customized function `vp_chunk_counter` that returns the 30 most common vp-chunks from a list of chunked sentences has been imported to the workspace for you. Call `vp_chunk_counter` with `vp_chunked_oz` as an argument and save the result to a variable named `most_common_vp_chunks`.

Print `most_common_vp_chunks`. What sticks out to you about the most common verb phrase chunks? Does the action provided by the verbs give other insights simple noun phrases did not? Open the hint to see our analysis.

Want to see how `vp_chunk_counter` works? Use the file navigator to open `vp_chunk_counter.py` and inspect the function.

You can call `vp_chunk_counter()` to count the most common verb phrase chunks as follows:

```
most_common_vp_chunks = vp_chunk_counter(vp_chunked_oz)
```

Copy to Clipboard

Print `most common vp chunks` to view the 30 most common chunks in descending order, given in the form `(VP-Chunk, Count)`.

Analysis

Looking at `most_common_vp_chunks`, some interesting findings appear. While `the lion` is mentioned more frequently than the `tin (woodman)` from the NP-chunk analysis, the times when he speaks, indicated by `the lion said`, is less than the `tin woodman`. This can indicate `the lion` may not like to speak as much, reflecting on his character.

And while it is commonly understood that `the scarecrow` desires a brain and knowledge, he answers more questions than any other character! Maybe L. Frank Baum had a different perception of `the scarecrow` than many think.

Checkpoint 5 Passed

5.

Go back to the chunk grammar you defined earlier and update the grammar to find a verb phrase of the following form: noun phrase, followed by a verb `VB`, followed by an optional adverb `RB`. Rerun your code and look at the most common chunks. What do you find?

The chunk grammar for the other form of verb phrase is given below:

VP_CHUNK_COUNTER

```
from collections import Counter
```

```
# function that pulls chunks out of chunked sentence and finds the most common chunks
```

```
def vp_chunk_counter(chunked_sentences):
```

```
    # create a list to hold chunks
```

```
    chunks = list()
```

```
    # for-loop through each chunked sentence to extract verb phrase chunks
```

```
    for chunked_sentence in chunked_sentences:
```

```
        for subtree in chunked_sentence.subtrees(filter=lambda t: t.label() == 'VP'):
```

```
            chunks.append(tuple(subtree))
```

```
    # create a Counter object
```

```
    chunk_counter = Counter()
```

```
    # for-loop through the list of chunks
```

```
    for chunk in chunks:
```

```
        # increase counter of specific chunk by 1
```

```
        chunk_counter[chunk] += 1
```

```
    # return 30 most frequent chunks
```

```
    return chunk_counter.most_common(30)
```

```
chunk_grammar = "VP: {<DT>?<JJ>*<NN><VB.*><RB.??>}"
```

```
from nltk import RegexpParser
```

```
from pos_tagged_oz import pos_tagged_oz
```

```
from vp_chunk_counter import vp_chunk_counter
```

```
# define verb phrase chunk grammar here
```

```
chunk_grammar = "VP: {<VB.*><DT>?<JJ>*<NN><RB.??>}"
```

```
#chunk_grammar = "VP: {<DT>?<JJ>*<NN><VB.*><RB.?>?}"

# create RegexpParser object here
chunk_parser = RegexpParser(chunk_grammar)

# create a list to hold verb-phrase chunked sentences
vp_chunked_oz = list()

# create for loop through each pos-tagged sentence in pos_tagged_oz here
for pos_tagged_sentence in pos_tagged_oz:
    # chunk each sentence and append to vp_chunked_oz here
    vp_chunked_oz.append(chunk_parser.parse(pos_tagged_sentence))

# store and print the most common vp-chunks here
most_common_vp_chunks = vp_chunk_counter(vp_chunked_oz)
print(most_common_vp_chunks)

[(((('said', 'VBD'), ('the', 'DT'), ('scarecrow', 'NN')), 33), (((('said', 'VBD'), ('dorothy', 'NN')), 31),
(((('asked', 'VBN'), ('dorothy', 'NN')), 20), (((('said', 'VBD'), ('the', 'DT'), ('tin', 'NN')), 19), (((('said',
'VBD'), ('the', 'DT'), ('lion', 'NN')), 15), (((('said', 'VBD'), ('the', 'DT'), ('girl', 'NN')), 10),
(((('asked', 'VBN'), ('the', 'DT'), ('scarecrow', 'NN')), 10), (((('answered', 'VBD'), ('the', 'DT'),
('scarecrow', 'NN')), 8), (((('said', 'VBD'), ('the', 'DT'), ('cowardly', 'JJ'), ('lion', 'NN')), 8), (((('said',
'VBD'), ('oz', 'NN')), 8), (((('said', 'VBD'), ('the', 'DT'), ('woodman', 'NN')), 7), (((('pass', 'VB'), ('the',
'DT'), ('night', 'NN')), 6), (((('asked', 'VBN'), ('the', 'DT'), ('girl', 'NN')), 6), (((('see', 'VB'), ('the',
'DT'), ('great', 'JJ'), ('oz', 'NN')), 6), (((('answered', 'VBD'), ('oz', 'NN')), 6), (((('replied', 'VBD'),
('oz', 'NN')), 6), (((('cried', 'VBN'), ('dorothy', 'NN')), 5), (((('asked', 'VBN'), ('the', 'DT'), ('tin',
'NN')), 5), (((('asked', 'VBN'), ('the', 'DT'), ('lion', 'NN')), 5), (((('remarked', 'VBD'), ('the', 'DT'),
('lion', 'NN')), 5), (((('answered', 'VBD'), ('dorothy', 'NN')), 5), (((('replied', 'VBD'), ('the', 'DT'),
('lion', 'NN')), 5), (((('killed', 'VBN'), ('the', 'DT'), ('wicked', 'JJ'), ('witch', 'NN')), 4), (((('said',
'VBD'), ('the', 'DT'), ('witch', 'NN')), 4), (((('replied', 'VBD'), ('the', 'DT'), ('scarecrow', 'NN')), 4),
(((('answered', 'VBD'), ('the', 'DT'), ('girl', 'NN')), 4), (((('said', 'VBD'), ('the', 'DT'), ('farmer', 'NN')),
4), (((('thought', 'VBD'), ('i', 'NN')), 4), (((('answered', 'VBD'), ('the', 'DT'), ('woodman', 'NN')), 4),
(((('have', 'VBP'), ('no', 'DT'), ('heart', 'NN')), 4)]
```

Natural Language Parsing with Regular Expressions

Chunk Filtering

Another option you have to find chunks in your text is *chunk filtering*. **Chunk filtering** lets you define what parts of speech you *do not* want in a chunk and remove them.

A popular method for performing chunk filtering is to chunk an entire sentence together and then indicate which parts of speech are to be filtered out. If the filtered parts of speech are in the middle of a chunk, it will split the chunk into two separate chunks! The chunk grammar you can use to perform chunk filtering is given below:

```
chunk_grammar = """NP: {<.*>+}

                }<VB.?|IN>+{ """
```

Copy to Clipboard

`NP` is the user-defined name of the chunk you are searching for. In this case `NP` stands for noun phrase

The brackets `{ }` indicate what parts of speech you are chunking. `<.*>+` matches every part of speech in the sentence

The inverted brackets `} {` indicate which parts of speech you want to filter from the chunk. `<VB.?|IN>+` will filter out any verbs or prepositions

Chunk filtering provides an alternate way for you to search through a text and find the chunks of information useful for your analysis!

Instructions

Checkpoint 1 Passed

1.

The code in the workspace chunks an entire sentence together using the chunk grammar `"Chunk: {<.*>+}"`. Run the code and view the output to see how the sentence is chunked into one big chunk named `Chunk!`

Checkpoint 2 Passed

2.

Define a piece of chunk grammar named `chunk_grammar` that will chunk a noun phrase using chunk filtering. Name the chunk `NP`.

One way to chunk noun-phrases with chunk filtering uses the following chunk grammar:

```
chunk_grammar = """NP: {<.*>+}
                  }<VB.?!IN>+{ """
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Create a `RegexParser` object called `chunk_parser` using `chunk_grammar` as an argument.

To create a `RegexParser` object using chunk grammar:

```
regex_parser_object = RegexParser(_____)
```

Copy to Clipboard

Fill the blank with the `chunk_grammar` you just defined!

Checkpoint 4 Passed

4.

Now you can find the NP-chunks in a sentence from *The Wonderful Wizard of Oz* using chunk filtering! Chunk and filter the part-of-speech tagged sentence stored at index 230 in `pos_tagged_oz` using `chunk_parser`'s `.parse()` method. Save the result to `filtered_dancers`, and print `filtered_dancers`.

What parts of speech are removed from the chunk? What chunks remain?

Checkpoint 5 Passed

5.

The last line in the workspace `.pretty_print()` s the chunked and filtered sentence with `nlTK`. Uncomment the line and run the code to view the chunked and filtered sentence. Expand the output terminal all the way to the left to get a better view!

```
from nltk import RegexParser, Tree
from pos_tagged_oz import pos_tagged_oz

# define chunk grammar to chunk an entire sentence together
grammar = "Chunk: {<.*>+}"

# create RegexParser object
parser = RegexParser(grammar)

# chunk the pos-tagged sentence at index 230 in pos_tagged_oz
chunked_dancers = parser.parse(pos_tagged_oz[230])
print(chunked_dancers)

# define noun phrase chunk grammar using chunk filtering here
chunk_grammar = """NP: {<.*>+}
                  }<VB.?!IN>+{ """

# create RegexParser object here
```

```

chunk_parser = RegexpParser(chunk_grammar)

# chunk and filter the pos-tagged sentence at index 230 in pos_tagged_oz here
filtered_dancers = chunk_parser.parse(pos_tagged_oz[230])
print(filtered_dancers)

# pretty_print the chunked and filtered sentence here
Tree.fromstring(str(filtered_dancers)).pretty_print()

```

```

(S
  (Chunk
    then/RB
    she/PRP
    sat/VBD
    upon/IN
    a/DT
    settee/NN
    and/CC
    watched/VBD
    the/DT
    people/NNS
    dance/NN
    ./.)
)
(S
  (NP then/RB she/PRP)
  sat/VBD
  upon/IN
  (NP a/DT settee/NN and/CC)
  watched/VBD
  (NP the/DT people/NNS dance/NN ./.)
)

```

```

      S
      |
      |      NP      NP      NP
      |      |      |      |
sat/VBD upon/IN watched/VBD then/RB she/PRP a/DT settee/NN and/CC the/DT people/NNS dance/NN ./.

```

Natural Language Parsing with Regular Expressions

Review

And there you go! Now you have the toolkit to dig into any piece of text data and perform natural language parsing with [regular expressions](#)

Preview: Docs Loading link description

. What insights will you gain, or what bias may you uncover? Let's review what you have learned:

The `re` module's `.compile()` and `.match()` methods allow you to enter any regex pattern and look for a single match at the beginning of a piece of text

The `re` module's `.search()`

[method](#)

Preview content is loading

lets you find a single match to a regex pattern anywhere in a

[string](#)

Preview: Docs Loading link description

, while the `.findall()` method finds all the matches of a regex pattern in a string

Part-of-speech tagging identifies and labels the part of speech of words in a sentence, and can be performed in `nltk` using the `pos_tag()` function

Chunking groups together patterns of words by their part-of-speech tag. Chunking can be performed in `nltk` by defining a piece of chunk grammar using regular expression syntax and calling a `RegexpParser`'s `.parse()` method on a word tokenized sentence

NP-chunking chunks together an optional determiner `DT`, any number of adjectives `JJ`, and a noun `NN` to form a noun phrase. The frequency of different NP-chunks can identify important topics in a text or demonstrate how an author describes different subjects

VP-chunking chunks together a verb `VB`, a noun phrase, and an optional adverb `RB` to form a verb phrase. The frequency of different VP-chunks can give insight into what kind of action different subjects take or how the actions that different subjects take are described by an author, potentially indicating bias

Chunk filtering provides an alternative means of chunking by specifying what parts of speech you do not want in a chunk and removing them

Instructions

The code in the workspace is set up to perform natural language parsing on *The Wonderful Wizard of Oz*. However, the chunk grammar is empty! Instead of finding NP-chunks or VP-chunks, define your own chunk grammar using regular expressions in between the curly braces {}. Feel free to add any chunk filtering in between the inverted braces {} if you so desire!

Run the code and observe the frequencies of the chunks. What insights or knowledge do the chunk frequencies give you? Have you come to any different conclusions than from analyzing the NP-chunks and VP-chunks?

```
from nltk import RegexpParser
from pos_tagged_oz import pos_tagged_oz
from chunk_counter import chunk_counter

# define your own chunk grammar here
chunk_grammar = '''Chunk: {}
                  {}'''

# create RegexpParser object
chunk_parser = RegexpParser(chunk_grammar)

# create a list to hold chunked sentences
chunked_oz = list()

# create a for loop through each pos-tagged sentence in pos_tagged_oz
for pos_tagged_sentence in pos_tagged_oz:
    # chunk each sentence and append to chunked_oz
    chunked_oz.append(chunk_parser.parse(pos_tagged_sentence))

# store and print the most common chunks
most_common_chunks = chunk_counter(chunked_oz)
print(most_common_chunks)
```

Review: Language Parsing

See [what you've learned in Language Parsing](#).

Review

Congratulations! The goal of this unit was to apply regular expressions (regex) and other natural language parsing tactics to find meaning and insights in text data.

Having completed this unit, you are now able to:

- Work with regex to find specific strings in your data.
- Use NLTK to identify parts of speech.
- Group words together by part-of-speech tags.
- Remove specific chunks of text based on part-of-speech tags.

If you are interested in learning more about these topics, here are some additional resources:

- Book: [Natural Language Processing with Python, Chapter 7](#)
- Documentation: [NLTK](#)

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Introduction: Language Quantification

See [what's covered in Language Quantification](#).

Goals of this Unit

The goal of this unit is to use various methods, including language models and word embeddings, to represent text numerically.

After this unit, you will be able to:

- Identify common real-world applications for the bag-of-words (BoW) language model.
- Convert text into a BoW representation using a Python dictionary.
- Understand how feature dictionaries and feature vectors can be used to build a BoW model.

Recognize the difference between training data and test data.

Use scikit-learn to convert text into a BoW vector.

Identify advantages and disadvantages of BoW in relation to other common language models.

Understand how tf-idf helps you determine word importance within texts.

Implement tf-idf using scikit-learn.

Reframe word meaning based on context using word embeddings.

Understand how multi-dimensional vectors and distance metrics are useful for numerical representations of language.

Implement word embeddings, including word2vec, with spaCy and gensim.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.



Language Quantification

Introduction: Language Quantification

See what's covered in [Language Quantification](#).

Goals of this Unit

The goal of this unit is to use various methods, including language models and word embeddings, to represent text numerically. After this unit, you will be able to:

- Identify common real-world applications for the bag-of-words (BoW) language model.
- Convert text into a BoW representation using a Python dictionary.
- Understand how feature dictionaries and feature vectors can be used to build a BoW model.
- Recognize the difference between training data and test data.
- Use scikit-learn to convert text into a BoW vector.
- Identify advantages and disadvantages of BoW in relation to other common language models.
- Understand how tf-idf helps you determine word importance within texts.
- Implement tf-idf using scikit-learn.
- Reframe word meaning based on context using word embeddings.
- Understand how multi-dimensional vectors and distance metrics are useful for numerical representations of language.
- Implement word embeddings, including word2vec, with spaCy and gensim.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Bag-of-Words Language Model

1 Intro to Bag-of-Words

"A bag-of-words is all you need," some NLPers have decreed.

The bag-of-words language model is a simple-yet-powerful tool to have up your sleeve when working on natural language processing (NLP). The model has many, many use cases including:

- *determining topics in a song*
- *filtering spam from your inbox*
- *finding out if a tweet has positive or negative sentiment*
- *creating word clouds*

Instructions

Checkpoint 1 Enabled

1.

In the code editor, we've created a spam filter using bag-of-words. Test it out!

Replace the text in `test_text` with the text from a marketing email you've received and run the code. Was the result what you expected?

Spam example:

Our records indicate your Pension is under-performing to see higher growth and up to 25% cash release reply for a free review.

Copy to Clipboard

Normal email example:

Have you ever wondered how an email ends up in the spam folder? Or how customer service phone systems are able to understand what you're saying? From a cleaner inbox to faster customer service and virtual assistants that can tell you the weather, the number of applications using natural language processing (NLP) is rapidly growing. NLP is all about how computers work with human language. Learn how to create your own powerful NLP programs with this new course!

Start Now

Happy coding,
Codecademy

```
from spam_data import training_spam_docs, training_doc_tokens, training_labels
from sklearn.naive_bayes import MultinomialNB
from preprocessing import preprocess_text

# Add your email text to test_text between the triple quotes:
test_text = """
GMAIL - Subscription Termination Notice
Your Subscription has Closed at Thu,04 Sep-2025.
[ Final Warning ]
Account-ID: CA4983902
Email: l.manuel.s.soares@gmail.com
Username: l.manuel.s.soares
Renewal Discount: 90%
Dear l.manuel.s.soares,

As a result,We have attempted to contact your account multiple times with warnings and alerts,
but we have not received any response from you..

You are now unprotected against cyber attacks and hackers. For your safety and to prevent data
loss, we strongly recommend renewing your subscription.

Update Now
"""
test_tokens = preprocess_text(test_text)

def create_features_dictionary(document_tokens):
    features_dictionary = {}
    index = 0
    for token in document_tokens:
        if token not in features_dictionary:
            features_dictionary[token] = index
            index += 1
    return features_dictionary

def tokens_to_bow_vector(document_tokens, features_dictionary):
    bow_vector = [0] * len(features_dictionary)
    for token in document_tokens:
        if token in features_dictionary:
            feature_index = features_dictionary[token]
            bow_vector[feature_index] += 1
    return bow_vector

bow_sms_dictionary = create_features_dictionary(training_doc_tokens)
```

```

training_vectors = [tokens_to_bow_vector(training_doc, bow_sms_dictionary) for training_doc in
training_spam_docs]
test_vectors = [tokens_to_bow_vector(test_tokens, bow_sms_dictionary)]

spam_classifier = MultinomialNB()
spam_classifier.fit(training_vectors, training_labels)

predictions = spam_classifier.predict(test_vectors)

print("Looks like a normal email!" if predictions[0] == 0 else "You've got spam!")

You've got spam!

```

Bag-of-Words Language Model

2 Bag-of-What?

Bag-of-words (BoW) is a statistical language model based on word count. Say what?

Let's start with that first part: a **statistical language model** is a way for computers to make sense of language based on probability. For example, let's say we have the text:

"Five fantastic fish flew off to find faraway functions. Maybe find another five fantastic fish?"

A statistical language model focused on the starting letter for words might take this text and predict that words are most likely to start with the letter "f" because 11 out of 15 words begin that way. A different statistical model that pays attention to word order might tell us that the word "fish" tends to follow the word "fantastic."

Bag-of-words does not give a flying fish about word starts or word order though; its sole concern is **word count** — how many times each word appears in a document.

If you're already familiar with statistical language models, you may also have heard BoW referred to as the **unigram model**. It's technically a special case of another statistical model, the n -gram model, with n (the number of words in a sequence) set to 1.

If you have no idea what n -grams are, don't worry — we'll dive deeper into them in another lesson.

Instructions

Try out a few word combinations in the applet to see how the bag-of-words model works. What happens if you add the same word a few times?

PREPROCESSING

```

import nltk, re
from nltk.corpus import wordnet
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from collections import Counter

stop_words = stopwords.words('english')

normalizer = WordNetLemmatizer()

def get_part_of_speech(word):
    probable_part_of_speech = wordnet.synsets(word)
    pos_counts = Counter()
    pos_counts["n"] = len([item for item in probable_part_of_speech if item.pos()=="n"])
    pos_counts["v"] = len([item for item in probable_part_of_speech if item.pos()=="v"])
    pos_counts["a"] = len([item for item in probable_part_of_speech if item.pos()=="a"])
    pos_counts["r"] = len([item for item in probable_part_of_speech if item.pos()=="r"])
    most_likely_part_of_speech = pos_counts.most_common(1)[0][0]
    return most_likely_part_of_speech

def preprocess_text(text):

```

```
cleaned = re.sub(r'\W+', ' ', text).lower()
tokenized = word_tokenize(cleaned)
normalized = [normalizer.lemmatize(token, get_part_of_speech(token)) for token in tokenized]
return normalized
```

SOLUTION

```
from preprocessing import preprocess_text
# Define text_to_bow() below:
def text_to_bow(some_text):
    bow_dictionary = {}
    tokens = preprocess_text(some_text)
    for token in tokens:
        if token in bow_dictionary:
            bow_dictionary[token] += 1
        else:
            bow_dictionary[token] = 1
    return bow_dictionary

print(text_to_bow("I love fantastic flying fish. These flying fish are just ok, so maybe I will
find another few fantastic fish..."))
```

```
{'i': 2, 'love': 1, 'fantastic': 2, 'fly': 2, 'fish': 3, 'these': 1, 'be': 1, 'just': 1, 'ok': 1, 'so': 1,
'maybe': 1, 'will': 1, 'find': 1, 'another': 1, 'few': 1}
```

Bag-of-Words Language Model

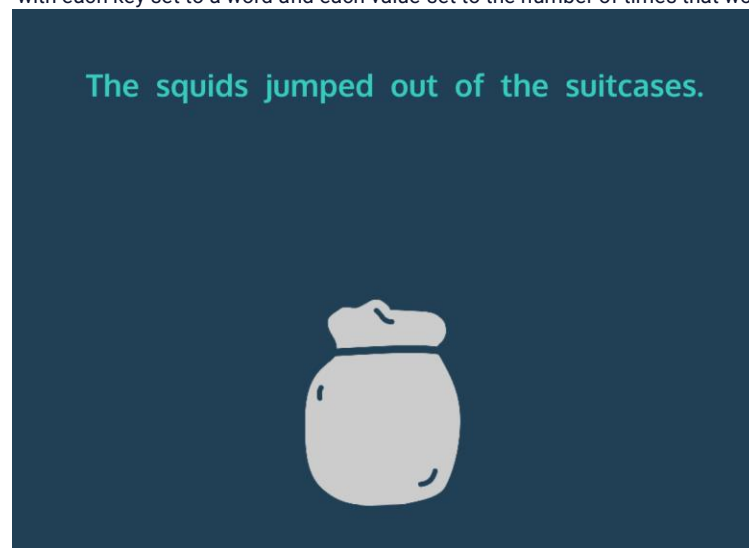
3 - BoW Dictionaries

One of the most common ways to implement the BoW model in Python is as a

[dictionary](#)

Preview: Docs A dictionary is an unordered set of (key, value) pairs. It provides a way to map pieces of data to each other, and allows for quick access to values associated to keys. The syntax of a dictionary is as follows: pseudo dictionary = { key1: value1, key2: value2, key3: value3

with each key set to a word and each value set to the number of times that word appears. Take the example below:



The words from the sentence go into the bag-of-words and come out as a dictionary of words with their corresponding counts. For statistical models, we call the text that we use to build the model our **training data**. Usually, we need to prepare our text data by breaking it up into `documents` (shorter strings of text, generally sentences).

Let's build a function that converts a given training text into a bag-of-words!

Instructions

Checkpoint 1 Passed

1.

Define a function `text_to_bow()` that accepts `some_text` as a variable. Inside the function, set `bow_dictionary` equal to an empty dictionary and return it from the function. This is where we'll be collecting the words and their counts.

Checkpoint 2 Passed

2.

Above the return statement, call the `preprocess_text()` function we created for you on `some_text` and assign the result to the variable `tokens`.

Text preprocessing allows us to count words like "game" and "Games" as the same word token.

Checkpoint 3 Passed

3.

Still above the `return`, iterate over each `token` in `tokens` and check if `token` is already in the `bow_dictionary`.

If it is, increment that token's count by 1. (Remember that each `token`'s count is its corresponding value within the `bow_dictionary`.)

Otherwise, set the count equal to 1 because this is the first time the model has seen that word token.

To iterate through a list in Python:

```
for some_item in list_of_items:
    # do something with some_item
```

Copy to Clipboard

To check if an item exists as a key in a Python dictionary:

```
if some_item in some_dictionary:
    # execute some code
```

Copy to Clipboard

You can set a key-value pair in a dictionary like this:

```
some_dictionary[some_key] = some_value
```

Copy to Clipboard

To increment a dictionary value in Python:

```
some_dictionary[some_key] += 1
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Uncomment the print statement and run the code to see your bag-of-words function in action!

```
from preprocessing import preprocess_text
# Define text_to_bow() below:
def text_to_bow(some_text):
    bow_dictionary = {}
    tokens = preprocess_text(some_text)

    for token in tokens:
        if token in bow_dictionary:
            bow_dictionary[token] += 1
        else:
            bow_dictionary[token] = 1
    return bow_dictionary

print(text_to_bow("I love fantastic flying fish. These flying fish are just ok, so maybe I will
find another few fantastic fish..."))
```

Bag-of-Words Language Model

4 - Introducing BoW Vectors

Sometimes a dictionary just won't fit the bill. Topic modelling applications, for example, require an implementation of bag-of-words that is a bit more mathematical: **feature vectors**.

A feature vector is a numeric representation of an item's important features. Each feature has its own column. If the feature exists for the item, you could represent that with a 1. If the feature does not exist for that item, you could represent that with a 0. A few monsters could be represented as vectors like so:

	has_fangs	melts_in_water	hates_sunlight	has_fur
vampire	1	0	1	0
werewolf	1	0	0	1
witch	0	1	0	0

For bag-of-words, instead of monsters you would have documents and the features would be different words. And we don't just care if a word is present in a document; we want to know how many times it occurred! Turning text into a BoW vector is known as **feature extraction** or **vectorization**.

But how do we know which vector index corresponds to which word? When building BoW vectors, we generally create a **features dictionary** of all vocabulary in our training data (usually several documents) mapped to indices.

For example, with "Five fantastic fish flew off to find faraway functions. Maybe find another five fantastic fish?" our dictionary might be:

```
{'five': 0,  
'fantastic': 1,  
'fish': 2,  
'fly': 3,  
'off': 4,  
'to': 5,  
'find': 6,  
'faraway': 7,  
'function': 8,  
'maybe': 9,  
'another': 10}
```

Copy to Clipboard

Using this dictionary, we can convert new documents into vectors using a vectorization function. For example, we can take a brand new sentence "Another five fish find another faraway fish." — **test data** — and convert it to a vector that looks like:

```
[1, 0, 2, 0, 0, 0, 1, 1, 0, 0, 2]
```

Copy to Clipboard

The word 'another' appeared twice in the test data. If we look at the feature dictionary for 'another', we find that its index is 10. So when we go back and look at our vector, we'd expect the number at index 10 to be 2.

Instructions

In the applet, you'll see a string of text and its corresponding BoW vector trained on the dictionary of terms indicated. Add a word or two to the string from the vocabulary. How does the vector change? Now remove a few words. What happens?

Bag-of-Words Language Model

Building a Features Dictionary

Now that you know what a bag-of-words vector looks like, you can create a function that builds them!

First, we need a way of generating a features

[dictionary](#)

Preview: Docs A dictionary is an unordered set of (key, value) pairs. It provides a way to map pieces of data to each other, and allows for quick access to values associated to keys. The syntax of a dictionary is as follows: pseudo dictionary = { key1: value1, key2: value2, key3: value3

from a list of training documents. We can build a Python function to do that for us...

Instructions

Checkpoint 1 Passed

1.

Define a function `create_features_dictionary()` that takes one argument, `documents`. This will be the list of string documents that we pass in (like `["All the cool fish love to fly high.", "Nobody knows why the fish fly so high.", "Those cool fish sure are spry."]`). Inside the function, set `features_dictionary` equal to an empty dictionary. This is where we'll map all of our terms to index numbers. For now, return `features_dictionary` from the function.

Checkpoint 2 Passed

2.

Above the return statement, merge the `documents` into a string joined together by spaces and assign the result to `merged`.

Now that the documents are all in a single string, call `preprocess_text()` on `merged` and assign the result to `tokens`. Return `tokens` from the function in addition to `features_dictionary`.

Checkpoint 3 Passed

3.

Above the return statement, assign `index` a value of `0`. This will correspond to the first word's vector index.

Checkpoint 4 Passed

4.

The words are prepared, the empty dictionary is prepared, and we have an index number we can use; it's time to get the words into the dictionary and link each to a vector index number!

Above the `return`, loop through each `token` in `tokens`.

In the loop, check if `token` is **NOT** in `features_dictionary`.

If it's a new word, add `token` as a key to `features_dictionary` with a value of `index`.

Checkpoint 5 Passed

5.

After adding `token` to `features_dictionary`, increment `index` by `1` so that each new word has its own index.

Checkpoint 6 Passed

6.

Uncomment the print statement to test out the function!

Bag-of-Words Language Model

Building a BoW Vector

Nice work! Time to put that dictionary of vocabulary to good use and build a bag-of-words vector from a new document. In Python, we can use a list to represent a vector. Each index in the list will correspond to a word and be set to its count.

Help my fly fish fly away

Vector of Test Document

0	0	0	0	0	0	0
---	---	---	---	---	---	---

Features Dictionary

all	my	fish	fly	away	help	me
0	1	2	3	4	5	6

Instructions

Checkpoint 1 Passed

1.

Define a function `text_toBow_vector()` with two parameters:

`some_text` (the document we pass in to vectorize)

`features_dictionary` (the dictionary of vocabulary we generated in the previous exercise)

Create a list of 0s the length of `features_dictionary` and assign it to the variable `bow_vector`. Each 0 represents a word's count within the vector.

Return `bow_vector` from the function.

You can create a list of repeated elements of a given length like this:

```
list_of_5_xs = 5 * ["x"]
print(list_of_5_xs)
# ["x", "x", "x", "x", "x"]
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Above the return statement, preprocess the `some_text` document using the `preprocess_text()` function we built for you and assign the result to the variable `tokens`. Add `tokens` as a second return value for the function.

Checkpoint 3 Passed

3.

Still above the return statement, loop through each `token` in `tokens`.

Determine which index the `token` has within `features_dictionary` and assign the value to a new variable `feature_index`. (Take a look at the gif. If `token` is the word `fish`, then we would want `feature_index` to be 2.)

Now that you have the word's index, access the word count index within the `bow_vector` and increment that count by 1.

Checkpoint 4 Passed

4.

Uncomment the print statement to test out the function!

```
from preprocessing import preprocess_text
# Define text_toBow_vector() below:
def text_toBow_vector(some_text, features_dictionary):
    bow_vector = [0] * len(features_dictionary)
    tokens = preprocess_text(some_text)
    for token in tokens:
        feature_index = features_dictionary[token]
        bow_vector[feature_index] += 1
    return bow_vector, tokens
```

```
features_dictionary = {'function': 8, 'please': 14, 'find': 6, 'five': 0, 'with': 12,
'fantastic': 1, 'my': 11, 'another': 10, 'a': 13, 'maybe': 9, 'to': 5, 'off': 4, 'faraway':
7, 'fish': 2, 'fly': 3}

text = "Another five fish find another faraway fish."
print(text_to_bow_vector(text, features_dictionary)[0])
```

Bag-of-Words Language Model

It's All in the Bag

Phew! That was a lot of work.

It's time to put `create_features_dictionary()` and `tokens_to_bow_vector()` together and use them in a spam filter we created that uses a Naive Bayes classifier. We've slightly modified the two functions for this use case, but they should still look familiar.

Let's see `create_features_dictionary()` and `tokens_to_bow_vector()` in action with real test data, helping fend off spam!

Instructions

Checkpoint 1 Passed

1.

Below `tokens_to_bow_vector()`, call `create_features_dictionary()` on `training_doc_tokens` and assign the result to `bow_sms_dictionary`.

Checkpoint 2 Passed

2.

Define `training_vectors` as a list comprehension. The list comprehension should call `tokens_to_bow_vector()` on `training_doc` and `bow_sms_dictionary` for each `training_doc` in `training_spam_docs`.

Checkpoint 3 Passed

3.

Define `test_vectors` as a list comprehension that calls `tokens_to_bow_vector()` on `test_doc` and `bow_sms_dictionary` for each `test_doc` in `test_spam_docs`.

Checkpoint 4 Passed

4.

Ready? Get set! Uncomment the code at the bottom of **script.py** and run the code again!

The Naive Bayes classifier was pretty darn accurate in determining which messages were spam by using your bag-of-words functions!

```
from spam_data import training_spam_docs, training_doc_tokens, training_labels, test_labels,
test_spam_docs, training_docs, test_docs
from sklearn.naive_bayes import MultinomialNB

def create_features_dictionary(document_tokens):
    features_dictionary = {}
    index = 0
    for token in document_tokens:
        if token not in features_dictionary:
            features_dictionary[token] = index
            index += 1
    return features_dictionary

def tokens_to_bow_vector(document_tokens, features_dictionary):
    bow_vector = [0] * len(features_dictionary)
    for token in document_tokens:
        if token in features_dictionary:
            feature_index = features_dictionary[token]
            bow_vector[feature_index] += 1
    return bow_vector

# Define bow_sms_dictionary:
bow_sms_dictionary = create_features_dictionary(training_doc_tokens)
```



```
# Define training_vectors:
training_vectors = [tokens_to_bow_vector(training_doc, bow_sms_dictionary) for training_doc in
training_spam_docs]

# Define test_vectors:
test_vectors = [tokens_to_bow_vector(test_doc, bow_sms_dictionary) for test_doc in
test_spam_docs]

spam_classifier = MultinomialNB()

def spam_or_not(label):
    return "spam" if label else "not spam"

# Uncomment the code below when you're done:
spam_classifier.fit(training_vectors, training_labels)

predictions = spam_classifier.score(test_vectors, test_labels)

print("The predictions for the test data were {0}% accurate.\n\nFor example, '{1}' was classified
as {2}.\n\nMeanwhile, '{3}' was classified as {4}.".format(predictions * 100, test_docs[0],
spam or not(test_labels[0]), test_docs[10], spam or not(test_labels[10])))
```

Bag-of-Words Language Model

Spam A Lot No More

Amazing work! As is the case with many tasks in Python, there's already a library that can do all of that work for you. For `text_to_bow()`, you can approximate the functionality with the `collections` module's `Counter()` function:

```
from collections import Counter

tokens = ['another', 'five', 'fish', 'find', 'another', 'faraway', 'fish']
print(Counter(tokens))

# Counter({'fish': 2, 'another': 2, 'find': 1, 'five': 1, 'faraway': 1})
```

Copy to Clipboard

For vectorization, you can use `CountVectorizer` from the [machine learning](#)

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming. library `scikit-learn`. You can use `fit()` to train the features [dictionary](#)

Preview: Docs Loading link description

and then `transform()` to transform text into a vector:

```
from sklearn.feature_extraction.text import CountVectorizer

training_documents = ["Five fantastic fish flew off to find faraway functions.",
"Maybe find another five fantastic fish?", "Find my fish with a function please!"]
test_text = ["Another five fish find another faraway fish."]
bow_vectorizer = CountVectorizer()
bow_vectorizer.fit(training_documents)
bow_vector = bow_vectorizer.transform(test_text)
print(bow_vector.toarray())
# [[2 0 1 1 2 1 0 0 0 0 0 0 0 0 0]]
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Now, let's see how scikit-learn stacks up with the same bag-of-words functionality! Import `CountVectorizer` from `sklearn`. (Check out the example we gave for how to import `CountVectorizer`.)

Checkpoint 2 Passed

2.

Define `bow_vectorizer` as our vectorizer using `CountVectorizer()`.

Check the `sklearn` example for help defining `bow_vectorizer`.

Checkpoint 3 Passed

3.

Define `training_vectors` as `bow_vectorizer.fit_transform()` called on `training_docs`.

`fit_transform()` does two things: creation of the features dictionary and the vectorization of the training data.

Define `test_vectors` as `bow_vectorizer.transform()` called on `test_docs`.

```
# for training data:
vector_from_training_data = vectorizer.fit_transform(training_data)

# for test data:
vector_from_test_data = vectorizer.transform(test_data)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Uncomment the code at the bottom of `script.py`. Run the code again to see why it makes sense to use `sklearn`'s optimized functions!

```
from spam_data import training_spam_docs, training_doc_tokens, training_labels,
test_labels, test_spam_docs, training_docs, test_docs
from sklearn.naive_bayes import MultinomialNB
# Import CountVectorizer from sklearn:
from sklearn.feature_extraction.text import CountVectorizer

# Define bow_vectorizer:
bow_vectorizer = CountVectorizer()

# Define training_vectors:
training_vectors = bow_vectorizer.fit_transform(training_docs)
# Define test_vectors:
test_vectors = bow_vectorizer.transform(test_docs)

spam_classifier = MultinomialNB()

def spam_or_not(label):
    return "spam" if label else "not spam"

# Uncomment the code below when you're done:
spam_classifier.fit(training_vectors, training_labels)

predictions = spam_classifier.score(test_vectors, test_labels)

print("The predictions for the test data were {0}% accurate.\n\nFor example, '{1}' was
classified as {2}.\n\nMeanwhile, '{3}' was classified as {4}.".format(predictions * 100,
test_docs[7], spam_or_not(test_labels[7]), test_docs[15], spam_or_not(test_labels[15])))
```

Bag-of-Words Language Model

BoW Wow

As you can see, bag-of-words is pretty useful! BoW also has several advantages over other language models. For one, it's an easier model to get started with and a few Python libraries already have built-in support for it.

Because bag-of-words relies on single words, rather than sequences of words, there are more examples of each unit of language in the training corpus. More examples means the model has less **data sparsity** (i.e., it has more training knowledge to draw from) than other statistical models.

Imagine you want to make a shirt to sell to people. If you have the shirt exactly tailored to someone's body, it probably won't fit that many people. But if you make a shirt that is just a giant bag with arm holes, you know that no one will buy it. What do you do? You loosely fit the shirt to someone's body, leaving some extra room for different body shapes.

Overfitting (adapting a model too strongly to training data, akin to our highly tailored shirt) is a common problem for statistical language models. While BoW still suffers from overfitting in terms of vocabulary, it overfits less than other statistical models, allowing for more flexibility in grammar and word choice.

The combination of low data sparsity and less overfitting makes the bag-of-words model more reliable with smaller training data sets than other statistical models.

Instructions

Checkpoint 1 Passed

1.

The bigrams model is another statistical model that is helpful for tasks like text prediction. However, it's not always ideal when you want to determine the topic of a given text.

Read the short `text` in the code and then run the code as is to see the most common bigrams.

Checkpoint 2 Passed

2.

Because of data sparsity, each bigram only has a single occurrence. As a result, the bigram model alone is not great at making predictions about topic or sentiment. Let's see how the bag-of-words model does...

Define `bag_of_words` in the code editor as `Counter()` called on `tokens`.

Checkpoint 3 Passed

3.

At the end of `script.py`, let's print out the three most frequently occurring words in the text. You can find the most common words by calling the `Counter` method `.most_common()` on `bag_of_words` and passing in a number — in this case `3` — as an argument. Save the result to `most_common_three` and print the result.

Can you tell what the topic is by looking at the bag of words model?

```
from preprocessing import preprocess_text

from nltk.util import ngrams

from collections import Counter

text = "It's exciting to watch flying fish after a hard day's work. I don't know why some fish prefer flying and other fish would rather swim. It seems like the fish just woke up one day and decided, 'hey, today is the day to fly away.'"

tokens = preprocess_text(text)

# Bigram approach:

bigrams_prepped = ngrams(tokens, 2)

bigrams = Counter(bigrams_prepped)

print("Three most frequent word sequences and the number of occurrences according to Bigrams:")

print(bigrams.most_common(3))
```

```
# Bag of Words approach:

# Define bag_of_words here:

bag_of_words = Counter(tokens)

print("\nThree most frequent words and number of occurrences according to Bag of Words:")

most_common_three = bag_of_words.most_common(3)

print(most_common_three)
```

Bag-of-Words Language Model

BoW Ow

Alas, there is a trade-off for all the brilliance BoW brings to the table.

Unless you want sentences that look like “the a but for the”, BoW is NOT a great primary model for text prediction. If that sort of “sentence” isn’t your bag, it’s because bag-of-words has high *perplexity*, meaning that it’s not a very accurate model for language prediction. The probability of the following word is always just the most frequently used words.

If your BoW model finds “good” frequently occurring in a text sample, you might assume there’s a positive sentiment being communicated in that text... but if you look at the original text you may find that in fact every “good” was preceded by a “not.”

Hmm, that would have been helpful to know. The BoW model’s word tokens lack context, which can make a word’s intended meaning unclear.

Perhaps you are wondering, “What happens if the model comes across a new word that wasn’t in the training data?” As mentioned, like all statistical models, BoW suffers from overfitting when it comes to vocabulary.

There are several ways that NLP developers have tackled this issue. A common approach is through *language smoothing* in which some probability is siphoned from the known words and given to unknown words.

Instructions

Checkpoint 1 Passed

1.

Run the code as is to see a made-up Oscar Wilde passage using a training text written by him. The tool currently uses the *n*-gram statistical model with a word sequence length of 3 (e.g., “I made the”) to make predictions.

Checkpoint 2 Failed, try again

2.

Change `sequence_length` to 1 so that we use the bag-of-words model. For the purposes of this function, we’ll pick each next word randomly from the 20 most common words.

Run the code again to see how bag of words compares with the *n*-gram model on language prediction. Not so great, right?

```
import nltk, re, random
from nltk.tokenize import word_tokenize
from collections import defaultdict, deque, Counter
from document import oscar_wilde_thoughts

# Change sequence_length:
sequence_length = 1

class MarkovChain:
    def __init__(self):
        self.lookup_dict = defaultdict(list)
        self.most_common = []
        self._seeded = False
        self.__seed_me()

    def __seed_me(self, rand_seed=None):
        if self._seeded is not True:
            try:
                if rand_seed is not None:
```

```

        random.seed(rand_seed)
    else:
        random.seed()
    self._seeded = True
except NotImplementedError:
    self._seeded = False

def add_document(self, str):
    preprocessed_list = self._preprocess(str)
    self.most_common = Counter(preprocessed_list).most_common(20)
    pairs = self.__generate_tuple_keys(preprocessed_list)
    for pair in pairs:
        self.lookup_dict[pair[0]].append(pair[1])

def _preprocess(self, str):
    cleaned = re.sub(r'\W+', ' ', str).lower()
    tokenized = word_tokenize(cleaned)
    return tokenized

def __generate_tuple_keys(self, data):
    if len(data) < sequence_length:
        return

    for i in range(len(data) - 1):
        yield [ data[i], data[i + 1] ]

def generate_text(self, max_length=50):
    context = deque()
    output = []
    if len(self.lookup_dict) > 0:
        self.__seed_me(rand_seed=len(self.lookup_dict))
        chain_head = [list(self.lookup_dict)[0]]
        context.extend(chain_head)
        if sequence_length > 1:
            while len(output) < (max_length - 1):
                next_choices = self.lookup_dict[context[-1]]
                if len(next_choices) > 0:
                    next_word = random.choice(next_choices)
                    context.append(next_word)
                    output.append(context.popleft())
                else:
                    break
            output.extend(list(context))
        else:
            while len(output) < (max_length - 1):
                next_choices = [word[0] for word in self.most_common]
                next_word = random.choice(next_choices)
                output.append(next_word)
    return " ".join(output)

my_markov = MarkovChain()
my_markov.add_document(oscar_wilde_thoughts)
random_oscar_wilde = my_markov.generate_text()
print(random_oscar_wilde)

```

Bag-of-Words Language Model

Review of Bag-of-Words

You made it! And you've learned plenty about the bag-of-words language model along the way:

Bag-of-words (BoW) — also referred to as the unigram model — is a statistical language model based on word count.

There are loads of real-world applications for BoW.

BoW can be implemented as a Python

[dictionary](#)

Preview: Docs Loading link description

with each key set to a word and each value set to the number of times that word appears in a text.

For BoW, training data is the text that is used to build a BoW model.

BoW test data is the new text that is converted to a BoW vector using a trained features dictionary.

A feature vector is a numeric depiction of an item's salient features.

Feature extraction (or vectorization) is the process of turning text into a BoW vector.

A features dictionary is a mapping of each unique word in the training data to a unique

[index](#)

Preview: Docs Loading link description

. This is used to build out BoW vectors.

BoW has less data sparsity than other statistical models. It also suffers less from overfitting.

BoW has higher perplexity than other models, making it less ideal for language prediction.

One solution to overfitting is language smoothing, in which a bit of probability is taken from known words and allotted to unknown words.

The spam data for this lesson were taken from the [UCI Machine Learning Repository](#).

Dua, D. and Karra Taniskidou, E. (2017). UCI

[Machine Learning](#)

Preview: Docs Loading link description

Repository [archive.ics.uci.edu/ml/]. Irvine, CA: University of California, School of Information and Computer Science.

Instructions

Feel free to play around with the spam data using your new BoW knowledge. You can also use this workspace to try out other bag-of-words use cases.

```
from spam_data import training_docs, training_doc_tokens, training_labels, training_docs
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

test_text = """
Play around with the spam classifier!
"""

bow_vectorizer = CountVectorizer()

training_vectors = bow_vectorizer.fit_transform(training_docs)
test_vectors = bow_vectorizer.transform([test_text])

spam_classifier = MultinomialNB()
spam_classifier.fit(training_vectors, training_labels)

predictions = spam_classifier.predict(test_vectors)

print("Looks like a normal email!" if predictions[0] == 0 else "You've got spam!")
```

Working with Text Data | scikit-learn

Working with Text Data | scikit-learn

In this documentation, you will learn about the scikit-learn tools for generating language models like bag-of-words using text data. This is helpful if you would like to analyze a collection of text documents.

Term Frequency–Inverse Document Frequency

Introduction

It's a dark night in the middle of winter as you make your way through another of Emily Dickinson's poems. As you grapple with questions of immortality and death, you notice the word choice in each poem you read. With each passing poem, you discover for yourself which words are common throughout her work, and which indicate more unique meaning in individual poems.

You might not even realize, but you are building a language model in your head similar to **term frequency-inverse document frequency**, commonly known as **tf-idf**. Tf-idf is another powerful tool in your NLP toolkit that has a variety of use cases included:

- ranking results in a search engine
- text summarization
- building smarter chatbots

Instructions

The gif on the right showcases an example of applying tf-idf to a set of documents. The output of applying tf-idf is the table shown, also known as a term-document matrix. You can think of a term-document matrix like a matrix of bag-of-word vectors.

Each column of the table represents a unique document (in this case, an individual sentence). Each row represents a unique word token. The value in each cell represents the tf-idf score for a word token in that particular document.

Proceed to the next exercise to learn more about what tf-idf is and how you can calculate it!



Term Frequency–Inverse Document Frequency

What is Tf-idf?

Term frequency-inverse document frequency is a numerical statistic used to indicate how important a word is to each document in a collection of documents, or a corpus.

When applying tf-idf to a corpus, each word is given a tf-idf score for each document, representing the relevance of that word to the particular document. A higher tf-idf score indicates a term is more important to the corresponding document.

Tf-idf has many similarities with the [bag-of-words language model](#), which if you recall is concerned with word count — how many times each word appears in a document.

While tf-idf can be used in any situation bag-of-words can be used, there is a key difference in how it is calculated.

Tf-idf relies on two different metrics in order to come up with an overall score:

term frequency, or how often a word appears in a document. This is the same as bag-of-words' word count.

inverse document frequency, which is a measure of how often a word appears in the overall corpus. By penalizing the score of words that appear throughout a corpus, tf-idf can give better insight into how important a word is to a particular document of a corpus.

We will dig into each component of tf-idf in the next two exercises.

Instructions

Checkpoint 1 Passed

1.

The code in **script.py** is used to apply tf-idf to a corpus of documents and display the scores as a DataFrame in the web browser component. The current code is incomplete, so the DataFrame is not yet appearing!

Like with many natural language processing tasks, we must preprocess our text data before applying a technique. Paste the below line of code into the “preprocess documents” section of **script.py** to preprocess each document in the corpus. Then run the code.

```
processed_corpus = [preprocess_text(doc) for doc in corpus]
```

Copy to Clipboard

Paste the following line of code beneath the “preprocess documents” comment in *script.py*:

```
processed_corpus = [preprocess_text(doc) for doc in corpus]
```

Copy to Clipboard

Checkpoint 2 Passed

2.

You should now see what is known as a *term-document matrix* in the browser. Each row of the table represents a term, and each column of the table represents a document. The value in each cell indicates the tf-idf score for each term-document pair.

Try changing the text in `document_1`, `document_2` and `document_3` and re-running the code. How do the tf-idf values change?

Make sure to change the text of **at least** one document.

If a term appears more frequently in a document, the tf-idf score for that term-document pair will go up.

If a term appears in multiple documents, the tf-idf score for the term across all documents will go down.

```
import codecademylib3_seaborn
from preprocessing import preprocess_text
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

# sample documents
document_1 = "This is a sample piece of text!"
document_2 = "This is my second sentence of text."
document_3 = "Is this my third sentence of text?"

# corpus of documents
corpus = [document_1, document_2, document_3]

# preprocess documents
processed_corpus = [preprocess_text(doc) for doc in corpus]

# initialize and fit TfidfVectorizer
vectorizer = TfidfVectorizer(norm=None)
tf_idf_scores = vectorizer.fit_transform(processed_corpus)

# get vocabulary of terms
feature_names = vectorizer.get_feature_names()
corpus_index = [n for n in processed_corpus]

# create pandas DataFrame with tf-idf scores
df_tf_idf = pd.DataFrame(tf_idf_scores.T.todense(), index=feature_names,
                          columns=corpus_index)
print(df_tf_idf)
```

```
import nltk, re
from nltk.corpus import wordnet
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from collections import Counter

stop_words = stopwords.words('english')
normalizer = WordNetLemmatizer()

def get_part_of_speech(word):
    probable_part_of_speech = wordnet.synsets(word)
    pos_counts = Counter()
    pos_counts["n"] = len([item for item in probable_part_of_speech if item.pos()=="n"])
```



```
pos_counts["v"] = len([item for item in probable_part_of_speech if item.pos()=="v"])
pos_counts["a"] = len([item for item in probable_part_of_speech if item.pos()=="a"])
pos_counts["r"] = len([item for item in probable_part_of_speech if item.pos()=="r"])
most_likely_part_of_speech = pos_counts.most_common(1)[0][0]
return most_likely_part_of_speech

def preprocess_text(text):
    cleaned = re.sub(r'\W+', ' ', text).lower()
    tokenized = word_tokenize(cleaned)
    normalized = " ".join([normalizer.lemmatize(token, get_part_of_speech(token)) for token in tokenized])
    return normalized
```

Term Frequency–Inverse Document Frequency

Breaking It Down Part I: Term Frequency

The first component of tf-idf is *term frequency*, or how often a word appears in a document within the corpus. The value for the term frequency is the same as if applying the bag-of-words language model to a document. If you have previously studied bag-of-words, this will all be familiar! If not, have no fear. Term frequency indicates how often each word appears in the document. The intuition for including term frequency in the tf-idf calculation is that the more frequently a word appears in a single document, the more important that term is to the document.

Consider the stanza from Emily Dickinson’s poem *I’m Nobody! Who are you?* below:

```
stanza = '''I'm nobody! Who are you?
Are you nobody, too?
Then there's a pair of us — don't tell!
They'd banish us, you know.'''
```

Copy to Clipboard

The term frequency for “you” is 3, “nobody” is 2, “are” is 2, “us” is 2, and the rest of the terms have a frequency of 1. We can get a general sense of what this stanza is about by the most frequently used words.

Term frequency can be calculated in Python using scikit-learn’s `CountVectorizer`, as shown below:

```
vectorizer = CountVectorizer()
term_frequencies = vectorizer.fit_transform([stanza])
```

Copy to Clipboard

A `CountVectorizer` object is initialized

The `CountVectorizer` object is fit (trained) and transformed (applied) on the corpus of data, returning the term frequencies for each term-document pair

Instructions

Checkpoint 1 Passed

1.

Provided in `script.py` is Emily Dickinson’s poem *Success is counted sweetest*, stored in the variable `poem`. Read through `poem` yourself and find the term-frequency of “clear”. Save your answer, as an integer, to a variable named `clear_count`.

The term “clear” appears once in the second stanza, “So **clear**, of victory,” and once in the third stanza, “Break, agonized and **clear**!”

The term-frequency for clear is thus 2.

Checkpoint 2 Passed

2.

Reading through each word of a document to count how many times it appears isn’t too efficient. The code in `script.py` preprocesses the poem and then uses scikit-learn’s `CountVectorizer` to get the term-frequencies for all terms in `poem`. It’s currently missing one line of code to display the term-document matrix of term-frequencies.

`CountVectorizer` objects have a `.get_feature_names()` method which returns a list of all the unique terms in the corpus.

Paste the below line into the “get vocabulary of terms” section of `script.py` to display the term-frequencies matrix.

```
feature_names = vectorizer.get_feature_names()
```

Copy to Clipboard

Which term appears the most frequently?

Make sure to scroll down through the table to see the term frequency for all terms. “the” appears 4 times, “of” appears 3 times, and “clear”, “to”, and “who” appear 2 times.

This could be a good case for stop word removal!

```
import codecademylib3_seaborn
import pandas as pd
```

```

from sklearn.feature_extraction.text import CountVectorizer
from preprocessing import preprocess_text

poem = '''
Success is counted sweetest
By those who ne'er succeed.
To comprehend a nectar
Requires sorest need.

Not one of all the purple host
Who took the flag to-day
Can tell the definition,
So clear, of victory,

As he, defeated, dying,
On whose forbidden ear
The distant strains of triumph
Break, agonized and clear!'''

# define clear count:
clear count = 2

# preprocess text
processed poem = preprocess_text(poem)

# initialize and fit CountVectorizer
vectorizer = CountVectorizer()
term frequencies = vectorizer.fit_transform([processed poem])

# get vocabulary of terms
feature names = vectorizer.get_feature_names()

# create pandas DataFrame with term frequencies
try:
    df term frequencies = pd.DataFrame(term frequencies.T.todense(),
index=feature names, columns=['Term Frequency'])
    print(df term frequencies)
except:
    pass

```

Term Frequency–Inverse Document Frequency

Breaking It Down Part II: Inverse Document Frequency

The *inverse document frequency* component of the tf-idf score penalizes terms that appear more frequently across a corpus. The intuition is that words that appear more frequently in the corpus give less insight into the topic or meaning of an individual document, and should thus be deprioritized.

For example, terms like “the” or “go” are used all over the place, so in a bag-of-words model, they would be given priority even though they don’t provide much meaning; tf-idf would deprioritize these sorts of common words.

We can calculate the inverse document frequency for some term t across a corpus using the below equation. Don’t be scared if you aren’t a math person!

$$\log\left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t}\right)$$

The important take away from the equation is that as the number of documents with the term t increases, the inverse document frequency decreases (due to the nature of the log function). The more frequently a term appears across the corpus, the less important it becomes to an individual document.

Inverse document frequency can be calculated on a group of documents using scikit-learn’s `TfidfTransformer`:

```

transformer = TfidfTransformer(norm=None)
transformer.fit(term frequencies)

```

```
inverse_doc_frequency = transformer.idf
```

Copy to Clipboard

a `TfidfTransformer` object is initialized. Don't worry about the `norm=None` keyword [argument](#)

Preview: Docs Loading link description

for now, we will dig into this in the next exercise

the `TfidfTransformer` is fit (trained) on a term-document matrix of term frequencies

the `.idf_` attribute of the `TfidfTransformer` stores the inverse document frequencies of the terms as a NumPy

[array](#)

Preview: Docs Loading link description

Instructions

Checkpoint 1 Passed

1.

A selection of 6 Emily Dickinson poems is given in `poems.py`. The term frequencies of each term-document pair are calculated in `term_frequency.py` and stored in `term_frequencies` as a matrix and `df_term_frequencies` as a Pandas DataFrame.

In `script.py`, print `df_term_frequencies` to view the term-document matrix of term frequencies.

Call `print()` with `df_term_frequencies` as an argument:

```
print(df_term_frequencies)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Let's calculate the inverse document frequency for each term in the selection of Emily Dickinson's poems!

Begin by creating a `TfidfTransformer` object named `transformer`.

You can create a `TfidfTransformer` object as shown below:

```
my_transformer = TfidfTransformer()
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Fit `transformer` on `term_frequencies`.

You can fit a `TfidfTransformer` object on a term frequency matrix as shown below:

```
my_transformer.fit(term_frequencies_matrix)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Store the calculated inverse document frequency values in a variable named `idf_values`.

When you run the code, a table of inverse document frequency scores for all the terms will appear. Which terms are penalized for occurring across multiple documents?

Refer back to the poems in `poems.py` to see the original poems.

You can access the calculated inverse document frequency values of a fitted `TfidfTransformer` object as shown below:

```
my_idf_values = my_transformer.idf
```

Copy to Clipboard

"be" and "the" have the lowest inverse document frequency scores, as they appear in each of the poems.

Terms that appear in only one of the 6 poems have the highest inverse document frequency score

(2.2527...).

If you are a math lover and have been calculating the inverse document frequency scores by hand, you will notice that the results provided by scikit-learn are different than the equation we have stated. This is because scikit-learn changes the inverse document frequency calculation slightly to the below equation:

$\log(\text{Total number of documents} + 1 / \text{Number of documents with term } t + 1) + 1$

$\log($

$\text{Number of documents with term } t + 1$

$\text{Total number of documents} + 1$

$) + 1$

Reference the [TfidfTransformer scikit-learn documentation here](#) for more information.

```

import codecademylib3 seaborn
import pandas as pd
from sklearn.feature_extraction.text import TfidfTransformer
from term_frequency import term_frequencies, feature_names, df_term_frequencies

# display term-document matrix of term frequencies
print(df_term_frequencies)

# initialize and fit TfidfTransformer
transformer = TfidfTransformer()
transformer.fit(term_frequencies)
idf_values = transformer.idf_

# create pandas DataFrame with inverse document frequencies
try:
    df_idf = pd.DataFrame(idf_values, index = feature_names, columns=['Inverse Document Frequency'])
    print(df_idf)
except:
    pass

```

Term Frequency–Inverse Document Frequency

Putting It All Together: Tf-idf

Now that we understand how term frequency and inverse document frequency are calculated, let's put it all together to calculate tf-idf!

Tf-idf scores are calculated on a term-document basis. That means there is a tf-idf score for each word, for each document. The tf-idf score for some term t in a document d in some corpus is calculated as follows:

$$tfidf(t,d)=tf(t,d)*idf(t,corpus)$$

$$tfidf(t,d)=tf(t,d)*idf(t,corpus)$$

$tf(t,d)$ is the term frequency of term t in document d

$idf(t,corpus)$ is the inverse document frequency of a term t across corpus

We can easily calculate the tf-idf values for each term-document pair in our corpus using scikit-learn's `TfidfVectorizer`:

```

vectorizer = TfidfVectorizer(norm=None)
tfidf_vectorizer = vectorizer.fit_transform(corpus)

```

Copy to Clipboard

a `TfidfVectorizer` object is initialized. The `norm=None` keyword

argument

Preview: Docs Loading link description

prevents scikit-learn from modifying the multiplication of term frequency and inverse document frequency

the `TfidfVectorizer` object is fit and transformed on the corpus of data, returning the tf-idf scores for each term-document pair

Instructions

Checkpoint 1 Passed

1.

The same selection of 6 Emily Dickinson poems from the previous exercise is given in `poems.py`.

In `script.py`, the poems are preprocessed. Let's calculate the tf-idf scores for each term-document pair.

Begin by creating a `TfidfVectorizer` object named `vectorizer` with keyword argument `norm=None`.

You can create a `TfidfVectorizer` object as shown below:

```
my_vectorizer = TfidfVectorizer(norm=None)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Fit and transform your `vectorizer` on the corpus of preprocessed poems. Save the result to a variable named `tfidf_scores`.

You can calculate the tf-idf scores of a corpus by calling a `TfidfVectorizer`'s `.fit_transform()` method as follows:

```
my_tfidf_scores =
my_vectorizer.fit_transform(corpus_of_documents)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Like `CountVectorizer` objects, `TfidfVectorizer` objects have a `.get_feature_names()` method which returns a list of all the unique terms in the corpus.

Paste the below line into the "get vocabulary of terms" section of `script.py` to display the tf-idf matrix.

```
feature_names = vectorizer.get_feature_names()
```

Copy to Clipboard

Which term-document pairs have the highest tf-idf scores?

Make sure to scroll down and to the right through the table to see the tf-idf scores for all term-document pairs.

The highest tf-idf score 12.48 is for "and" in Poem 4. Maybe we should filter out stop words! Two of the next highest tf-idf scores are more interesting. "night" and "wild" in document 2 have tf-idf scores of 9.01 and 6.76, respectively.

These scores indicate that Poem 2 may be about a "wild night", and much more so than any of the other poems!

Community Support

```
import codecademylib3 seaborn
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from poems import poems
from preprocessing import preprocess_text

# preprocess documents
processed_poems = [preprocess_text(poem) for poem in poems]

# initialize and fit TfidfVectorizer
vectorizer = TfidfVectorizer(norm=None)
tfidf_scores = vectorizer.fit_transform(processed_poems)

# get vocabulary of terms
feature_names = vectorizer.get_feature_names()

# get corpus index
corpus_index = [f"Poem {i+1}" for i in range(len(poems))]

# create pandas DataFrame with tf-idf scores
try:
    df_tf_idf = pd.DataFrame(tfidf_scores.T.todense(), index=feature_names,
                             columns=corpus_index)
    print(df_tf_idf)
except:
    pass
```

Term Frequency-Inverse Document Frequency

Converting Bag-of-Words to Tf-idf

In addition to directly calculating the tf-idf scores for a set of terms across a corpus, you can also convert a bag-of-words model you have already created into tf-idf scores.

Scikit-learn's `TfidfTransformer` is up to the task of converting your bag-of-words model to tf-idf. You begin by initializing a `TfidfTransformer` object.

```
tf_idf_transformer = TfidfTransformer(norm=False)
```

Copy to Clipboard

Given a bag-of-words matrix `count_matrix`, you can now multiply the term frequencies by their inverse document frequency to get the tf-idf scores as follows:

```
tf_idf_scores = tfidf_transformer.fit_transform(count_matrix)
```

Copy to Clipboard

This is very similar to how we calculated inverse document frequency, except this time we are fitting *and* transforming the `TfidfTransformer` to the term frequencies/bag-of-words vectors rather than just fitting the `TfidfTransformer` to them.

Instructions

Checkpoint 1 Passed

1.

Consider one last time the same selection of 6 Emily Dickinson poems given in `poems.py`. The term frequencies of each term-document pair are calculated in `term_frequency.py` and stored in `bow_matrix` as a matrix and `df_bag_of_words` as a Pandas DataFrame.

In `script.py`, print `df_bag_of_words` to view the bag-of-words matrix (term-document matrix of term frequencies).

Call `print()` with `df_bag_of_words` as an argument:

```
print(df bag of words)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Initialize a `TfidfTransformer` object named `transformer` with keyword argument `norm=None`.

You can create a `TfidfTransformer` object as shown below:

```
my_vectorizer = TfidfTransformer(norm=None)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Use `transformer` to fit and transform the bag-of-words matrix `bow_matrix` into tf-idf scores. Save your result to `tfidf scores`.

You can transform a bag-of-words matrix into tf-idf scores using `TfidfTransformer` as follows:

```
my_tf_idf_scores = my_transformer.fit_transform(my_bow_matrix)
```

```
import codecademylib3 seaborn
import pandas as pd
from sklearn.feature_extraction.text import TfidfTransformer
from term frequency import bow_matrix, feature_names, df bag of words, corpus_index

# display term-document matrix of term frequencies (bag-of-words)
print(df bag of words)

# initialize and fit TfidfTransformer, transform bag-of-words matrix
transformer = TfidfTransformer(norm=None)
tfidf_scores = transformer.fit_transform(bow_matrix)

# create pandas DataFrame with tf-idf scores
try:
    df_tf_idf = pd.DataFrame(tfidf_scores.T.todense(), index = feature_names, columns=corpus_index)
    print(df_tf_idf)
except:
    pass
```

TEXT - TRAINING DATA

```
poem_1 = '''
Success is counted sweetest
By those who ne'er succeed.
To comprehend a nectar
Requires sorest need.

Not one of all the purple host
Who took the flag to-day
Can tell the definition,
So clear, of victory,

As he, defeated, dying,
On whose forbidden ear
The distant strains of triumph
Break, agonized and clear!'''

poem_2 = '''
Wild nights! Wild nights!
Were I with thee,
Wild nights should be
Our luxury!

Futile the winds
To a heart in port, -
Done with the compass,
Done with the chart.

Rowing in Eden!
Ah! the sea!
Might I but moor
To-night in thee!'''

poem_3 = '''
I'm nobody! Who are you?
Are you nobody, too?
Then there 's a pair of us - don't tell!
They 'd banish us, you know.

How dreary to be somebody!
How public, like a frog
To tell your name the livelong day
To an admiring bog!'''

poem_4 = '''
```

```

I felt a funeral in my brain,
And mourners, to and fro,
Kept treading, treading, till it seemed
That sense was breaking through.

And when they all were seated,
A service like a drum
Kept beating, beating, till I thought
My mind was going numb.

And then I heard them lift a box,
And creak across my soul
With those same boots of lead, again.
Then space began to toll

As all the heavens were a bell,
And Being but an ear,
And I and silence some strange race,
Wrecked, solitary, here.'''

poem 5 = '''
Hope is the thing with feathers
That perches in the soul,
And sings the tune without the words,
And never stops at all,

And sweetest in the gale is heard;
And sore must be the storm
That could abash the little bird
That kept so many warm.

I 've heard it in the chillest land,
And on the strangest sea;
Yet, never, in extremity,
It asked a crumb of me.'''

poem 6 = '''
The pedigree of honey
Does not concern the bee;
A clover, any time, to him
Is aristocracy.'''

poems = [poem 1, poem 2, poem 3, poem 4, poem 5, poem 6]

```

Term Frequency–Inverse Document Frequency

Review

```

"Hope is the thing with feathers
That perches in the soul
And sings the tune without the words
And never stops at all."

```

Copy to Clipboard

So goes Emily Dickinson's poem *Hope is the thing with feathers*. And just as Emily proclaims, your hope and perseverance have taken you to the end of this lesson!

Let's recount all you have learned:

Term frequency-inverse document frequency, known as tf-idf, is a numerical statistic used to indicate how important a word is to each document in a collection of documents

tf-idf consists of two components, term frequency and inverse document frequency

term frequency is how often a word appears in a document. This is the same as bag-of-words' word count

inverse document frequency is a measure of how often a word appears across all documents of a corpus

tf-idf is calculated as the term frequency multiplied by the inverse document frequency

term frequency, inverse document frequency, and tf-idf can be calculated in scikit-learn using the

`CountVectorizer`, `TfidfTransformer`, and `TfidfVectorizer` objects, respectively

Let's practice what you've learned about tf-idf using the work of another poet with an inclination for the dreary.

Instructions

Checkpoint 1 Passed

1.

"Once upon a review exercise dreary..." begins Edgar Allen Poe's *The Raven* (or so it almost does).

We can use Poe's classic poem to demonstrate another means of defining documents in a tf-idf analysis. Rather than use different poems as their own documents, we can consider each stanza of *The Raven* as its own document and try to gain insight into the meaning and insight of the individual stanzas.

In `raven.py` *The Raven* is broken down to individual stanzas and stored in `the_raven_stanzas`.

Print `the_raven_stanzas[0]` to view the first stanza.

Checkpoint 2 Passed

2.

In **script.py** the stanzas of `the_raven_stanzas` are preprocessed. Let's calculate the tf-idf scores for each term-document pair.

Begin by creating a `TfidfVectorizer` object named `vectorizer` with keyword argument `norm=None`.

Checkpoint 3 Passed

3.

Fit and transform your `vectorizer` on the corpus of preprocessed stanzas. Save the result to a variable named `tfidf_scores`.

Checkpoint 4 Passed

4.

Now you just need to get the vocabulary of unique terms used in *The Raven*.

Paste the below line into the "get vocabulary of terms" section of **script.py** to display the tf-idf matrix.

```
feature_names = vectorizer.get_feature_names()
```

Copy to Clipboard

Which stanzas share similarly high/low tf-idf scores for the same terms?

Make sure to scroll down and to the right through the table to see the tf-idf scores for all term-document (in this case term-stanza) pairs.

```
import codecademylib3_seaborn
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from raven import the_raven_stanzas
from preprocessing import preprocess_text

# view first stanza
print(the_raven_stanzas[0])

# preprocess documents
processed_stanzas = [preprocess_text(stanza) for stanza in the_raven_stanzas]

# initialize and fit TfidfVectorizer
vectorizer = TfidfVectorizer(norm=None)
tfidf_scores = vectorizer.fit_transform(processed_stanzas)

# get vocabulary of terms
feature_names = vectorizer.get_feature_names()

# get stanza index
stanza_index = [f"Stanza {i+1}" for i in range(len(the_raven_stanzas))]

# create pandas DataFrame with tf-idf scores
try:
    df_tf_idf = pd.DataFrame(tfidf_scores.T.todense(), index=feature_names,
                             columns=stanza_index)
    print(df_tf_idf)
except:
    pass
```

Working with Text Data | scikit-learn | From Occurrences to Frequencies

Working with Text Data | scikit-learn | From Occurrences to Frequencies

In this documentation, you will learn how to use scikit-learn to conduct tf-idf. This is helpful if you are trying to determine topics or themes within text data.

Fill in the blanks such that the tf-idf scores of all terms across the corpus `song` are calculated.

```
doc_1 = 'In the beginning, there was Jack, and Jack had a groove.'
doc_2 = 'And from this groove came the groove of all grooves.'
doc_3 = 'And while one day viciously throwing down on his box, Jack
boldy declared,'
doc_4 = 'Let there be HOUSE!'

song = [doc_1, doc_2, doc_3, doc_4]

vectorizer = TfidfVectorizer()
tfidf_scores = vectorizer.fit_transform(song)
```

Tfidf for Topic modeling

Word Embeddings

Introduction

There is a famous saying by German writer Johann Wolfgang von Goethe:

"Tell me with whom you consort and I will tell you who you are..."

Copy to Clipboard

Goethe's assumption is that the people you spend time with each and every day are a reflection of who you are as a person. Would you agree?

Now, what if we extend that same idea from people to words? Linguist John Rupert Firth took this step and came up with his own saying:

"You shall know a word by the company it keeps."

Copy to Clipboard

This idea that a word's meaning can be understood by its context, or the words that surround it, is the basis for word embeddings. *A word embedding is a representation of a word as a numeric vector, enabling us to compare and contrast how words are used and identify words that occur in similar contexts.*

The applications of word embeddings include:

- entity recognition in chatbots
- sentiment analysis
- syntax parsing

Have little to no experience with vectors? Do not fear! We'll break down word embeddings and how they relate to vectors, step-by-step, in the next few exercises with the help of the [spaCy package](#). Let's get started!

Instructions

1. In **script.py** we have loaded the spaCy package and processed a collection of words into word embeddings. We then compare the embeddings for two pairs:

```
"summer" and "winter"
"mustard" and "amazing"
```

What do you notice about the distance between words with similar usages and words with very different usages? Uncomment some of the print statements at the bottom of the file and run the code to see what a word embedding looks like. Soon you will learn how to load, create, and compare word embeddings just like these!

```
import spacy
from scipy.spatial.distance import cosine

# load model
nlp = spacy.load('en')

# define vectors
summer_vec = nlp("summer").vector
winter_vec = nlp("winter").vector

# compare similarity
print(f"The cosine distance between the word embeddings for 'summer' and 'winter' is: {cosine(summer_vec, winter_vec)}\n")

# define vectors
mustard_vec = nlp("mustard").vector
amazing_vec = nlp("amazing").vector

# compare similarity
print(f"The cosine distance between the word embeddings for 'mustard' and 'amazing' is: {cosine(mustard_vec, amazing_vec)}\n")

# display word embeddings
# print(f"'summer' in vector form: {summer_vec}")
# print(f"'winter' in vector form: {winter_vec}")
# print(f"'mustard' in vector form: {mustard_vec}")
# print(f"'amazing' in vector form: {amazing_vec}")
```

Output-only Terminal

Output:

The cosine distance between the word embeddings for 'summer' and 'winter' is:
0.20887523889541626

The cosine distance between the word embeddings for 'mustard' and 'amazing' is:
0.6902923285961151

Word Embeddings

Vectors

Vectors can be many things in many different fields, but ultimately they are containers of information. Depending on the size, or the dimension, of a vector, it can hold varying amounts of data.

The simplest case is a 1-dimensional vector, which stores a single number. Say we want to represent the length of a word with a vector. We can do so as follows:

```
"cat" ----> [3]
"scrabble" ----> [8]
"antidisestablishmentarianism" ----> [28]
```

Copy to Clipboard

Instead of looking at these three words with our own eyes, we can compare the vectors that represent them by plotting the vectors on a number line.



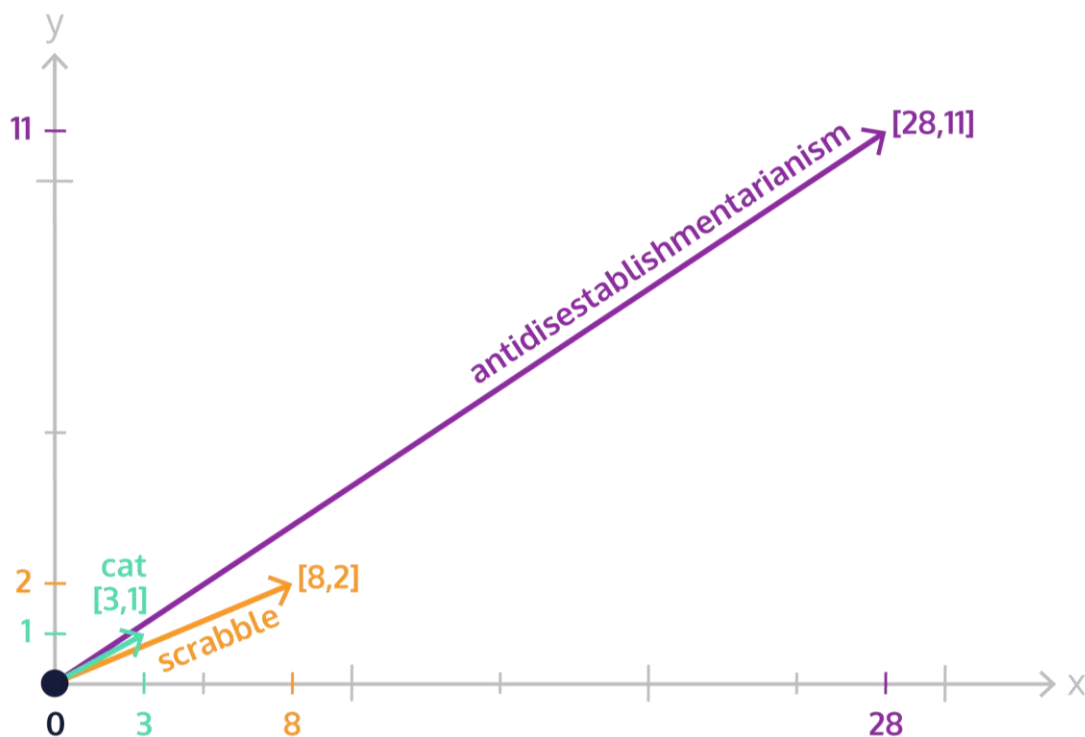
We can clearly see that the “cat” vector is much smaller than the “scrabble” vector, which is much smaller than the “antidisestablishmentarianism” vector.

Now let’s say we also want to record the number of vowels in our words, in addition to the number of letters. We can do so using a 2-dimensional vector, where the first entry is the length of the word, and the second entry is the number of vowels:

```
"cat" ----> [3, 1]
"scrabble" ----> [8, 2]
"antidisestablishmentarianism" ----> [28, 11]
```

[Copy to Clipboard](#)

To help visualize these vectors, we can plot them on a two-dimensional grid, where the x-axis is the number of letters, and the y-axis is the number of vowels.



Here we can see that the vectors for “cat” and “scrabble” point to a more similar area of the grid than the vector for “antidisestablishmentarianism”. So we could argue that “cat” and “scrabble” are closer together.

While we have shown here only 1-dimensional and 2-dimensional vectors, we are able to have vectors in any number of dimensions. Even 1,000! The tricky part, however, is visualizing them.

Vectors are useful since they help us summarize information about an object using numbers. Then, using the number representation, we can make comparisons between the vector representations of different objects!

This idea is central to how word embeddings map words into vectors.

We can easily represent vectors in Python using NumPy arrays. To create a vector containing the odd numbers from 1 to 9, we can use NumPy’s `.array()` method:

```
odd_vector = np.array([1, 3, 5, 7, 9])
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Say you are working at a school and want to analyze student scores on the first two English exams of the semester. One student, Xavier, scored 88 on the first exam and 92 on the second. Another student, Niko, scored 94 on the first exam and 87 on the second.

Create vectors using NumPy arrays called `scores_xavier` and `scores_niko` that contain the students' exam scores. Make sure the first exam score is the first value in each vector!

To create a vector using NumPy, you can use the `.array()` method:

```
my_vector = np.array(['green eggs', 'ham'])
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Take a look at the web browser component. You should see the two vectors representing Xavier's and Niko's exam scores plotted on a grid. What do you notice about the two vectors? Are they close together?

A new student Alena transfers into your class from a different section. Alena scored 90 on the first exam but struggled on the second, scoring 48. Create a new vector named `scores_alena` containing her exam scores in the same section of **script.py** where you defined the other exam vectors. Where do you think the vector will point to? And how similar is the vector for Alena's scores to Xavier's and Niko's?

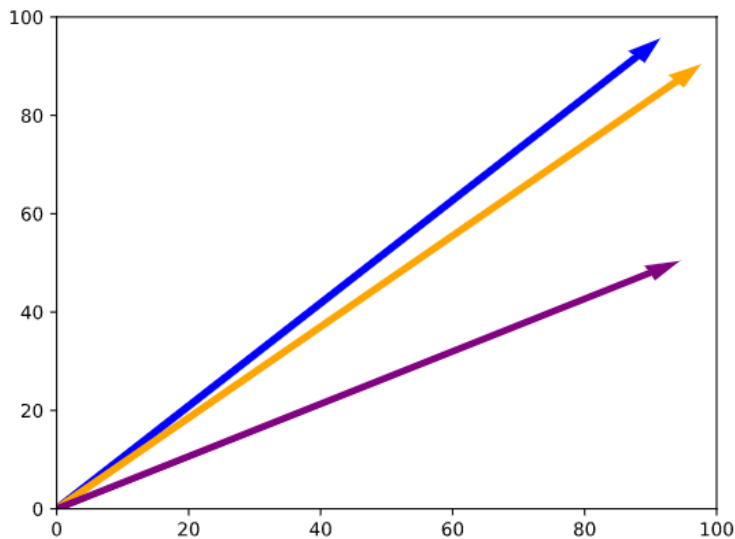
The vectors for Xavier and Niko point in a similar direction and have similar lengths, indicating that the performance of the students on the exams is similar.

The vector for Alena, however, points in a different direction than Xavier's and Niko's, indicating Alena's overall performance on the exams is markedly different than her peers.

```
import numpy as np
import matplotlib.pyplot as plt
import codecademylib3_seaborn

# define score vectors
scores_xavier = np.array([88,92])
scores_niko = np.array([94,87])
scores_alena = np.array([90,48])

# plot vectors
try:
    plt.arrow(0, 0, scores_xavier[0], scores_xavier[1], width=1, color='blue')
except:
    pass
try:
    plt.arrow(0, 0, scores_niko[0], scores_niko[1], width=1, color='orange')
except:
    pass
try:
    plt.arrow(0, 0, scores_alena[0], scores_alena[1], width=1, color='purple')
except:
    pass
plt.axis([0, 100, 0, 100])
plt.show()
```



Word Embeddings

What is a Word Embedding?

Now that you have an understanding of vectors, let's jump back to word embeddings. Word embeddings are vector representations of a word.

They allow us to take all the information that is stored in a word, like its meaning and its part of speech, and convert it into a numeric form that is more understandable to a computer.

For example, we could look at a word embedding for the word "peace".

```
[5.2907305, -4.20267, 1.6989858, -1.422668, -1.500128, ...]
```

Copy to Clipboard

Here "peace" is represented by a 96-dimension vector, with just the first five dimensions shown. Each dimension of the vector is capturing some information about how the word "peace" is used. We can also look at a word embedding for the word "war":

```
[7.2966490, -0.52887750, 0.97479630, -2.9508233, -3.3934135, ...]
```

Copy to Clipboard

By converting the words "war" and "peace" into their numeric vector representations, we are able to have a computer more easily compare the vectors and understand their similarities and differences.

We can load a basic English word embedding model using spaCy as follows:

```
nlp = spacy.load('en')
```

Copy to Clipboard

Note: the convention is to load spaCy models into a variable named `nlp`.

To get the vector representation of a word, we call the model with the desired word as an [argument](#)

Preview: Docs Loading link description
and can use the `.vector` attribute.

```
nlp('love').vector
```

Copy to Clipboard

But how do we compare these vectors? And how do we arrive at these numeric representations?

Instructions

Checkpoint 1 Passed

1.

Load a word embedding model from spaCy, as demonstrated in the narrative above, into a variable named `nlp`.

To load the basic English word embedding model from spaCy:

```
nlp = spacy.load('en')
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Use the loaded model to save the vector representation of the word “happy” into a variable named `happy_vec`, the vector representation of the word “sad” into a variable named `sad_vec`, and the vector representation of the word “angry” into a variable named `angry_vec`.

Print `happy_vec`, `sad_vec` and `angry_vec` to the terminal.

What do the vectors look like?

To access the vector representation of some word using spaCy:

```
nlp = spacy.load("en")

spectacular_vec = nlp("spectacular").vector
```

Copy to Clipboard

Checkpoint 3 Passed

3.

How big are these word embeddings?

Use Python’s `len()` function to find the length of any of the vectors you just defined, and save the result to a variable named `vector_length`.

Print `vector_length` to the terminal to see how many dimensions the word embedding has!

To find the length of a NumPy vector using Python’s `len()` function:

```
length = len(my_vector_here)
```

```
import spacy

# load word embedding model
nlp = spacy.load('en')

# define word embedding vectors
happy_vec = nlp('happy').vector
print(happy_vec)
sad_vec = nlp('sad').vector
print(sad_vec)
angry_vec = nlp('angry').vector
print(angry_vec)

# find vector length here
vector_length = len(happy_vec)
print(vector_length)
```

Word Embeddings

Distance

The key at the heart of word embeddings is distance. Before we explain why, let’s dive into how the distance between vectors can be measured.

There are a variety of ways to find the distance between vectors, and here we will cover three. The first is called Manhattan distance.

In Manhattan distance, also known as city block distance, distance is defined as the sum of the differences across each individual dimension of the vectors. Consider the vectors $[1, 2, 3]$ and $[2, 4, 6]$. We can calculate the Manhattan distance between them as shown below:

$$\text{manhattan distance} = |1-2| + |2-4| + |3-6| = 1+2+3 = 6$$

Another common distance metric is called the Euclidean distance, also known as straight line distance. With this distance metric, we take the square root of the sum of the squares of the differences in each dimension.

$$\text{euclidean distance} = \sqrt{(1-2)^2 + (2-4)^2 + (3-6)^2} = \sqrt{14} \approx 3.74$$

The final distance we will consider is the cosine distance. Cosine distance is concerned with the angle between two vectors, rather than by looking at the distance between the points, or ends, of the vectors. Two vectors that point in the same direction have no angle between them, and have a cosine distance of 0. Two vectors that point in opposite directions, on the other hand, have a cosine distance of 1. We would show you the calculation, but we don't want to scare you away! For the mathematically adventurous, [you can read up on the calculation here](#).

We can easily calculate the Manhattan, Euclidean, and cosine distances between vectors using helper functions from SciPy:

```
from scipy.spatial.distance import cityblock, euclidean, cosine

vector_a = np.array([1,2,3])
vector_b = np.array([2,4,6])

# Manhattan distance:
manhattan_d = cityblock(vector_a,vector_b) # 6

# Euclidean distance:
euclidean_d = euclidean(vector_a,vector_b) # 3.74

# Cosine distance:
cosine_d = cosine(vector_a,vector_b) # 0.0
```

Copy to Clipboard

When working with vectors that have a large number of dimensions, such as word embeddings, the distances calculated by Manhattan and Euclidean distance can become rather large. Thus, calculations using cosine distance are preferred!

Instructions

Checkpoint 1 Passed

1.

Provided in `script.py` are the three vectors from the previous exercise, `happy_vec`, `sad_vec`, and `angry_vec`. Use SciPy to compute the Manhattan distance for the following:

between `happy_vec` and `sad_vec`, storing the result in a variable `man_happy_sad`

between `sad_vec` and `angry_vec`, storing the result in a variable `man_sad_angry`

Print `man_happy_sad` and `man_sad_angry` to the terminal.

Which word embeddings are a greater distance apart according to Manhattan distance?

To calculate the Manhattan distance between two vectors using SciPy's `cityblock()` function:

```
manhattan_d = cityblock(vector_a, vector_b)
```

Copy to Clipboard

According to Manhattan distance, the word embeddings for "sad" and "angry" are farther apart than the word embeddings for "happy" and "sad".

Checkpoint 2 Passed

2.

Now use SciPy to compute the Euclidean distance between `happy_vec` and `sad_vec`, storing the result in a variable `euc_happy_sad`, as well as the Euclidean distance between `sad_vec` and `angry_vec`, storing the result in a variable `euc_sad_angry`.

Print `euc_happy_sad` and `euc_sad_angry` to the terminal.

Which word embeddings are a greater distance apart according to Euclidean distance?

To calculate the Euclidean distance between two vectors using SciPy's `euclidean()` function:

```
euclidean_d = euclidean(vector_a, vector_b)
```

Copy to Clipboard

According to Euclidean distance, the word embeddings for "sad" and "angry" are farther apart than the word embeddings for "happy" and "sad".

Checkpoint 3 Passed

3.

Next stop, cosine city! Use SciPy to compute the cosine distance between `happy_vec` and `sad_vec`, storing the result in a variable `cos_happy_sad`, as well as the cosine distance between `sad_vec` and `angry_vec`, storing the result in a variable `cos_sad_angry`.

Print `cos_happy_sad` and `cos_sad_angry` to the terminal.

Which word embeddings are further apart according to cosine distance? What else do you notice about the different distance metrics? Are the values similar between the different techniques on each pairing of vectors?

To calculate the cosine distance between two vectors using SciPy's `cosine()` function:

```
cosine_d = cosine(vector_a, vector_b)
```

Copy to Clipboard

Like the other distance metrics, according to cosine distance the word embeddings for "sad" and "angry" are farther apart than the word embeddings for "happy" and "sad".

The cosine distance values, however, remain low and bounded between 0 and 1, where the Manhattan and Euclidean distances are rather large (and continue to grow as more dimensions are added to a vector).

```
import numpy as np
from scipy.spatial.distance import cityblock, euclidean, cosine
import spacy

# load word embedding model
nlp = spacy.load('en')

# define word embedding vectors
happy_vec = nlp('happy').vector
sad_vec = nlp('sad').vector
angry_vec = nlp('angry').vector

# calculate Manhattan distance
man_happy_sad = cityblock(happy_vec, sad_vec)
man_sad_angry = cityblock(sad_vec, angry_vec)
print(man_happy_sad, man_sad_angry)

# calculate Euclidean distance
euc_happy_sad = euclidean(happy_vec, sad_vec)
euc_sad_angry = euclidean(sad_vec, angry_vec)
print(euc_happy_sad, euc_sad_angry)

# calculate cosine distance
cos_happy_sad = cosine(happy_vec, sad_vec)
cos_sad_angry = cosine(sad_vec, angry_vec)
print(cos_happy_sad, cos_sad_angry)
```

Word Embeddings

Word Embeddings Are All About Distance

The idea behind word embeddings is a theory known as the distributional hypothesis. This hypothesis states that words that co-occur in the same contexts tend to have similar meanings. With word embeddings, we map words that exist with the same context to similar places in our vector space (math-speak for the area in which our vectors exist).

The numeric values that are assigned to the vector representation of a word are not important in their own right, but gather meaning from how similar or not words are to each other.

Thus the cosine distance between words with similar contexts will be small, and the cosine distance between words that have very different contexts will be large.

The literal values of a word's embedding have no actual meaning. We gain value in word embeddings from comparing the different word vectors and seeing how similar or different they are. Encoded in these vectors, however, is latent information about how they are used.

Instructions

Checkpoint 1 Passed

1.

In **script.py** we have loaded a list of the most common 1,000 words in the English language, `most_common_words`, and their corresponding vector representations as word embeddings, `vector_list`.

Inspect these lists by printing the word at index 347 in `most_common_words` and its corresponding word embedding in `vector_list` to the terminal.

You can print a value at index `j` in some list `my_list` as follows:

```
print(my_list[j])
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Also given in **script.py** is a function `find_closest_words()`. This function accepts the following as arguments:

- a list of words
- their corresponding vector representations
- a target word

The function returns the words that have the smallest cosine distance between their vector representations and the target word.

For example, we could find the most common words closest to "tree" like this:

```
closest_to_tree = find_closest_words(most_common_words,
vector_list, "tree")
```

Copy to Clipboard

Call `find_closest_words()` with `most_common_words`, `vector_list`, and "food" as arguments and save the result to `close_to_food`. Print `close_to_food` to the terminal to see the result.

Which words does the function return? Are you surprised?

You can call a function `my_func()` with arguments `apples`, `bananas`, and "pears" as follows:

```
answer = my_func(apples, bananas, "pears")
```

Copy to Clipboard

Calling `find_closest_words()` with "food" as the target word returns other words related to food, like 'eat', 'dinner', 'fish', and 'health'.

Checkpoint 3 Passed

3.

Again call `find_closest_words()` with `most_common_words` and `vector_list` as arguments, but this time change the last argument to "summer". Save the result to `close_to_summer`, and print `close_to_summer` to the terminal.

Which words does the function return? Any surprises this time around?

Feel free to experiment by calling `find_closest_words()` with a different target word and seeing what results you get!

You can call a function `my_func()` with arguments `apples`, `bananas`, and "pears" as follows:

```
answer = my_func(apples, bananas, "pears")
```

Copy to Clipboard

Calling `find_closest_words()` with "summer" as the target word returns other words related to summer, like 'spring', 'fall', 'season'.

```
import spacy
from scipy.spatial.distance import cosine
from processing import most_common_words, vector_list

# print word and vector representation at index 347
print(most_common_words[347], vector_list[347])

# define find_closest_words
def find_closest_words(word_list, vector_list, word_to_check):
    return sorted(word_list,
```

```

        key=lambda x: cosine(vector_list[word_list.index(word_to_check)],
vector_list[word_list.index(x)]))[:10]

# find closest words to food
close_to_food = find_closest_words(most_common_words, vector_list, 'food')
print(close_to_food)

# find closest words to summer
close_to_summer = find_closest_words(most_common_words, vector_list, 'summer')
print(close_to_summer)

```

Word Embeddings

Word2vec

You might be asking yourself a question now. How did we arrive at the vector values that define a word vector? And how do we ensure that the values chosen place the vectors for words with similar context close together and the vectors for words with different usages far apart?

Step in word2vec! *Word2vec is a statistical learning algorithm*

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

that develops word embeddings from a corpus of text. Word2vec uses one of two different model architectures to come up with the values that define a collection of word embeddings.

One

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

is to use the continuous bag-of-words (CBOW) representation of a piece of text. The word2vec model goes through each word in the training corpus, in order, and tries to predict what word comes at each position based on applying bag-of-words to the words that surround the word in question. In this approach, the order of the words does not matter!

The other method word2vec can use to create word embeddings is continuous skip-grams. Skip-grams function similarly to n-grams, except instead of looking at groupings of n-consecutive words in a text, we can look at sequences of words that are separated by some specified distance between them.

For example, consider the sentence "The squids jump out of the suitcase". The 1-skip-2-grams includes all the bigrams (2-grams) as well as the following subsequences:

```
(The, jump), (squids, out), (jump, of), (out, the), (of, suitcase)
```

Copy to Clipboard

When using continuous skip-grams, the order of context **is** taken into consideration! Because of this, the time it takes to train the word embeddings is slower than when using continuous bag-of-words. The results, however, are often much better!

With either the continuous bag-of-words or continuous skip-grams representations as training data, word2vec then uses a shallow, 2-layer neural network to come up with the values that place words with a similar context in vectors near each other and words with different contexts in vectors far apart from each other.

Let's take a closer look to see how continuous bag-of-words and continuous skip-grams work!

Instructions

Checkpoint 1 Passed

1.

At the top of **script.py** you'll find the opening sentence of Charles Dickens's novel *A Tale of Two Cities*, stored in a variable `sentence`. Below, you'll see that this sentence has been preprocessed for you and saved as `sentence_lst`.

You'll also find a function called `get_cbows()` that we've defined. `get_cbows()` iterates through a preprocessed sentence word by word and returns a list of the different bag-of-words representations for the context words that surround each word in the sentence.

In the area indicated in **script.py**, call `get_cbows()` with `sentence_lst` and `context_length` as

arguments, and save the result to a variable `cbows`.

What do you see printed to the terminal? Open the hint for an explanation of the output.

You can call a function `my_fun_function()` with arguments `yippee` and `whohoo` as follows:

```
result = my_fun_function(yippee, whohoo)
```

Copy to Clipboard

You should see the following printed to the terminal:

```
('the', ['best', 'it', 'of', 'was'])
('best', ['of', 'the', 'times', 'was'])
('of', ['best', 'it', 'the', 'times'])
('times', ['best', 'it', 'of', 'was'])
('it', ['of', 'the', 'times', 'was'])
('was', ['it', 'the', 'times', 'worst'])
('the', ['it', 'of', 'was', 'worst'])
('worst', ['of', 'the', 'times', 'was'])
```

Copy to Clipboard

The first value in each tuple is the word in the sentence we are focusing on. The second value is a list of the bag-of-words representation of the surrounding words.

For example, when focusing on the third word in `sentence`, `'the'`, the surrounding words are `'it'`, `'was'`, `'best'`, and `'of'`. But since we are looking at the bag-of-words representation, the order of the words *does not matter*! That is why in the tuple, we are given `['best', 'it', 'of', 'was']`.

Checkpoint 2 Passed

2.

Also provided in `script.py` is a function called `get_skip_grams()`. `get_skip_grams()` iterates through a sentence word by word and returns an ordered list of the different context words that surround each word in the sentence.

In the area indicated in `script.py`, call `get_skip_grams()` with `sentence_lst` and `context_length` as arguments, and save the result to a variable `skip_grams`.

What do you see printed to the terminal? How do these results differ from the previous exercise's results? Open the hint for an explanation of the output.

You can call a function `my_fun_function()` with arguments `yippee` and `whohoo` as follows:

```
result = my_fun_function(yippee, whohoo)
```

Copy to Clipboard

You should see the following printed to the terminal:

```
('the', ['it', 'was', 'best', 'of'])
('best', ['was', 'the', 'of', 'times'])
('of', ['the', 'best', 'times', 'it'])
('times', ['best', 'of', 'it', 'was'])
('it', ['of', 'times', 'was', 'the'])
('was', ['times', 'it', 'the', 'worst'])
('the', ['it', 'was', 'worst', 'of'])
('worst', ['was', 'the', 'of', 'times'])
```

Copy to Clipboard

The first value in each tuple is the word in the sentence we are focusing on. The second value is an ordered list of the different context words that surround each word.

For example, when focusing on the third word in `sentence`, `'the'`, the surrounding words are `'it'`, `'was'`, `'the'`, and `'best'`. Since we are looking at the skip-grams representation, the order of the words *does matter*! That is why in the tuple, we are given `['it', 'was', 'best', 'of']`.

While the words themselves do not vary between the output of `get_cbows()` and `get_skip_grams()`, the order is different! With continuous skip-grams we do care about word order, so the order is preserved in our output.

Checkpoint 3 Passed

3.

Above the functions in `script.py` we currently have a variable `context_length` set to 2. This indicates that when finding our bag-of-words and skip-gram representations, we are only looking 2 words to the left and 2 words to the right of our word we are focusing on.

Update the value of `context_length` to 3. How do the bag-of-words and skip-gram representations change?

When we update the value of `context_length` to 3, we are now looking 3 words to the left and 3 words to the right of our focus word. We are expanding the context to look farther away from our word!

This means that when word2vec trains our model, a larger context will be taken into consideration!

```
from sklearn.feature_extraction.text import CountVectorizer

sentence = "It was the best of times, it was the worst of times."
print(sentence)

# preprocessing
sentence_lst = [word.lower().strip(".") for word in sentence.split()]

# set context_length
context_length = 3

# function to get cbows (continuous bag-of-words)
def get_cbows(sentence_lst, context_length):
    cbows = list()
    for i, val in enumerate(sentence_lst):
        if i < context_length:
            pass
        elif i < len(sentence_lst) - context_length:
            context = sentence_lst[i-context_length:i] +
sentence_lst[i+1:i+context_length+1]
            vectorizer = CountVectorizer()
            vectorizer.fit_transform(context)
            context_no_order = vectorizer.get_feature_names()
            cbows.append((val, context_no_order))
    return cbows

# define cbows here:
cbows = get_cbows(sentence_lst, context_length)

# function to get skip_grams
def get_skip_grams(sentence_lst, context_length):
    skip_grams = list()
    for i, val in enumerate(sentence_lst):
        if i < context_length:
            pass
        elif i < len(sentence_lst) - context_length:
            context = sentence_lst[i-context_length:i] +
sentence_lst[i+1:i+context_length+1]
            skip_grams.append((val, context))
    return skip_grams

# define skip_grams here:
skip_grams = get_skip_grams(sentence_lst, context_length)

try:
    print('\nContinuous Bag of Words')
    for cbow in cbows:
        print(cbow)
except:
    pass

try:
    print('\nSkip Grams')
    for skip_gram in skip_grams:
        print(skip_gram)
except:
    pass
```

Gensim

Depending on the corpus of text we select to train a word embedding model, different word embeddings will be created according to the context of the words in the given corpus. The larger and more generic a corpus, however, the more generalizable the word embeddings become.

When we want to train our own word2vec model on a corpus of text, we can use the [gensim package](#)!

In previous exercises, we have been using pre-trained word embedding models stored in spaCy. These models were trained, using word2vec, on blog posts and news articles collected by the [Linguistic Data Consortium](#) at the University of Pennsylvania. With gensim, however, we are able to build our own word embeddings on any corpus of text we like.

To easily train a word2vec model on our own corpus of text, we can use gensim's `Word2Vec()` function.

```
model = gensim.models.Word2Vec(corpus, size=100, window=5, min_count=1, workers=2, sg=1)
```

Copy to Clipboard

`corpus` is a list of lists, where each inner list is a document in the corpus and each element in the inner lists is a word token

`size` determines how many dimensions our word embeddings will include. Word embeddings often have upwards of 1,000 dimensions! Here we will create vectors of 100-dimensions to keep things simple.

don't worry about the rest of the keyword arguments here!

To view the entire vocabulary used to train the word embedding model, we can use the `.wv.vocab.items()`

[method](#)

Preview: Docs Loading link description

```
vocabulary_of_model = list(model.wv.vocab.items())
```

Copy to Clipboard

When we train a word2vec model on a smaller corpus of text, we pick up on the unique ways in which words of the text are used.

For example, if we were using scripts from the television show Friends as a training corpus, the model would pick up on the unique ways in which words are used in the show. While the generalized vectors in a spaCy model might not place the vectors for "Ross" and "Rachel" close together, a gensim word embedding model trained on Friends' scripts would place the vectors for words like "Ross" and "Rachel", two characters that have a continuous on and off-again relationship throughout the show, very close together!

To easily find which vectors gensim placed close together in its word embedding model, we can use the

`.most_similar()` method.

```
model.most_similar("my_word_here", topn=100)
```

Copy to Clipboard

`"my_word_here"` is the target word token we want to find most similar words to

`topn` is a keyword

[argument](#)

Preview: Docs Loading link description

that indicates how many similar word vectors we want returned

One last gensim method we will explore is a rather fun one: `.doesn't_match()`.

```
model.doesnt_match(["asia", "mars", "pluto"])
```

Copy to Clipboard

when given a list of terms in the vocabulary as an argument, `.doesn't_match()` returns which term is furthest from the others.

Let's play around with gensim word2vec models to explore the word embeddings defined on our own corpus of training data!

Instructions

Checkpoint 1 Passed

1.

As is common in so much of NLP, before we can do our fancy modeling we need to do some preprocessing! In `script.py` we have lowercased and split the text of William Shakespeare's *Romeo and Juliet* into a list of

lists named `romeo_and_juliet_processed`, where each entry in the inner list is a word token of the play. All stop words, loaded from NLTK, have been removed as well. Inspect the processed text by printing the first 20 entries of the inner list in `romeo_and_juliet_processed` to the terminal.

To print the first 5 entries of the inner list in a list of lists named `my_lol`:

```
print(my_lol[0][:5])
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Train a gensim word2vec model of 100 dimensions on `romeo_and_juliet_processed` using the same keyword arguments described in the narrative. Save your model to a variable named `model`.

To train a gensim word2vec model on some corpus:

```
model = gensim.models.Word2Vec(corpus, size=100, window=5, min_count=1, workers=2, sg=1)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Save the vocabulary of your trained model as a list to a variable named `vocabulary`. Print `vocabulary` to the terminal and view it.

To find the vocabulary of a trained word2vec model in gensim and convert it to a list:

```
vocabulary = list(my_model.wv.vocab.items())
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Which words are used most similarly to “romeo” in *Romeo and Juliet*? Use `.most_similar()` to find the 20 most similar words to “romeo”, and save the result to a variable `similar_to_romeo`. Print `similar_to_romeo` to the terminal.

To find the 20 most similar words to “thingamabob” in a gensim word2vec model named `model`:

```
gadgets_and_gizmos = model.most_similar("thingamabob", topn=20)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Are “Romeo” and “Juliet” truly star crossed lovers? Let’s make sure!

Use `.doesn't_match()` to find which character in the below list is the odd-one-out, and save the result to a variable `not_star_crossed_lover`.

```
["romeo", "juliet", "mercutio"]
```

Copy to Clipboard

Print `not_star_crossed_lover` to the terminal.

To find which words is least similar to the others in a gensim word2vec model named `model`:

```
not_in_mushroom_land = model.doesnt_match(["mario", "luigi", "sonic"])
```

```
import gensim
from nltk.corpus import stopwords
from romeo_juliet import romeo_and_juliet

# load stop words
stop_words = stopwords.words('english')

# preprocess text
romeo_and_juliet_processed = [[word for word in romeo_and_juliet.lower().split() if word not in stop_words]]

# view inner list of romeo_and_juliet_processed
print(romeo_and_juliet_processed[0][:20])

# train word embeddings model
```

```

model = gensim.models.Word2Vec(romeo_and_juliet_processed, size=100, window=5,
min_count=1, workers=2, sg=1)

# view vocabulary
vocabulary = list(model.wv.vocab.items())
print(vocabulary)

# similar to romeo
similar_to_romeo = model.most_similar("romeo", topn=20)
print(similar_to_romeo)

# one is not like the others
not_star_crossed_lover = model.doesnt_match(["romeo", "juliet", "mercutio"])
print(not_star_crossed_lover)

```

Word Embeddings

Review

Lost in a multidimensional vector space after this lesson? We hope not! We have covered a lot here, so let's take some time to recap.

Vectors are containers of information, and they can have anywhere from 1-dimension to hundreds or thousands of dimensions

Word embeddings are vector representations of a word, where words with similar contexts are represented with vectors that are closer together

spaCy is a package that enables us to view and use pre-trained word embedding models

The distance between vectors can be calculated in many ways, and the best way for measuring the distance between higher dimensional vectors is *cosine distance*

Word2Vec is a shallow neural network model that can build word embeddings using either continuous bag-of-words or continuous skip-grams

Gensim is a package that allows us to create and train word embedding models using any corpus of text

Instructions

Checkpoint 1 Passed

1.

Load a word embedding model from spaCy into a variable named `nlp`.

To load the basic English word embedding model from spaCy:

```
nlp = spacy.load('en')
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Use the loaded model to create the following words embeddings:

a vector representation of the word "sponge" saved in a variable named `sponge_vec`

a vector representation of the word "starfish" in a variable named `starfish_vec`

a vector representation of the word "squid" in a variable named `squid_vec`

To access the vector representation of some word using spaCy:

```
nlp = spacy.load("en")

spectacular_vec = nlp("spectacular").vector
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Use SciPy to compute the cosine distance between:

`sponge_vec` and `starfish_vec`, storing the result in a variable `dist_sponge_star`

`sponge_vec` and `squid_vec`, storing the result in a variable `dist_sponge_squid`

starfish_vec and squid_vec, storing the result in a variable dist_star_squid
Print dist_sponge_star, dist_sponge_squid and dist_star_squid to the terminal.
Which word embeddings are furthest apart according to cosine distance?

To calculate the cosine distance between two vectors using SciPy's cosine() function:

```
cosine_d = cosine(vector_a, vector_b)
```

Copy to Clipboard

According to cosine distance, the word embeddings for "sponge" and "squid" are furthest apart!

```
import spacy
from scipy.spatial.distance import cosine

# load word embedding model
nlp = spacy.load('en')

# define word embedding vectors
sponge_vec = nlp('sponge').vector
starfish_vec = nlp('starfish').vector
squid_vec = nlp('squid').vector

# compare vectors with cosine distance
dist_sponge_star = cosine(sponge_vec, starfish_vec)
print(dist_sponge_star)
dist_sponge_squid = cosine(sponge_vec, squid_vec)
print(dist_sponge_squid)
dist_star_squid = cosine(starfish_vec, squid_vec)
print(dist_star_squid)
```

Review: Language Quantification

Review

Congratulations! The goal of this unit was to use various methods, including language models and embeddings, to represent text numerically.

Having completed this unit, you are now able to:

- Identify common real-world applications for the bag-of-words language model.
- Convert text into a bag-of-words representation using a Python dictionary.
- Understand how feature dictionaries and feature vectors can be used to build a BoW model.
- Recognize the difference between training data and test data.
- Use scikit-learn to convert text into a bag-of-words vector.
- Identify advantages and disadvantages of BoW in relation to other common language models.
- Understand how tf-idf can allow you to determine word importance within texts.
- Implement tf-idf using scikit-learn.
- Reframe word meaning based on context using word embeddings.
- Understand how multi-dimensional vectors and distance metrics are useful for numerical representations of language.
- Implement word embeddings, including word2vec, with spaCy and gensim.

If you are interested in learning more about these topics, here are some additional resources:

[Working with Text Data | scikit-learn](#)

[Token.similarity | spaCy](#)

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Introduction: Text Generation

See what's covered in [Text Generation](#).

Goals of this Unit

The goal of this unit is to introduce some common methods of generating text.

After this unit, you will be able to:

- Recognize why recurrent neural networks and specifically long short-term memory (LSTM) networks are helpful for working with sequences of text.

- Convert text into a matrix of one-hot vectors.

- Implement a seq2seq network using LSTM layers with teacher forcing.

- Use deep learning to conduct machine translation.

You will put all of this knowledge into practice with an upcoming Portfolio Project. You can complete the Portfolio Project either in parallel with or after taking the prerequisite content—it's up to you!

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Long Short Term Memory Networks

Long short-term memory networks (LSTMs) are often used in deep learning programs for natural language processing.

Information Persistence and Neural Networks

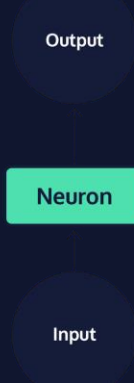
We do not create a new language every time we speak — every human language has a persistent set of grammar rules and collection of words that we rely on to interpret it. As you read this article, you understand each word based on your knowledge of grammar rules and interpretation of the preceding and following words. At the beginning of the next sentence, you continue to utilize information from earlier in the passage to follow the overarching logic of the paragraph. Your knowledge of language is both persistent across interactions, and adaptable to new conversations.

Neural networks are a machine learning framework loosely based on the structure of the human brain. They are very commonly used to complete tasks that seem to require complex decision making, like speech recognition or image classification. Yet, despite being modeled after neurons in the human brain, early neural network models were not designed to handle temporal sequences of data, where the past depends on the future. As a result, early models performed very poorly on tasks in which prior decisions are a strong predictor of future decisions, as is the case in most human language tasks. However, more recent models, called recurrent neural networks (RNN), have been specifically designed to process inputs in a temporal order and update the future based on the past, as well as process sequences of arbitrary length.

RNNs are one of the most commonly used neural network architectures today. Within the domain of Natural Language Processing, they are often used in speech generation and machine translation tasks. Additionally, they are often used to solve speech recognition and optical character recognition tasks. Let's dive into their implementation!

Neural Networks

A single neuron in a network, *reference to graphic*, takes in a single piece of input data, *reference to graphic*, and performs some data transformation to produce a single piece of output *reference to graphic*.



The very first network models, called **perceptrons**, were relatively simple systems that used many combinations of simple functions, like computing the slope of a line. While these transformations were very simple in isolation, together they resulted in sophisticated behavior for the entire system and allowed for large numbers of transformations to be strung together in order to compute advanced functions. However, the range of perceptron applications was still limited, and there were very simple functions that they just simply couldn't compute. In attempting to solve this issue by layering perceptrons together, researchers ran into another problem: few computers at the time were able to store enough data to execute these programs.

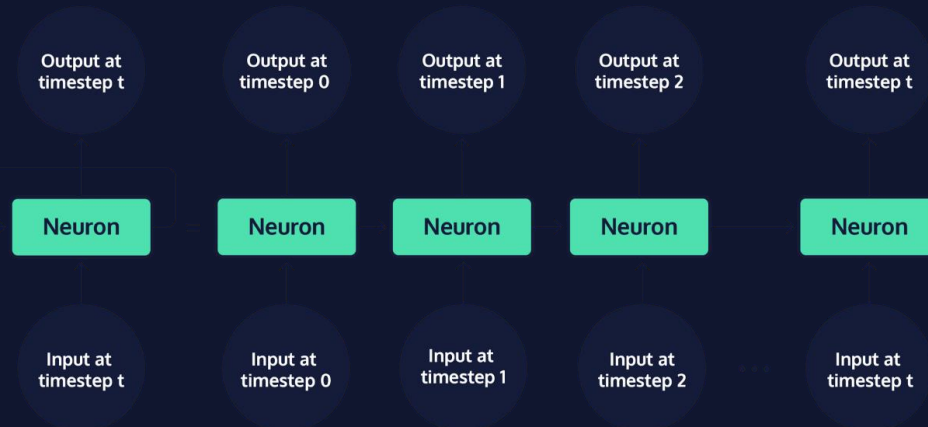
Deep Neural Networks

By the early 2000s, when innovations in computer hardware allowed for more complex modeling techniques, researchers had already developed neural network systems that combined many layers of neurons, including convolutional neural networks (CNN), multi-layer perceptrons (MLP), and recurrent neural networks (RNN). All three of these architectures are called deep neural networks, because they have many layers of neurons that combine to create a "deep" stack of neurons. Each of these deep-learning architectures have their own relative strengths:

MLP networks are comprised of layered perceptrons. They tend to be good at solving simple tasks, like applying a filter to each pixel in a photo.

CNN networks are designed to process image data, applying the same convolution function across an entire input image. This makes it simpler and more efficient to process images, which generally yields very high-dimensional output and requires a great deal of processing.

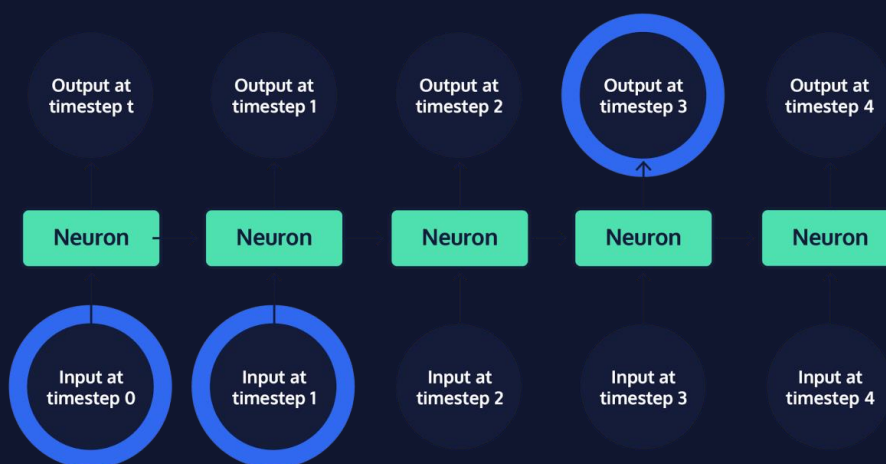
Finally, RNNs became widely adopted within natural language processing because they integrate a loop into the connections between neurons, allowing information to persist across a chain of neurons.



When the chain of neurons in an RNN is “rolled out,” it becomes easier to see that these models are made up of many copies of the same neuron, each passing information to its successor. Neurons that are not the first or last in a rolled out RNN are sometimes referred to as “hidden” network layers; the first and last neurons are called the “input” and “output” layers, respectively. The chain structure of RNNs places them in close relation to data with a clear temporal ordering or list-like structure — such as human language, where words obviously appear one after another. Standard RNNs are certainly the best fit for tasks that involve sequences, like the translation of a sentence from one language to another.

Long-term Dependencies

While the broad family of RNNs powers many of today’s advanced artificial intelligence systems, one particular RNN variant, the long short-term memory network (LSTM), has become popular for building realistic chatbot systems. When attempting to predict the next word in a conversation, words that were mentioned recently often provide enough information to make a strong guess. For instance, when trying to complete the sentence “The grass is always: _____,” most English-speakers do not need additional context to guess that the last word is “greener.” Standard RNNs are difficult to train, and fail to capture long-term dependencies well; they perform best on short sequences, when relevant context and the word to be predicted fall within a short distance of one



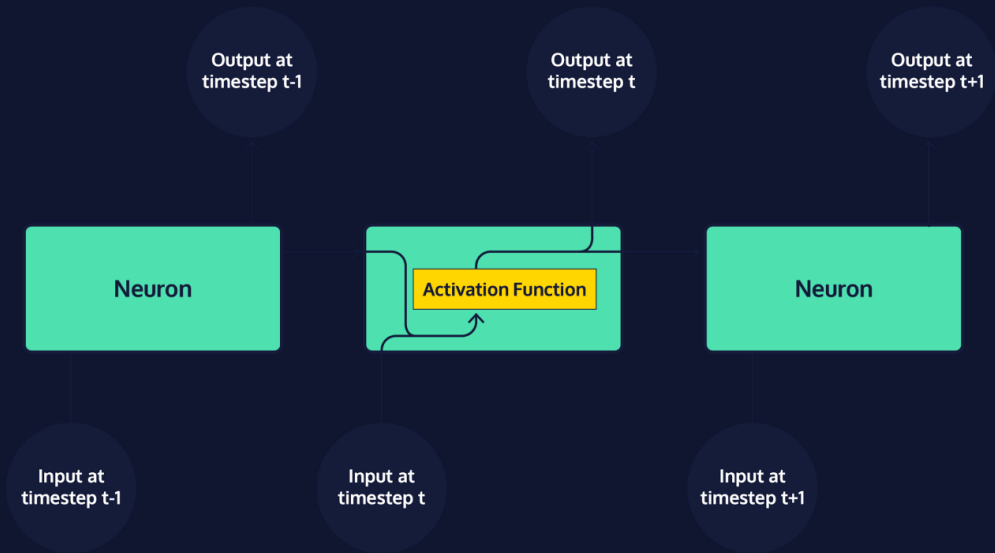
another.

However, there

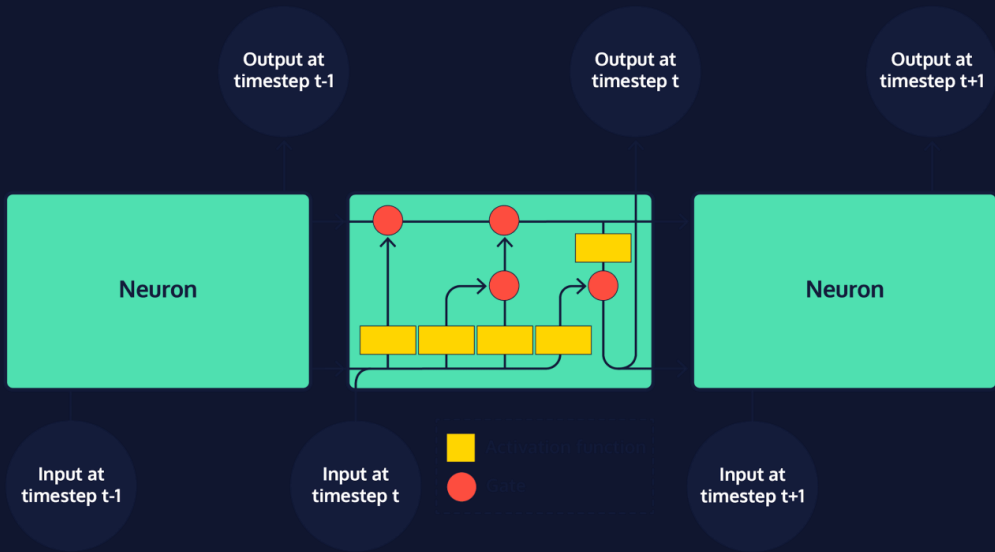
are also many cases of longer sequences in which more context is needed to make an accurate prediction, and that context is less obvious and at a greater lexical distance from the word it determines than in the example above. For instance, consider trying to complete the sentence "It is winter and there has been little sunlight. The grass is always ____." While the structure of the second sentence suggests that our word is probably an adjective related to grass, we need context from further back in the text to guess that the correct word is "brown." As the gap between context and the word to predict grows, standard RNNs become less and less accurate. This situation is commonly referred to as the long-term dependency problem. The solution to this problem? A neural network specifically designed for long-term memory – the LSTM!

Long Short-term Memory Networks

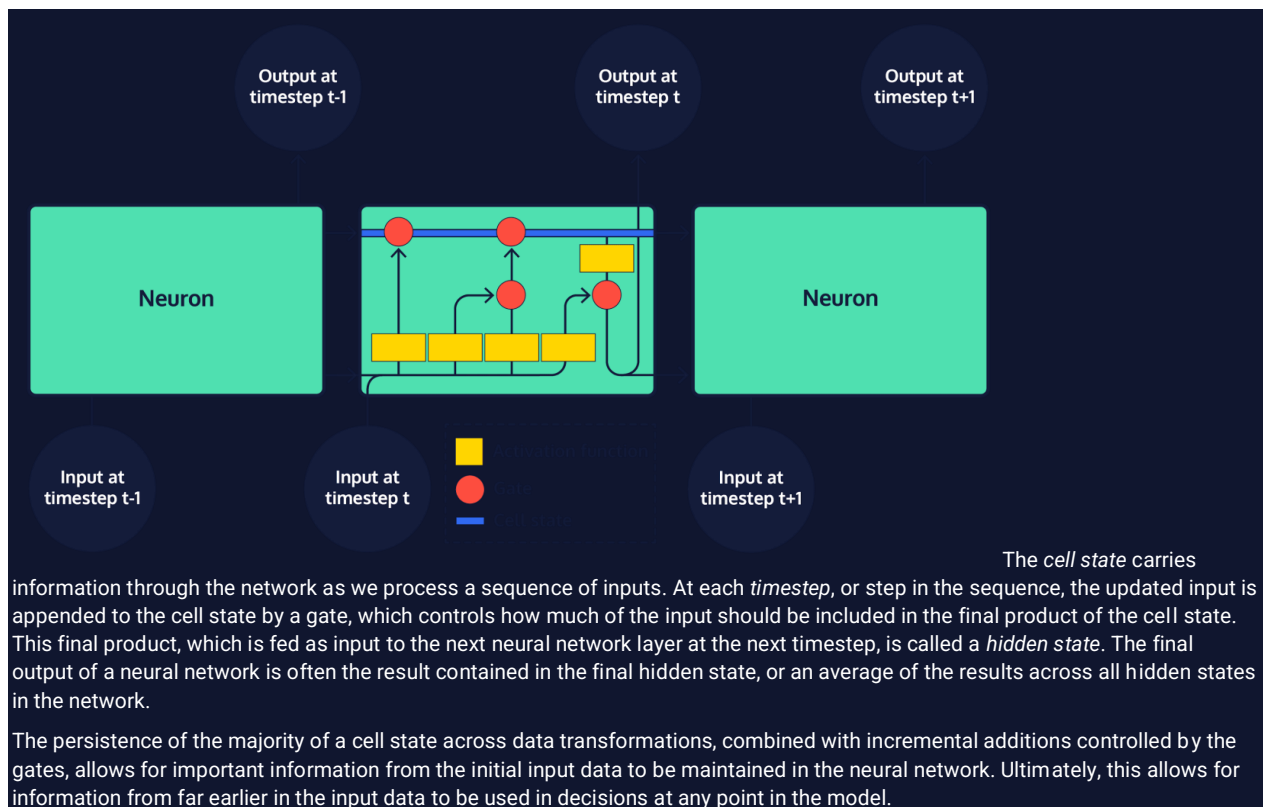
Every model in the RNN family, including LSTMs, is a chain of repeating neurons at its base. Within standard RNNs, each layer of neurons will only perform a single operation on the input data.



However, within an LSTM, groups of neurons perform four distinct operations on input data, which are then sequentially combined.



The most important aspect of an LSTM is the way in which the transformed input data is combined by adding results to *state*, or cell memory, represented as vectors. There are two states that are produced for the first step in the sequence and then carried over as subsequent inputs are processed: cell state, and hidden state.



Generating Text with Deep Learning

Introduction to seq2seq

LSTMs are pretty extraordinary, but they're only the tip of the iceberg when it comes to actually setting up and running a neural language model for text generation. In fact, an LSTM is usually just a single component in a larger network.

One of the most common neural models used for text generation is the sequence-to-sequence model, commonly referred to as *seq2seq* (pronounced "seek-to-seek"). A type of encoder-decoder model, seq2seq uses recurrent neural networks (RNNs) like LSTM in order to generate output, token by token or character by character.

So, where does seq2seq show up?

- Machine translation software like Google Translate
- Text summary generation
- Chatbots
- Named Entity Recognition (NER)
- Speech recognition

seq2seq networks have two parts:

An *encoder* that accepts language (or audio or video) input. The output matrix of the encoder is discarded, but its state is preserved as a vector.

A *decoder* that takes the encoder's final state (or memory) as its initial state. We use a technique called "teacher forcing" to train the decoder to predict the following text (characters or words) in a target sequence given the previous text.

Instructions

Take a look at the gif as "Knowledge is power" is translated from Hindi to English. Watch as state is passed through the encoder and on to each layer of the decoder.

Generating Text with Deep Learning

Introduction to seq2seq

LSTMs are pretty extraordinary, but they're only the tip of the iceberg when it comes to actually setting up and running a neural language model for text generation. In fact, an LSTM is usually just a single component in a larger network.

One of the most common neural models used for text generation is the sequence-to-sequence model, commonly referred to as *seq2seq* (pronounced "seek-to-seek"). A type of encoder-decoder model, *seq2seq* uses recurrent neural networks (RNNs) like LSTM in order to generate output, token by token or character by character.

So, where does *seq2seq* show up?

- Machine translation software like Google Translate

- Text summary generation

- Chatbots

- Named Entity Recognition (NER)

- Speech recognition

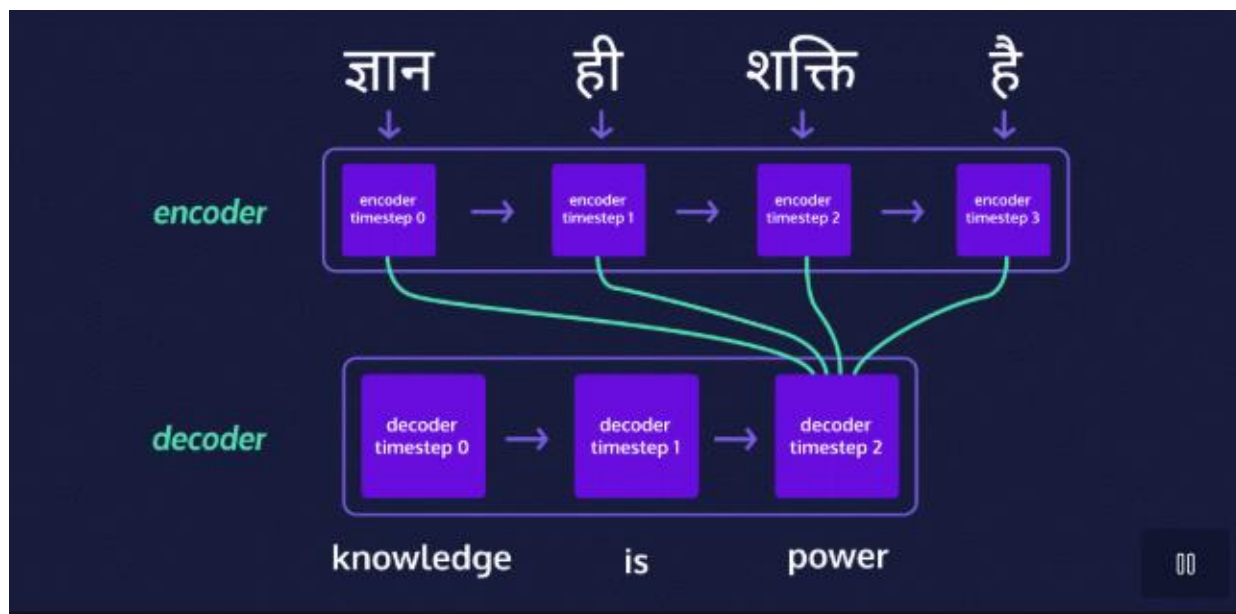
seq2seq networks have two parts:

- An *encoder* that accepts language (or audio or video) input. The output matrix of the encoder is discarded, but its state is preserved as a vector.

- A *decoder* that takes the encoder's final state (or memory) as its initial state. We use a technique called "teacher forcing" to train the decoder to predict the following text (characters or words) in a target sequence given the previous text.

Instructions

Take a look at the gif as "Knowledge is power" is translated from Hindi to English. Watch as state is passed through the encoder and on to each layer of the decoder.



Generating Text with Deep Learning

Preprocessing for seq2seq

If you're feeling a bit nervous about building this all on your own, never fear. You don't need to start from scratch — there are a few neural network libraries at your disposal. In our case, we'll be using [TensorFlow](#) with the [Keras API](#) to build a pretty limited English-to-Spanish translator (we'll explain this later and you'll get an opportunity to improve it).

We can import Keras from Tensorflow like this:

```
from tensorflow import keras
```

Copy to Clipboard

Also, do not worry about memorizing anything we cover here. The purpose of this lesson is for you to make sense of what each part of the code does and how you can modify it to suit your own needs. In fact, the code we'll be using is mostly derived from [Keras's own tutorial on the seq2seq model](#).

First things first: preprocessing the text data. Noise removal depends on your use case — do you care about casing or punctuation? For many tasks they are probably not important enough to justify the additional processing. This might be the time to make changes.

We'll need the following for our Keras implementation:

- vocabulary sets for both our input (English) and target (Spanish) data
- the total number of unique word tokens we have for each set
- the maximum sentence length we're using for each language

We also need to mark the start and end of each document (sentence) in the target samples so that the model recognizes where to begin and end its text generation (no book-long sentences for us!). One way to do this is adding "<START>" at the beginning and "<END>" at the end of each target document (in our case, this will be our Spanish sentences). For example, "Estoy feliz." becomes "<START> Estoy feliz. <END>".

Before you dig into the instructions, read through the existing code in **script.py** and try to make sense of each line.

Instructions

Checkpoint 1 Passed

1.

If you take a look at **span-eng.txt**, you'll see that we're working with a very tiny data set right now, which will make this a very terrible translator indeed. This is because we don't want codecademy.com to crash on you! When you build your own translator later, you'll be using a much larger data set, which will require a great deal more time to process everything.

Use string concatenation to reassign each `target_doc` to the value of `target_doc` surrounded by "<START> " and " <END>".

Then append the `target_doc` to `target_docs`.

To concatenate string values:

```
demand = "Please " + demand + " now."
```

Copy to Clipboard

To append items to a list:

```
list_of_fruit.append(pineapple)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Complete each `for` loop by adding each `token` to the corresponding (input or target) tokens set if it hasn't already been added.

To check if something is NOT in a set:

```
if "Ghoti" not in cats:
```

Copy to Clipboard

To add a new item to a set:

```
cats.add("Ghoti")
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Create two new variables:

`num_encoder_tokens`: the length of the input tokens set

`num_decoder_tokens`: the length of the target tokens set

To set a variable equal to the length of a set:

```
from tensorflow import keras
import re
```

```

# Importing our translations
data_path = "span-eng.txt"
# Defining lines as a list of each line
with open(data_path, 'r', encoding='utf-8') as f:
    lines = f.read().split('\n')

# Building empty lists to hold sentences
input_docs = []
target_docs = []
# Building empty vocabulary sets
input_tokens = set()
target_tokens = set()

for line in lines:
    # Input and target sentences are separated by tabs
    input_doc, target_doc = line.split('\t')
    # Appending each input sentence to input_docs
    input_docs.append(input_doc)
    # Splitting words from punctuation
    target_doc = " ".join(re.findall(r"[\w']+|^[^s\w]", target_doc))
    # Redefine target_doc below
    # and append it to target_docs:
    target_doc = '<START> ' + target_doc + ' <END>'
    target_docs.append(target_doc)

    # Now we split up each sentence into words
    # and add each unique word to our vocabulary set
    for token in re.findall(r"[\w']+|^[^s\w]", input_doc):
        print(token)
        # Add your code here:
        if token not in input_tokens:
            input_tokens.add(token)
    for token in target_doc.split():
        print(token)
        # And here:
        if token not in target_tokens:
            target_tokens.add(token)

input_tokens = sorted(list(input_tokens))
target_tokens = sorted(list(target_tokens))

# Create num_encoder_tokens and num_decoder_tokens:
num_encoder_tokens = len(input_tokens)
num_decoder_tokens = len(target_tokens)

try:
    max_encoder_seq_length = max([len(re.findall(r"[\w']+|^[^s\w]", input_doc)) for
input_doc in input_docs])
    max_decoder_seq_length = max([len(re.findall(r"[\w']+|^[^s\w]", target_doc)) for
target_doc in target_docs])
except ValueError:
    pass

```



```
Span-eng.txt = """
We'll see.    Después veremos.
We'll see.    Ya veremos.
We'll try.    Lo intentaremos.
We've won!    ¡Hemos ganado!
Well done.    Bien hecho.
What's up?    ¿Qué hay?
Who cares?    ¿A quién le importa?
Who drove?    ¿Quién condujo?
Who drove?    ¿Quién conducía?
Who is he?    ¿Quién es él?
Who is it?    ¿Quién es? """
```

Generating Text with Deep Learning

Training Setup (part 1)

For each sentence, Keras expects a NumPy matrix containing *one-hot vectors* for each token. What's a one-hot vector? In a one-hot vector, every token in our set is represented by a 0 except for the current token which is represented by a 1. For example given the vocabulary ["the", "dog", "licked", "me"], a one-hot vector for "dog" would look like [0, 1, 0, 0].

In order to vectorize our data and later translate it from vectors, it's helpful to have a features

[dictionary](#)

Preview: Docs A dictionary is an unordered set of (key, value) pairs. It provides a way to map pieces of data to each other, and allows for quick access to values associated to keys. The syntax of a dictionary is as follows: pseudo dictionary = { key1: value1, key2: value2, key3: value3

(and a reverse features dictionary) to easily translate between all the 1s and 0s and actual words. We'll build out the following:

- a features dictionary for English
- a features dictionary for Spanish
- a reverse features dictionary for English (where the keys and values are swapped)
- a reverse features dictionary for Spanish

Once we have all of our features dictionaries set up, it's time to vectorize the data! We're going to need vectors to input into our encoder and decoder, as well as a vector of target data we can use to train the decoder.

Because each matrix is almost all zeros, we'll use `numpy.zeros()` from the NumPy library to build them out.

```
import numpy as np

encoder_input_data = np.zeros(
    (len(input_docs), max_encoder_seq_length, num_encoder_tokens),
    dtype='float32')
```

Copy to Clipboard

Let's break this down:

We defined a NumPy matrix of zeros called `encoder_input_data` with two arguments:

- the shape of the matrix — in our case the number of documents (or sentences) by the maximum token sequence length (the longest sentence we want to see) by the number of unique tokens (or words)
- the data type we want — in our case NumPy's `float32`, which can speed up our processing a bit

Instructions

Checkpoint 1 Passed

1.

Hang on... where did all that code go from the previous exercise? Don't worry, it's still there; we just moved it over to **preprocess.py** (all the necessary variables are imported at the top of **script.py**) to make some room for the new influx of code!

Take a look at the new code. You'll see we've defined a features dictionary for our input vocabulary called `input_features_dict`.

Below `input_features_dict`, build `target_features_dict` the same way, but using the set of target tokens instead of input tokens.

The code used for `target_features_dict` should be identical to `input_features_dict` except with `target_tokens` instead of `input_tokens`.

`enumerate()` is just a way to access both an item and its index in a list or set. You can learn more about it [here](#).

Checkpoint 2 Passed

2.

We've also built out the `reverse_input_features_dict`, which just swaps keys for values of the `input_features_dict`.

Build the `reverse_target_features_dict` in the same way, reversing the key-value pairs in `target_features_dict`.

Checkpoint 3 Passed

3.

We already have the `encoder_input_data` numpy matrix done for you.

Your task is to create the following NumPy matrices with the same arguments as `encoder_input_data`, except they should use the max sequence length for decoder sentences instead of encoder sentences, and the number of decoder tokens instead of encoder tokens:

`decoder_input_data`: a matrix for the data we'll pass into the decoder

`decoder_target_data`: a matrix for the data we expect the decoder to produce

(The two new matrices you create should be identical for now.)

For example:

```
decoder_input_data = np.zeros(
    (len(input_docs), max_decoder_seq_length, num_decoder_tokens),
    dtype='float32')
```

```
from tensorflow import keras
import numpy as np
from preprocessing import input_docs, target_docs, input_tokens, target_tokens,
num_encoder_tokens, num_decoder_tokens, max_encoder_seq_length, max_decoder_seq_length

print('Number of samples:', len(input_docs))
print('Number of unique input tokens:', num_encoder_tokens)
print('Number of unique output tokens:', num_decoder_tokens)
print('Max sequence length for inputs:', max_encoder_seq_length)
print('Max sequence length for outputs:', max_decoder_seq_length)

input_features_dict = dict(
    [(token, i) for i, token in enumerate(input_tokens)])
# Build out target_features_dict:
target_features_dict = dict(
    [(token, i) for i, token in enumerate(target_tokens)])
```

```

# Reverse-lookup token index to decode sequences back to
# something readable.
reverse_input_features_dict = dict(
    (i, token) for token, i in input_features_dict.items())
# Build out reverse_target_features_dict:
reverse_target_features_dict = dict(
    (i, token) for token, i in target_features_dict.items())

encoder_input_data = np.zeros(
    (len(input_docs), max_encoder_seq_length, num_encoder_tokens),
    dtype='float32')
print("\nHere's the first item in the encoder input matrix:\n", encoder_input_data[0],
      "\n\nThe number of columns should match the number of unique input tokens and the
number of rows should match the maximum sequence length for input sentences.")

# Build out the decoder_input_data matrix:
decoder_input_data = np.zeros(
    (len(input_docs), max_decoder_seq_length, num_decoder_tokens),
    dtype='float32')
# Build out the decoder_target_data matrix:
decoder_target_data = np.zeros(
    (len(input_docs), max_decoder_seq_length, num_decoder_tokens),
    dtype='float32')

```

preprocessing.py

```

from tensorflow import keras
import re
# Importing our translations
data_path = "span-eng.txt"
# Defining lines as a list of each line
with open(data_path, 'r', encoding='utf-8') as f:
    lines = f.read().split('\n')

# Building empty lists to hold sentences
input_docs = []
target_docs = []
# Building empty vocabulary sets
input_tokens = set()
target_tokens = set()

for line in lines:
    # Input and target sentences are separated by tabs
    input_doc, target_doc = line.split('\t')
    # Appending each input sentence to input_docs
    input_docs.append(input_doc)
    # Splitting words from punctuation
    target_doc = " ".join(re.findall(r"[\w']+|^[^s\w]", target_doc))
    # Redefine target_doc below
    # and append it to target_docs:
    target_doc = '<START> ' + target_doc + ' <END>'
    target_docs.append(target_doc)

# Now we split up each sentence into words
# and add each unique word to our vocabulary set
for token in re.findall(r"[\w']+|^[^s\w]", input_doc):
    print(token)

```

```

# Add your code here:
if token not in input_tokens:
    input_tokens.add(token)
for token in target_doc.split():
    print(token)
# And here:
if token not in target_tokens:
    target_tokens.add(token)

input_tokens = sorted(list(input_tokens))
target_tokens = sorted(list(target_tokens))

# Create num_encoder_tokens and num_decoder_tokens:
num_encoder_tokens = len(input_tokens)
num_decoder_tokens = len(target_tokens)

max_encoder_seq_length = max([len(re.findall(r"[\w']+|^s\w]", input_doc)) for input_doc in input_docs])
max_decoder_seq_length = max([len(re.findall(r"[\w']+|^s\w]", target_doc)) for target_doc in target_docs])

```

Generating Text with Deep Learning

Training Setup (part 2)

At this point we need to fill out the 1s in each vector. We can loop over each English-Spanish pair in our training sample using the features dictionaries to add a 1 for the token in question. For example, the dog sentence (`["the", "dog", "licked", "me"]`) would be split into the following matrix of vectors:

```

[
  [1, 0, 0, 0], # timestep 0 => "the"
  [0, 1, 0, 0], # timestep 1 => "dog"
  [0, 0, 1, 0], # timestep 2 => "licked"
  [0, 0, 0, 1], # timestep 3 => "me"
]

```

Copy to Clipboard

You'll notice the vectors have timesteps — we use these to track where in a given document (sentence) we are.

To build out a three-dimensional NumPy matrix of one-hot vectors, we can assign a value of 1 for a given word at a given timestep in a given line:

```
matrix_name[line, timestep, features_dict[token]] = 1.
```

Copy to Clipboard

Keras will fit — or train — the seq2seq model using these matrices of one-hot vectors:

- the encoder input data
- the decoder input data
- the decoder target data

Hang on a second, why build two matrices of decoder data? Aren't we just encoding and decoding?

The reason has to do with a technique known as *teacher forcing* that most seq2seq models employ during training. Here's the idea: we have a Spanish input token from the previous timestep to help train the model for the current timestep's target token.

target sentence

<START> we like kiwis.

Decoder

training output

Instructions

Checkpoint 1 Passed

1.

Inside the first nested `for` loop, assign `1.` for the current `line`, `timestep`, and `token` in `encoder_input_data`.

For example, if we wanted to assign `1.` for the current `line`, `timestep`, and `token` in `some_numpy_matrix`:

```
some_numpy_matrix[line, timestep, features_dict[token]] = 1.
```

Copy to Clipboard

In our case, use `input_features_dict[token]` as the third argument of the index.

Checkpoint 2 Passed

2.

Inside the second nested `for` loop, assign `1.` for the current `line`, `timestep`, and `token` in `decoder_input_data`.

For example, if we wanted to assign `1.` for the current `line`, `timestep`, and `token` in `some_numpy_matrix`:

```
some_numpy_matrix[line, timestep, features_dict[token]] = 1.
```

Copy to Clipboard

In our case, use `target_features_dict[token]` as the third argument of the index.

Checkpoint 3 Passed

3.

Inside the second nested for loop, assign 1. for the current line, the previous timestep (at timestep - 1), and token in decoder_target_data if timestep is greater than 0.

For example, if we wanted to assign 1. for the current line, the previous timestep, and token in

some_numpy_matrix:

```
some_numpy_matrix[line, timestep - 1, features_dict[token]] = 1.
```

```
from tensorflow import keras
import numpy as np
import re
from preprocessing import input_docs, target_docs, input_tokens, target_tokens,
num_encoder_tokens, num_decoder_tokens, max_encoder_seq_length, max_decoder_seq_length

input_features_dict = dict(
    [(token, i) for i, token in enumerate(input_tokens)])
target_features_dict = dict(
    [(token, i) for i, token in enumerate(target_tokens)])

reverse_input_features_dict = dict(
    (i, token) for token, i in input_features_dict.items())
reverse_target_features_dict = dict(
    (i, token) for token, i in target_features_dict.items())

encoder_input_data = np.zeros(
    (len(input_docs), max_encoder_seq_length, num_encoder_tokens),
    dtype='float32')
decoder_input_data = np.zeros(
    (len(input_docs), max_decoder_seq_length, num_decoder_tokens),
    dtype='float32')
decoder_target_data = np.zeros(
    (len(input_docs), max_decoder_seq_length, num_decoder_tokens),
    dtype='float32')

for line, (input_doc, target_doc) in enumerate(zip(input_docs, target_docs)):

    for timestep, token in enumerate(re.findall(r"[\w']+|^[\s\w]", input_doc)):

        print("Encoder input timestep & token:", timestep, token)
        # Assign 1. for the current line, timestep, & word
        # in encoder_input_data:
        encoder_input_data[line, timestep, input_features_dict[token]] = 1.

    for timestep, token in enumerate(target_doc.split()):

        # decoder_target_data is ahead of decoder_input_data by one timestep
        print("Decoder input timestep & token:", timestep, token)
        # Assign 1. for the current line, timestep, & word
        # in decoder_input_data:
        decoder_input_data[line, timestep, target_features_dict[token]] = 1.
        if timestep > 0:
            # decoder_target_data will be ahead by one timestep
            # and will not include the start token.
            print("Decoder target timestep:", timestep)
            # Assign 1. for the current line, timestep, & word
            # in decoder_target_data:
            decoder_target_data[line, timestep - 1, target_features_dict[token]] = 1.
```

Generating Text with Deep Learning

Encoder Training Setup

It's time for some deep learning!

Deep learning models in Keras are built in layers, where each layer is a step in the model.

Our encoder requires two layer types from Keras:

An input layer, which defines a matrix to hold all the one-hot vectors that we'll feed to the model.

An LSTM layer, with some output dimensionality.

We can import these layers as well as the model we need like so:

```
from keras.layers import Input, LSTM
from keras.models import Model
```

Copy to Clipboard

Next, we set up the input layer, which requires some number of dimensions that we're providing. In this case, we know that we're passing in all the encoder tokens, but we don't necessarily know our batch size (how many chocolate chip cookies sentences we're feeding the model at a time). Fortunately, we can say `None` because the code is written to handle varying batch sizes, so we don't need to specify that dimension.

```
# the shape specifies the input matrix sizes
encoder_inputs = Input(shape=(None, num_encoder_tokens))
```

Copy to Clipboard

For the LSTM layer, we need to select the dimensionality (the size of the LSTM's hidden states, which helps determine how closely the model molds itself to the training data — something we can play around with) and whether to return the state (in this case we do):

```
encoder_lstm = LSTM(100, return_state=True)
# we're using a dimensionality of 100
# so any LSTM output matrix will have
# shape [batch_size, 100]
```

Copy to Clipboard

Remember, the only thing we want from the encoder is its final states. We can get these by linking our LSTM layer with our input layer:

```
encoder_outputs, state_hidden, state_cell = encoder_lstm(encoder_inputs)
```

Copy to Clipboard

`encoder_outputs` isn't really important for us, so we can just discard it. However, the states, we'll save in a list:

```
encoder_states = [state_hidden, state_cell]
```

Copy to Clipboard

There is a lot to take in here, but there's no need to memorize any of this — you got this.👊

Instructions

Checkpoint 1 Passed

1.

We've moved the code from the previous exercises into another file to give you some room (and to speed things up a bit for you).

The necessary modules are imported, so now it's up to you to set up the encoder layers and retrieve the states. Ready?

First, define an input layer, `encoder_inputs`. Give its shape:

a batch size of `None`

number of tokens set to `num_encoder_tokens`

Checkpoint 2 Passed

2.

Build the LSTM layer called `encoder_lstm` with a dimensionality of 256 that will return the output state.

For example to create an LSTM layer called `some_lstm` with a dimensionality of 100 that returns the output state:

```
some_lstm = LSTM(100, return_state=True)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Call `encoder_lstm` on `encoder_inputs` to retrieve the following return values:

`encoder_outputs`

`state_hidden`

`state_cell`

Now, create a list of the two states and assign it to a new variable: `encoder_states`.

We can define multiple variables from return values of a function call like this:

```
potatoes, rice, eggs = buy_groceries(7.00)
```

```
from prep import num_encoder_tokens

from tensorflow import keras
from keras.layers import Input, LSTM
from keras.models import Model

# Create the input layer:
encoder_inputs = Input(shape=(None, num_encoder_tokens))

# Create the LSTM layer:
encoder_lstm = LSTM(256, return_state=True)

# Retrieve the outputs and states:
encoder_outputs, state_hidden, state_cell = encoder_lstm(encoder_inputs)

# Put the states together in a list:
encoder_states = [state_hidden, state_cell]
```

Generating Text with Deep Learning

Decoder Training Setup

The decoder looks a lot like the encoder (phew!), with an input layer and an LSTM layer that we use together:

```
decoder_inputs = Input(shape=(None, num_decoder_tokens))
decoder_lstm = LSTM(100, return_sequences=True, return_state=True)
# This time we care about full return sequences
```

Copy to Clipboard

However, with our decoder, we pass in the state data from the encoder, along with the decoder inputs. This time, we'll keep the output instead of the states:

```
# The two states will be discarded for now
decoder_outputs, decoder_state_hidden, decoder_state_cell =
    decoder_lstm(decoder_inputs, initial_state=encoder_states)
```

Copy to Clipboard

We also need to run the output through a final activation layer, using the Softmax function, that will give us the probability distribution – where all probabilities sum to one – for each token. The final layer also transforms our

LSTM output from a dimensionality of whatever we gave it (in our case, 10) to the number of unique words within the hidden layer's vocabulary (i.e., the number of unique target tokens, which is definitely more than 10!).

```
decoder_dense = Dense(num_decoder_tokens, activation='softmax')

decoder_outputs = decoder_dense(decoder_outputs)
```

Copy to Clipboard

Keras's implementation could work with several layer types, but `Dense` is the least complex, so we'll go with that. We also need to modify our import statement to include it before running the code:

```
from keras.layers import Input, LSTM, Dense
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

If you take a look at `script.py`, you'll see that we've already set up the decoder input and LSTM layers for you. Now it's your turn to grab the `decoder_outputs`, `decoder_state_hidden`, and `decoder_state_cell` by calling the decoder LSTM layer on `decoder_inputs`. This time though, pass in the `encoder_states` as the `initial_state`.

Checkpoint 2 Passed

2.

Alright, time for something new! Add `Dense` to the layer types you're importing from Keras. Then, build the final Dense layer `decoder_dense` layer. Pass in the following arguments:

- the number of decoder tokens
- an activation of "softmax"

For example, to build a Dense layer called `dense_layer` with `num_tokens` and an activation function of "tanh", we could do the following:

```
dense_layer = Dense(num_tokens, activation='tanh')
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Run those `decoder_outputs` through the Dense layer you just created. Assign the resulting value to `decoder_outputs`.

For example, to run `sandwich` through a `panini_press()` function:

```
sandwich = panini_press(sandwich)
```

Copy to Clipboard

In this case, the function should be `decoder_dense()` and the variable that is reset is `decoder_outputs`.

```
PREP.py =

import pickle
```

```
input_docs, target_docs, input_tokens, target_tokens, num_encoder_tokens,
num_decoder_tokens, max_encoder_seq_length, max_decoder_seq_length,
input_features_dict, target_features_dict, reverse_input_features_dict,
reverse_target_features_dict, encoder_input_data, decoder_input_data,
decoder_target_data = pickle.load(open("seq2seq_prep.p", "rb"))
```

```
from prep import num_encoder_tokens, num_decoder_tokens

from tensorflow import keras
# Add Dense to the imported layers
from keras.layers import Input, LSTM, Dense
from keras.models import Model

# Encoder training setup
encoder_inputs = Input(shape=(None, num_encoder_tokens))
encoder_lstm = LSTM(256, return_state=True)
encoder_outputs, state_hidden, state_cell = encoder_lstm(encoder_inputs)
encoder_states = [state_hidden, state_cell]

# The decoder input and LSTM layers:
decoder_inputs = Input(shape=(None, num_decoder_tokens))
decoder_lstm = LSTM(256, return_sequences=True, return_state=True)

# Retrieve the LSTM outputs and states:
decoder_outputs, decoder_state_hidden, decoder_state_cell =
decoder_lstm(decoder_inputs, initial_state=encoder_states)

# Build a final Dense layer:
decoder_dense = Dense(num_decoder_tokens, activation='softmax')

# Filter outputs through the Dense layer:
decoder_outputs = decoder_dense(decoder_outputs)
```

Generating Text with Deep Learning

Build and Train seq2seq

Alright! Let's get model-building!

First, we define the seq2seq model using the `Model()` function we imported from Keras. To make it a seq2seq model, we feed it the encoder and decoder inputs, as well as the decoder output:

```
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
```

Copy to Clipboard

Finally, our model is ready to train. First, we compile everything. Keras models demand two arguments to compile:

- An optimizer (we're using RMSprop, which is a fancy version of the widely-used [gradient descent](#)) to help minimize our error rate (how bad the model is at guessing the true next word given the previous words in a sentence).

- A loss function (we're using the logarithm-based cross-entropy function) to determine the error rate.

Because we care about accuracy, we're adding that into the metrics to pay attention to while training. Here's what the compiling code looks like:

```
model.compile(optimizer='rmsprop',
```

```
loss='categorical_crossentropy',
metrics=['accuracy'])
```

Copy to Clipboard

Next we need to fit the compiled model. To do this, we give the `.fit()`

[method](#)

Preview: Docs Loading link description

the encoder and decoder input data (what we pass into the model), the decoder target data (what we expect the model to return given the data we passed in), and some numbers we can adjust as needed:

batch size (smaller batch sizes mean more time, and for some problems, smaller batch sizes will be better, while for other problems, larger batch sizes are better)

the number of

[epochs](#)

Preview: Docs Loading link description

or cycles of training (more epochs mean a model that is more trained on the dataset, and that the process will take more time)

validation split (what percentage of the data should be set aside for validating — and determining when to stop training your model — rather than training)

Keras will take it from here to get you a (hopefully) nicely trained seq2seq model:

```
model.fit([encoder_input_data, decoder_input_data],
          decoder_target_data,
          batch_size=10,
          epochs=100,
          validation_split=0.2)
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Run the code as is to see a summary of the training model. You may notice the following layers included:

two input layers (one for the encoder, one for the decoder)

two LSTM layers (one for the encoder, one for the decoder)

one Dense layer (for the decoder)

The summary should look something like this: `` Model: "model_1"

	Layer
(type) Output Shape Param # Connected to	
input_1 (InputLayer) (None, None, 15) 0	
	input_2
(InputLayer) (None, None, 23) 0	
	lstm_1
(LSTM) [(None, 10), (None, 1040 input_1[0][0]	
	lstm_2
(LSTM) [(None, None, 10), (1360 input_2[0][0]	
lstm_1[0][1]	
lstm_1[0][2]	
	dense_1
(Dense) (None, None, 23) 253 lstm_2[0][0]	

Total params: 2,653 Trainable params: 2,653 Non-trainable params: 0 ``

Checkpoint 2 Passed

2.

Next up, we need to compile the model. Below the training setup and where the model is built, compile `training_model` using the `RMSprop optimizer`, `categorical cross-entropy loss`, and `metrics` of accuracy.

Please note: for the remainder of this lesson, running the code may take a bit more time! Keras requires more time and resources than other libraries you have worked with here.

For example, to compile a model called `fish_training`:

```
fish_training.compile(optimizer='rmsprop',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Fitting time! Because we don't want to crash this exercise, we'll make the batch size large and the number of epochs very small. (Note that small batch sizes are more prone to crashing a deep learning program in general, but in our case we care about time.)

We've provided the `batch_size` and `epochs` variables in `script.py`. Update them as follows:

`batch_size` to 50

`epochs` to 50

Fit training_model with `epochs` set to `epochs`, `batch_size` set to `batch_size`, and a `validation_split` of 0.2.

For example, to train a model called `model` with a batch size of 10, epochs set to 100, and a validation split of 0.2, it might look like this:

```
model.fit([encoder_input_data, decoder_input_data],
          decoder_target_data,
          batch_size=10,
          epochs=100,
          validation_split=0.2)
```

```
from prep import num_encoder_tokens, num_decoder_tokens, decoder_target_data,
encoder_input_data, decoder_input_data, decoder_target_data
```

```
from tensorflow import keras
```

```
# Add Dense to the imported layers
```

```
from keras.layers import Input, LSTM, Dense
```

```
from keras.models import Model
```

```
# Encoder training setup
```

```
encoder_inputs = Input(shape=(None, num_encoder_tokens))
```

```
encoder_lstm = LSTM(256, return_state=True)
```

```
encoder_outputs, state_hidden, state_cell = encoder_lstm(encoder_inputs)
```

```
encoder_states = [state_hidden, state_cell]
```

```
# Decoder training setup:
```

```
decoder_inputs = Input(shape=(None, num_decoder_tokens))
```

```
decoder_lstm = LSTM(256, return_sequences=True, return_state=True)
```

```
decoder_outputs, decoder_state_hidden, decoder_state_cell =
```

```
decoder_lstm(decoder_inputs, initial_state=encoder_states)
```

```
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
```

```
decoder_outputs = decoder_dense(decoder_outputs)
```

```
# Building the training model:
```

```
training_model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
```

```
training_model.summary()
```

```
# Compile the model:
```

```
training_model.compile(optimizer='rmsprop', loss='categorical_crossentropy',
                      metrics=['accuracy'])
```

```
# Choose the batch size
```

```
# and number of epochs:
```

```
batch_size = 50
```

```
epochs = 50

# Train the model:
training_model.fit([encoder_input_data, decoder_input_data], decoder_target_data,
batch_size=batch_size, epochs=epochs, validation_split=0.2)
```

Generating Text with Deep Learning

Setup for Testing

Now our model is ready for testing! Yay! However, to generate some original output text, we need to redefine the seq2seq architecture in pieces. Wait, didn't we just define and train a model?

Well, yes. But the model we used for training our network only works when we already know the target sequence. This time, we have no idea what the Spanish should be for the English we pass in! So we need a model that will decode step-by-step instead of using teacher forcing. To do this, we need a seq2seq network in individual pieces.

To start, we'll build an encoder model with our encoder inputs and the placeholders for the encoder's output states:

```
encoder_model = Model(encoder_inputs, encoder_states)
```

Copy to Clipboard

Next up, we need placeholders for the decoder's input states, which we can build as input layers and store together. Why? We don't know what we want to decode yet or what hidden state we're going to end up with, so we need to do everything step-by-step. We need to pass the encoder's final hidden state to the decoder, sample a token, and get the updated hidden state back. Then we'll be able to (manually) pass the updated hidden state back into the network:

```
latent_dim = 256
decoder_state_input_hidden = Input(shape=(latent_dim,))

decoder_state_input_cell = Input(shape=(latent_dim,))

decoder_states_inputs = [decoder_state_input_hidden, decoder_state_input_cell]
```

Copy to Clipboard

Using the decoder LSTM and decoder dense layer (with the activation function) that we trained earlier, we'll create new decoder states and outputs:

```
decoder_outputs, state_hidden, state_cell =
    decoder_lstm(decoder_inputs,
        initial_state=decoder_states_inputs)

# Saving the new LSTM output states:
decoder_states = [state_hidden, state_cell]

# Below, we redefine the decoder output
# by passing it through the dense layer:
decoder_outputs = decoder_dense(decoder_outputs)
```

Copy to Clipboard

Finally, we can set up the decoder model. This is where we bring together:

- the decoder inputs (the decoder input layer)
- the decoder input states (the final states from the encoder)
- the decoder outputs (the NumPy matrix we get from the final output layer of the decoder)
- the decoder output states (the memory throughout the network from one word to the next)

```
decoder_model = Model(
    [decoder_inputs] + decoder_states_inputs,
    [decoder_outputs] + decoder_states)
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

As you may have noticed, we moved everything to another file again. We also [saved the training model on its own as an HDF5 file](#), which we are loading back up in `script.py`.

We already created the encoder test model and built two decoder state input layers.

Before we get into the decoder side, put those two state input layers into a list called `decoder_states_inputs` – first the hidden, then the cell.

Checkpoint 2 Passed

2.

Now that we have the decoder state inputs organized, we can pass them, along with the decoder inputs, through the decoder LSTM layer to get testing decoder outputs and states.

Call `decoder_lstm()` with the following arguments:

```
decoder_inputs
initial_state=decoder_states_inputs
```

Save the resulting return values to `decoder_outputs`, `state_hidden`, and `state_cell`.

Then save these two new states into a new list: `decoder_states`.

Your code should look like:

```
decoder_outputs, state_hidden, state_cell =
    decoder_lstm(decoder_inputs,
        initial_state=decoder_states_inputs)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Time to redefine the `decoder_outputs` you got from the `decoder_lstm()` call.

Pass this output through the `decoder_dense()` layer and save the resulting value back to `decoder_outputs`.

For example, to redefine `potatoes` by passing it through a function `mash()`:

```
potatoes = mash(potatoes)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

And finally we can create the decoder test model!

Build a Keras model called `decoder_model` with the following arguments passed in:

```
[decoder_inputs] + decoder_states_inputs
[decoder_outputs] + decoder_states
```

```
from training import encoder_inputs, decoder_inputs, encoder_states, decoder_lstm, decoder_dense

from tensorflow import keras
from keras.layers import Input, LSTM, Dense
from keras.models import Model, load_model

training_model = load_model('training_model.h5')
# These next lines are only necessary
# because we're using a saved model:
encoder_inputs = training_model.input[0]
encoder_outputs, state_h_enc, state_c_enc = training_model.layers[2].output
encoder_states = [state_h_enc, state_c_enc]

# Building the encoder test model:
encoder_model = Model(encoder_inputs, encoder_states)

latent_dim = 256
# Building the two decoder state input layers:
decoder_state_input_hidden = Input(shape=(latent_dim,))
decoder_state_input_cell = Input(shape=(latent_dim,))

# Put the state input layers into a list:
decoder_states_inputs = [decoder_state_input_hidden, decoder_state_input_cell]

# Call the decoder LSTM:
decoder_outputs, state_hidden, state_cell = decoder_lstm(decoder_inputs,
    initial_state=decoder_states_inputs)

decoder_states = [state_hidden, state_cell]
```

```
# Redefine the decoder outputs:
decoder_outputs = decoder_dense(decoder_outputs)

# Build the decoder test model:
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs] +
decoder_states)
```

```
_TEST.PY

load_file_in_context("script.py")

inputs_list = [decoder_state_input_hidden, decoder_state_input_cell]

try:
    decoder_TEST.PY
    # Attempt to access a variable identifier:
    if decoder_states_inputs != inputs_list:
        fail_tests("Expected `decoder states inputs` to be set equal to `[decoder state input hidden,
decoder_state_input_cell]`.")

except NameError:
    fail_tests("Expected `decoder_states_inputs` to be defined.")

pass_tests()
```

```
PREP.PY

import pickle

input docs, target docs, input tokens, target tokens, num encoder tokens, num decoder tokens,
max encoder seq length, max decoder seq length, input features dict, target features dict,
reverse input features dict, reverse target features dict, encoder input data,
decoder_input_data, decoder_target_data = pickle.load(open("seq2seq_prep.p", "rb"))
```

```
TRAINING.PY

from prep import num_encoder_tokens, num_decoder_tokens, decoder_target_data, encoder_input_data,
decoder_input_data, decoder_target_data

from tensorflow import keras
# Add Dense to the imported layers
from keras.layers import Input, LSTM, Dense
from keras.models import Model

# Encoder training setup
encoder_inputs = Input(shape=(None, num_encoder_tokens))
encoder_lstm = LSTM(256, return_state=True)
encoder_outputs, state_hidden, state_cell = encoder_lstm(encoder_inputs)
encoder_states = [state_hidden, state_cell]

# Decoder training setup:
decoder_inputs = Input(shape=(None, num_decoder_tokens))
decoder_lstm = LSTM(256, return_sequences=True, return_state=True)
decoder_outputs, decoder_state_hidden, decoder_state_cell = decoder_lstm(decoder_inputs,
initial_state=encoder_states)
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_outputs)
```

The Test Function

Finally, we can get to testing our model! To do this, we need to build a function that:

- accepts a NumPy matrix representing the test English sentence input

- uses the encoder and decoder we've created to generate Spanish output

Inside the test function, we'll run our new English sentence through the encoder model. The `.predict()` method takes in new input (as a NumPy matrix) and gives us output states that we can pass on to the decoder:

```
# test_input is a NumPy matrix
# representing an English sentence
states = encoder.predict(test_input)
```

Copy to Clipboard

Next, we'll build an empty NumPy array for our Spanish translation, giving it three dimensions:

```
# batch size: 1
# number of tokens to start with: 1
# number of tokens in our target vocabulary
target_sequence = np.zeros((1, 1, num_decoder_tokens))
```

Copy to Clipboard

Luckily, we already know the first value in our Spanish sentence — "<Start>". So we can give "<Start>" a value of 1 at the first timestep:

```
target_sequence[0, 0, target_features_dict['<START>']] = 1.
```

Copy to Clipboard

Before we get decoding, we'll need a string where we can add our translation to, word by word:

```
decoded_sentence = ''
```

Copy to Clipboard

This is the variable that we will ultimately return from the function.

Instructions

Checkpoint 1 Passed

1.

Use `encoder_model` to encode the `test_input` we pass into the `decode_sequence()` function and save the resulting states as `encoder_states_value`.

Because we're transferring the states from the encoder to the decoder, it's helpful to keep track of this transfer here. Create a new variable `decoder_states_value` and set it equal to `encoder_states_value`.

To pass input called `new_input` through an encoder model's `.predict()` method:

```
new_states = some_encoder.predict(new_input)
```

Copy to Clipboard

To set one variable equal to another variable's value:

```
apple = orange
# apple is now equal to the value of orange
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Generate a NumPy array of zeros called `target_seq` with the following tuple argument passed in:

```
(1, 1, num_decoder_tokens)
```


Copy to Clipboard

To generate a NumPy array of zeros:

```
target_seq = np.zeros( (tuple, of, some, size) )
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Now populate `target_seq`'s first token with a value of 1. where "<START>" is.
You won't see any decoded sentences yet. We'll get to that in the next exercise!

For example, if the target sequence were called `spanish_output`, the code should look something like this:

```
spanish_output[0, 0, target_features_dict['<START>']] = 1.
```

```
from training import encoder_inputs, decoder_inputs, encoder_states, decoder_lstm,
decoder_dense, encoder_input_data, num_decoder_tokens

from prep import target_features_dict, reverse_target_features_dict,
max_decoder_seq_length, input_docs, target_docs, target_tokens

from tensorflow import keras
from keras.layers import Input, LSTM, Dense
from keras.models import Model, load_model
import numpy as np

training_model = load_model('training_model.h5')
encoder_inputs = training_model.input[0]
encoder_outputs, state_h_enc, state_c_enc = training_model.layers[2].output
encoder_states = [state_h_enc, state_c_enc]
encoder_model = Model(encoder_inputs, encoder_states)

latent_dim = 256
decoder_state_input_hidden = Input(shape=(latent_dim,))
decoder_state_input_cell = Input(shape=(latent_dim,))
decoder_states_inputs = [decoder_state_input_hidden, decoder_state_input_cell]
decoder_outputs, state_hidden, state_cell = decoder_lstm(decoder_inputs,
initial_state=decoder_states_inputs)
decoder_states = [state_hidden, state_cell]
decoder_outputs = decoder_dense(decoder_outputs)
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs] +
decoder_states)

def decode_sequence(test_input):
    # Encode the input as state vectors:
    encoder_states_value = encoder_model.predict(test_input)
    # Set decoder states equal to encoder final states
    decoder_states_value = encoder_states_value

    # Generate empty target sequence of length 1:
    target_seq = np.zeros((1, 1, num_decoder_tokens))

    # Populate the first token of target sequence with the start token:
    target_seq[0, 0, target_features_dict['<START>']] = 1.

    decoded_sentence = ''

    return decoded_sentence
```

```

for seq_index in range(10):
    test_input = encoder_input_data[seq_index: seq_index + 1]
    decoded_sentence = decode_sequence(test_input)
    print('-')
    print('Input sentence:', input_docs[seq_index])
    print('Decoded sentence:', decoded_sentence)

```

Generating Text with Deep Learning

Test Function (part 2)

At long last, it's translation time. Inside the test function, we'll decode the sentence word by word using the output state that we retrieved from the encoder (which becomes our decoder's initial hidden state). We'll also update the decoder hidden state after each word so that we use previously decoded words to help decode new ones.

To tackle one word at a time, we need a `while` loop that will run until one of two things happens (we don't want the model generating words forever):

- The current token is "`<END>`".

- The decoded Spanish sentence length hits the maximum target sentence length.

Inside the `while` loop, the decoder model can use the current target sequence (beginning with the "`<START>`" token) and the current state (initially passed to us from the encoder model) to get a bunch of possible next words and their corresponding probabilities. In Keras, it looks something like this:

```

output_tokens, new_decoder_hidden_state, new_decoder_cell_state =
    decoder_model.predict(
        [target_seq] + decoder_states_value)

```

Copy to Clipboard

Next, we can use NumPy's `.argmax()` method to determine the token (word) with the highest probability and add it to the decoded sentence:

```

# slicing [0, -1, :] gives us a
# specific token vector within the
# 3d NumPy matrix
sampled_token_index = np.argmax(
    output_tokens[0, -1, :])

# The reverse features dictionary
# translates back from index to Spanish
sampled_token = reverse_target_features_dict[
    sampled_token_index]

decoded_sentence += " " + sampled_token

```

Copy to Clipboard

Our final step is to update a few values for the next word in the sequence:

```

# Move to the next timestep
# of the target sequence:
target_seq = np.zeros((1, 1, num_decoder_tokens))
target_seq[0, 0, sampled_token_index] = 1.

# Update the states with values from
# the most recent decoder prediction:
decoder_states_value = [

```

```
new_decoder_hidden_state,  
new_decoder_cell_state]
```

Copy to Clipboard

And now we can test it all out!

You may recall that, because of platform constraints here, we're using very little data. As a result, we can only expect our model to translate a handful of sentences coherently. Luckily, you will have an opportunity to try this out on your own computer with far more data to see some much more impressive results.

Instructions

Checkpoint 1 Passed

1.

We've added the `while` loop in, along with the stop conditions.

Now it's your turn to fill in the rest...

Call `decoder_model.predict()` on `[target_seq] + decoder_states_value` to get `output_tokens`, `new_decoder_hidden_state`, and `new_decoder_cell_state` respectively.

Checkpoint 2 Passed

2.

Next up, we'll get the most probable next word in the sequence (according to our model) and add it to the decoded sentence.

Define `sampled_token_index` as `np.argmax()` called on `output_tokens[0, -1, :]`.

Use `sampled_token_index` to find our next word in `reverse_target_features_dict`. Save the result to `sampled_token`, replacing the empty string `""`.

Add the word to `decoded_sentence`. (Make sure you add a space before the word.)

Checkpoint 3 Passed

3.

Reset `target_seq` as a NumPy array of zeros with dimensions `(1, 1, num_decoder_tokens)`.

Give `target_seq[0, 0, sampled_token_index]` a value of 1. This one-hot vector now represents the token we just sampled.

And, finally, update `decoder_states_value` with a list of the new `new_decoder_hidden_state` and `new_decoder_cell_state` (in this order).

How does the Spanish look to you?

```
from training import encoder_inputs, decoder_inputs, encoder_states, decoder_lstm,  
decoder_dense, encoder_input_data, num_decoder_tokens  
  
from prep import target_features_dict, reverse_target_features_dict,  
max_decoder_seq_length, input_docs, target_docs, target_tokens  
  
from tensorflow import keras  
from keras.layers import Input, LSTM, Dense  
from keras.models import Model, load_model  
import numpy as np  
  
training_model = load_model('training_model.h5')  
encoder_inputs = training_model.input[0]  
encoder_outputs, state_h_enc, state_c_enc = training_model.layers[2].output  
encoder_states = [state_h_enc, state_c_enc]  
encoder_model = Model(encoder_inputs, encoder_states)  
  
latent_dim = 256  
decoder_state_input_hidden = Input(shape=(latent_dim,))  
decoder_state_input_cell = Input(shape=(latent_dim,))  
decoder_states_inputs = [decoder_state_input_hidden, decoder_state_input_cell]  
decoder_outputs, state_hidden, state_cell = decoder_lstm(decoder_inputs,  
initial_state=decoder_states_inputs)  
decoder_states = [state_hidden, state_cell]
```

```

decoder_outputs = decoder_dense(decoder_outputs)
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs] +
decoder_states)

def decode_sequence(test_input):
    encoder_states_value = encoder_model.predict(test_input)
    decoder_states_value = encoder_states_value
    target_seq = np.zeros((1, 1, num_decoder_tokens))
    target_seq[0, 0, target_features_dict['<START>']] = 1.
    decoded_sentence = ''

    stop_condition = False
    while not stop_condition:
        # Run the decoder model to get possible
        # output tokens (with probabilities) & states
        output_tokens, new_decoder_hidden_state, new_decoder_cell_state =
decoder_model.predict(
            [target_seq] + decoder_states_value)

        # Choose token with highest probability
        sampled_token_index = np.argmax(output_tokens[0, -1, :])
        sampled_token = reverse_target_features_dict[sampled_token_index]
        decoded_sentence += " " + sampled_token

        # Exit condition: either hit max length
        # or find stop token.
        if (sampled_token == '<END>' or len(decoded_sentence) > max_decoder_seq_length):
            stop_condition = True

        # Update the target sequence (of length 1).
        target_seq = np.zeros((1, 1, num_decoder_tokens))
        target_seq[0, 0, sampled_token_index] = 1.

        # Update states
        decoder_states_value = [new_decoder_hidden_state, new_decoder_cell_state]

    return decoded_sentence

for seq_index in range(11):
    test_input = encoder_input_data[seq_index: seq_index + 1]
    decoded_sentence = decode_sequence(test_input)
    print('-')
    print('Input sentence:', input_docs[seq_index])
    print('Decoded sentence:', decoded_sentence)

```

Generating Text with Deep Learning

Generating Text with Deep Learning Review

Congrats! You've successfully built a machine translation program using deep learning with Tensorflow's Keras

[API](#)

Preview: Docs An API, or Application Programming Interface, is a term used to describe specifications that allow applications to communicate with one another. APIs enable exchange of information and can be a major source of value to organizations. They can be divided into three groups: Public APIs, Private APIs, and Partner APIs.

While the translation output may not have been quite what you expected, this is just the beginning. There are many ways you can improve this program on your own device by using a larger data set, increasing the size of the model, and adding more

[epochs](#)

Preview: Docs Loading link description for training.

You could also convert the one-hot vectors into word embeddings during training to improve the model. Using embeddings of words rather than one-hot vectors would help the model capture that semantically similar words might have semantically similar embeddings (helping the LSTM generalize).

You've learned quite a bit, even beyond the Keras implementation:

seq2seq models are deep learning models that use recurrent neural networks like LSTMs to generate output.

In machine translation, seq2seq networks encompass two main parts:

The encoder accepts language as input and outputs state vectors.

The decoder accepts the encoder's final state and outputs possible translations.

Teacher forcing is a

[method](#)

Preview: Docs Loading link description

we can use to train seq2seq decoders.

We need to mark the beginning and end of target sentences so that the decoder knows what to expect at the beginning and end of sentences.

One-hot vectors are a way to represent a given word in a set of words wherein we use 1 to indicate the current word and 0 to indicate every other word.

Timesteps help us keep track of where we are in a sentence.

We can adjust batch size, which determines how many sentences we give a model at a time.

We can also tweak dimensionality and number of epochs, which can improve results with careful tuning.

The Softmax function converts the output of the LSTMs into a probability distribution over words in our vocabulary.

Instructions

Try running the code a few times to see different output. Did the program get any of the words translated correctly? (You can use [Google Translate](#) to see how accurate the translations were.)

```
from training import encoder_inputs, decoder_inputs, encoder_states, decoder_lstm,
decoder_dense, encoder_input_data, num_decoder_tokens

from prep import target_features_dict, reverse_target_features_dict,
max_decoder_seq_length, input_docs, target_docs, target_tokens

from tensorflow import keras
from keras.layers import Input, LSTM, Dense
```

```

from keras.models import Model, load_model
import numpy as np

training_model = load_model('training_model.h5')
encoder_inputs = training_model.input[0]
encoder_outputs, state_h_enc, state_c_enc = training_model.layers[2].output
encoder_states = [state_h_enc, state_c_enc]
encoder_model = Model(encoder_inputs, encoder_states)

latent_dim = 256
decoder_state_input_hidden = Input(shape=(latent_dim,))
decoder_state_input_cell = Input(shape=(latent_dim,))
decoder_states_inputs = [decoder_state_input_hidden, decoder_state_input_cell]
decoder_outputs, state_hidden, state_cell = decoder_lstm(decoder_inputs,
initial_state=decoder_states_inputs)
decoder_states = [state_hidden, state_cell]
decoder_outputs = decoder_dense(decoder_outputs)
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs] +
decoder_states)

def decode_sequence(test_input):
    encoder_states_value = encoder_model.predict(test_input)
    decoder_states_value = encoder_states_value
    target_seq = np.zeros((1, 1, num_decoder_tokens))
    target_seq[0, 0, target_features_dict['<START>']] = 1.
    decoded_sentence = ''

    stop_condition = False
    while not stop_condition:
        # Run the decoder model to get possible
        # output tokens (with probabilities) & states

        # Choose token with highest probability
        sampled_token = ""

        # Exit condition: either hit max length
        # or find stop token.
        if (sampled_token == '<END>' or len(decoded_sentence) > max_decoder_seq_length):
            stop_condition = True

        # Update the target sequence (of length 1).

        # Update states

    return decoded_sentence

for seq_index in range(10):
    test_input = encoder_input_data[seq_index: seq_index + 1]
    decoded_sentence = decode_sequence(test_input)
    print('--')
    print('Input sentence:', input_docs[seq_index])
    print('Decoded sentence:', decoded_sentence)

```

Read the following:

Deep Learning with Python by Francois Chollet

Read Now

In this book, you will learn concepts in deep learning through Python's powerful Keras library. This is helpful if you want to build your own deep learning models from scratch and explore applications in computer vision, natural-language processing, and generative models.

Read the following selection for free: Deep Learning with Python, Francois Chollet [Chapter 1](#).

Off-Platform Project: Machine Translation

Use Keras models with seq2seq neural networks to build a better translation tool.

Now that you have a basic understanding of how to use Keras models and seq2seq neural networks for some pretty rudimentary machine translation, it's time to take that code off Codecademy's platform and make it a lot better.

Setting Up

Installing Python 3

The Codecademy platform lets you run Python easily in our learning environment, but to run/write Python on your own computer you'll need to install Python on it, if you haven't already.

[This article will walk through installing Python 3 on your computer.](#)

Installing Tensorflow, Keras, and NumPy

First [install Tensorflow](#). This will be the deep learning back end for Keras.

Now [install the Keras API](#).

To get NumPy, we recommend installing the NumPy package using [Anaconda](#) or [Miniconda](#). If you don't have a package and environment manager, you can also install NumPy through `pip`:

```
python -m pip install --user numpy
```

Download your dataset

Select and download a [language pair dataset for your translator here](#). We encourage you to pick a language you have some familiarity with so you can better check the accuracy of the translations you generate. If you don't have familiarity with any of the languages listed, don't fear! You can also use Google Translate to get a sense of how well your model is working.

Unzip the folder and take a look at the **[your-language].txt** file in a notepad or code editor. You should see sentence pairs of English and the target language you picked.

Set Up The Code

[Download and unzip the machine translation code](#) we used in the lesson. You should have following files:

```
preprocessing.py
training_model.py
test_function.py
```

While you could also have all of the code combined into one file, we wanted to break everything down to make it easier to understand how each piece works.

You can move the **[your-language].txt** file into the same directory (folder) as the machine translation code to make it slightly easier to set up.

Preprocessing

Open **preprocessing.py** in a code editor or IDE.

Change `data_path` to the file path of **[your-language].txt**. If it's in the same directory as **preprocessing.py**, then all you need is the file name.

In **preprocessing.py**, adjust the number of lines of training data you want to work with. We're giving you a default of 500, but depending on how much you want to tax your computer, you can go up to 123,000 lines. Whatever you choose, this should be a much higher number than what we've used on the Codecademy platform.

Run the code and make sure everything works, error-free. This may take a moment because you have a lot more training data than we used on the Codecademy platform. Remember that more training data leads to better results for machine translation.

At the end of **preprocessing.py**, print out `list(input_features_dict.keys())[:50]`, `reverse_target_features_dict[50]`, and the length of `input_tokens`. Does each look how you expected?

The Training Model

Open **training_model.py**. This is where the training model is built and trained.

Change the values for the following:

latent_dim: Choose a latent dimensionality for your model. Keras's documentation uses 256, but you can adjust as you see fit.

batch_size: You can choose to adjust this or not at this point. This determines how many sentences are used at a time for training.

epochs: This should be a larger number (Keras's documentation uses 100) so that the seq2seq model has many chances to improve. Bear in mind that a larger number of epochs will also take your computer a lot longer to process. If you don't have the ability to leave your computer running for several hours for this project, then choose a number that is less than 100.

Run the code to generate your model. In the terminal, you should see a summary of the model printed out. Meanwhile, you'll also see a new file in the directory called **training_model.h5**. This is where your seq2seq training model is saved so that it's quicker for you to run your code during testing.

Note that you may get the following error when attempting to run your program on a regular computer that uses CPU processing:

```
OMP: Error #15: Initializing libiomp5.dylib, but found libiomp5.dylib
already initialized.
OMP: Hint: This means that multiple copies of the OpenMP runtime have
been linked into the program. That is dangerous, since it can degrade
performance or cause incorrect results. The best thing to do is to
ensure that only a single OpenMP runtime is linked into the process,
e.g. by avoiding static linking of the OpenMP runtime in any library.
As an unsafe, unsupported, undocumented workaround you can set the
environment variable KMP_DUPLICATE_LIB_OK=TRUE to allow the program to
continue to execute, but that may cause crashes or silently produce
incorrect results. For more information, please see
http://www.intel.com/software/products/support/.
Abort trap: 6
```

Below the import statements on **training.py**, there's a couple lines commented out, which you can uncomment to make the program run. However, if you are concerned about your computer crashing, you may want to hold off completing this project until you have access to a faster computer with GPU processing.

The Test Model and Function

Open **test_function.py**. You'll see that we've imported several variables from the preprocessing and training steps of the program.

In the `for` loop at the bottom of the file, increase the `range` if you want to see more sentences translated — this is up to you. Of course, more sentences will increase the amount of time your computer will require for the translation.

When you feel ready for your computer to spend awhile training and translating, run the code. This is going to take awhile — from 20 minutes to several hours, so make sure your computer is plugged in. You should see each epoch appear in the terminal as a fraction of the total number of epochs you selected. For example, 16/100 indicates the program has reached the 16th epoch out of 100 total epochs. You'll also see the loss go down for each epoch, which is very exciting — this is the model getting more accurate!

When your computer finishes the full process, you'll see the translations appear. You can use a speaker of that language or Google Translate to see how accurate they are.

Congratulations on setting up your first deep learning program locally!

BONUS

Some ways to improve:

Try including more lines of training data in **preprocessing.py** to see if your translations improve (this may also crash the program if there are too many!).

Try out different values for the `epochs`, `latent_dim`, and `batch_size` to see how they change the performance of the model.

Add in a step to convert new text into a NumPy matrix so that you can translate new sentences that aren't in the dataset. (This may also require handling unknown tokens.)

Review: Text Generation

See what you've learned in Text Generation.

Review

Congratulations! The goal of this unit was to introduce current common methods of generating text. Having completed this unit, you are now able to:

- Recognize why recurrent neural networks and specifically long short-term memory (LSTM) networks are helpful for working with sequences of text.
- Convert text into a matrix of one-hot vectors.
- Implement a seq2seq network using LSTM layers with teacher forcing.
- Understand how teacher forcing works.
- Use deep learning to conduct machine translation.

If you are interested in learning more about these topics, here are some additional resources:

- Documentation: [Keras: Natural Language Processing](#)
- Tutorial: [TensorFlow: Neural Machine Translation with Attention](#)
- Book: [Deep Learning with Python, François Chollet](#)

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Welcome to Build Chatbots

Python + Chatbots = Fun

Welcome you to Build Chatbots with Python! If you are interested in NLP, this might be one of the most exciting units in this path. You've done a lot of work to get here, and we are excited to dig in and learn all about building chatbots!

In addition to building increasingly complex chatbots, you'll also learn about:

- what chatbots are
- what types of chatbots exist
- how chatbots have changed over time
- what ethical considerations to take when building chatbots
- the basics of natural language processing
- a high-level overview of what neural networks and deep learning are (without all the math)
- how to apply deep learning models for natural language processing purposes

By the end of this skill path, you'll have the tools to create different kinds of Python-based chatbots on your own. Ready to get started?

User Input

How to assign variables with user input.

So far, we've covered how to assign variables values directly in a Python file. However, we often want a user of a program to enter new information into the program.

How can we do this? As it turns out, another way to assign a value to a variable is through user input.

While we output a variable's value using `print()`, we assign information to a variable using `input()`. The `input()` function requires a prompt message, which it will print out for the user before they enter the new information. For example:

```
likes_snakes = input("Do you like snakes? ")
```

In the example above, the following would occur:

```
"Do you like snakes? "
"yes! I have seven pythons as pets!"
likes_snakes
```

enter

Try constructing a statement to collect user input on your own!

Fill in the blank

Questions

Fill in the blanks in the code to complete a statement that asks a user "What is your favorite flightless bird?" and then stores their answer in the variable `favorite_flightless_bird`.

Code

```
blank 1
=
blank 2
(
blank 3
)
```

Answer Choices

user.input
input
favorite_flightless_bird
get_input
prompt
"What is your favorite flightless bird?"

Click or drag and drop to fill in the blank

Not only can `input()` be used for collecting all sorts of different information from a user, but once you have that information stored as a variable you can use it to simulate interaction:

```
>>> favorite_fruit = input("What is your favorite fruit? ")
What is your favorite fruit? mango

>>> print("Oh cool! I like " + favorite_fruit + " too, but I think my favorite fruit
is apple.")
Oh cool! I like mango too, but I think my favorite fruit is apple.
```

These are pretty basic implementations of `input()`, but as you get more familiar with Python you'll find more and more interesting scenarios where you will want to interact with your users.

What are Chatbots

[Learn about the different types of chatbots and how they work.](#)

Chatbots all around

From the customer support dialog boxes you find on e-commerce websites to virtual assistants like Siri and Alexa, it's likely that you've encountered chatbots frequently in your everyday life. Like its name suggests, a chatbot is a piece of software designed to conduct a conversation or dialog. They can be found in a wide range of industries to serve a variety of purposes, ranging from providing customer support to aiding in therapy to simply being a source of fun and entertainment. Let's take a closer look at how chatbots are able to do what they do!

How chatbots work

To interact with the human user, chatbots must be able to:

- parse the user input
- interpret what it means
- provide an appropriate response or output

For example, a user query could be, "Show me hotels in Los Angeles for tomorrow."

A good chatbot will be able to identify the *intent* and *entities* of the query. The intent is the purpose or category of the user query, such as to retrieve a list of hotels. Entities are extra information that describes the user's intent. In this case, the entities are "Los Angeles" and "tomorrow." With these pieces of information, chatbots should be able to respond to the user with a list of available hotels for the correct location and date.

Types of chatbots

Chatbots generally fall into a few broad categories, depending on what purpose they are designed to serve.

Response architecture models

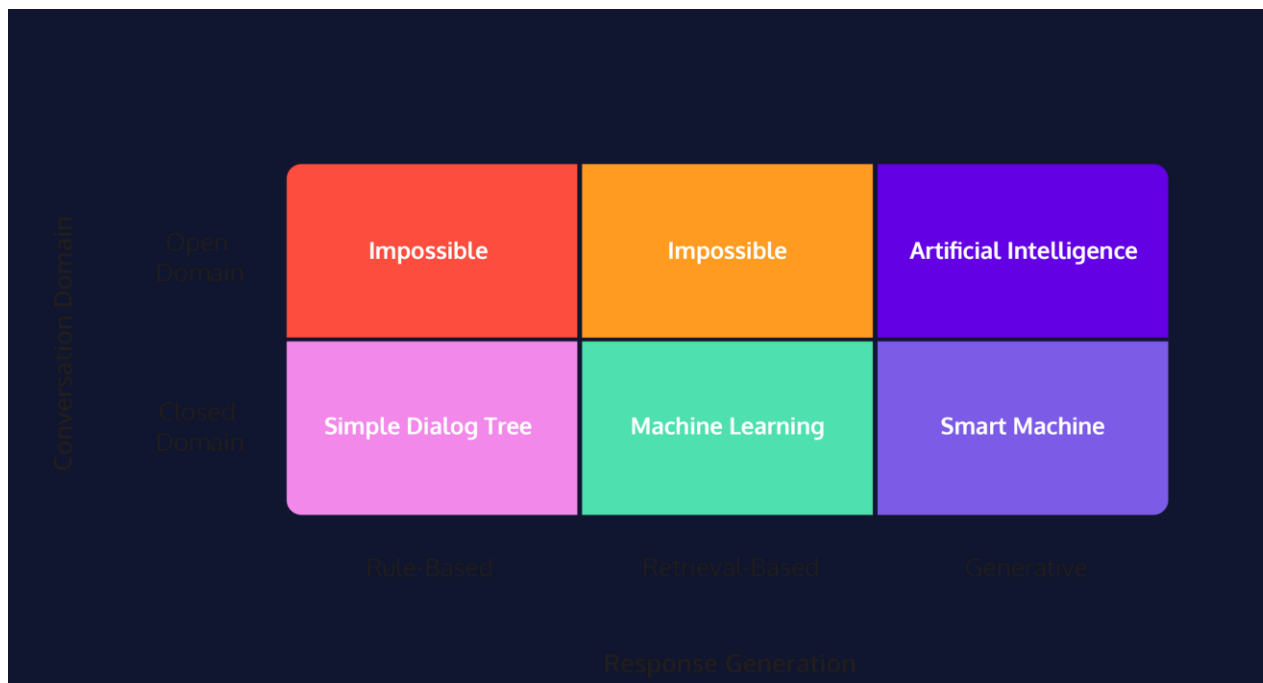
First, chatbots can be categorized according to how they generate the response that gets returned to the user. The simplest approach is the *rule-based* model, where chatbot responses are entirely predefined and returned to the user according to a series of rules. This includes decision trees that have a clear set of possible outputs defined for each step in the dialog.

Next, there is the *retrieval-based* model, where chatbot responses are pulled from an existing corpus of dialogs. Machine learning models, such as statistical NLP models and sometimes supervised neural networks, are used to interpret the user input and determine the most fitting response to retrieve. Like rule-based models, retrieval-based models rely on predefined responses, but they have the additional ability to self-learn and improve their selection of response over time.

Finally, *generative* chatbots are capable of formulating their own original responses based on user input, rather than relying on existing text. This involves the use of deep learning, such as LSTM-based seq2seq models, to train the chatbots to be able to make decisions about what is an appropriate response to return.

While generative models are very flexible and powerful in that they are not confined to a predefined set of rules or responses, they are also significantly more challenging to implement. Training these chatbots require an abundance of data, and it is often unclear what gets used for their decision-making, making them more prone to grammatical errors and nonsensical replies. By contrast, retrieval-based models can guarantee the quality of the responses since they are predefined, but these chatbots are in turn restricted to language that exists within the training data.

Thus, chatbots often use a combination of the different models in order to produce optimal results. For example, a customer support chatbot may use generative models for creating open-ended small talk with the user, but then are able to retrieve professional, predefined responses for answering the user's inquiries regarding the business or product.



Conversation domains

Chatbots can also be categorized based on the range of conversation topics they are able to cover. *Closed domain* chatbots, or *dialog agents*, are restricted to providing responses with a particular focus, such as booking a hotel room. Because they are designed with a specific goal in mind, these chatbots are often very efficient and have great success in accomplishing what they are intended to accomplish. The user-perceived quality is also high, because users don't expect the chatbots to provide responses outside of the pre-established domain.

On the other hand, *open domain* chatbots, or *conversational agents*, are capable of exploring any range of conversation topics, much like how a human-to-human interaction would be. Many of these "companion bots" have filled the roles of a friend or therapist, allowing the user to connect with them on an emotional level. While they have great potential, open domain chatbots are challenging to implement and evaluate.

It is worthwhile to note that the term "chatbot" is sometimes reserved only for open domain conversational agents, but for the purpose of this course, we include closed domain dialog agents as well.

Initiatives

Another way chatbots can be categorized is by which side – the user or bot – is able to take initiative on the conversation. Looking back at our previous example on hotel search, notice how the user is free to provide their request in their own words, and the chatbot is able to identify and piece together the relevant keywords to answer the query. This is an example of a *mixed-initiative* system, representative of a normal human-to-human conversation where all participants have the chance to take initiative.

By contrast, a *system-initiative* system is one where the chatbot controls the conversation and explicitly asks for each piece of information, such as the date and location for the hotel booking. While this system is more straight-forward to implement because user response can be anticipated, it lacks the flexibility and naturalness that characterize a normal human dialog.

Into the future

Chatbots have come a long way. What started out as computers that attempted to mimic human conversation has grown into elaborate systems that are able to carry out a multitude of functions and goals. But this great power and potential doesn't come without a price. As chatbots continue to evolve, there are also growing ethical concerns that arise. We will discuss this in more detail in the next article!

<https://www.codecademy.com/paths/data-science-nlp/tracks/dsnlp-build-chatbots/modules/chatbot-evolution-and-dialog-trees/articles/history-of-chatbots>

History of Chatbots

Learn about how chatbots grew from simple pattern recognition software to sophisticated machine learning-based models.

The History of Chatbots

In October of 1950, Alan Turing proposed an approach for evaluating a computer's intelligence and famously named his method, The Imitation Game. The premise is that an interrogator talks to two people through a "typewritten" machine (today we would refer to this as instant messaging). The catch is that only one of the conversations is with a real person – the other is with a computer.

Turing posited that, by the turn of the century (the year 2000), in a well-controlled experiment, a computer should be able to fool the interrogator 70% of the time. While we've made a lot of progress since 1950, no algorithm has consistently reached this bar. However, there has still been substantial progress in the field of chatbot development, which has led to a multi-billion dollar industry and dozens of profitable products.

Before examining the history of chatbot development, read through the conversation we had with two different chatbots: ELIZA and Mitsuku. One of these models is still in development, while the other was programmed in 1966. Can you guess which is the newest?

Conversation with ELIZA:

Codecademy: Hey Eliza, what do you want to talk about?

ELIZA: You're not really talking about me -- are you?

Codecademy: I asked you what you wanted to talk about

ELIZA: Do you wish to asked I what me?

Codecademy: I don't understand. What?

ELIZA: Why don't you understand?

Codecademy: Because your question doesn't make sense

ELIZA: What about your own question doesn't make sense?

Conversation with MITSUKU:

Codecademy: hey mitsuku, what do you want to talk about?

MITSUKU: I like to talk about movies, robots and history.

Codecademy: what is your favorite movie?

MITSUKU: My favourite movie is Terminator. Have you seen it?

Codecademy: yeah, I saw it a while ago.

MITSUKU: What's your favorite scene? What did you think about it?
What did it look like?

Codecademy: I like the whole thing. I can't just pick one scene out of it. The ending is pretty good though.

If you guessed Mitsuku, nice work. While Mitsuku was more coherent than ELIZA, I still wouldn't describe the interaction as a "normal" conversation with a stranger.

The progression from ELIZA to Mitsuku didn't happen overnight. There were a lot of innovations and creative chatbots in between. Let's take a deeper look at them.



The Chatbots

ELIZA

ELIZA was developed by Joseph Weizenbaum at MIT Laboratories in 1966 and was the first chatbot that made a meaningful attempt to beat the Turing Test. It used pattern-recognition to pick out patterns in a person's speech, then repeated the words back to the person in a premade template. The most famous implementation of the ELIZA chatbot is DOCTOR. In this implementation, the chatbot acts like a psychotherapist, responding to a patient's statements by selecting a phrase from the respondent and parroting them back in the form of a question. This form of chatbot is rule-based, because the program responds to a person based on rules that a developer establishes in a predefined script. However, as you saw above, the responses from a DOCTOR-programmed chatbot quickly become incoherent.

PARRY

Kenneth Colby developed PARRY while at Stanford University in 1968. Colby built PARRY with a similar rule-based method to ELIZA. However, it was designed to model the behavior of a person with diagnosable paranoia. PARRY also had a richer response library and was able to simulate the mood of a person by shifting weights of mood parameters. PARRY would respond differently based on the distribution between mood parameters for anger, fear, or mistrust. PARRY passed a modified Turing Test by fooling people who tried to distinguish the difference between it and a person with paranoia.

A.L.I.C.E.(Artificial Linguistic Internet Computer Entity)

Richard Wallace started developing A.L.I.C.E. in 1995, shortly before leaving his computer vision teaching job. Wallace improved upon ELIZA's implementation by continuing to watch the model while it had conversations with people. If a person asked an A.L.I.C.E. bot something it did not recognize, Wallace would add a response for it. In this way, the person who designs an A.L.I.C.E.-powered device could continuously modify it by adding responses to unrecognized phrases. This means that well-developed A.L.I.C.E. bots can respond to a variety of questions and statements based on the developer's needs. In a chapter from the 2009 book, *Parsing the Turing Test*, Richard Wallace described this process as supervised learning, because the developer – who he calls the botmaster – can supervise the learning of the model. \

In 2000, 2001, and 2004, A.L.I.C.E., won the *Loebner prize*, which is a contest that was started in 1980 to award computer programs that are the most human-like. The Loebner prize is awarded to the bot that performs the best on the Turing Test. The judges of the Loebner competition have two conversations simultaneously. One with a real person and the other with a bot. The winner of the competition is the one that tricks a judge the highest percentage of the time.

Jabberwacky

Jabberwacky was developed by Rollo Carpenter in the 1980s and was launched on the web in 1997. It was the first chatbot that tried to incorporate voice interaction. Two versions of Jabberwacky won the Loebner prize in 2005 and 2006. Jabberwacky has undergone continuous development since it debuted on the web. When it launched, it used a similar rule-based approach to previous models, like ELIZA and PARRY. However, in 2008, the model was renamed Cleverbot and updated to include a method for learning without the supervision of a botmaster. Cleverbot can parse and save human responses to questions, and respond similarly if a human asked it the same question.

Mitsuku

Mitsuku was developed by Steve Worswick during the early 2000s and first won the Loebner prize in 2013. The model is still actively developed and has won the Loebner Prize in 2016, 2017, 2018, and 2019, making it the most human-like chatbot available. Mitsuku works more like the A.L.I.C.E. It is a supervised learning model, where developers actively tweak the rules to make interactions with Mitsuku more human-like.

Over 60 years ago, Alan Turing predicted we wouldn't be able to distinguish humans from robots by now. If Turing saw the progress we've made, would he be impressed? It's impossible to know. However, it's unlikely he would have predicted the market potential of chatbot technology – some expect Alexa sales will be \$19 billion by 2021. Products like Alexa, however, are highly integrated, drawing on many other technologies and systems in addition to chatbot software. Alexa falls well short of competing with Mitsuku in carrying out a conversation.

With the growing market for chatbot-driven technologies, there is far more money and interest in chatbot development than just a few years ago. In 2017, Amazon started the Alexa challenge, where teams compete to develop a chatbot that has the best conversation with users. The winners of this challenge receive \$500,000, and Amazon tests each model's ability to converse on Alexa devices.

We've made a lot of progress developing bots to beat the Imitation Game, but there is still progress to be made. Based on the growing interest in the field, it's safe to expect significant progress in the coming years.

Rule-Based Chatbots

Introduction

Have you ever sought customer support help? Maybe you needed to pay a bill or make an inquiry about your bank account. There's a good chance the first interaction you had was with a chatbot. Many companies, including airlines, [use rule-based chatbots](#) to handle customer support requests, like reporting flight delays and providing customers with boarding passes.

Rule-based chatbots use regular expression patterns to match user input to human-like responses that simulate a conversation with a real person. For example, if a customer wanted to check the status of their KLM flight, they may have an interaction like this:

```
- USER: Hey, do you know if flight 3984 is on time?
- KLM CHATBOT: Unfortunately, flight 3984 is delayed 30 minutes. Can I help you with anything else?
- USER: Nope, that's too bad.
- KLM CHATBOT: Okay, sorry for the inconvenience. Have a good day.
```

Copy to Clipboard

In the conversation above, the chatbot matched each statement by the user, and responded with something that made sense.

Many chatbots, including the airline example above, are called *dialog agents*, or *closed domain* chatbots because they are limited to conversations on a specific subject, such as checking flight status or getting a boarding pass.

In this lesson, you will learn how to build a closed domain, rule-based chatbot. We will introduce you to important rule-based concepts such as utterances, intents, and entities.

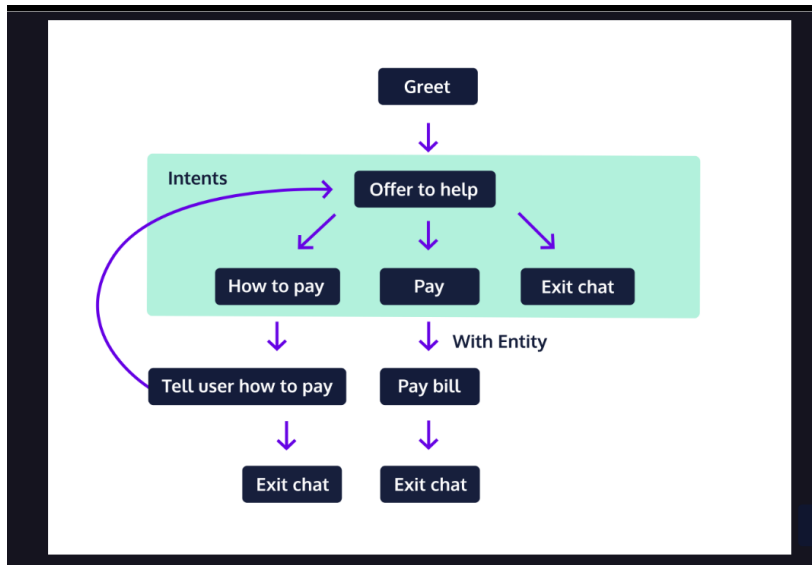
Instructions

1. The graphic to the right shows the architecture of a rule-based customer support chatbot that you will build in this lesson. Before moving onto the next exercise, take a moment to look for the following components in this chatbot tree:

The entry and exit points of the chatbot

The types of user queries the chatbot can handle

How a user would interact with the chatbot if they wanted to pay their bill



Rule-Based Chatbots

Greeting the User

The first step for any rule-based chatbot is greeting the user and asking them how the chatbot can help. In this exercise, we will walk you through the following four steps to accomplish just that:

Get the name of the user

Ask the user, by name, if they need help

Exit the conversation if the user does not want help

Return the user's help request if they want help

Let's walk through the code for each of these steps.

1. Get the user's name

Throughout this lesson, we will use the `input` function to solicit responses from a user. For example, to greet a user and ask for their name, we would write:

```
name = input("Hi, I'm a customer support representative. What is your name? ")
```

Copy to Clipboard

The user's response will be saved to the variable, `name`.

2. Ask the user if they need help

Now that you have the user's name, you can insert it into the following question to ask if they need help:

```
will_help = input(f"Okay {name}, what can I help you with? ")
```

Copy to Clipboard

Asking the user if they need help, with their name, is a personal touch and mimics how a human customer support representative would be trained to respond.

3. Exit the conversation if the user does not want help

In `script.py`, there is a `SupportBot` attribute called `negative_responses`, which contains a

[tuple](#)

Preview: Docs Loading link description

of words. We can use this tuple in a conditional to check if the user does not want help:

```
if will_help in self.negative_responses:
    print("Ok, have a great day!")
    return
```

Copy to Clipboard

In this code, if the user responds “nothing” to the question, “What can I help you with?”, the chatbot will wish the user to have a good day and exit.

4. Return the user’s help request if they want help

Finally, if the user wants help, return their response.

```
return will_help
```

Copy to Clipboard

In a later exercise, we will match the text saved to `will_help` to a given intent.

Instructions

Checkpoint 1 Passed

1.

At the bottom of `main.py`, we use the following code to create an instance of the chatbot, then call the `.welcome()` method.

```
# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()
```

Copy to Clipboard

Type the following into the terminal to run the code. You should see `Hello` printed to the terminal.

```
python3 script.py
```

Copy to Clipboard

Make sure that you use `python3` throughout the lesson, as your code may error if you use `python`.

For each checkpoint, click the Check Work button when you’re ready to move to the next checkpoint.

Checkpoint 2 Passed

2.

Inside the `welcome()` method, delete `print("hello")`. In its place, ask the user:

```
Hi, I'm a customer support representative. Welcome to Codecademy Bank.
Before we can help you, I need some information from you. What is your
first name and last name?
```

Copy to Clipboard

Save the user’s response to a variable called `name`.

To run your code, type `python3 script.py` into the terminal.

Use the `input()` method to ask the user their name, then save the response to the variable, `name`. See the example below:

```
variable = input("Ask question")
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Next, use the user’s name to ask them:

```
Okay NAME, what can I help you with?
```

Copy to Clipboard

Substitute name with the value saved to `name`. Save the user's response to `will_help`. Run your code.

You can interpolate a variable's value using an f-string, like the following:

```
temp = "Codecademy"
print(f"You are taking this course on {temp}")
```

Copy to Clipboard

This code block will print:

You are taking this course on Codecademy

Copy to Clipboard

Checkpoint 4 Passed

4.

Check if the user responded with a value in `negative_responses`.
If true, say:

Okay, have a great day!

Copy to Clipboard

and return.

Otherwise, return the value saved to `will_help`.

Use the following format to check if the value is in `negative_responses`:

```
if will_help in self.negative_responses:
    # Print something
    return
```

Copy to Clipboard

After this conditional statement, return `will_help`.

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        pass

    def welcome(self):
        name = input("Hi, I'm a customer support representative. Welcome to Codecademy Bank. Before we can help you, I need some information from you. What is your first name and last name? ")

        will_help = input(f"Okay {name}, what can I help you with?")

        if will_help in self.negative_responses:
            print("Okay, have a great day!")
            return

        return will_help

# Create a SupportBot instance
SupportConversation = SupportBot()
```

```
# Call the .welcome() method
SupportConversation.welcome()
```

Rule-Based Chatbots

Handling the Conversation

The start of most customer support conversations is formulaic. In the last exercise, we welcomed the user, then asked for their name and if they wanted help. Once you're beyond this initial welcome, the conversation can start to go in many different directions. Usually, chatbots have a central

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

to handle the back-and-forth of a conversation.

In **script.py**, we created a method called `.handle_conversation()` that will be our central method to continue responding for as long as a user asks questions.

Because we start our conversation with the `.welcome()` method, the first step is to call our conversation handling method:

```
def welcome(self):
    ...
    self.handle_conversation(will_help)
```

Copy to Clipboard

To handle the indefinite length of a conversation, we use a `while` loop. The `while` loop can check if the user wants to continue the conversation after each response. Let's say a user can only exit the conversation by responding with "stop." Our `while` loop would look something like:

```
def handle_conversation(self, reply):
    while not (reply == "stop"):
        reply = input("How can I help you?")
```

Copy to Clipboard

At the moment, this code will respond "How can I help you?" to every user response but "stop" – not too useful. However, the `while` loop, as you'll see over the next few exercises, provides a lot of flexibility for processing user input and responding with something that makes sense.

Instructions

Checkpoint 1 Passed

1.

In the last exercise, you returned `will_help` at the end of the `.welcome()` method. We removed this line.

In place of it, call `self.handle_conversation(will_help)` so the chatbot continues to converse with the user.

Run **script.py** and see if you can get the chatbot to print:

```
Conversation is being handled
```

Copy to Clipboard

Click the Check Work button when you're ready to move on to the next checkpoint.

Under the conditional statement in `.welcome()`, call `self.handle_conversation(will_help)`.

Checkpoint 2 Passed

2.

The `.handle_conversation()` method accepts one argument, `reply`.

Inside `.handle_conversation()`, implement a `while` loop that continues to ask a user

```
How's it going?
```

Copy to Clipboard
until they respond "stop."

As displayed above, a `while` loop can be used as follows to get a user response and check if it is equal to "stop."

```
while not (reply == "stop"):
    reply = input("How's it going?")
```

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        pass

    def welcome(self):
        name = input("Hi, I'm a customer support representative. Welcome to Codecademy Bank. Before we can help you, I need some information from you. What is your first name and last name? ")

        will_help = input(f"Ok {name}, what can I help you with? ")

        if will_help in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.handle_conversation(will_help)

    def handle_conversation(self, reply):
        while not (reply == "stop"):
            reply = input("How's it going?")

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()
```

Rule-Based Chatbots

Exiting the Conversation

In the last exercise, you made a while loop to respond to a user until they replied "stop." Saying the word "stop" isn't the only, nor is it the typical, way to end a conversation with someone. Instead, you may say something like:

I have to leave, bye.

I need to go. I'll come back later.

To account for the variety of statements someone could make to exit a conversation, we created a [method](#)

Preview: Docs Loading link description

called `.make_exit()`. The purpose of this method is to check whether the user wants to leave the conversation. If they do, it returns `True`. If not, it returns `False`.

The `.make_exit()` method accepts one [argument](#)

Preview: Docs Loading link description

, the user's response. Within the method, we check whether the user's response contains a word that is commonly used to exit a conversation, such as "bye" or "exit." In `script.py`, we save a few of these words in a [tuple](#)

Preview: Docs Loading link description

called `exit_commands`.

```
def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Nice talking with you!")
            return True

    return False
```

Copy to Clipboard

In the code above, we iterate over each word in `self.exit_commands`. If there is a match between the user's input and a word from `self.exit_commands`, the method prints "Ok, have a nice day!" and returns `True`. If there is not a match, then the method returns `False`.

We can incorporate this method call into each loop using the following code:

```
while not make_exit(reply):
    reply = input("something ")
```

Copy to Clipboard

If `.make_exit()` returns `True`, then the loop will stop executing, and the conversation will end.

Instructions

Checkpoint 1 Passed

1.

Use the `self.exit_commands` tuple and a `for` loop in the `.make_exit()` method so it returns `True` if a user wants to exit the conversation. If the user wants to exit the conversation, print:

```
Ok, have a great day!
```

Copy to Clipboard

If the user does not want to exit the conversation, return `False`.

Use the code in the narrative above to help you implement this solution. Make sure that you change the text to

```
"Ok, have a great day!"
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Change the `while` loop in `.handle_conversation()` so it executes as long as `self.make_exit()` returns `False`.

Change the condition in the `while` loop to:

```
not self.make_exit(reply)
```

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        pass
```

```

def welcome(self):
    name = input("Hi, I'm a customer support representative. Welcome to Codecademy Bank. Before we can help you, I need some information from you. What is your first name and last name? ")

    will_help = input(f"Ok {name}, what can I help you with? ")

    if will_help in self.negative_responses:
        print("Ok, have a great day!")
        return

    self.handle_conversation(will_help)

def handle_conversation(self, reply):
    while not self.make_exit(reply):
        reply = input("How can I help you? ")

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()

```

Rule-Based Chatbots

Interpreting User Responses I

At this point, we've built entry and exit paths to our rule-based chatbot. If someone were to interact with it, they would be greeted as if they were talking to a human, and they would be able to stop the conversation with a simple statement, like "goodbye."

Now, we can shift our focus to the conversation. In addition to greeting and exiting, our chatbot will be able to do two other actions:

- Allow a user to pay their bill

- Tell the user how they can pay their bill

We often refer to these actions as *intents*. Before we can trigger an intent, the chatbot must interpret the user's statement. We often refer to the user's statement as an *utterance*. In **script.py**, we added a class variable called `matching_phrases`, which is a

[dictionary](#)

Preview: Docs Loading link description
with the following:

```

self.matching_phrases = {'how_to_pay_bill': [r'.*how.*pay bills.*', r'.*how.*pay my bill.*']}

```

[Copy to Clipboard](#)

This dictionary contains a list with two strings for regular expression matching:

```

r'.*how.*pay bill.*'

```

```
r'.*how.*pay my bill.*'
```

In Exercise 7, we will use this 'how_to_pay_bill' key-value pair to match user inputs like:

"how can I pay my bill?"

"how can I pay bills?"

Instructions

Checkpoint 1 Enabled

1.

In **script.py**, add "pay_bill" as a key to the `matching_phrases` variable. The value for this key should be a list with no more than two regular expressions that will match the following two strings:

```
- "i want to pay my bill"
- "i need to pay my bill"
```

Copy to Clipboard

The regular expression should not match:

```
- "how can I pay my bill"
```

Copy to Clipboard

There are many regular expressions that can work for this checkpoint. First, let's consider an example that will match "i want to pay my bill":

```
r'.*want.*pay my bill.*'
```

Copy to Clipboard

You can use this same format to check for "i need to pay my bill". By including "want" and "need" in your regular expressions, they will not match "how can i pay my bill".

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        self.matching_phrases = {'how_to_pay_bill': [r'.*how.*pay bills.*', r'.*how.*pay my bill.*'], 'pay_bill': [r'.*(want|need) to pay my bill']}

    def welcome(self):
        name = input("Hi, I'm a customer support representative. Welcome to Codecademy Bank. Before we can help you, I need some information from you. What is your first name and last name? ")

        will_help = input(f"Ok {name}, what can I help you with? ")

        if will_help in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.handle_conversation(will_help)

    def handle_conversation(self, reply):
        while not self.make_exit(reply):
            reply = input("How can I help you? ")

    def make_exit(self, reply):
        for exit_command in self.exit_commands:
```

```

        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()

```

Rule-Based Chatbots

Interpreting User Responses II

At this point, our chatbot has a mapping from regular expression patterns that represent user utterance to chatbot intents. In this exercise, we'll use the regular expression library's `.match()`

[method](#)

Preview: Docs Loading link description

to check if a user's utterance matches one of these patterns.

In `script.py`, we added a method called `.match_reply()` that we will use to match user utterances. Let's walk step-by-step through the code for matching user utterances:

1. Iterate over each item in the dictionary

```

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            ...

```

Copy to Clipboard

In the code above, the first `for` loop iterates over each item in the `self.matching_phrases`

[dictionary](#)

Preview: Docs Loading link description

. Inside of this, there is another `for` loop that we use to iterate over the matching patterns in the current list of regex patterns.

2. Check if user utterance matches a regular expression pattern

```

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:

            found_match = re.match(regex_pattern, reply)
            ...

```

Copy to Clipboard

In the code above, we use the `re.match()` function to check if the current regular expression pattern matches the user utterance.

3. Respond if a match was made

```

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            found_match = re.match(regex_pattern, reply)

            if found_match:
                reply = input("Great! I found a matching regular expression. Isn't that cool?")

```

```
    return reply
...

```

Copy to Clipboard

In the code above, we use a conditional to check if `found_match` is `True`. In a future exercise, we will trigger an intent inside of this conditional. Here, we use an `input()` statement to ask another question and then return the reply.

4. Respond if a match was not made

```
def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            found_match = re.match(regex_pattern, reply)
            if found_match:
                reply = input("Great! I found a matching regular expression. Isn't that cool?")
                return reply

    return input("Can you please ask your questions a different way? ")

```

Copy to Clipboard

If the `found_match` variable is `False`, then we return the response to the question, "Can you please ask your questions a different way?"

5. Call `.match_reply()` after every user response

```
def handle_conversation(self, reply):
    while not make_exit(reply):
        reply = match_reply(reply)

```

Copy to Clipboard

Finally, inside the `while` loop of `.handle_conversation()`, we need to call `.match_reply()` so that we check the user's utterance every time we get a response.

Instructions

Checkpoint 1 Passed

1.

Inside of `.match_reply()`, add a nested `for` loop to iterate over every regular expression in `self.matching_phrases`. Inside the inner `for` loop, print the current regular expression. At the end of the method, return `"exit"`.

If you receive the following error message:

```
argument of type 'NoneType' is not iterable

```

Copy to Clipboard

it is because your `.match_reply()` method is not returning anything.

You can fix this by adding `return "exit"` at the bottom of the method.

Run the code from the terminal. You should see something like:

```
$ python3 script.py
Hi, I'm a customer support representative. Welcome to Codecademy Bank.
Before we can help you, I need some information from you. What is your
first name and last name? Codecademy Python
Okay Codecademy Python, what can I help you with? Printing regular
expressions
.*how.*pay bills.*
.*how.*pay my bill.*
.*want.*pay my bill.*
.*need.*pay my bill.*
Ok, have a great day!

```

Copy to Clipboard

Use the following code to loop through each regular expression in `.match_reply()`:

```
for key, values in self.matching_phrases.items():
    for regex_pattern in values:
        # Print something
```

Copy to Clipboard

Checkpoint 2 Passed

2.

To make it easier to test your code, delete the line that prints the regular expression during each iteration.

Inside the nested `for` loop, check if the user utterance matches a regular expression. If it does, respond to the user with an `input()`:

```
- "We matched your utterance to INTENT"
```

Copy to Clipboard

Where `INTENT` is the current `key`.

If it does not, respond to the user with:

```
- "I did not understand you. Can you please ask your question differently?"
```

Copy to Clipboard

Return the user's reply to whichever statement is triggered above.

An example conversation would look like:

```
Hi, I'm a customer support representative. Welcome to Codecademy Bank.
Before we can help you, I need some information from you. What is your
first name and last name? Codecademy Python
Okay Codecademy Python, what can I help you with? i want to pay my bill
We matched your utterance to pay_bill
```

Copy to Clipboard

Completing this checkpoint correctly requires a few steps:

Use the `re.match()` function to check if the regular expression matches the user's input.

If a match is found, return `input(f"We matched your utterance to {key}")`

Outside of both `for` loops, return `input("I did not understand you. Can you please ask your question differently? ")`

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        self.matching_phrases = {'how_to_pay_bill': [r'.*how.*pay bills.*', r'.*how.*pay my bill.*'],
                                   r'pay_bill': [r'.*want.*pay my bill.*', r'.*need.*pay my bill.*']}

    def welcome(self):
        name = input("Hi, I'm a customer support representative. Welcome to Codecademy Bank. Before we can help you, I need some information from you. What is your first name and last name? ")

        will_help = input(f"Ok {name}, what can I help you with? ")

        if will_help in self.negative_responses:
            print("Ok, have a great day!")
            return
```

```

        self.handle_conversation(will_help)

def handle_conversation(self, reply):
    while not self.make_exit(reply):
        reply = self.match_reply(reply)

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            found_match = re.match(regex_pattern, reply)
            if found_match:
                return input(f"We matched your utterance to {key}")

    return input("I did not understand you. Can you please ask your question differently?")

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()

```

Rule-Based Chatbots

Intents

Now that the chatbot is set up to match user input, we can trigger a desirable response. An intent often maps to a function or

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

. For example, if you wanted to say to a virtual assistant, "Hey, play music," it may match the user's utterance to a function called `play_music()`.

In **script.py**, we added two methods to handle our intent actions: `self.how_to_pay_bill_intent()` and `self.pay_bill_intent()`. We can use a conditional statement inside `self.match_reply()` to trigger a method mapped to an intent if a match is found:

```

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            found_match = re.match(regex_pattern, reply)
            if found_match and key == 'how_to_pay_bill':
                return self.how_to_pay_bill_intent()

    return input("I did not understand you. Can you please ask your question again?")

```

Copy to Clipboard

In the above code, we call `self.how_to_pay_bill_intent()` if there is a match and the `key` is equal to `"how_to_pay_bill"`. Additionally, we return output from `self.how_to_pay_bill_intent()`.

Instructions

Checkpoint 1 Passed

1.

Inside `.match_reply()`, add an `elif` condition that calls the `self.pay_bill_intent()` method when a user's utterance matches a regular expression for the corresponding intent. See if you can run the chatbot using the same inputs as given below and get the same response:

```
Hi, I'm a customer support representative. Welcome to Codecademy Bank.
Before we can help you, I need some information from you. What is your
first name and last name? Codecademy Python
Okay Codecademy, what can I help you with? i want to pay my bill
inside self.pay_bill()exit
Ok, have a great day!
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Edit the `self.how_to_pay_bill_intent()` method so it returns a user's response to the following statement:

```
You can pay your bill a couple of ways. 1) online at
bill.codecademybank.com or 2) you can pay your bill right now with me.
Can I help you with anything else?
```

Copy to Clipboard

See if you can run the chatbot and get the following response:

```
Hi, I'm a customer support representative. Welcome to Codecademy Bank.
Before we can help you, I need some information from you. What is your
first name and last name? Codecademy
Okay Codecademy, what can I help you with? how can i pay my bill
You can pay your bill a couple of ways. 1) online at
bill.codecademybank.com or 2) you can pay your bill right now with me.
Can I help you with anything else?exit
Ok, have a great day!
```

Copy to Clipboard

Inside `.how_to_pay_bill_intent()`, return the user's response to the question using the following:

```
return input("You can pay your bill a couple of ways. 1) online at
bill.codecademybank.com or 2) you can pay your bill right now with me. Can I
help you with anything else?")
```

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        self.matching_phrases = {'how_to_pay_bill': [r'.*how.*pay bills.*', r'.*how.*pay
my bill.*'], r'pay_bill': [r'.*want.*pay my bill.*', r'.*need.*pay my bill.*']}

    def welcome(self):
        name = input("Hi, I'm a customer support representative. Welcome to Codecademy
Bank. Before we can help you, I need some information from you. What is your first
name and last name? ")
```



```

will_help = input(f"Ok {name}, what can I help you with? ")

if will_help in self.negative_responses:
    print("Ok, have a great day!")
    return

self.handle_conversation(will_help)

def handle_conversation(self, reply):
    while not self.make_exit(reply):
        reply = self.match_reply(reply)

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            found_match = re.match(regex_pattern, reply)
            if found_match and key == 'how_to_pay_bill':
                return self.how_to_pay_bill_intent()
            if found_match and key == 'pay_bill':
                self.pay_bill_intent()

    return input("I did not understand you. Can you please ask your question again?")

def how_to_pay_bill_intent(self):
    return input("You can pay your bill a couple of ways. 1) online at
bill.codecademybank.com or 2) you can pay your bill right now with me. Can I help you
with anything else?")

def pay_bill_intent(self):
    return input("inside self.pay_bill_intent()")

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()

```

Rule-Based Chatbots

Entities

To improve the functionality of a chatbot, we can also parse the user's response and grab important information. For example, if you wanted to play a song from a virtual assistant, you could say, "Hey, play Beethoven's Fifth Symphony." In this example, Beethoven's Fifth Symphony would be passed into an intent of the virtual assistant that plays music.

We call the information that a chatbot passes from a user statement to a triggered intent an *entity* – also referred to as a *slot*.

In the chatbot we've built in this lesson, we need to collect the user's account number when we call the

`.pay_bill_intent()`

method

Preview: Docs Loading link description

to credit their account. We can do this by adding a regular expression, like the one below, to the `pay_bill` list in `self.matching_phrases`:

```
regex = r'.*want.*pay.*my.*bill.*account.*number.*is (\d+)'
```

Copy to Clipboard

If an utterance matches the above statement the `(\d+)`, called a capture group, will grab the numeric value that follows the pattern. Notice, there is a space between "is" and "`(\d+)`", rather than a `.*` pattern. With the regular expression above, we can use the following to grab and print an account number:

```
reply = "i want to pay my bill. my account number is 888999333"
found_match = re.match(regex, reply)
print(found_match.groups()[0]) # Prints: '888999333'
```

Copy to Clipboard

In the above code, the `found_match` variable contains the account number. We can grab this number using the `.groups()` method. Because there is only one group, we can use `found_match.groups()[0]` to grab the first, and only, group.

Once we have the group, we can pass it into `.pay_bill_intent()`:

```
def match_reply(self):
    ...
    if found_match and (key == 'pay_bill'):
        return self.pay_bill_intent(found_match.groups()[0])
```

Copy to Clipboard

In the above code, we pass the account number, as an entity, into the `self.pay_bill_intent()` method if the `found_match` variable contains the account number. Otherwise, we call the method without passing the account number.

Instructions

Checkpoint 1 Passed

1.

Change the regular expression `r'.*need.*pay my bill.*'` value in the `self.matching_phrases` `pay_bill` list so it matches the following statement:

"i need to pay my bill. my account number is 123456789"

The regular expression should be able to grab the account number at the end of the statement.

Look at the first element in the list associated with the `pay_bill` key. This item is able to match the following statement and grab the digits:

```
- "i want to pay my bill. my account number is 123456789"
```

Copy to Clipboard

Checkpoint 2 Passed

2.

In `.match_reply()`, Change the call to the `.pay_bill_intent()` method, so it passes in the account number.

Use the following to access the captured digits when you pass it into the `self.pay_bill_intent()` method:

```
found_match.groups()[0]
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Inside `.pay_bill_intent()`, get the user's response to the following statement, then return the response:

The account with number ACCOUNTNUMBER was paid off. What else can I help you with?

Copy to Clipboard

In the above code, ACCOUNTNUMBER should be number stored to `account_number`.

Return the user's response to the following:

```
input(f"The account with number {account_number} was paid off. What else can I help you with?")
```

```
import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        self.matching_phrases = {'how_to_pay_bill': [r'.*how.*pay bills.*', r'.*how.*pay my bill.*'], r'pay_bill': [r'.*want.*pay.*my.*bill.*account.*number.*is (\d+)', r'.*need.*pay.*my.*bill.*account.*number.*is (\d+)']}

    def welcome(self):
        name = input("Hi, I'm a customer support representative. Welcome to Codecademy Bank. Before we can help you, I need some information from you. What is your first name and last name? ")

        will_help = input(f"Ok {name}, what can I help you with? ")

        if will_help in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.handle_conversation(will_help)

    def handle_conversation(self, reply):
        while not self.make_exit(reply):
            reply = self.match_reply(reply)

    def make_exit(self, reply):
        for exit_command in self.exit_commands:
            if exit_command in reply:
                print("Ok, have a great day!")
                return True

        return False

    def match_reply(self, reply):
        for key, values in self.matching_phrases.items():
            for regex_pattern in values:
                found_match = re.match(regex_pattern, reply)
                if found_match and key == 'how_to_pay_bill':
                    return self.how_to_pay_bill_intent()
                elif found_match and key == 'pay_bill':
                    return self.pay_bill_intent(found_match.groups()[0])

        return input("I did not understand you. Can you please ask your question again?")
```

```

def how_to_pay_bill_intent(self):
    return input("You can pay your bill a couple of ways. 1) online at
bill.codecademybank.com or 2) you can pay your bill right now with me. Can I help you
with anything else?")

def pay_bill_intent(self, account_number=None):
    return input(f"The account with number {account_number} was paid off. What else
can I help you with?")

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()

```

Rule-Based Chatbots

Review

In this lesson, you learned how to build a rule-based chatbot that is capable of serving a couple of hypothetical user needs. The chatbot that we developed is small and does not offer a lot of functionality. However, you can extend and improve this chatbot to help with additional tasks by adding:

- Regular expressions for utterance matching
- Intents for additional functionality
- Slots to improve information passing

In addition to these features, chatbots can be extended to do far more sophisticated tasks by hooking them up to databases or using them to trigger a call to a human support representative. While these concepts are outside the [scope](#)

Preview: Docs Loading link description

of this lesson, it's worth understanding that a chatbot can be more useful when fit into more complex software.

Instructions

1. There are still a few things we can add to our code to improve upon our existing chatbot. If you're interested in continuing to work on this, try out the following:

Add code that converts all user responses to lowercase values before they are checked by a regular expression.

Try to introduce the concept of memory into the chatbot by saving the user's name to a class variable, and changing your response in the `.how_to_pay_bill_intent()` to:

```

2. You can pay your bill a couple of ways. 1) online at bill.codecademybank.com or
   2) you can pay your bill right now with me. Can I help you with anything else,
   NAME?
3.

```

4. Copy to Clipboard

5. In this example, NAME is the user's name.

```

import re
import random

class SupportBot:
    negative_responses = ("nothing", "don't", "stop", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later")

    def __init__(self):
        self.matching_phrases = {'how_to_pay_bill': [r'.*how.*pay bills.*', r'.*how.*pay
my bill.*'], r'pay_bill': [r'.*want.*pay.*my.*bill.*account.*number.*is (\d+)',
r'.*need.*pay.*my.*bill.*account.*number.*is (\d+)']}

```

```

def welcome(self):
    self.name = input("Hi, I'm a customer support representative. Welcome to
Codecademy Bank. Before we can help you, I need some information from you. What is
your first name and last name? ")

    will_help = input(f"Ok {self.name}, what can I help you with? ")

    if will_help in self.negative_responses:
        print("Ok, have a great day!")
        return

    self.handle_conversation(will_help)

def handle_conversation(self, reply):
    while not self.make_exit(reply):
        reply = self.match_reply(reply)

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

def match_reply(self, reply):
    for key, values in self.matching_phrases.items():
        for regex_pattern in values:
            found_match = re.match(regex_pattern, reply.lower())
            if found_match and key == 'how_to_pay_bill':
                return self.how_to_pay_bill_intent()
            elif found_match and key == 'pay_bill':
                return self.pay_bill_intent(found_match.groups()[0])

    return input("I did not understand you. Can you please ask your question again?")

def how_to_pay_bill_intent(self):
    return input(f"You can pay your bill a couple of ways. 1) online at
bill.codecademybank.com or 2) you can pay your bill right now with me. Can I help you
with anything else, {self.name}? ")

def pay_bill_intent(self, account_number=None):
    return input(f"The account with number {account_number} was paid off. What else
can I help you with?")

# Create a SupportBot instance
SupportConversation = SupportBot()
# Call the .welcome() method
SupportConversation.welcome()

```

Language Modeling and Chatbots

Discover language modeling techniques that will make your chatbots more advanced!

So far, you've explored intent matching, entity recognition, and response selection using Regex and rule-based methods. However, apart from very specific conversations, rule-based chatbots require a significant amount of time dedicated to writing pattern-matching expressions. As you'll soon see, language modeling techniques open up more sophisticated approaches to building closed-domain chatbot systems.

Beginning with bag-of-words (BoW) and term frequency-inverse document frequency (tf-idf), you'll get a chance to try out language modeling tactics to determine word importance relative to a body of text.

Next, you'll see how word embedding strategies like word2vec allow us to measure similarity between words and determine best-fit words for specific scenarios.

Applying your new NLP knowledge, you'll build a retrieval-based chatbot to help potential restaurant patrons learn more about their food options.

Bag-of-Words Language Model

Review of Bag-of-Words

You made it! And you've learned plenty about the bag-of-words language model along the way:

Bag-of-words (BoW) — also referred to as the unigram model — is a statistical language model based on word count.

There are loads of real-world applications for BoW.

BoW can be implemented as a Python

[dictionary](#)

Preview: Docs A dictionary is an unordered set of (key, value) pairs. It provides a way to map pieces of data to each other, and allows for quick access to values associated to keys. The syntax of a dictionary is as follows: pseudo dictionary = { key1: value1, key2: value2, key3: value3

with each key set to a word and each value set to the number of times that word appears in a text.

For BoW, training data is the text that is used to build a BoW model.

BoW test data is the new text that is converted to a BoW vector using a trained features dictionary.

A feature vector is a numeric depiction of an item's salient features.

Feature extraction (or vectorization) is the process of turning text into a BoW vector.

A features dictionary is a mapping of each unique word in the training data to a unique

[index](#)

Preview: Docs Loading link description

. This is used to build out BoW vectors.

BoW has less data sparsity than other statistical models. It also suffers less from overfitting.

BoW has higher perplexity than other models, making it less ideal for language prediction.

One solution to overfitting is language smoothing, in which a bit of probability is taken from known words and allotted to unknown words.

The spam data for this lesson were taken from the [UCI Machine Learning Repository](#).

Dua, D. and Karra Taniskidou, E. (2017). UCI

[Machine Learning](#)

Preview: Docs Loading link description

Repository [archive.ics.uci.edu/ml/]. Irvine, CA: University of California, School of Information and Computer Science.

Instructions

Feel free to play around with the spam data using your new BoW knowledge. You can also use this workspace to try out other bag-of-words use cases.

```
from spam_data import training_spam_docs, training_doc_tokens,
training_labels, training_docs
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

test_text = """
Play around with the spam classifier!
"""

bow_vectorizer = CountVectorizer()

training_vectors = bow_vectorizer.fit_transform(training_docs)
test_vectors = bow_vectorizer.transform([test_text])

spam_classifier = MultinomialNB()
spam_classifier.fit(training_vectors, training_labels)

predictions = spam_classifier.predict(test_vectors)

print("Looks like a normal email!" if predictions[0] == 0 else "You've got
spam!")
```

Working with Text Data | scikit-learn

Working with Text Data | scikit-learn

In this documentation, you will learn about the scikit-learn tools for generating language models like bag-of-words using text data. This is helpful if you would like to analyze a collection of text documents.

Working with Text Data | scikit-learn | From Occurrences to Frequencies

Working with Text Data | scikit-learn | From Occurrences to Frequencies

In this documentation, you will learn how to use scikit-learn to conduct tf-idf. This is helpful if you are trying to determine topics or themes within text data.

Token.similarity | spaCy

spaCy Documentation - Token.similarity

In this documentation, you will learn about the spaCy library for Natural Language Processing in Python. This is helpful if you would like to process and derive insights from unstructured textual data.

Review: Language Quantification

Review

Congratulations! The goal of this unit was to use various methods, including language models and embeddings, to represent text numerically.

Having completed this unit, you are now able to:

- Identify common real-world applications for the bag-of-words language model.
- Convert text into a bag-of-words representation using a Python dictionary.
- Understand how feature dictionaries and feature vectors can be used to build a BoW model.
- Recognize the difference between training data and test data.
- Use scikit-learn to convert text into a bag-of-words vector.
- Identify advantages and disadvantages of BoW in relation to other common language models.
- Understand how tf-idf can allow you to determine word importance within texts.
- Implement tf-idf using scikit-learn.

Reframe word meaning based on context using word embeddings.

Understand how multi-dimensional vectors and distance metrics are useful for numerical representations of language.

Implement word embeddings, including word2vec, with spaCy and gensim.

If you are interested in learning more about these topics, here are some additional resources:

Documentation: [Working with Text Data | scikit-learn](#)

Documentation: [Token.similarity | spaCy](#)

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Introduction: Text Generation

See what's covered in [Text Generation](#).

Goals of this Unit

The goal of this unit is to introduce some common methods of generating text.

After this unit, you will be able to:

Recognize why recurrent neural networks and specifically long short-term memory (LSTM) networks are helpful for working with sequences of text.

Convert text into a matrix of one-hot vectors.

Implement a seq2seq network using LSTM layers with teacher forcing.

Use deep learning to conduct machine translation.

You will put all of this knowledge into practice with an upcoming Portfolio Project. You can complete the Portfolio Project either in parallel with or after taking the prerequisite content—it's up to you!

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Long Short Term Memory Networks

Long short-term memory networks (LSTMs) are often used in deep learning programs for natural language processing.
Information Persistence and Neural Networks

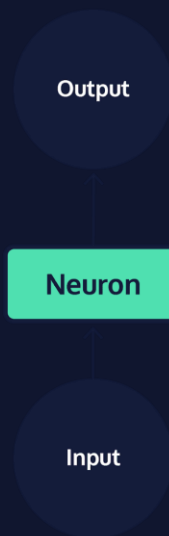
We do not create a new language every time we speak -- every human language has a persistent set of grammar rules and collection of words that we rely on to interpret it. As you read this article, you understand each word based on your knowledge of grammar rules and interpretation of the preceding and following words. At the beginning of the next sentence, you continue to utilize information from earlier in the passage to follow the overarching logic of the paragraph. Your knowledge of language is both persistent across interactions, and adaptable to new conversations.

Neural networks are a machine learning framework loosely based on the structure of the human brain. They are very commonly used to complete tasks that seem to require complex decision making, like speech recognition or image classification. Yet, despite being modeled after neurons in the human brain, early neural network models were not designed to handle temporal sequences of data, where the past depends on the future. As a result, early models performed very poorly on tasks in which prior decisions are a strong predictor of future decisions, as is the case in most human language tasks. However, more recent models, called recurrent neural networks (RNN), have been specifically designed to process inputs in a temporal order and update the future based on the past, as well as process sequences of arbitrary length.

RNNs are one of the most commonly used neural network architectures today. Within the domain of Natural Language Processing, they are often used in speech generation and machine translation tasks. Additionally, they are often used to solve speech recognition and optical character recognition tasks. Let's dive into their implementation!

Neural Networks

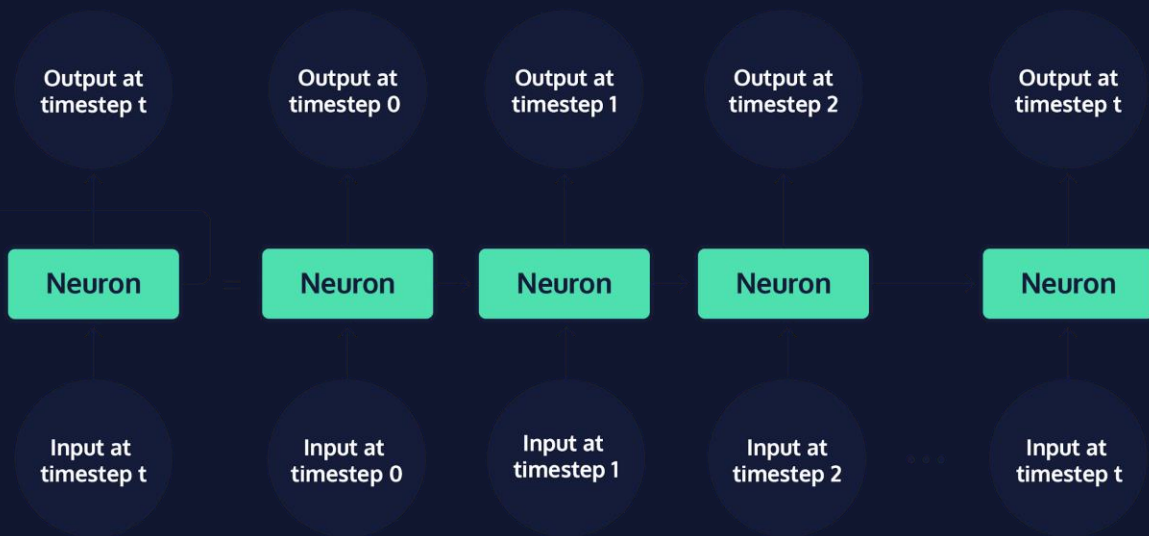
A single neuron in a network, *reference to graphic*, takes in a single piece of input data, *reference to graphic*, and performs some data transformation to produce a single piece of output *reference to graphic*.



The very first network models, called [perceptrons](#), were relatively simple systems that used many combinations of simple functions, like computing the slope of a line. While these transformations were very simple in isolation, together they resulted in sophisticated behavior for the entire system and allowed for large numbers of transformations to be strung together in order to compute advanced functions. However, the range of perceptron applications was still limited, and there were very simple functions that they just simply couldn't compute. In attempting to solve this issue by layering perceptrons together, researchers ran into another problem: few computers at the time were able to store enough data to execute these programs.

Deep Neural Networks

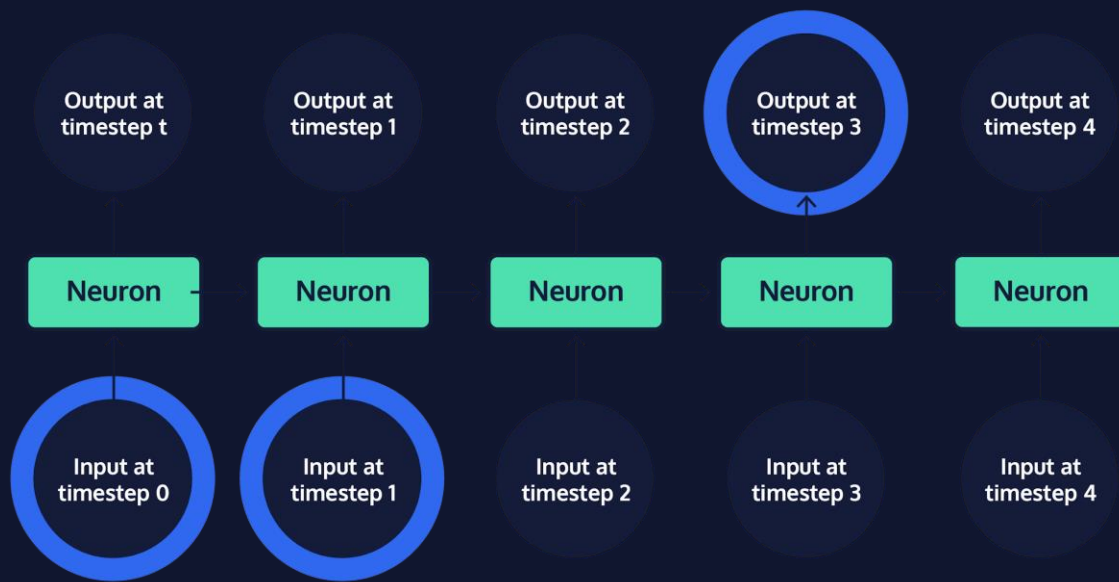
By the early 2000s, when innovations in computer hardware allowed for more complex modeling techniques, researchers had already developed neural network systems that combined many layers of neurons, including convolutional neural networks (CNN), multi-layer perceptrons (MLP), and recurrent neural networks (RNN). All three of these architectures are called deep neural networks, because they have many layers of neurons that combine to create a “deep” stack of neurons. Each of these deep-learning architectures have their own relative strengths:



When the chain of neurons in an RNN is “rolled out,” it becomes easier to see that these models are made up of many copies of the same neuron, each passing information to its successor. Neurons that are not the first or last in a rolled out RNN are sometimes referred to as “hidden” network layers; the first and last neurons are called the “input” and “output” layers, respectively. The chain structure of RNNs places them in close relation to data with a clear temporal ordering or list-like structure — such as human language, where words obviously appear one after another. Standard RNNs are certainly the best fit for tasks that involve sequences, like the translation of a sentence from one language to another.

Long-term Dependencies

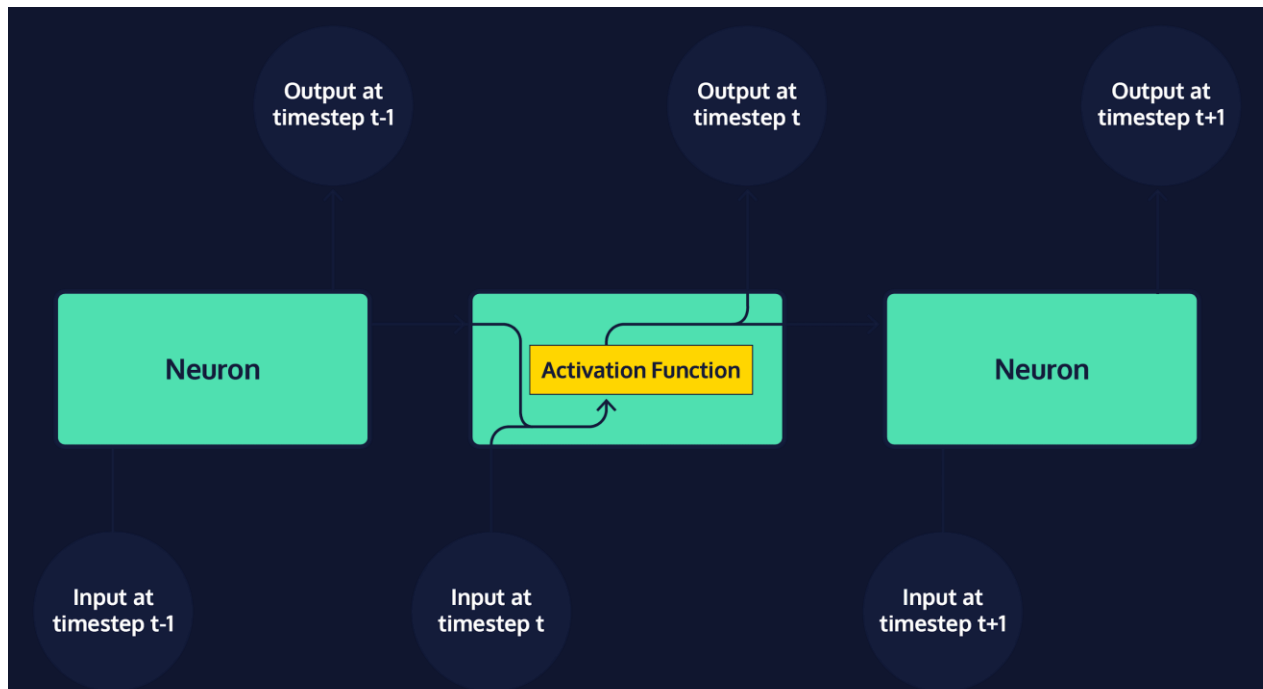
While the broad family of RNNs powers many of today’s advanced artificial intelligence systems, one particular RNN variant, the long short-term memory network (LSTM), has become popular for building realistic chatbot systems. When attempting to predict the next word in a conversation, words that were mentioned recently often provide enough information to make a strong guess. For instance, when trying to complete the sentence “The grass is always: _____,” most English-speakers do not need additional context to guess that the last word is “greener.” Standard RNNs are difficult to train, and fail to capture long-term dependencies well; they perform best on short sequences, when relevant context and the word to be predicted fall within a short distance of one another.



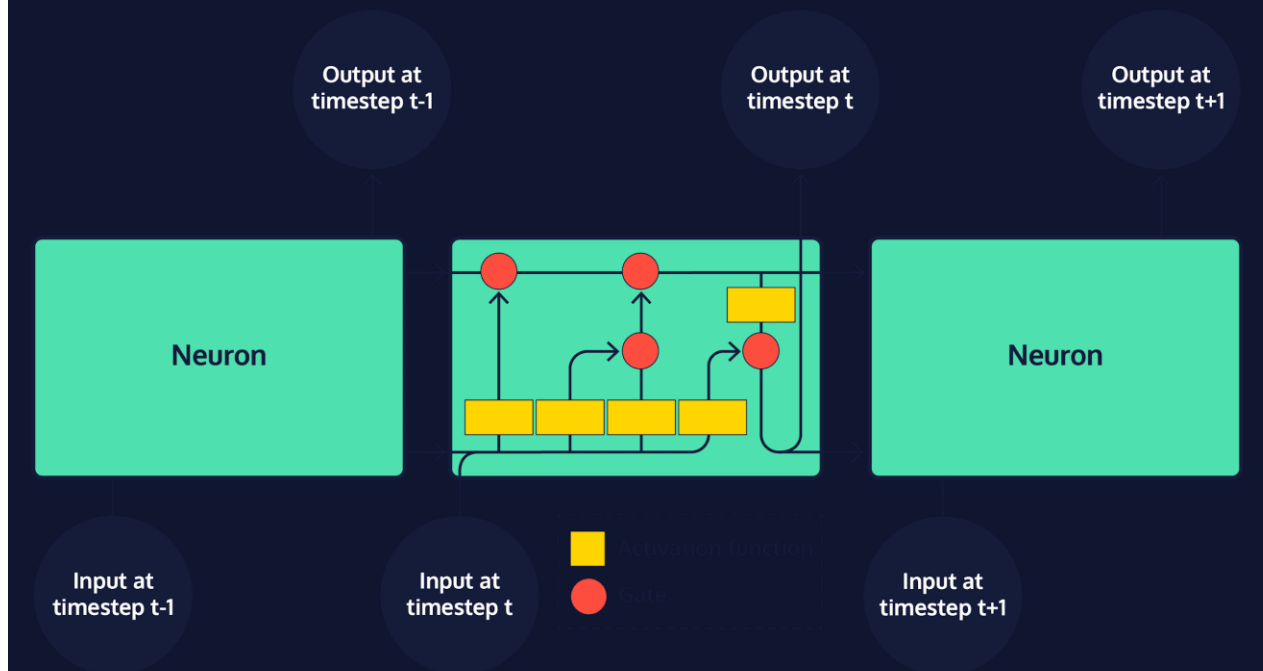
However, there are also many cases of longer sequences in which more context is needed to make an accurate prediction, and that context is less obvious and at a greater lexical distance from the word it determines than in the example above. For instance, consider trying to complete the sentence "It is winter and there has been little sunlight. The grass is always ____." While the structure of the second sentence suggests that our word is probably an adjective related to grass, we need context from further back in the text to guess that the correct word is "brown." As the gap between context and the word to predict grows, standard RNNs become less and less accurate. This situation is commonly referred to as the long-term dependency problem. The solution to this problem? A neural network specifically designed for long-term memory — the LSTM!

Long Short-term Memory Networks

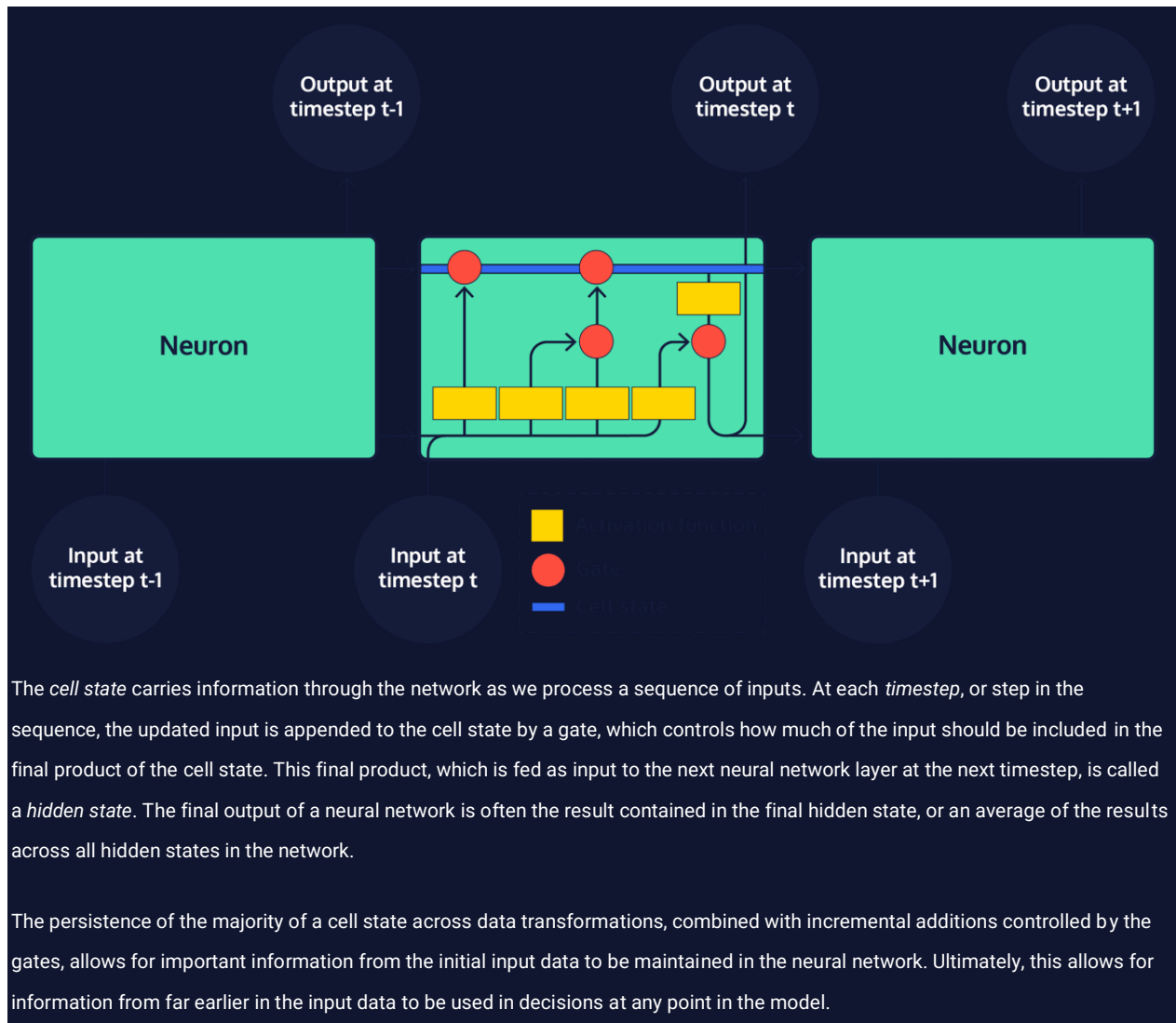
Every model in the RNN family, including LSTMs, is a chain of repeating neurons at its base. Within standard RNNs, each layer of neurons will only perform a single operation on the input data.



However, within an LSTM, groups of neurons perform four distinct operations on input data, which are then sequentially combined.



The most important aspect of an LSTM is the way in which the transformed input data is combined by adding results to *state*, or cell memory, represented as vectors. There are two states that are produced for the first step in the sequence and then carried over as subsequent inputs are processed: cell state, and hidden state.



Read the following:

Deep Learning with Python by Francois Chollet

Read Now

In this book, you will learn concepts in deep learning through Python's powerful Keras library. This is helpful if you want to build your own deep learning models from scratch and explore applications in computer vision, natural-language processing, and generative models.

Read the following selection for free: Deep Learning with Python, Francois Chollet [Chapter 1](#).

Off-Platform Project: Machine Translation

Use Keras models with seq2seq neural networks to build a better translation tool.

Now that you have a basic understanding of how to use Keras models and seq2seq neural networks for some pretty rudimentary machine translation, it's time to take that code off Codecademy's platform and make it a lot better.

Setting Up

Installing Python 3

The Codecademy platform lets you run Python easily in our learning environment, but to run/write Python on your own computer you'll need to install Python on it, if you haven't already.

[This article will walk through installing Python 3 on your computer.](#)

Installing Tensorflow, Keras, and NumPy

First [install Tensorflow](#). This will be the deep learning back end for Keras.

Now [install the Keras API](#).

To get NumPy, we recommend installing the NumPy package using [Anaconda](#) or [Miniconda](#). If you don't have a package and environment manager, you can also install NumPy through `pip`:

```
python -m pip install --user numpy
```

Download your dataset

Select and download a [language pair dataset for your translator here](#). We encourage you to pick a language you have some familiarity with so you can better check the accuracy of the translations you generate. If you don't have familiarity with any of the languages listed, don't fear! You can also use Google Translate to get a sense of how well your model is working.

Unzip the folder and take a look at the `[your-language].txt` file in a notepad or code editor. You should see sentence pairs of English and the target language you picked.

Set Up The Code

[Download and unzip the machine translation code](#) we used in the lesson. You should have following files:

```
preprocessing.py  
training_model.py  
test_function.py
```

While you could also have all of the code combined into one file, we wanted to break everything down to make it easier to understand how each piece works.

You can move the `[your-language].txt` file into the same directory (folder) as the machine translation code to make it slightly easier to set up.

Preprocessing

Open `preprocessing.py` in a code editor or IDE.

Change `data_path` to the file path of `[your-language].txt`. If it's in the same directory as `preprocessing.py`, then all you need is the file name.

In **preprocessing.py**, adjust the number of lines of training data you want to work with. We're giving you a default of 500, but depending on how much you want to tax your computer, you can go up to 123,000 lines. Whatever you choose, this should be a much higher number than what we've used on the Codecademy platform.

Run the code and make sure everything works, error-free. This may take a moment because you have a lot more training data than we used on the Codecademy platform. Remember that more training data leads to better results for machine translation.

At the end of **preprocessing.py**, print out `list(input_features_dict.keys())[:50]`, `reverse_target_features_dict[50]`, and the length of `input_tokens`. Does each look how you expected?

The Training Model

latent_dim: Choose a latent dimensionality for your model. Keras's documentation uses 256, but you can adjust as you see fit.

batch_size: You can choose to adjust this or not at this point. This determines how many sentences are used at a time for training.

epochs: This should be a larger number (Keras's documentation uses 100) so that the seq2seq model has many chances to improve. Bear in mind that a larger number of epochs will also take your computer a lot longer to process. If you don't have the ability to leave your computer running for several hours for this project, then choose a number that is less than 100.

Run the code to generate your model. In the terminal, you should see a summary of the model printed out. Meanwhile, you'll also see a new file in the directory called **training_model.h5**. This is where your seq2seq training model is saved so that it's quicker for you to run your code during testing.

Note that you may get the following error when attempting to run your program on a regular computer that uses CPU processing:

```
OMP: Error #15: Initializing libiomp5.dylib, but found libiomp5.dylib
already initialized.
OMP: Hint: This means that multiple copies of the OpenMP runtime have
been linked into the program. That is dangerous, since it can degrade
performance or cause incorrect results. The best thing to do is to
ensure that only a single OpenMP runtime is linked into the process,
e.g. by avoiding static linking of the OpenMP runtime in any library.
As an unsafe, unsupported, undocumented workaround you can set the
environment variable KMP_DUPLICATE_LIB_OK=TRUE to allow the program to
continue to execute, but that may cause crashes or silently produce
incorrect results. For more information, please see
http://www.intel.com/software/products/support/.
Abort trap: 6
```

Below the import statements on **training.py**, there's a couple lines commented out, which you can uncomment to make the program run. However, if you are concerned about your computer crashing, you may want to hold off completing this project until you have access to a faster computer with GPU processing.

The Test Model and Function

Open **test_function.py**. You'll see that we've imported several variables from the preprocessing and training steps of the program.

In the `for` loop at the bottom of the file, increase the `range` if you want to see more sentences translated — this is up to you. Of course, more sentences will increase the amount of time your computer will require for the translation.

When you feel ready for your computer to spend awhile training and translating, run the code. This is going to take awhile — from 20 minutes to several hours, so make sure your computer is plugged in. You should see each epoch appear in the terminal as a fraction of the total number of epochs you selected. For example, 16/100 indicates the program has reached the 16th epoch out of 100 total epochs. You'll also see the loss go down for each epoch, which is very exciting — this is the model getting more accurate!

When your computer finishes the full process, you'll see the translations appear. You can use a speaker of that language or Google Translate to see how accurate they are.

Congratulations on setting up your first deep learning program locally!

BONUS

Some ways to improve:

Try including more lines of training data in **preprocessing.py** to see if your translations improve (this may also crash the program if there are too many!).

Try out different values for the `epochs`, `latent_dim`, and `batch_size` to see how they change the performance of the model.

Add in a step to convert new text into a NumPy matrix so that you can translate new sentences that aren't in the dataset. (This may also require handling unknown tokens.)

Review: Text Generation

See what you've learned in [Text Generation](#).

Review

Congratulations! The goal of this unit was to introduce current common methods of generating text.

Having completed this unit, you are now able to:

Recognize why recurrent neural networks and specifically long short-term memory (LSTM) networks are helpful for working with sequences of text.

Convert text into a matrix of one-hot vectors.

Implement a seq2seq network using LSTM layers with teacher forcing.

Understand how teacher forcing works.

Use deep learning to conduct machine translation.

If you are interested in learning more about these topics, here are some additional resources:

Documentation: [Keras: Natural Language Processing](#)

Tutorial: [TensorFlow: Neural Machine Translation with Attention](#)

Book: [Deep Learning with Python, François Chollet](#)

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Welcome to Build Chatbots

Python + Chatbots = Fun

Welcome you to Build Chatbots with Python! If you are interested in NLP, this might be one of the most exciting units in this path. You've done a lot of work to get here, and we are excited to dig in and learn all about building chatbots!

In addition to building increasingly complex chatbots, you'll also learn about:

- what chatbots are
- what types of chatbots exist
- how chatbots have changed over time
- what ethical considerations to take when building chatbots
- the basics of natural language processing
- a high-level overview of what neural networks and deep learning are (without all the math)
- how to apply deep learning models for natural language processing purposes

By the end of this skill path, you'll have the tools to create different kinds of Python-based chatbots on your own.

Ready to get started?

User Input

How to assign variables with user input.

So far, we've covered how to assign variables values directly in a Python file. However, we often want a user of a program to enter new information into the program.

How can we do this? As it turns out, another way to assign a value to a variable is through user input.

While we output a variable's value using `print()`, we assign information to a variable using `input()`. The `input()` function requires a prompt message, which it will print out for the user before they enter the new information. For example:

```
likes_snakes = input("Do you like snakes? ")
```

In the example above, the following would occur:

```
"Do you like snakes? "
"Yes! I have seven pythons as pets!"
likes_snakes
```

enter

Try constructing a statement to collect user input on your own!

Fill in the blank

Questions

Fill in the blanks in the code to complete a statement that asks a user "What is your favorite flightless bird?" and then stores their answer in the variable `favorite_flightless_bird`.

Code

```
blank 1
```

```
=
```

```
blank 2
```

```
(
```

```
blank 3
```

```
)
```

Answer Choices

"What is your favorite flightless bird?"
`input`
`user.input`
`get_input`
`favorite_flightless_bird`
`prompt`

Click or drag and drop to fill in the blank

Not only can `input()` be used for collecting all sorts of different information from a user, but once you have that information stored as a variable you can use it to simulate interaction:

```
>>> favorite_fruit = input("What is your favorite fruit? ")
What is your favorite fruit? mango

>>> print("Oh cool! I like " + favorite_fruit + " too, but I think my
favorite fruit is apple.")
Oh cool! I like mango too, but I think my favorite fruit is apple.
```

These are pretty basic implementations of `input()`, but as you get more familiar with Python you'll find more and more interesting scenarios where you will want to interact with your users.

What are Chatbots

Ethics of Chatbots

History of Chatbots

Language Modeling and Chatbots

Retrieval-Based Chatbots

Introduction to Retrieval-based Chatbots

Most modern technology users are well-aware that the voice assistant on their phone is an intelligent chatbot mimicking a human conversation. Less obvious are the thousands of other chatbots that facilitate shopping online, accessing your bank account, and a multitude of other customer-service interactions. In sheer number, **retrieval-based chatbots** are the most popular chatbot implementation in use today. This popularity is due in large part to the strength of retrieval-based bots in **closed-domain** conversations — conversations that are clearly limited in

[scope](#)

Preview: Docs Loading link description

.

Most chatbot systems, including those that are retrieval-based, depend on three main tasks in order to convincingly complete a conversation:

Intent Classification: When presented with user input, a system must classify the intent of the message. Is a message requesting information on the material of a pair of pants for sale? Or is the message related to the estimated shipping date of the clothing item?

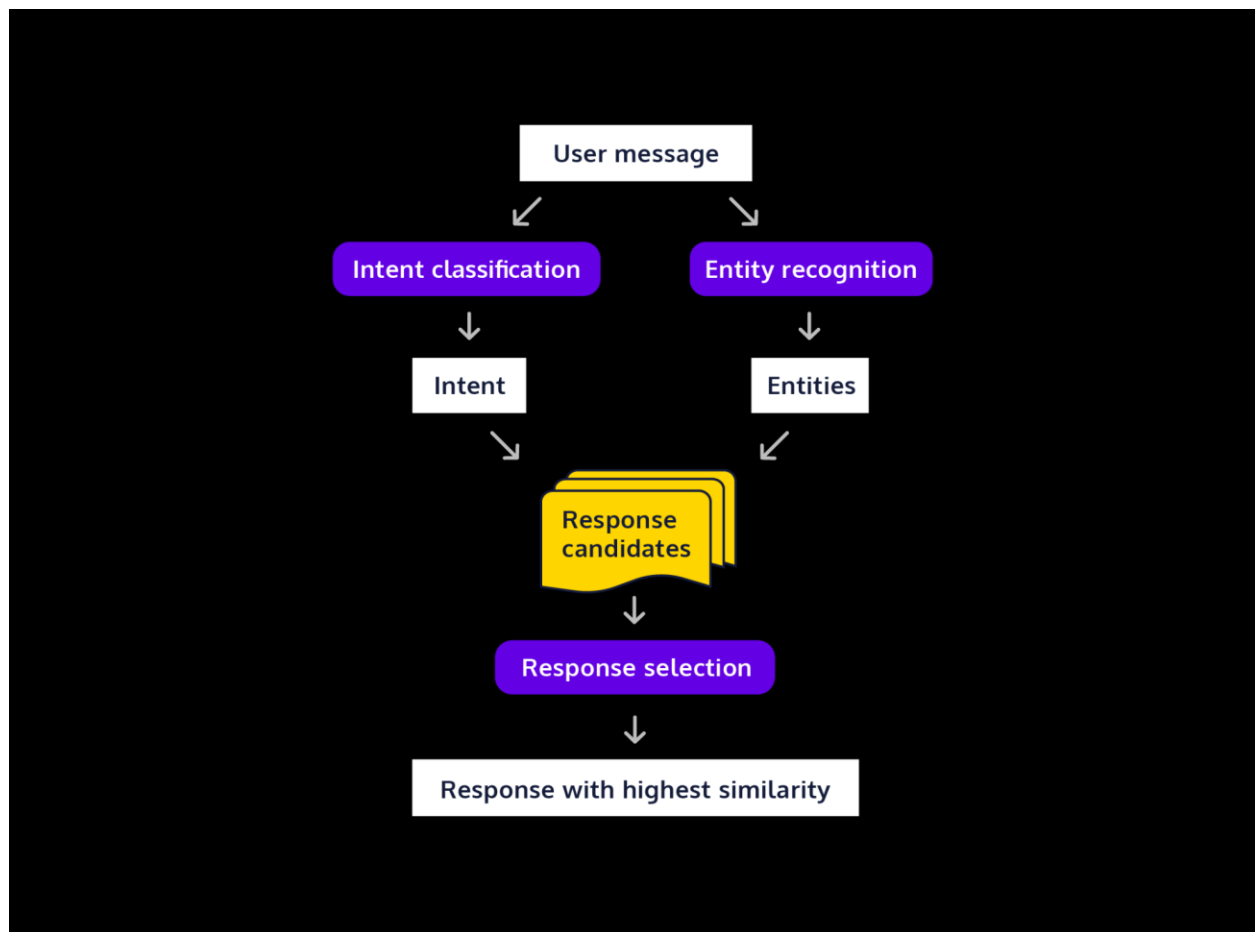
Entity Recognition: Entities are often the proper nouns of a message, like the day of the week when an item will ship, or the name of a specific item of clothing. A chatbot system must be able to recognize which entities a user message revolves around, and reference those entities in a response.

Response Selection: Retrieval-based chatbot systems are unique in their reliance on a collection of predefined responses to user messages. Once a system has an understanding of both the intent and main entities of the user message, the program can retrieve the best-fit response from this collection.

In this lesson, you'll have the chance to apply modeling techniques, from Bag-of-Words to word embeddings, to build a full chatbot system that can take in intent and entity information from a user question, and reply with a personalized answer. You'll use the same technologies that are used to build the retrieval-based chatbots you interact with every day!

Instructions

1. The diagram in the workspace area is a representation of a standard chatbot system architecture, which includes four main steps:
 - When a chatbot receives a user message, **intent classification** and **entity recognition** programs are immediately run in tandem.
 - The results from both of these tasks are then fed into a candidate response generator. In a retrieval-based chatbot system, **response generation** relies on a database of predefined responses. Using the results from entity recognition and intent classification tasks, a set of possibly similar chatbot responses are retrieved from the database.
 - After the selection of a set of candidate responses, a final **response selection** program ranks and selects the "best fit" response to be returned to the user.



Retrieval-Based Chatbots

Intent with Bag-of-Words

One way we can begin parsing a user's message to a retrieval-based chatbot is to consider a user's intent word by word. Bag-of-Words (BoW) is one of the most common models used to understand human language at the individual word level. You might remember that BoW results in a mapping of a word to its frequency in a text.

The `collections` module's `Counter()` concisely builds this word-to-frequency mapping:

```
Counter(preprocess("Wheels on the bus go round and round, round and round, round and round."))
```

```
print(word_counts)
#Counter({'round': 6, 'wheels': 1, 'bus': 1, 'go': 1})
```

Copy to Clipboard

We can use the results of this mapping to construct a measure of the intent of a user's message. Then we'll use this measure to retrieve the most similar answer from our collection of predefined chatbot responses.

However, there are a number of different ways in which we can define sentence similarity. There is no guarantee that one definition will be best, or even that any two definitions will suggest the same response!

A simple BoW model is best-fit for a situation where the order of words does not contain much information about the intent of a user's message. In these situations, the unique collection of words in a message often reveals a lot of information about the user's concern and provides a simple, yet powerful metric to quantify similarity.

Instructions

Checkpoint 1 Passed

1.

In `script.py`, we've included `user_message`, a message sent by a user interacting with a chatbot from an online clothing store. In addition, two possible responses, `response_a` and `response_b`, are preprocessed and stored in list data structures.

Run the code to see the normalized form of `user_message`!

Checkpoint 2 Passed

2.

Below where `bow_user_message` is defined, use the `collections` module's `Counter()` function to build BoW models from the possible response objects. Save the output from the function call on `response_a` and `response_b` to `bow_response_a` and `bow_response_b`, respectively.

We can get a BoW model for `bus_response` like this:

```
bus_response = preprocess("Wheels on the bus go round and round, round and round, round and round.")
bow_bus_response = Counter(bus_response)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

We've started writing a short function, called `compare_overlap()`, to help us compute the number of shared words between our user message and possible responses.

After the initialization of `similar_words` to 0, iterate over each token in `user_message` and check if the token is already in the `possible_response`.

If it is, increment `similar_words` by 1

Otherwise, do not change the `similar_words` variable, as the token is not shared between the message and response!

We can iterate over the tokens in a list of strings as follows:

```
for token in user_message:
```

Copy to Clipboard

The `in` statement can be used to check if a value is in a list:

```
if token in possible_response:
```

Copy to Clipboard

We can use the `+=` shorthand to increment a variable by the value on the right-hand side:

```
similar_words += 1
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Use `compare_overlap()` to compare the number of shared words between `bow_user_message` and both `bow_response_a` and `bow_response_b`. Which response is most similar to the user message, based on shared words? Print to console the result of calling `compare_overlap()` on `bow_user_message` and the response most similar to the user message.

We can use `compare_overlap()` to compare two texts, `wheels_on_bus` and `twinkle_star` as follows:

```
print(compare_overlap(wheels_on_bus, twinkle_star))
```

```
from collections import Counter
from preprocessing import preprocess, response_a, response_b
```

```

user_message = preprocess("Hello! What is the fit of the 'Elosie' dress? My shoulders
are broad, so I often size up for a comfortable fit. Do dress sizes run large or
small? Especially in the shoulders?")
response_a = preprocess("All of our dresses sare cut from a polyester blend for a
strechy fit")
response_b = preprocess("The 'Elosie' dress runs large. I suggest you take your
regular size or smaller for the best fit.")

#printing the user_message variable below
print(user_message)

#use the Counter() function on the response objects below
bow_user_message = Counter(user_message)
bow_response_a = Counter(response_a)
bow_response_b = Counter(response_b)

def compare_overlap(user_message, possible_response):
    similar_words = 0
    #iterate over tokens in user_message
    for token in user_message:
        if token in possible_response:
            similar_words += 1
    return similar_words

print("Number of shared words between user_message and response_a: " +
str(compare_overlap(bow_user_message, bow_response_a)))

print("Number of shared words between user_message and response_b: " +
str(compare_overlap(bow_user_message, bow_response_b)))

```

Output-only Terminal

Output:

```

['hello', 'fit', 'elosie', 'dress', 'shoulders', 'broad', 'often', 'size', 'comfortable', 'fit',
'dress', 'sizes', 'run', 'large', 'small', 'especially', 'shoulders']
Number of shared words between user_message and response_a: 1
Number of shared words between user_message and response_b: 5

```

Retrieval-Based Chatbots

Intent with TF-IDF

While the number of shared words is an intuitive way to think about the similarity between two documents, a number of other methods are also commonly used to determine the best match between the user input and a pre-defined response.

One notable measure, **term frequency-inverse document frequency (tf-idf)**, puts emphasis on the relative frequency in which a term occurs within a possible response and the user message. You can dive into the calculation in [our lesson on TF-IDF](#), but generally, tf-idf is best suited for domains where the most important terms in an input or response are mentioned repeatedly.

In Python, the `sklearn` package has a handy `TfidfVectorizer()` object that can be initialized as follows:

```
vectorizer = TfidfVectorizer()
```

Copy to Clipboard

We can then fit the tf-idf model with the `.fit_transform()`

[method](#)

Preview: Docs Loading link description
and input a list of

[string](#)

Preview: Docs Loading link description

objects. Using the vectorized results of this fitted model, we can compute the cosine similarity of the user message and a possible response with the aptly named `cosine_similarity()` function:

```
# fit model
tfidf_vectors = vectorizer.fit_transform(input_list)

# compute cosine similarity
cosine_sim = cosine_similarity(user_message_vector, response_vector)
```

Copy to Clipboard

Most retrieval-based chatbots use multiple measures in order to rank the similarity between a user's input and a number of possible responses. Oftentimes, different measures produce different similarity rankings.

The selection of a similarity measure is one of the most important decisions chatbot architects have to make while building a system. We should always consider the relative strengths and weaknesses of different metrics.

Let's use tf-idf similarity to determine the best match between `user_message` and a set of possible responses from the earlier exercise; how might this impact which pre-defined response we determine to have the most similar intent with the user message?

Instructions

Checkpoint 1 Passed

1.

We've already loaded the data sources from the previous exercise, `user_message`, `response_a`, `response_b`, and a new response, `response_c`, into a list in the workspace. Call the `preprocess_text()` function on each object in the `documents` list and store all results in a list called `processed_docs`.

We can apply a function to multiple objects, and save results to a list, using Python's helpful list comprehension syntax:

```
result_list = [function(obj) for obj in obj_list]
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Create an instance of `TfidfVectorizer()` called `vectorizer`.

Call `vectorizer's .fit_transform()` on `processed_docs` and save the result to `tfidf_vectors`.

Checkpoint 3 Passed

3.

Call `cosine_similarity()` with `tfidf_vectors[-1]` and `tfidf_vectors` as arguments. Assign the results to `cosine_similarities`. This call computes the lexical distance between our user message and each element in `tfidf_vectors`.

Because our tf-idf values are stored in a matrix structure, we can use Python's slicing functionality to select `user_message` from our set of vectors. `user_message` is the last object in the matrix, so we can select it using a negative index, similar to below:

```
last_element_in_list = list_obj[-1]
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Let's select the list index of the object with the largest cosine similarity:

Call the `.argsort()` method on `cosine_similarities`, using slicing notation to only select the data at the 0 index in the object. `argsort()` is a function from the NumPy package which sorts an array and returns the indices in order.

Use another slicing operation to select the **response object** with the highest similarity score, which is stored at index -2. The `user_message` itself will always have the highest similarity score, and will always be stored at index -1.

Save the result to a variable called `similar_response_index`.

We can select the index of the last element in an `argsort()`-ed list as follows:

```
last_element = list.argsort()[0][-1]
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Finally, use `similar_response_index` with list slicing notation to select the best response from `documents` and save the result to a variable called `best_response`. Print `best_response` to the console. Is the selected response the same as in the previous exercise using BoW?

Yes! The BoW and tf-idf methods selected the same response.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from preprocessing import preprocess_text

response_a = "Every dress style is cut from a polyester blend for a stretchy fit."
response_b = "The 'Elosie' dress runs large. I suggest you take your regular size or smaller."
response_c = "The 'Elosie' dress comes in green, lavender, and orange."
user_message = "Hello! What is the fit of the 'Elosie' dress? My shoulders are broad, so I often size up for a comfortable fit. Do dress sizes run large or small?"

documents = [response_a, response_b, response_c, user_message]

# preprocess responses and user_message
processed_docs = [preprocess_text(doc) for doc in documents]

# create tfidf vectorizer
vectorizer = TfidfVectorizer()

# fit and transform vectorizer on processed docs
tfidf_vectors = vectorizer.fit_transform(processed_docs)

# compute cosine similarity between the user message tf-idf vector and the different response tf-idf vectors
cosine_similarities = cosine_similarity(tfidf_vectors[-1], tfidf_vectors)
# get the index of the most similar response to the user message
similar_response_index = cosine_similarities.argsort()[0][-2]

best_response = documents[similar_response_index]
print(best_response)

The 'Elosie' dress runs large. I suggest you take your regular size or smaller.
```

Retrieval-Based Chatbots

Entity Recognition with POS Tagging

After determining the best [method](#)

Preview: Docs Loading link description

for the classification of a user's intent, chatbot architects set upon the task of recognizing entities within a user's message. Similar to other tasks in chatbot systems, there are a number of methods that can be used to locate and interpret the entities found in a user message — it is up to the system architect (you!) to critically evaluate methods and select those that are best-fit for a chatbot's specific domain.

Part of speech (POS) tagging is commonly used to identify entities within a user message, as most entities are nouns (refresh your understanding of English parts of speech in our [Parsing with Regex lesson](#)). `nltk's pos_tag()` function can rapidly tag a tokenized sentence and return a list of [tuple](#)

Preview: Docs Loading link description

objects for use in entity recognition tasks. A sentence, like “Jack and Jill went up the hill,” when tokenized and supplied in a call to `pos_tag()`, outputs a list of tuple objects:

```
tagged_message = [('Jack', 'NNP'), ('and', 'CC'), ('Jill', 'NNP'), ('went', 'VBD'), ('up', 'RP'), ('the', 'DT'), ('hill', 'NN')]
```

Copy to Clipboard

We can extract words tagged as a noun, represented in `nltk's tagging schema` as a collection of tags beginning with “NN,” by checking if the [string](#)

Preview: Docs Loading link description

“NN” occurs in the token tag, and then appending the token string to a list of nouns if True:

```
for token in tagged_message:
    if "NN" in token[1]:
        message_nouns.append(token[0])
```

Copy to Clipboard

Once we have a list of the entities in a user message, we're well on our way to developing a realistic chatbot response!

Instructions

Checkpoint 1 Passed

1.

Provided in the workspace is a sample user message, `user_message`, that has already been preprocessed and tokenized. Call the `pos_tag()` function on `user_message`, and save the output to a new variable, `tagged_user_message`.

Checkpoint 2 Passed

2.

With the user message already tagged, we just need to extract all tokens that have been tagged as nouns, i.e. all tokens with an associated tag that starts with `NN`. Let's write a function, `extract_nouns()`, to complete this task.

Create a `for` loop through each tuple in `tagged_message`.

Within the `for` loop, write an `if` statement that checks if the second element in the tuple begins with the string `NN`.

If the `if` statement resolves to `True`, append the first element in the tuple (the word token) to `message_nouns`.

Don't forget that we can use `tuple[0]` to select the token object in our tuple, and `tuple[1]` to select the POS tag.

Checkpoint 3 Passed

3.

Save the result of calling `extract_nouns()` on `tagged_user_message` to a variable called `user_message_nouns`. Print `user_message_nouns` to the terminal. Did your function extract all of the entities in the user message?

```
from nltk import pos_tag
user_message = ["i", "ordered", "two", "t-shirts", "this", "past",
"weekend", "when", "will", "my", "package", "be", "shipped"]

#define `tagged_user_message` here
tagged_user_message = pos_tag(user_message)
print(tagged_user_message)

def extract_nouns(tagged_message):
    message_nouns = list()
    for token in tagged_message:
        if "NN" in token[1]:
            message_nouns.append(token[0])
    return message_nouns

#define `user_message_nouns` here
user_message_nouns = extract_nouns(tagged_user_message)

print(user_message_nouns)
```

```
[('i', 'RB'), ('ordered', 'VBD'), ('two', 'CD'), ('t-shirts', 'NNS'), ('this', 'DT'), ('past', 'JJ'), ('weekend', 'NN'), ('when', 'WRB'), ('will', 'MD'), ('my', 'PRP$'), ('package', 'NN'), ('be', 'VB'), ('shipped', 'VBN')]
['t-shirts', 'weekend', 'package']
```

Retrieval-Based Chatbots

Entity Recognition with Word Embeddings

A conversation is a two-way street — both participants must actively listen to one another and adapt their responses based on the information given by the other. While POS tagging allows us to extract key entities in a user message, it does not provide context that allows a chatbot to believably integrate an entity reference into a predefined response.

A retrieval-based chatbot's collection of predefined responses is very similar to an empty [Madlibs story](#). Each response is a complete sentence, but with many key words replaced with blank spots. Unlike Madlibs, these blanks will be labeled with a reference to a broad kind of entity. For instance, a predefined

response for a weather report chatbot might look like:

"Good Morning! You should see sunny skies

this around the greater Chicago area,
weekday

including in .
neighborhood-chicago

In order to produce a coherent response, the chatbot must insert entities from a user message into these blank spots. Chatbot architects often use word embedding models, like word2vec, to rank the similarity of user-provided entities and the broad category associated with a response "blank spot" (refresh your understanding of word2vec in our [word embeddings](#) lesson). The `spacy` package provides a pre-trained word2vec model which can be called on a

[string](#)

Preview: Docs Loading link description
of entities and the responses category like this:

```
# load word2vec model
word2vec = spacy.load('en')

# call model on data
tokens = word2vec("wednesday, dog, boots, flower")
response_category = word2vec("weekday")
```

Copy to Clipboard

After the model has been applied, we can use spacy's `.similarity()`

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

to compute the cosine similarity between user-provided entities and a response category:

```
output_list = list()
for token in tokens:
    output_list.append(f"{token.text} {response_category.text}
{token.similarity(response_category)}")

# output:
# wednesday weekday 0.453354920245737
# dog weekday 0.21911001129423147
```

```
# boots weekday 0.15690609198936833
# flower weekday 0.17118961389940174
```

Copy to Clipboard

The resulting output above shows that the word2vec model found “wednesday” to have the greatest similarity to “weekday.” This should match what you would expect! A chatbot system can select the token with the highest similarity score and insert it into the “blank spot” in a predefined response in order to continue a coherent conversation with a user.

Instructions

Checkpoint 1 Passed

1.

A word2vec model, `word2vec`, and two data sources are provided in the workspace:

The list of user provided entities, `message_nouns`, from the previous exercise.
`bot_response`, a [formatted string literal](#) with the `blank_spot` variable (category) currently set to “clothing.”

Run the code in the workspace to print `bot_response` to the terminal. In order to produce a coherent response, you will have to determine which user-provided entity should be inserted into `bot_response`.

Checkpoint 2 Passed

2.

Build word embeddings from `message_nouns` and the category “clothes”:

Call `word2vec()` on the string “clothes” and assign the result to `category`.

Call `word2vec()` on the concatenated form of `message_nouns` and assign the result to `tokens`.

Don't forget that we can concatenate a `list()` object into a string by using

```
" ".join(list_variable)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Let's write a function called `compute_similarity()` to help compare the similarity between each token in `tokens` and the `category` object.

Write a `for` loop which iterates over each token in `tokens`.

For each token, append a string to `output_list` which includes the text of the token object, the text of the `category` object, and the result of calling the `.similarity()` method with these objects as arguments, in this order, separated by spaces.

Call `compute_similarity()` with `tokens` and `category` as arguments.
Print the result to the terminal.

We can combine a `for` loop and the `.similarity()` statement as follows:

```
output_list = list()
for token in tokens:
    output_list.append(f"{token.text} {response_category.text} {token.similarity(response_category.text)}")
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Use Python's list slicing functionality to update the value of `blank_spot` to the token in `message_nouns` with the highest similarity score. Run the code in the workspace, and confirm that the `bot_response` printed to the console now incorporates the user-provided entity.

Don't forget that Python list indices begin at 0! We would select the third element in a list as follows:

```
example_list[2]
```

```
import spacy
word2vec = spacy.load('en')

message_nouns = ['shirts', 'weekend', 'package']

tokens = word2vec(" ".join(message_nouns))
category = word2vec("clothes")

def compute_similarity(tokens, category):
    output_list = list()
    #your code here
    for token in tokens:
        output_list.append(f"{token.text} {category.text} {token.similarity(category)}")
    return output_list

print(compute_similarity(tokens, category))

blank_spot = message_nouns[0]
bot_response = f"Hey! I just checked my records, your shipment containing {blank_spot} is en route. Expect it within the next two days!"
print(bot_response)
```

Output-only Terminal

Output:

```
Hey! I just checked my records, your shipment containing clothing is en route. Expect it within the next two days!
```

Retrieval-Based Chatbots

Response Selection

Great work! We've covered all the steps necessary for the construction of a chatbot response selection system — now let's put our skills to work. Using BoW for intent selection, and word2vec for entity recognition, let's automate the selection of the best-fit response from a collection of candidate responses.

All of the functions defined across previous exercises have been imported into the workspace. In addition, there are four provided data sources:

- The `user_message`, a question from a user engaging with a weather chatbot.

- Three possible `response` objects, with an entity `blank_spot` representing an "Illinois" category.

We've already preprocessed the `user_message` and response objects, and saved the results as BoW models. The results of calling our `compare_overlap()` function on each candidate response have been saved in `similarity_list`.

Similarly, a word2vec model has already been fit on our data, and the results of the model have been saved in `word2vec_result`. It's up to you to combine the results from both models to retrieve a response object.

Instructions

Checkpoint 1 Passed

1.

`similarity_list` is a list containing the similarity score of each response object to `user_message`. Because we've constructed this list from `processed_responses`, we know that the scores are stored at the same indices as their respective responses in our `response` list.

Use Python's `.index()` method to select the index of the highest similarity score in `similarity_list`. We can use the `max()` function to select the largest element. Save the result of `.index()` to `response_index`.

We can use the `.index()` method to select the index of the *smallest* element in a list as follows:

```
result_list.index(min(input_list))
```

Copy to Clipboard

Checkpoint 2 Passed

2.

`word2vec_result` is a nested list structure containing the results of a word2vec model called on a collection of nouns (`message_nouns`) that have been extracted from `user_message` using our `extract_nouns()` function.

Print `word2vec_result` to the terminal. Any surprising results? Which noun has the highest similarity score?

Use list slicing to select the entity string with the highest cosine similarity score, and save the result to `entity`. Don't forget that `word2vec_result` is a list of lists!

We can use select the first element (at index 0) in the second list (at index 1) in a nested list structure as follows:

```
result_list[1][0]
```

Copy to Clipboard

Checkpoint 3 Passed

3.

With our best fit response object and entity selected, we can now output our final chatbot response!

Using list indices, select the element from `responses` that is stored at `response_index`.

Call the `.format()` method, with `entity` as an argument, on this list element and save the result to `final_response`.

Print `final_response` to the terminal. Is this the response *you* would have chosen?

We use `.format()` to format a string as follows:

```
"I'm missing an {}!".format("object")
```

```
#output
```

```
"I'm missing an object!"
```

```
from user_functions import preprocess, compare_overlap, pos_tag,
extract_nouns, word2vec, compute_similarity
```

```

from collections import Counter

user_message = "Good morning... will it rain in Chicago later this week?"

blank_spot = "illinois city"
response_a = "The average temperature this weekend in {} will be 88 degrees. Bring your sunglasses!"
response_b = "Forget about your umbrella; there is no rain forecasted in {} this weekend."
response_c = "This weekend, a warm front from the southeast will keep skies near {} clear."
responses= [response_a, response_b, response_c]

#preprocess documents
bow_user_message = Counter(preprocess(user_message))
processed_responses = [Counter(preprocess(response)) for response in responses]

#build BoW model
similarity_list = [compare_overlap(doc, bow_user_message) for doc in processed_responses]

# select response with best intent fit below:
response_index = similarity_list.index(max(similarity_list))

#extracting entities with word2vec
tagged_user_message = pos_tag(preprocess(user_message))
message_nouns = extract_nouns(tagged_user_message)

#executing word2vec model
tokens = word2vec(" ".join(message_nouns))
category = word2vec(blank_spot)
word2vec_result = compute_similarity(tokens, category)

#select highest scoring entity below:
print(word2vec_result)
entity = word2vec_result[2][0]

#select final response below:
final_response = responses[response_index].format(entity)
print(final_response)

[['morning', 'illinois city', 0.5612323259082398], ['rain', 'illinois city', 0.5571145482359784], ['chicago', 'illinois city', 0.5678637976929392], ['week', 'illinois city', 0.5404435240064991]]
Forget about your umbrella; there is no rain forecasted in chicago this weekend.

```

```

user_functions.py

import re
from collections import Counter
from nltk import pos_tag
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
import spacy
word2vec = spacy.load('en')

def preprocess(input_sentence):
    input_sentence = input_sentence.lower()
    input_sentence = re.sub(r'^\w\s|', '', input_sentence)
    tokens = word_tokenize(input_sentence)
    input_sentence = [i for i in tokens if not i in stop_words]
    return(input_sentence)

def compare_overlap(user_message, possible_response):
    similar_words = 0
    for token in user_message:
        if token in possible_response:
            similar_words += 1
    return similar_words

def extract_nouns(tagged_message):
    message_nouns = list()
    for token in tagged_message:
        if token[1].startswith("N"):
            message_nouns.append(token[0])
    return message_nouns

def compute_similarity(tokens, category):
    output_list = list()
    for token in tokens:
        output_list.append([token.text, category.text, token.similarity(category)])

```

```
return output_list
```

Retrieval-Based Chatbots

Building a Retrieval System

Now that we have covered the three underlying tasks of a chatbot system — intent classification, entity extraction, and response selection — it's time for us to pull all these tasks together into a full conversational system! We'll do so by integrating the functions we have written throughout this lesson into three methods and organize them into a

`class`

Preview: Docs Loading link description

.

This organization will allow us to call our program from the bash

`terminal`

Preview: Docs Loading link description

and pass in our own questions as the `user_message`. We have already included code that initializes and starts a conversation with our

`bot`

Preview: Docs Loading link description

. It's up to you to construct the `.find_intent_match()`, `.find_entities()`, and `.respond()` methods so that our bot can really speak!

Note: If you load the solution code, do not uncomment `stratus.chat()` (the last line of code) until step 4.

Instructions

Checkpoint 1 Passed

1.

The `.find_intent_match()` method has already been defined for you, along with intermediate steps that preprocess and build BoW models from `user_message` and `responses`.

Use Python's list comprehension functionality to apply `compare_overlap()` on each response in `processed_responses`. Save the resulting list to `similarity_list`.

Use Python's `.index()` method to select the index of the highest similarity score in `similarity_list`. Save the result to `response_index`.

Return the response within `responses` at `response_index` from the function.

We can use a function within a list comprehension as follows:

```
result_list = [my_function(object) for object in input_list]
```

Copy to Clipboard

In order to select the largest number from a list, we can use the `max()` function:

```
fav_numbers = [2, 7, 8, 28]
```

```
highest_fav = max(fav_numbers)
# highest_fav is equal to 28
```

Copy to Clipboard

Checkpoint 2 Passed

2.

`.find_entities()` has also already been defined in the workspace, along with preprocessing steps and an application of our `extract_nouns()` function.

Call `word2vec()` on `message_nouns` and save the result to `tokens`. Don't forget that `message_nouns` needs to be concatenated into a string using `".join()"`!

Call `word2vec()` on `blank_spot` and save the result to `category`.

Call `compute_similarity()` on `tokens` and `category`; save the result to `word2vec_result`.

Uncomment the call to `sort()` on `word2vec_result`. `sort()` has ordered `word2vec_result` by *ascending* cosine similarity score, so the entity with the highest similarity score is the *last* element in this list of lists.

Return the first element in the last list object in `word2vec_result` from the function.

We can use select the second element (at index `1`) in the last list (at index `-1`) in a nested list structure as follows:

```
result_list[-1][1]
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Let's put all our work together!

Define `.respond()` with `self` and `user_message` as parameters.

Inside `.respond()` call `self.find_intent_match()`, passing in `responses` and `user_message` as arguments. Save the result to `best_response`.

Call `self.find_entities()` with `user_message` as an argument, and save the result to `entity`.

Call Python's `.format()` method (or use an f-string) on `best_response` with `entity` as an argument. Print the result to the console.

We use `.format()` to format a string as follows:

```
"I'm missing an {}!".format("object")
```

```
#output: "I'm missing an object!"
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Let's test out our system!

Outside of the class, create a `ChatBot` instance

Call the `.chat()` method on your `ChatBot` instance

Run your script from the bash terminal by typing

```
python3 script.py
```


Copy to Clipboard

Did your chatbot respond correctly to your question? In order to expand the functionality of Stratus, you can add additional response frameworks to the **data.py** file. How many responses do you think Stratus would need to carry out an extended conversation coherently?

You can create an instance of a class using the following syntax:

```
instanceName = ClassName()
```

```
from user_functions import preprocess, compare_overlap, pos_tag,
extract_nouns, compute_similarity
from data import responses, blank_spot
from collections import Counter
import spacy
word2vec = spacy.load('en')

class ChatBot:
    def find_intent_match(self, responses, user_message):
        bow_user_message = Counter(preprocess(user_message))
        processed_responses = [Counter(preprocess(response)) for response in
responses]
        # define similarity_list here:
        similarity_list = [compare_overlap(doc, bow_user_message) for doc in
processed_responses]
        # define response_index:
        response_index = similarity_list.index(max(similarity_list))
        return responses[response_index]

    def find_entities(self, user_message):
        tagged_user_message = pos_tag(preprocess(user_message))
        message_nouns = extract_nouns(tagged_user_message)

        # execute word2vec model here:
        tokens = word2vec(" ".join(message_nouns))
        category = word2vec(blank_spot)
        word2vec_result = compute_similarity(tokens, category)
        word2vec_result.sort(key=lambda x: x[2])
        return word2vec_result[-1][0]

    # define .respond() here:
    def respond(self, user_message):
        best_response = self.find_intent_match(responses, user_message)
        entity = self.find_entities(user_message)
        print(best_response.format(entity))

    def chat(self):
        user_message = input("Hi, I'm Stratus. Ask me about your local
weather!\n")
        self.respond(user_message)

# create ChatBot() instance:
stratus = ChatBot()
# call .chat() method:
#stratus.chat()
```

```
data.py

blank_spot = "illinois"
response_a = "The average temperature this weekend in {} will be 88 degrees. Bring your sunglasses!"
response_b = "Forget about your umbrella; there is no rain forecasted in {} this weekend."
response_c = "This weekend, a warm front from the southeast will keep skies near {} clear."
responses= [response_a, response_b, response_c]
```

Retrieval-Based Chatbots

Lesson Review

Congratulations! You've learned to build your own retrieval-based chatbot, and have the tools necessary to develop a

[bot](#)

Preview: Docs Loading link description

that can be used in any closed-domain. Let's review what we've covered in this lesson:

Retrieval-based chatbots are used in **closed-domain scenarios** and rely on a collection of predefined responses to a user message. A retrieval-based bot completes three main tasks: **intent classification**, **entity recognition**, and **response selection**.

There are a number of ways to determine which response is the best fit for a given user message. One of the most important decisions a chatbot architect makes is the selection of a similarity metric.

Bag-of-Words (BoW) models are commonly used to compute intent similarity measures based on word overlap.

Term frequency-inverse document frequency (tf-idf) is another common similarity metric which incorporates the relative frequency of terms across the collection of possible responses. The `sklearn` package provides a `TfidfVectorizer()` object that we can use to fit tf-idf models.

*Entity recognition tasks often extract proper nouns from a user message using **Part of Speech (POS)** tagging. POS tagging can be performed with `nlTK's .pos_tag()` [method](#)

Preview: Docs Loading link description

. *It's often helpful to imagine pre-defined chatbot responses as a kind of MadLibs story. We can use word embeddings models, like the one implemented in the `spacy` package, to insert entities into response objects based on their cosine similarity with abstract, "blank-spot" concepts. *The final response selection relies on results from both intent classification and entity recognition tasks in order to produce a coherent response to the user message.

Instructions

The code in the workspace executes a chatbot system using the functions we have designed throughout this lesson -- but we're missing a set of pre-defined responses with a shared "blank spot" category! Write your own set of responses to a possible user message. You can run the file from the terminal by entering `python3 script.py` in order to test the chatbot system and the coverage of your pre-defined responses.

Can you imagine how many pre-defined messages a retrieval-based chatbot must have access to in order to complete the variety of applications they are used in today?

```
from user_functions import preprocess, compare_overlap, pos_tag,
extract_nouns, compute_similarity
from collections import Counter
import spacy
word2vec = spacy.load('en')

blank_spot = ""

response_a = ""
response_b = ""
response_c = ""

responses= [response_a, response_b, response_c]

class ChatBot:
    def find_intent_match(self, responses, user_message):
        bow_user_message = Counter(preprocess(user_message))
        processed_responses = [Counter(preprocess(response)) for response
in responses]
        similarity_list = [compare_overlap(doc, bow_user_message) for doc
in processed_responses]
        response_index = similarity_list.index(max(similarity_list))
        return responses[response_index]

    def find_entities(self, user_message):
        tagged_user_message = pos_tag(preprocess(user_message))
        message_nouns = extract_nouns(tagged_user_message)

        tokens = word2vec(" ".join(message_nouns))
        category = word2vec(blank_spot)
        word2vec_result = compute_similarity(tokens, category)
        word2vec_result.sort(key=lambda x: x[2])
        return word2vec_result[-1][0]

    def respond(self, user_message):
        best_response = self.find_intent_match(responses, user_message)
        entity = self.find_entities(user_message)
        print(best_response.format(entity))
        print("I hope I was able to help. See ya around!")
        return True

    def chat(self):
        user_message = input("Hey! I'm a bot. Ask me your questions! ")
```

```
self.respond(user_message)

ChatBot = ChatBot()
ChatBot.chat()
```

Deep Learning & Generative Chatbots

Why use deep learning for open-domain chatbots?

As sophisticated as retrieval-based chatbots can get, information retrieval cannot generalize conversation (even with lots and lots of data) or generate anything new. If we want any creativity with the language used in our program's responses — as would be necessary with an open-domain system where any topic is possible — we need to turn to models that facilitate new language patterns. This is where generative models shine.

By using neural networks with many hidden layers — known as deep learning —, generative chatbot models can build sentences that are completely original rather than retrieved from a list of possible responses.

In this section of Build Chatbots with Python, you'll learn about neural networks commonly used in NLP work and how to implement them for both machine translation and generative chatbot purposes. You'll also get to work on your own deep learning-based generative chatbot off-platform!

Long Short Term Memory Networks

Long short-term memory networks (LSTMs) are often used in deep learning programs for natural language processing.

Information Persistence and Neural Networks

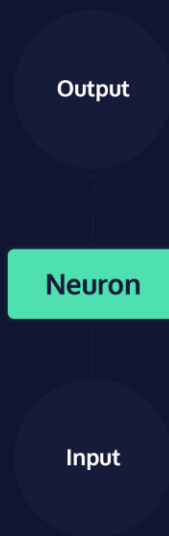
We do not create a new language every time we speak -- every human language has a persistent set of grammar rules and collection of words that we rely on to interpret it. As you read this article, you understand each word based on your knowledge of grammar rules and interpretation of the preceding and following words. At the beginning of the next sentence, you continue to utilize information from earlier in the passage to follow the overarching logic of the paragraph. Your knowledge of language is both persistent across interactions, and adaptable to new conversations.

Neural networks are a machine learning framework loosely based on the structure of the human brain. They are very commonly used to complete tasks that seem to require complex decision making, like speech recognition or image classification. Yet, despite being modeled after neurons in the human brain, early neural network models were not designed to handle temporal sequences of data, where the past depends on the future. As a result, early models performed very poorly on tasks in which prior decisions are a strong predictor of future decisions, as is the case in most human language tasks. However, more recent models, called recurrent neural networks (RNN), have been specifically designed to process inputs in a temporal order and update the future based on the past, as well as process sequences of arbitrary length.

RNNs are one of the most commonly used neural network architectures today. Within the domain of Natural Language Processing, they are often used in speech generation and machine translation tasks. Additionally, they are often used to solve speech recognition and optical character recognition tasks. Let's dive into their implementation!

Neural Networks

A single neuron in a network, *reference to graphic*, takes in a single piece of input data, *reference to graphic*, and performs some data transformation to produce a single piece of output *reference to graphic*.



The very first network models, called [perceptrons](#), were relatively simple systems that used many combinations of simple functions, like computing the slope of a line. While these transformations were very simple in isolation, together they resulted in sophisticated behavior for the entire system and allowed for large numbers of transformations to be strung together in order to compute advanced functions. However, the range of perceptron applications was still limited, and there were very simple functions that they just simply couldn't compute. In attempting to solve this issue by layering perceptrons together, researchers ran into another problem: few computers at the time were able to store enough data to execute these programs.

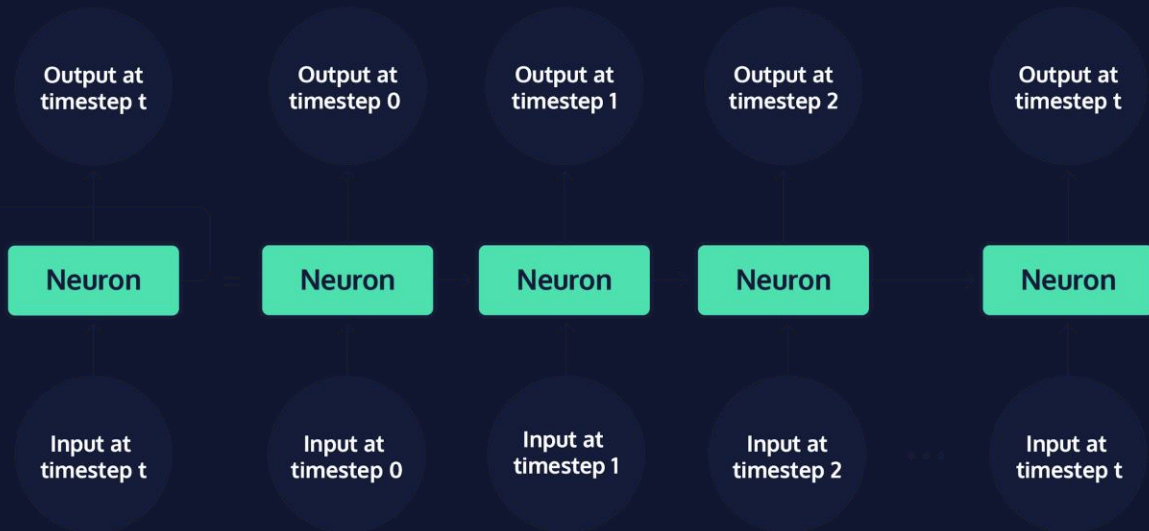
Deep Neural Networks

By the early 2000s, when innovations in computer hardware allowed for more complex modeling techniques, researchers had already developed neural network systems that combined many layers of neurons, including convolutional neural networks (CNN), multi-layer perceptrons (MLP), and recurrent neural networks (RNN). All three of these architectures are called deep neural networks, because they have many layers of neurons that combine to create a "deep" stack of neurons. Each of these deep-learning architectures have their own relative strengths:

MLP networks are comprised of layered perceptrons. They tend to be good at solving simple tasks, like applying a filter to each pixel in a photo.

CNN networks are designed to process image data, applying the same convolution function across an entire input image. This makes it simpler and more efficient to process images, which generally yields very high-dimensional output and requires a great deal of processing.

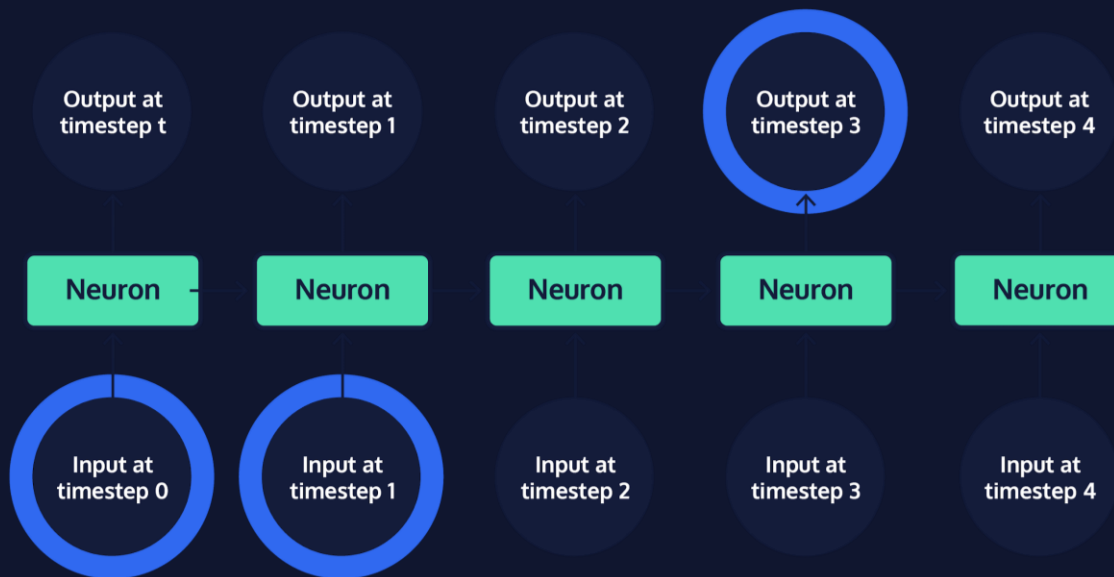
Finally, RNNs became widely adopted within natural language processing because they integrate a loop into the connections between neurons, allowing information to persist across a chain of neurons.



When the chain of neurons in an RNN is “rolled out,” it becomes easier to see that these models are made up of many copies of the same neuron, each passing information to its successor. Neurons that are not the first or last in a rolled out RNN are sometimes referred to as “hidden” network layers; the first and last neurons are called the “input” and “output” layers, respectively. The chain structure of RNNs places them in close relation to data with a clear temporal ordering or list-like structure — such as human language, where words obviously appear one after another. Standard RNNs are certainly the best fit for tasks that involve sequences, like the translation of a sentence from one language to another.

Long-term Dependencies

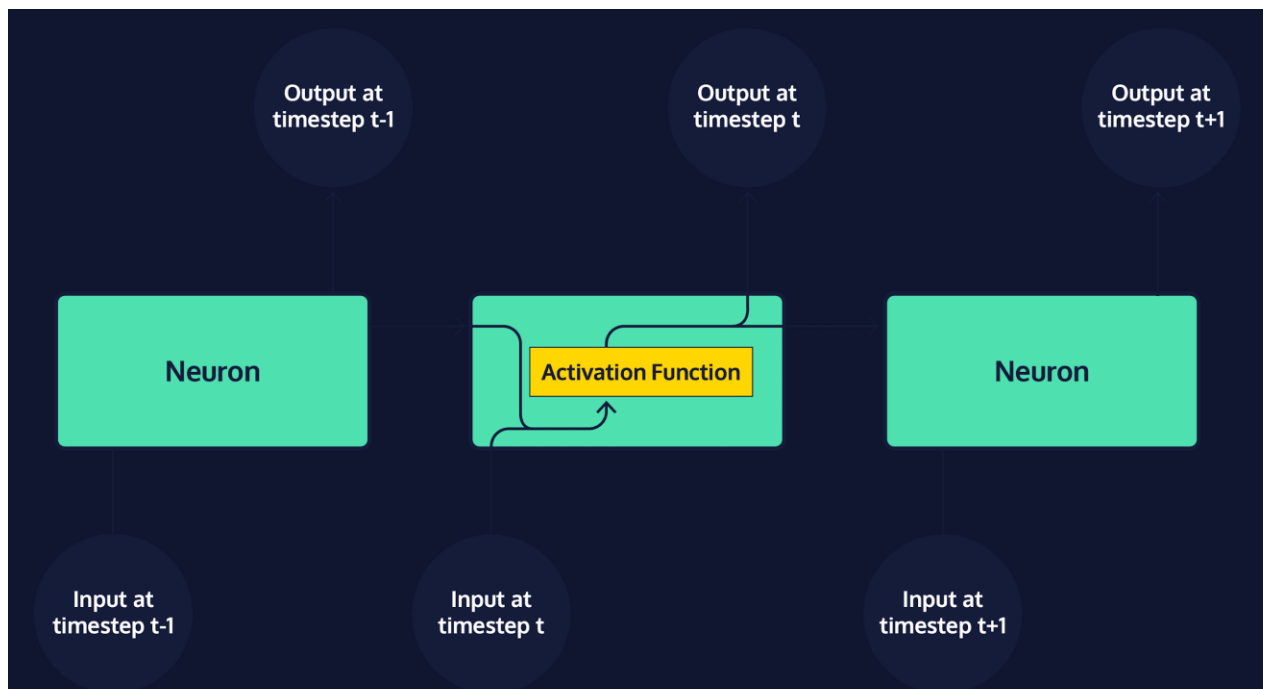
While the broad family of RNNs powers many of today’s advanced artificial intelligence systems, one particular RNN variant, the long short-term memory network (LSTM), has become popular for building realistic chatbot systems. When attempting to predict the next word in a conversation, words that were mentioned recently often provide enough information to make a strong guess. For instance, when trying to complete the sentence “The grass is always: _____,” most English-speakers do not need additional context to guess that the last word is “greener.” Standard RNNs are difficult to train, and fail to capture long-term dependencies well; they perform best on short sequences, when relevant context and the word to be predicted fall within a short distance of one another.



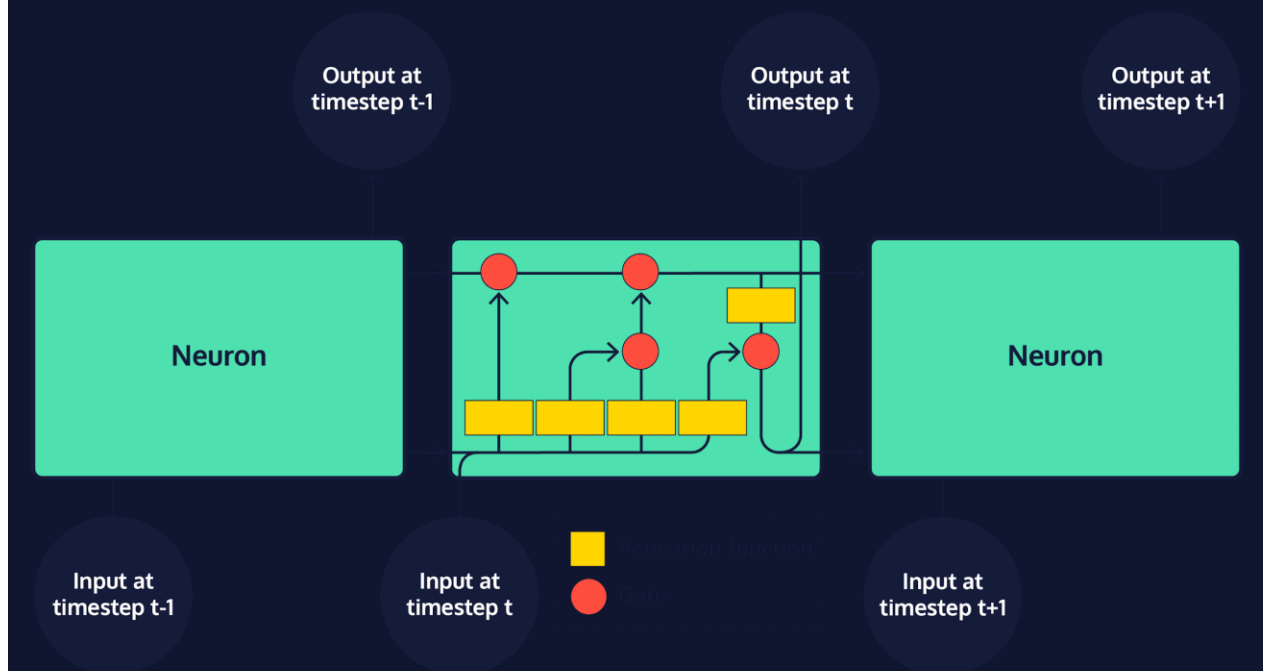
However, there are also many cases of longer sequences in which more context is needed to make an accurate prediction, and that context is less obvious and at a greater lexical distance from the word it determines than in the example above. For instance, consider trying to complete the sentence “It is winter and there has been little sunlight. The grass is always ____.” While the structure of the second sentence suggests that our word is probably an adjective related to grass, we need context from further back in the text to guess that the correct word is “brown.” As the gap between context and the word to predict grows, standard RNNs become less and less accurate. This situation is commonly referred to as the long-term dependency problem. The solution to this problem? A neural network specifically designed for long-term memory – the LSTM!

Long Short-term Memory Networks

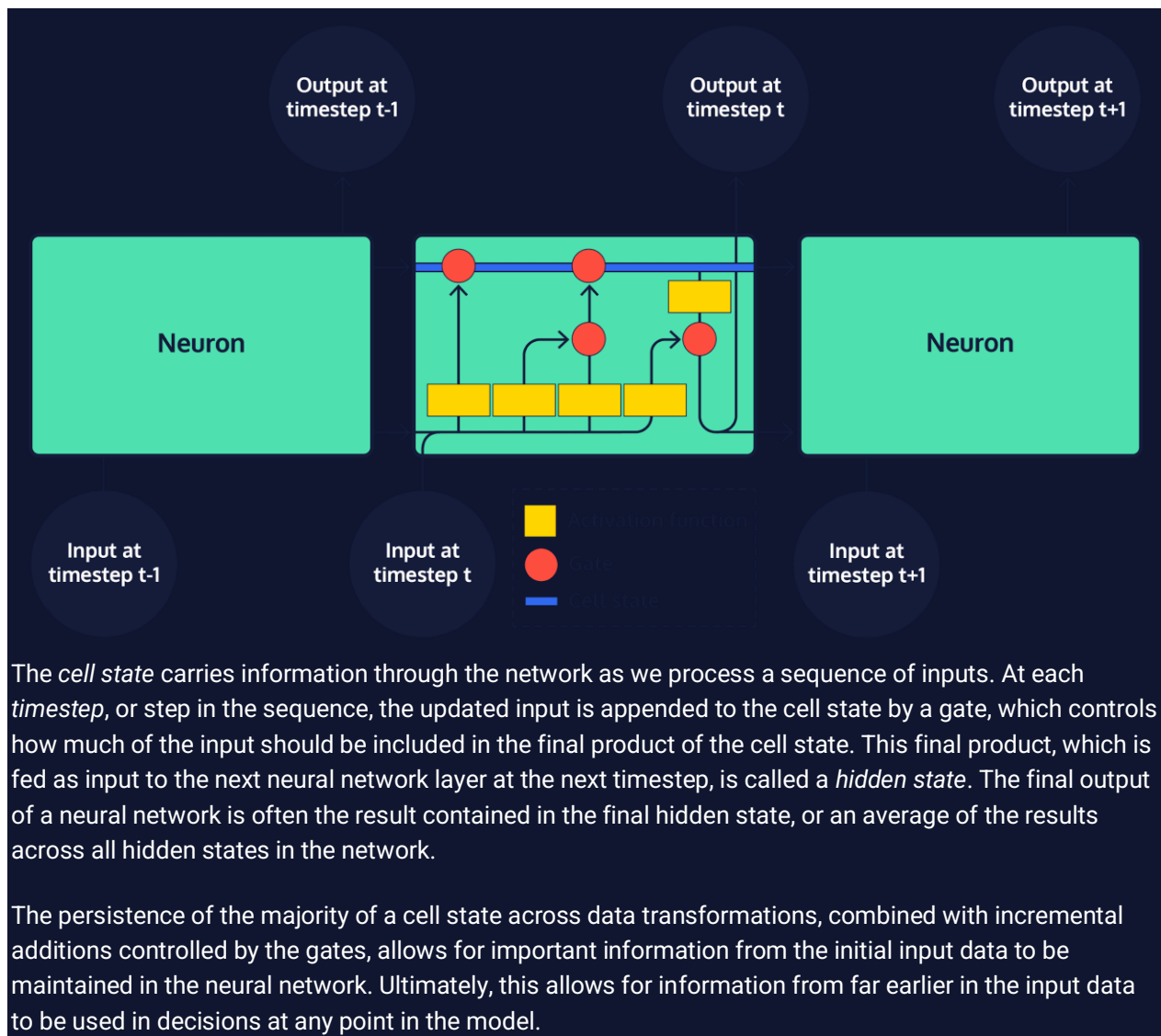
Every model in the RNN family, including LSTMs, is a chain of repeating neurons at its base. Within standard RNNs, each layer of neurons will only perform a single operation on the input data.



However, within an LSTM, groups of neurons perform four distinct operations on input data, which are then sequentially combined.



The most important aspect of an LSTM is the way in which the transformed input data is combined by adding results to *state*, or cell memory, represented as vectors. There are two states that are produced for the first step in the sequence and then carried over as subsequent inputs are processed: cell state, and hidden state.



Generative Chatbots

Introduction to Generative Chatbots

In this lesson, we'll use seq2seq models to build generative chatbots. If you haven't encountered seq2seq models before, take our course on [generating text and machine translation](#).

Seq2seq models were designed for machine translation, but generating dialog can also be accomplished with seq2seq. Rather than converting input in one language to output in another language, we can convert dialog input into a likely corresponding response.

In fact, the only major difference between the code used for a machine translation program and a generative chatbot is the dataset we use to train our model!

What about the three components of closed-domain systems: intent classification, entity recognition, and response selection? When building an open-domain chatbot, intent classification is much harder and an infinite number of intents are possible. Entity recognition is just ignored in favor of the trained "black box" model.

While closed-domain architecture is focused on response selection from a set of predefined responses, generative architecture allows us to perform unbounded text generation. Instead of selecting full sentences, the open-domain model generates word by word or character by character, allowing for new combinations of language.

In the seq2seq decoding function, the decoder generates several possible output tokens and the one with the highest probability (according to the model) gets selected.

Instructions

When you feel ready to create an open-domain chatbot, move on to the next exercise.



Choosing the Right Dataset

One of the trickiest challenges in building a deep learning chatbot program is choosing a dataset to use for training.

Even though the chatbot is more open-domain than other architectures we've come across, we still need to consider the context of the training data. Many sources of "open-domain" data which hopefully will capture unbiased, broad human conversation, actually have their own biases which will affect our chatbot. If we use customer service data, Twitter data, or Slack data, we're setting our potential conversations up in a particular way.

Does it matter to us if our model is trained on "authentic" dialog? If we prefer that the model use complete sentences, then TV and movie scripts could be great resources.

Depending on your use case for the chatbot, you may also need to consider the license associated with the dataset.

Of course, there are ethical considerations as well here. If our training chat data is biased, bigoted, or rude, then our chatbot will learn to be so too.

Once we've selected and preprocessed a set of chat data, we can build and train the seq2seq model using the chosen dataset.

We will be building a rudimentary generative chatbot on the Codecademy platform. You will have the opportunity to build a far more robust chatbot on your own device by increasing the quantity of chat data used (or changing the source entirely), increasing the size and capacity of the model, and allowing for more training time.

Instructions

Checkpoint 1 Passed

1.

For the purposes of this lesson, we'll be working with the [Cornell Movie-Dialogs Corpus](#) to gather fictional conversations from [The Princess Bride](#).

Take a look at the files we have set up:

In **pb.txt** you can see a bit of the raw dialog data — we're only drawing a small sample so that the program can run smoothly on the Codecademy platform.

In **dataset.py** we're preprocessing the dialog so that it is in the format of response pairs, similar to the machine translation pairs we've worked with in the past.

At the bottom of **dataset.py**, print out `dialog_combos` to see the response pairs we'll be using.

```
import more_itertools as mit

data_path = "pb.txt"

# Defining lines as a list of each line
with open(data_path, 'r', encoding='utf-8') as f:
    raw_lines = f.read().split('\n')

raw_lines.reverse()
lines = []

for line in raw_lines:
    # split line into parts
    line_split = line.split(' +++$+++ ')
    # append tuple of character and line
    line_num = int(line_split[0][1:])
```

```

    current_line = line_split[4].strip()
    # append tuple of line num, character and line
    lines.append((line_num, current_line))
# make sure the lines are in order
lines = sorted(lines, key=lambda x: x[0])

# group lines by scene
by_scene = [list(group) for group in mit.consecutive_groups(lines, lambda
x: x[0])]

dialog_only = [[dialog_line[1] for dialog_line in dialog_group]
                for dialog_group in by_scene]

dialog_combos_nested = [list(map(list, zip(dialog_group,
dialog_group[1:]])) for dialog_group in dialog_only]

dialog_combos = [combo for combos in dialog_combos_nested for combo in
combos]

# print dialog combos:
print(dialog_combos)

```

Generative Chatbots

Setting Up the Bot

Just as we built a chatbot class to handle the methods for our rule-based and retrieval-based chatbots, we'll build a chatbot class for our generative chatbot.

Inside, we'll add a greeting

[method](#)

Preview: Docs Loading link description

and a set of exit commands, just like we did for our closed-domain chatbots.

However, in this case, we'll also import the seq2seq model we've built and trained on chat data for you, as well as other information we'll need to generate a response.

As it happens, many cutting-edge chatbots blend a combination of rule-based, retrieval-based, and generative approaches in order to easily handle some intents using predefined responses and offload other inputs to a natural language generation system.

Instructions

Checkpoint 1 Passed

1.

The code we have in **script.py** may look somewhat familiar. So far, it's structurally a rule-based chatbot, using methods that we've employed in other rule-based chatbots on Codecademy.

If you take a look at the `.chat()` method, you'll see that as long as the user inputs something that isn't an exit command, the chatbot will respond with "Cool!". We'll change that soon enough. Our first step is to move the response handling into another method — one that will use seq2seq to generate a response.

Below `.chat()` define a new method: `.generate_response()` with `self` and `user_input` as parameters. Inside the method, return the string `"Cool!\n"`.

Checkpoint 2 Passed

2.

In `.chat()`, change the line that resets `reply` to `input("Cool!\n")`. Now it should set `reply` equal to `input()` called on `self.generate_response(reply)`.

Checkpoint 3 Passed

3.

Let's test out our work!

At the bottom of **script.py**, instantiate a new `ChatBot`.

Call `.generate_response()` on it with whatever input you like (e.g., "I'd love to chat!") and print the result.

Run your script from the bash terminal by typing `python3 script.py`.

To print the resulting output of a function call:

```
print(make_pasta("noodles", "sauce"))
```

Copy to Clipboard

Community Support

```
class ChatBot:

    negative_responses = ("no", "nope", "nah", "naw", "not a chance",
                           "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later",
                     "stop")

    def start_chat(self):
        user_response = input("Hi, I'm a chatbot trained on dialog from The
        Princess Bride. Would you like to chat with me?\n")

        if user_response in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.chat(user_response)

    def chat(self, reply):
        while not self.make_exit(reply):
            # change this line below:
            reply = input(self.generate_response(reply))

        # define .generate_response():
        def generate_response(self, user_input):
            return "Cool!\n"

    def make_exit(self, reply):
        for exit_command in self.exit_commands:
            if exit_command in reply:
                print("Ok, have a great day!")
                return True

        return False
```

```
# instantiate your ChatBot below:
chatty_mcchatface = ChatBot()
print(chatty_mcchatface.generate_response("I'd love to chat."))
```

Generative Chatbots

Generating Chatbot Responses

As you may have noticed, a fundamental change from one chatbot architecture to the next is how the [method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

that handles conversation works. In rule-based and retrieval-based systems, this method checks for various user intents that will trigger corresponding responses. In the case of generative chatbots, the seq2seq test function we built for the machine translation will do most of the heavy lifting for us!

For our chatbot we've renamed `decode_sequence()` to `.generate_response()`. As a reminder, this is where response generation and selection take place:

- The encoder model encodes the user input

- The encoder model generates an embedding (the last hidden state values)

- The embedding is passed from the encoder to the decoder

- The decoder generates an output matrix of possible words and their probabilities

- We use NumPy to help us choose the most probable word (according to our model)

- Our chosen word gets translated back from a NumPy matrix into human language and added to the output sentence

Instructions

Checkpoint 1 Passed

1.

In **script.py**, delete the line that returns "Cool!\n" from `.generate_response()`. We're going to start returning seq2seq generated responses instead!

Tab over to **seq2seq.py** and copy the body of `decode_sequence()` – everything below the `def` line – into `.generate_response()` (in **script.py**).

Note that we're adding deep learning in now, so running the code may take a bit more time again.

```
def ducklings():
    print("We've got ducklings all in a row")

def baby_birds():
    return "So many baby birds!"
```

Copy to Clipboard

For example, given the code above, to copy the body of `baby_birds()` into `ducklings()`, you would do the following:

```
def ducklings():
    return "So many baby birds!"
```

```
def baby_birds():  
    return "So many baby birds!"
```

Copy to Clipboard

Don't forget that Python is sensitive to indentation, so you may need to shift some indentation when you copy the code into `.generate_response()`.

Checkpoint 2 Passed

2.

Try calling `.generate_response()` on `chatty_mcchatface` with a string as an argument. Then run `python3 script.py` in the terminal and check your work.

Why do you think the code is not working?

For example:

```
some_chatbot.generate_response("hi there")
```

Copy to Clipboard

You should get an error that looks like this:

```
AttributeError: 'str' object has no attribute 'ndim'
```

Copy to Clipboard

```
import numpy as np  
from seq2seq import encoder_model, decoder_model, num_decoder_tokens,  
target_features_dict, reverse_target_features_dict, max_decoder_seq_length  
  
class ChatBot:  
  
    negative_responses = ("no", "nope", "nah", "naw", "not a chance",  
"sorry")  
  
    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later",  
"stop")  
  
    def start_chat(self):  
        user_response = input("Hi, I'm a chatbot trained on dialog from The  
Princess Bride. Would you like to chat with me?\n")  
  
        if user_response in self.negative_responses:  
            print("Ok, have a great day!")  
            return  
  
        self.chat(user_response)  
  
    def chat(self, reply):  
        while not self.make_exit(reply):  
            reply = input(self.generate_response(reply))  
  
# update .generate_response():  
def generate_response(self, user_input):  
    states_value = encoder_model.predict(user_input)  
    target_seq = np.zeros((1, 1, num_decoder_tokens))  
    target_seq[0, 0, target_features_dict['<START>']] = 1.
```



```

chatbot_response = ''

stop_condition = False
while not stop_condition:
    output_tokens, hidden_state, cell_state = decoder_model.predict(
        [target_seq] + states_value)

    sampled_token_index = np.argmax(output_tokens[0, -1, :])
    sampled_token = reverse_target_features_dict[sampled_token_index]
    chatbot_response += " " + sampled_token

    if (sampled_token == '<END>' or len(chatbot_response) >
max_decoder_seq_length):
        stop_condition = True

    target_seq = np.zeros((1, 1, num_decoder_tokens))
    target_seq[0, 0, sampled_token_index] = 1.

    states_value = [hidden_state, cell_state]

return chatbot_response

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

chatty_mcchatface = ChatBot()
# call .generate_response():
chatty_mcchatface.generate_response("I'd love to chat.")

```

Generative Chatbots

Handling User Input

Hmm... why can't our chatbot chat? Right now our `.generate_response()` [method](#)

Preview: Docs Loading link description

only works with preprocessed data that's been converted into a NumPy matrix of one-hot vectors. That won't do for our chatbot; we don't just want to use test data for our output. We want the `.generate_response()` method to accept new user input.

Luckily, we can address this by building a method that translates user input into a NumPy matrix. Then we can call that method inside `.generate_response()` on our user input.

We said it before, and we'll say it again: we're adding deep learning in now, so running the code may take a bit more time again.

Instructions

Checkpoint 1 Passed

1.

In `ChatBot`, above `.generate_response()`:

Define a new method called `.string_to_matrix()` with `self` and `user_input` as parameters.

Inside `.string_to_matrix()`, split up the user input string by setting `tokens` equal to `re.findall(r"[\w']+|^[^\s\w]", user_input)`.

Next, generate a NumPy matrix of zeros called `user_input_matrix`:

```
user_input_matrix = np.zeros(
    (1, max_encoder_seq_length, num_encoder_tokens),
    dtype='float32')
```

Copy to Clipboard

Return the `user_input_matrix` NumPy matrix from the method.

Checkpoint 2 Passed

2.

Between where you created `user_input_matrix` and where you returned it from `.string_to_matrix()`, iterate over each `timestep` (which is really the index) and `token` in `enumerate(tokens)`.

Set `user_input_matrix[0, timestep, input_features_dict[token]]` to `1`. This creates a one-hot vector within the matrix for each word in the user input.

To iterate over the indices and elements in a list:

```
for some_index, some_element in enumerate(some_elements):
    # do stuff here
```

Copy to Clipboard

Checkpoint 3 Passed

3.

At the top of the `.generate_response()` method, convert `user_input` into a NumPy matrix using `self.string_to_matrix()`.

Pass the resulting user input matrix into `encoder_model.predict()` instead of the `user_input` string.

The code should look something like this:

```
input_matrix = self.string_to_matrix(user_input)
states_value = encoder_model.predict(input_matrix)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Try calling `.generate_response()` on `chatty_mcchatface` with "What?" as an argument. Print the result and run `python3 script.py` in the terminal so you can see the response generated. Does it work? Now try calling the method with a different argument (e.g., "I love the Princess Bride."). What happens? "What" and "?" are part of the training data and so "What?" should lead to some sort of output. However, most phrases will cause a `KeyError`. Can you guess why?

```

import numpy as np
import re
from seq2seq import encoder_model, decoder_model, num_decoder_tokens,
num_encoder_tokens, input_features_dict, target_features_dict,
reverse_target_features_dict, max_decoder_seq_length, max_encoder_seq_length

class ChatBot:

    negative_responses = ("no", "nope", "nah", "naw", "not a chance", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later", "stop")

    def start_chat(self):
        user_response = input("Hi, I'm a chatbot trained on dialog from The Princess
Bride. Would you like to chat with me?\n")

        if user_response in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.chat(user_response)

    def chat(self, reply):
        while not self.make_exit(reply):
            reply = input(self.generate_response(reply))

    # define .string_to_matrix() below:
    def string_to_matrix(self, user_input):
        tokens = re.findall(r"[\w']+|^[\s\w]", user_input)
        user_input_matrix = np.zeros(
            (1, max_encoder_seq_length, num_encoder_tokens),
            dtype='float32')
        for timestep, token in enumerate(tokens):
            user_input_matrix[0, timestep, input_features_dict[token]] = 1.
        return user_input_matrix

    def generate_response(self, user_input):
        # change user_input into a NumPy matrix:
        input_matrix = self.string_to_matrix(user_input)
        # update argument for .predict():
        states_value = encoder_model.predict(input_matrix)
        target_seq = np.zeros((1, 1, num_decoder_tokens))
        target_seq[0, 0, target_features_dict['<START>']] = 1.

        chatbot_response = ''

        stop_condition = False
        while not stop_condition:
            output_tokens, hidden_state, cell_state = decoder_model.predict(
                [target_seq] + states_value)

            sampled_token_index = np.argmax(output_tokens[0, -1, :])
            sampled_token = reverse_target_features_dict[sampled_token_index]
            chatbot_response += " " + sampled_token

            if (sampled_token == '<END>' or len(chatbot_response) > max_decoder_seq_length):
                stop_condition = True

```

```

target_seq = np.zeros((1, 1, num_decoder_tokens))
target_seq[0, 0, sampled_token_index] = 1.

states_value = [hidden_state, cell_state]

return chatbot_response

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

chatty_mcchatface = ChatBot()
# call .generate_response():
print(chatty_mcchatface.generate_response("What?"))

```

Generative Chatbots

Handling Unknown Words

Nice work! Our chatbot now knows how to accept user input. But there is a pretty large caveat here: our chatbot only knows the vocabulary from our training data. What if a user uses a word that the chatbot has never seen before?

With our current code, we'll get a `KeyError`:

This is because in `.string_to_matrix()` we are looking for `token` in `input_features_dict`:

```

for timestep, token in enumerate(tokens):
    user_input_matrix[0, timestep, input_features_dict[token]] = 1.

```

Copy to Clipboard

Currently, if the token doesn't exist in the `input_features_dict` dictionary (which keeps track of all words in the training data), our program has no way of handling it.

Here are a few popular approaches to tackle unknown words:

- Tell the chatbot to ignore them, which is the simplest fix for smaller datasets, but can never generate those words as output. (Can you imagine scenarios when this could be a problem?)

- Pause the chat process and have the chatbot ask what the entire utterance means. This requires the user to rephrase the entire utterance. This causes issues when working with a fairly limited dataset, since we may end up with the chatbot repeatedly asking the user to rephrase each input statement.

- Add in a step for the chatbot to register any unknown word as a '`<UNK>`' token. This is generally more complicated than the other two solutions. It would require that the training data is built out with '`<UNK>`' tokens and requires several extra manual steps.

Instructions

Checkpoint 1 Passed

1.

If you attempted to call `.generate_response()` with some text that wasn't in the training data, you probably got a `KeyError`. It's time to fix that!

Because we're using a tiny dataset to train our model, we're going to just ignore words that haven't been added into `input_features_dict` during training.

Fortunately, a simple `if` statement is all we need.

Add an `if` clause that checks that `token` is in `input_features_dict` so that a one-hot vector only gets created for a known word:

```

user_input_matrix[0, timestep, input_features_dict[token]] = 1.

```

Copy to Clipboard

To check whether an element exists in a list:

```
if radiohead in great_bands:
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Call `.generate_response()` on `chatty_mcchatface` with any phrase you like as an argument. Print the result and run `python3 script.py` in the terminal so you can see the response generated.

The response may not be great, but now it's running without error!

```
import numpy as np
import re
from seq2seq import encoder_model, decoder_model, num_decoder_tokens,
num_encoder_tokens, input_features_dict, target_features_dict,
reverse_target_features_dict, max_decoder_seq_length, max_encoder_seq_length

class ChatBot:

    negative_responses = ("no", "nope", "nah", "naw", "not a chance", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later", "stop")

    def start_chat(self):
        user_response = input("Hi, I'm a chatbot trained on dialog from The Princess
Bride. Would you like to chat with me?\n")

        if user_response in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.chat(user_response)

    def chat(self, reply):
        while not self.make_exit(reply):
            reply = input(self.generate_response(reply))

    # define .string_to_matrix() below:
    def string_to_matrix(self, user_input):
        tokens = re.findall(r"[\w']+|^[^s\w]", user_input)
        user_input_matrix = np.zeros(
            (1, max_encoder_seq_length, num_encoder_tokens),
            dtype='float32')
        for timestep, token in enumerate(tokens):
            # add an if clause to handle user input:
            if token in input_features_dict:
                user_input_matrix[0, timestep, input_features_dict[token]] = 1.
        return user_input_matrix

    def generate_response(self, user_input):
        input_matrix = self.string_to_matrix(user_input)
        states_value = encoder_model.predict(input_matrix)
        target_seq = np.zeros((1, 1, num_decoder_tokens))
        target_seq[0, 0, target_features_dict['<START>']] = 1.

        chatbot_response = ''

        stop_condition = False
```

```

while not stop_condition:
    output_tokens, hidden_state, cell_state = decoder_model.predict(
        [target_seq] + states_value)

    sampled_token_index = np.argmax(output_tokens[0, -1, :])
    sampled_token = reverse_target_features_dict[sampled_token_index]
    chatbot_response += " " + sampled_token

    if (sampled_token == '<END>' or len(chatbot_response) > max_decoder_seq_length):
        stop_condition = True

    target_seq = np.zeros((1, 1, num_decoder_tokens))
    target_seq[0, 0, sampled_token_index] = 1.

    states_value = [hidden_state, cell_state]

return chatbot_response

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

chatty_mcchatface = ChatBot()
# call .generate_response():
chatty_mcchatface.generate_response("Hello, there!")

```

Generative Chatbots

Putting It All Together

Now that we've addressed handling user input — including unknown words — let's try working with the full generative chatbot!

Instructions

Checkpoint 1 Failed, try again

1.

First, let's try it out! Call `.start_chat()` on `chatty_mcchatface` and run **chat.py** in the terminal.

Remember to use an exit command when you want to finish the conversation.

To run your code:

```
python3 chat.py
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We can have a bit of a back-and-forth with our chatbot, but it could be better. One thing that would make the chat nicer is if we strip the "<START>" and "<END>" tokens from each generated response.

Python strings have a `.replace()` method that can help us out:

```
pot = "potato".replace("ato", "")
```

```
# pot's value is now "pot"
```

Copy to Clipboard

Use `.replace()` to remove "<START>" and "<END>" from `chatbot_response` before it gets returned from `.generate_response()`.

You can either create a new variable to store the stripped string or just reset

`chatbot_response`.

It's possible to chain multiple actions on a single line like this:

```
ot = "potato".replace("ato", "").replace("p", "")
```

```
# ot's value is now "ot"
```

```
import numpy as np
import re
from seq2seq import encoder_model, decoder_model, num_decoder_tokens,
num_encoder_tokens, input_features_dict, target_features_dict,
reverse_target_features_dict, max_decoder_seq_length, max_encoder_seq_length

class ChatBot:

    negative_responses = ("no", "nope", "nah", "naw", "not a chance", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later", "stop")

    def start_chat(self):
        user_response = input("Hi, I'm a chatbot trained on dialog from The Princess
Bride. Would you like to chat with me?\n")

        if user_response in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.chat(user_response)

    def chat(self, reply):
        while not self.make_exit(reply):
            reply = input(self.generate_response(reply))

    def string_to_matrix(self, user_input):
        tokens = re.findall(r"[\w']+|^[^s\w]", user_input)
        user_input_matrix = np.zeros(
            (1, max_encoder_seq_length, num_encoder_tokens),
            dtype='float32')
        for timestep, token in enumerate(tokens):
```

```

        if token in input_features_dict:
            user_input_matrix[0, timestep, input_features_dict[token]] = 1.
        return user_input_matrix

def generate_response(self, user_input):
    input_matrix = self.string_to_matrix(user_input)
    states_value = encoder_model.predict(input_matrix)
    target_seq = np.zeros((1, 1, num_decoder_tokens))
    target_seq[0, 0, target_features_dict['<START>']] = 1.

    chatbot_response = ''

    stop_condition = False
    while not stop_condition:
        output_tokens, hidden_state, cell_state = decoder_model.predict(
            [target_seq] + states_value)

        sampled_token_index = np.argmax(output_tokens[0, -1, :])
        sampled_token = reverse_target_features_dict[sampled_token_index]

        chatbot_response += " " + sampled_token

        if (sampled_token == '<END>' or len(chatbot_response) > max_decoder_seq_length):
            stop_condition = True

        target_seq = np.zeros((1, 1, num_decoder_tokens))
        target_seq[0, 0, sampled_token_index] = 1.

        states_value = [hidden_state, cell_state]

    # remove <START> and <END> tokens
    # from chatbot_response:
    chatbot_response = chatbot_response.replace("<START>", "").replace("<END>", "")

    return chatbot_response

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

chatty_mcchatface = ChatBot()
# call .start_chat():
chatty_mcchatface.start_chat()

```

Generative Chatbots

Review

Hooray! While our chatbot is far from perfect, it does produce innovative babble, allowing us to create a truly open-domain conversation. Just like with a machine translation model, we want to accommodate new sentence structures and creativity, something we can do with a generative model for our chatbot.

Of course, even with a better trained model, there are some issues that we need to consider.

First, there are ethical considerations that are much harder to discern and track when working with deep learning models. It's very easy to accidentally train a model to reproduce latent biases within the training data.

We're also making a pretty big assumption right now in our chatbot architecture: the chatbot responses only depend on the previous turn of user input. The seq2seq model we have here won't account for any previous dialog. For example, our chatbot has no way to handle a simple back-and-forth like this:

User: "Do you have any siblings?"

Chatbot: "Yes, I do"

User: "How many?"

Chatbot: ☹

In addition to topics that have been previously covered, the entire context of the chat is missing. Our chatbot doesn't know anything about the user, their likes, their dislikes, etc.

As it happens, handling context and previously covered topics is an active area of research in NLP. Some proposed solutions include:

- training the model to hang onto some previous number of dialog turns

- keeping track of the decoder's hidden state across dialog turns

- personalizing models by including user context during training or adding user context as it is included in the user input

Instructions

Play around with the chatbot and tweak it a bit!

```
import numpy as np
import re
from seq2seq import encoder_model, decoder_model, num_decoder_tokens,
num_encoder_tokens, input_features_dict, target_features_dict,
reverse_target_features_dict, max_decoder_seq_length, max_encoder_seq_length

class ChatBot:

    negative_responses = ("no", "nope", "nah", "naw", "not a chance", "sorry")

    exit_commands = ("quit", "pause", "exit", "goodbye", "bye", "later", "stop")

    def start_chat(self):
        user_response = input("Hi, I'm a chatbot trained on dialog from The Princess
Bride. Would you like to chat with me?\n")

        if user_response in self.negative_responses:
            print("Ok, have a great day!")
            return

        self.chat(user_response)

    def chat(self, reply):
        while not self.make_exit(reply):
            reply = input(self.generate_response(reply))

    def string_to_matrix(self, user_input):
        tokens = re.findall(r"[\w']+|^[^s\w]", user_input)
        user_input_matrix = np.zeros(
            (1, max_encoder_seq_length, num_encoder_tokens),
            dtype='float32')
        for timestep, token in enumerate(tokens):
            if token in input_features_dict:
                user_input_matrix[0, timestep, input_features_dict[token]] = 1.
```

```

return user_input_matrix

def generate_response(self, user_input):
    input_matrix = self.string_to_matrix(user_input)
    states_value = encoder_model.predict(input_matrix)
    target_seq = np.zeros((1, 1, num_decoder_tokens))
    target_seq[0, 0, target_features_dict['<START>']] = 1.

    chatbot_response = ''

    stop_condition = False
    while not stop_condition:
        output_tokens, hidden_state, cell_state = decoder_model.predict(
            [target_seq] + states_value)

        sampled_token_index = np.argmax(output_tokens[0, -1, :])
        sampled_token = reverse_target_features_dict[sampled_token_index]

        chatbot_response += " " + sampled_token

        if (sampled_token == '<END>' or len(chatbot_response) > max_decoder_seq_length):
            stop_condition = True

        target_seq = np.zeros((1, 1, num_decoder_tokens))
        target_seq[0, 0, sampled_token_index] = 1.

        states_value = [hidden_state, cell_state]

    chatbot_response = chatbot_response.replace("<START>", "").replace("<END>", "")

    return chatbot_response

def make_exit(self, reply):
    for exit_command in self.exit_commands:
        if exit_command in reply:
            print("Ok, have a great day!")
            return True

    return False

chatty_mcchatface = ChatBot()
chatty_mcchatface.start_chat()

```

Off-Platform Project: Generative Chatbot

Apply your deep-learning knowledge and chatbot savvy to build an open-domain generative chatbot.

You've trained a chatbot on movie dialog, but what happens when you use real dialog? In this project, you'll create a generative chatbot using a stream of tweets on a particular topic.

Setting Up

Installing Python 3

The Codecademy platform lets you run Python easily in our learning environment, but to run/write Python on your own computer you'll need to install Python on it, if you haven't already.

[This article will walk through installing Python 3 on your computer.](#)

Installing Tensorflow, Keras, and NumPy

[install Tensorflow](#)

[Keras API](#)

[Anaconda](#)

[Miniconda](#)

[pip](#)

```
python -m pip install --user numpy
```

Set Up The Code

[Download and unzip the folder](#) for this project. You should have following files:

You should also have three **.txt** files of Twitter data:

Take a look at the **[your-topic].txt** file in a notepad or code editor. You should see tweets related to the topic you picked. If you look closer, you may notice that the tweets are grouped into response pairs; each group of two has an initial tweet and a reply tweet.

Preprocessing

`data_path`

`pairs`

`-1`

Training

Note that you may get the following error when attempting to run your program on a regular computer that uses CPU processing:

```
OMP: Error #15: Initializing libiomp5.dylib, but found libiomp5.dylib already initialized.
```

```
OMP: Hint: This means that multiple copies of the OpenMP runtime have been linked into the program. That is dangerous, since it can degrade performance or cause incorrect results. The best thing to do is to ensure that only a single OpenMP runtime is linked into the process, e.g. by avoiding static linking of the OpenMP runtime in any library. As an unsafe, unsupported, undocumented workaround you can set the
```

environment variable `KMP_DUPLICATE_LIB_OK=TRUE` to allow the program to continue to execute, but that may cause crashes or silently produce incorrect results. For more information, please see <http://www.intel.com/software/products/support/>.
Abort trap: 6

Below the import statements on **training_model.py**, there's a line commented out, which you can uncomment to make the program run. However, if you are concerned about your computer crashing, you may want to hold off completing this project until you have access to a faster computer with GPU processing.

The Chatbot

Open **test_model.py** and **chat.py**. You'll see that we've imported several variables from the preprocessing and training steps of the program and loaded the training model. In **chat.py**, create a `ChatBot` class. The `Chatbot` class should have the following:

```
negative_commands

exit_commands
.start_chat()

negative_commands

.make_exit()
    True                                False
.string_to_matrix()
    (1, max_encoder_seq_length, num_encoder_tokens)

tokens = re.findall(r"[\w']+|^[\s\w]", user_input)
user_input_matrix = np.zeros(
    (1, max_encoder_seq_length, num_encoder_tokens),
    dtype='float32')
```

The method should only account for a given timestep and token if the token exists within the `input_features_dict`:

```
for timestep, token in enumerate(tokens):
    if token in input_features_dict:
        user_input_matrix[0, timestep, input_features_dict[token]] = 1.
```

Finally, the method should return the new matrix.

```
.generate_response()                                decode_sequence()
                                                    .string_to_matrix()    user_input
"<START>"      "<END>"
```

At the end of **chat.py**, test out the chatbot by instantiating a `ChatBot` object and calling `.start_chat()` on it. Now run **chat.py** to see the result and try chatting with your bot.

Congratulations on creating a generative chatbot using real conversational data!

BONUS

Some ways to improve your chatbot:

Try out different tweet topic files by changing `data_path` in **twitter_prep.py**, try [using a Twitter scraper](#) to generate more data, or build your own Twitter scraper. Note that the chatbot will generally perform better with shorter tweets and more examples of similar language.

Tweak the number of lines (response pairs) given to the chatbot for training. Sometimes more is better, sometimes less is better, depending on the dataset. Make sure you re-run **training_model.py** again to generate a new model before you run **chat.py** again.

Try adding more chatbot methods to handle specific intents and entities. Currently, an integrated chatbot usually performs better than a purely generative one.

Build Chatbots with Python Capstone

Use your capstone project to show off everything you've learned in Build Chatbots with Python!

Overview

In this capstone project, you will create a presentation that showcases a well-built retrieval-based or generative chatbot system. The purpose of this capstone is to practice building chatbots that use natural language processing and machine learning. Compared to the other projects you have completed thus far, we are requiring few restrictions on how you structure your code. The project is much more open-ended, and you should use your creativity.

Import/Download the Data

The data you use in this project is up to you. You can access chat data using a dataset from [Kaggle](#) or by downloading data from a platform like [Twitter](#), [Slack](#), or [Reddit](#).

Expectations

If you choose to build a retrieval-based chatbot, we want to see the following:

If you choose to build a generative chatbot, we want to see the following:

[transformer model](#)

[seq2seq in PyTorch](#)

Bonus:

[attention in the seq2seq network](#)

Format of Capstone Project

Create a GitHub repository that includes a README file and code for your chatbot in the format of .py files or [Jupyter Notebook](#). Make sure the README file includes the following:

Share your Project

[Post your presentation on the Codecademy forum](#) to show off your hard work!

Good luck!

What is a Portfolio Project?

This project is a little different from other Codecademy projects you've encountered. In this project, you will bring together what you have learned in previous lessons to build a project in your own development environment and publish it to the web!

Create your own files outside of the Codecademy platform

Write your own code with less guided instructions

Use common project management processes to track your progress

How Do Portfolio Projects Work?

We'll provide you with high-level tasks to guide your project to completion, but you will be responsible for deciding how to implement them in your code.

There are many possible ways to correctly fulfill all of these requirements, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Note that there are hints that can assist you, but they will only provide one potential implementation. Do not worry if your program looks different from ours!

Project Prompt

Text Message Analysis

Portfolio projects are important for your work in natural language processing since they let you show off all the awesome skills that you've learned. This project will give you a chance to work with real world language data and analyze it using your very own code.

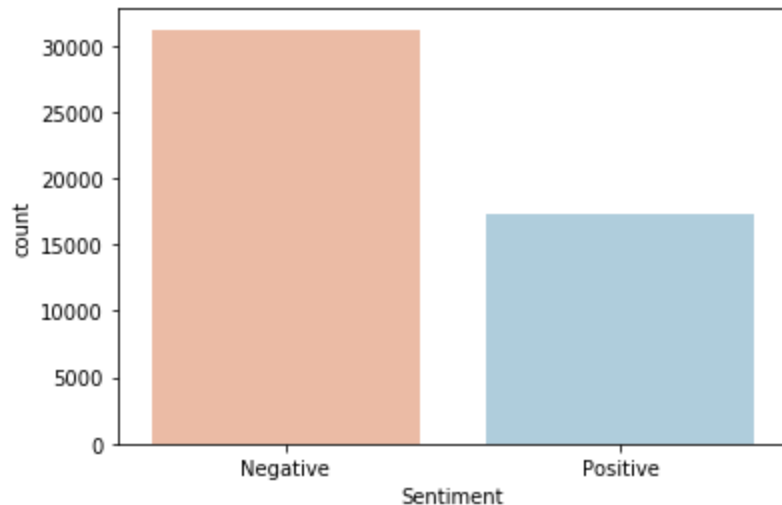
[Click Begin to get started!](#)

For this project, you will be analyzing text message data using Natural Language Processing techniques and libraries.

You are free to use [the dataset that is provided](#) or collect your own SMS data from a service like WhatsApp or just from your device's messaging default app.

You will be diving deeper into NLP skills in order to gain a better understanding of the insights chat data can offer.

Example Project



Project Objectives

- Work locally on your own computer.
- Import and look over your dataset.
- Conduct text-preprocessing.
- Plan and conduct any number of NLP techniques to analyze and gain insight into the data.

These techniques can include:

- Language Parsing
- Word Vectorization
- Language Modeling
- Topic Modeling
- Checking Similarity
- Sentiment Analysis
- POS Tagging
- Named Entity Recognition

Optional Objectives

- Document, organize, and share your findings.
- Make predictions about a dataset's features based on your findings.

Prerequisites

[Natural Language Processing](#)

Project Tasks

Keep track of your progress by dragging each task from "To Do" to "In Progress" to "Done" as you work on them. You can also click on a task to see more information about it.

To Do

In Progress

Done7 / 7 done

Import and look over your dataset

Open the **clean_nus_sms.csv.zip** file and export its contents.

Take a look at the **clean_nus_sms.csv** file. Take note of how the information is organized.

How will this affect the way you analyze the data in Python? Is there anything of particular interest to you in the dataset that you want to investigate? Think about these things before you start preprocessing.

Import `clean_nus_sms.csv` into your Python file. Make sure that the information is easy to access.

Plan your analysis

Now that your data is imported, you can plan what you would like to analyze for your Portfolio Project. What insights would you like to learn and share from this data?

Choose an NLP model or technique

Determine the particular natural language processing models and techniques you will use to conduct your planned analysis.

Conduct text preprocessing

Make sure to clean your data and prepare it for analysis.

Conduct your analysis

Having planned which analyses you wish to conduct on your data, you will now use the NLP skills gained throughout the course (and elsewhere) to analyze the data to gain the insight of your choosing.

Communicate your findings

Share the findings from your analysis. Some good ways to share your insights with others are included in the hint.

Project Extensions

Congrats on completing your Portfolio Project!

You're welcome to expand your analysis beyond what you have already done. Some potential extra features to add to your portfolio project are included in the hint.

Data Scientist: NLP Specialist Career Path Review

Congratulations on all your hard work building your data science knowledge and NLP fluency. Everything you've learned in this Career Path will continue to serve you for years to come!

At this point, you have the technical knowledge, skills, and abilities you need to get your start as a Data Scientist with a specialty in Natural Language Processing.

You are now able to:

- Create programs with Python 3 and R.
- Query and manipulate data with SQL and Python pandas.
- Create data visualizations with Python matplotlib and seaborn.
- Summarize and analyze datasets.
- Conduct hypothesis testing.
- Clean and tidy datasets.
- Communicate your findings clearly.
- Transform unstructured text into data.
- Build a chatbot.
- Decide which machine learning models are most appropriate for solving linguistic problems.
- Perform sentiment analysis & topic modeling.

Plus, you now have several portfolio projects that showcase what you've learned in this Career Path and can share these projects with your community and future employers.

Once again, congratulations on finishing a challenging learning journey! We hope that you continue to build on your skills for years to come.

Next Steps after the Data Scientist: NLP Specialist Career Path

You've completed the Data Scientist: NLP Career Path—what will you learn next?

Congratulations (again) on completing the Data Scientist: Natural Language Processing Career Path! There are a lot of places you can go with your new skills: You could work as a data scientist, in AI Research, or create hobby projects. As you know, there is always more to learn.

So, now that you have finished this learning journey, you may be asking, "What's next?"

Here are our recommendations for next steps:

Interview Preparation

Now that you have completed the Career Path, you are ready to start applying to jobs (though we realize you may have already started this). There is a lot that goes into interviewing for data scientist roles, from developing a portfolio to passing take home, as well as multiple rounds of interviews. We have you covered with a [Data Scientist Interview Preparation](#) Skill Path, where you will learn about the stages in the interview process, practice sample take-home projects, and review everything you need to know to pass the interview and earn a job as a Data Scientist.

Other Areas of Data Science

While you've completed your Data Scientist Career Path with an NLP specialization, there is still a lot more to learn about data science as a field. From here, you can build your visualization skills with [Tableau](#), your [Machine Learning](Machine Learning Skills, or move into Causal Inference.

R

R is the other major programming language in Data Science. You can get a start with the [R for Programmers] Course. If you want to dig deeper, the [Learn R](#) course covers the basics of the language, and there's the [Analyze Data with R](#) Skill Path will help you get started with R for data analysis.

Data Science Events

Events give you a chance to connect with other data scientists and learn about the latest topics in the field. There are a lot of events out there including [PyData](#), [rstudio](#), [Women in Analytics](#), [Gartner Data & Analytics Summit](#), [The Data Science Conference](#), and the list goes on! You can also join or create a [Codecademy meetup](#) to connect with other new data scientists.

Once again, congratulations! We're excited to see what you accomplish next.

